

→ Java Backend Interview Notes

V90 • Samjho, Ratto Nahi • 2026

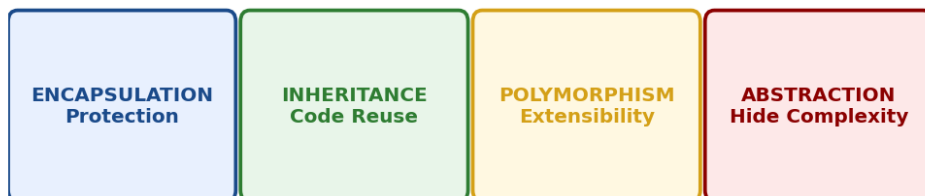
#	Section	Status
1	Core Java & OOP — Pillars, String, HashMap	[OK] Done
2	JDBC — Database Connectivity	[OK] 100%
3	Hibernate & JPA — ORM Framework	[OK] 100%
4	Caching — L1 & L2 Deep Dive	[OK] 100%
5	HQL — Hibernate Query Language	[OK] 100%
6	Spring Core & Dependency Injection	[OK] 100%
7	Spring MVC — Web Framework	[OK] 100%
8	Spring Security — Complete	[OK] 100%
9	JWT — JSON Web Token	[OK] 100%
10	N+1 Problem & LazyInitializationException	[OK] Added
11	Interview Traps — Read Before Every Interview	Before Interview
12	Code Snippets — Ready to Explain	Ready
13	Power Phrases — Say This!	Memorize
14	Introduction Template	Memorize
15	SOLID Principles	[OK] Complete
16	Core Java Extras — Interview Favourites	[OK] Done
17	Multithreading — Ek Saath Kaam	[OK] Done
18	Pass by Value — Java ka Rule	[OK] Done
19	Collections Framework	[OK] Done
20	Design Patterns — Reusable Solutions	[OK] Complete
21	Java 8 — Lambda, FI, Streams, Optional	[OK] Done
22	REST API — Status Codes, Idempotency	[OK] 10/10
23	Brain Ticklers — Deep Concept Traps	[OK] Done
24	MySQL — Database Interview Essentials	[OK] 12/12
25	@Transactional Deep — Proxy + Propagation	[OK] 16/16
26	Spring Data JPA — Repository + Queries	[OK] 16/16
27	@ControllerAdvice — Global Exception	[OK] 16/16
28	Basic Differences — Interview Favorites	[OK] Done
29	Extra Interview Topics — Quick Reference	[OK] Done
30	Extra Concepts — Quick Reference	[OK] Done
31	Microservices — Distributed Architecture	[OK] Part 1
32	Phase 2: Code Visualization — Real Code	[OK] In Progress

Core Java & OOP

OOP Kya Hai

→ WHY: OOP KYUN aaya? C mein data alag (global variables), functions alag. Koi connection nahi. 1000 variables tairte, koi bhi access kare, koi bhi corrupt kare. OOP = data + behaviour EK jagah (class). Protection (private), Grouping (related data), Reuse (new object). C ke 3 problems EK concept se solve.

4 Pillars of OOP



▲ OOP ke 4 Pillars: Encapsulation (data protect), Inheritance (code reuse), Polymorphism (same method, different behaviour), Abstraction (hide HOW, show WHAT)

- Ek baar ek codebase mila — 500 lines, ek hi file.
- User ka naam, age, email — sab alag alag variables.
- Functions mein 10 parameters. Kuch fix karo toh doosri jagah toot jaati thi.
- Tab samjha — data aur kaam alag alag nahi chahiye.
- Ek cheez ke saare data aur kaam ek jagah rakho.
- Ye hai OOP ka asli fayda.
- Object = data + behaviour ek saath.
- Jira ticket: fields=data, resolve/assign=behaviour. Java mein sab object hai.

OOP = real world cheezein code mein. Data + behaviour ek saath ek class mein.

- "OOP represents real-world entities as objects"
- data and behaviour bundled together in a single class."

4 Pillars — OOP ke Backbone

→ WHY: 4 Pillars KYUN? Har pillar EK problem solve karta: Encapsulation = PROTECTION (koi bhi -5000 set na kare). Inheritance = CODE REUSE (100 animals mein eat() copy-paste nahi). Polymorphism = EXTENSIBILITY (naya shape add karo bina if-else). Abstraction = SIMPLICITY (payment.pay() — UPI ya NEFT, caller ko fark nahi). 4 problems, 4 solutions.

- Manager ne bola
- — "salary field public rakha hai
- koi bhi -5000 set kar sakta hai." Tune private kiya getter setter diya
- — Encapsulation.
- Phir bola
- "Manager bhi User hai
- saara User ka code dobara likhna pad raha hai." Tune extends kiya
- — Inheritance.
- Phir ek loop mein sab employees ka bonus calculate karna tha
- Developer, Manager, Intern
- — alag alag formula, ek hi method naam — Polymorphism.
- Aur HR ko sirf result chahiye tha, andar ka formula nahi — Abstraction.
- Chaar alag problems. Chaar alag solutions.
- ENCAPSULATION → Private fields + public methods.
- ATM: buttons dabao, paisa milta hai. Andar kya hota hai → hide.

| **Encapsulation** → Private fields + public getters/setters. Andar ki baat andar.

→ "Encapsulation hides internal state with private fields and exposes behaviour through public methods —

→ like an ATM where you press buttons without touching the circuit."

→ **INHERITANCE** → Child extends Parent → parent ke methods reuse.

→ Dog extends Animal → eat() + sleep() free mein.

| **Inheritance** → Child extends Parent → parent ke sab features mil jaate hain.

→ "Inheritance lets a child class reuse parent class code —

→ Dog extends Animal and gets eat() and sleep() without rewriting them."

→ **POLYMORPHISM** → Same method naam, alag behavior.

→ shape.draw() → Circle hai toh circle, Square hai toh square.

→ Runtime decide.

| **Polymorphism** → Same naam, alag behavior. Runtime pe decide.

→ "Polymorphism means the same method name behaves differently at runtime depending on the object —

→ shape.draw() draws a circle or a square."

→ **ABSTRACTION** → WHAT dikhao, HOW chhupao. TV remote: ON press karo, kaam ho gaya.

→ Andar circuit → hide.

| **Abstraction** → Show WHAT, hide HOW.

→ "Abstraction exposes what a class does, not how —

→ like a TV remote, you press ON without knowing the internal circuit."

Abstract Class vs Interface

→ **WHY:** KAB kya use? Abstract class = partial implementation + STATE (fields). IS-A relationship (Dog IS-A Animal). Interface = contract, CAN-DO behaviour. Multiple interfaces implement ho sakte. Rule: "kya hai" = abstract class, "kya kar sakta" = interface. Java 8 se interface mein default methods aaye lekin STATE abhi bhi sirf abstract class mein.

→ Notification system banate waqt problem aayi.

→ Email, SMS, Push — teeno mein retry logic same, logging same, timestamp same.

→ Teeno mein copy-paste kiya.

→ Kal retry logic badli — teeno jagah change karna pada.

→ Fix: Abstract Class banaya — shared code ek jagah, teeno extend karo.

→ Lekin ek aur cheez — kuch notifications "Approvable" thi, kuch nahi.

→ Ye IS-A nahi, CAN-DO tha.

→ Isiye Interface banaya.

→ Rule: Shared code chahiye = Abstract Class. Capability define karni = Interface.

→ Shared code chahiye → Abstract Class (IS-A).

→ Capability define karni hai → Interface (CAN-DO).

→ Ek class → sirf 1 abstract extend, multiple interface implement.

Feature	Abstract Class	Interface
Constructor	YES (via super())	NO
Methods	abstract + normal dono	Java 8+: default+static bhi
Inheritance	SIRF ek parent	Multiple interfaces allowed
Variables	koi bhi type	public static final ONLY
Use when	Shared code + partial impl	Define a capability/contract

[TRAP] TRAP: Abstract Class = IS-A relationship. Interface = CAN-DO capability.

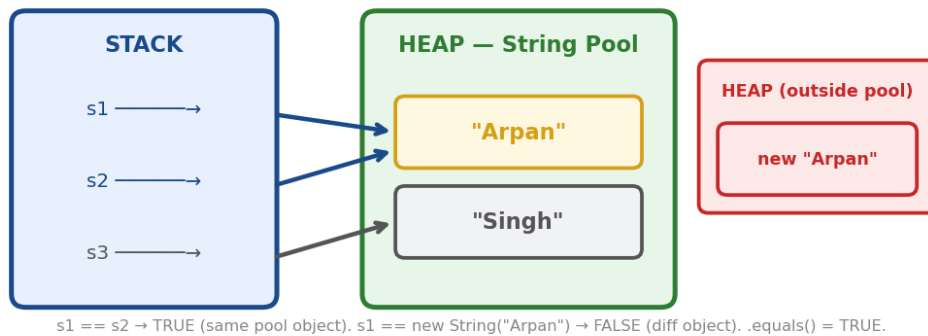
→ "Abstract class is for shared base implementation — IS-A relationship.

→ Interface is a capability contract — CAN-DO.

→ A class can implement multiple interfaces but extend only one abstract class."

String — Immutable Kyun Hai?

String Pool — WHY Immutable + Pool Works



▲ s1="Arpan", s2="Arpan" = SAME pool object (s1==s2 TRUE). new String("Arpan") = DIFFERENT heap object (== FALSE). .equals() = TRUE.

- Ek baar login system mein bug aaya.
- Username validate karne ka function tha — usme pass kiya "arpan".
- Function ne internally modify kiya.
- Wapas aaya toh original corrupt tha, wrong audit log gaya.
- Tab socha — String agar mutable hoti toh ye problem hamesha rehti.
- Java ne isliye String ko immutable banaya — ek baar bana, koi nahi badal sakta.
- Aur String Pool tab hi kaam karta hai jab immutable ho
- warna ek ne badla, sab ke liye badal gaya.
- s1 = "Arpan", s2 = s1 → dono same object.
- Agar mutable hoti → s1 change karo, s2 bhi change!
- Isliye immutable — safe sharing.

```
s = s + " Singh" "Arpan" wala object safe  
naya "Arpan Singh" bana →  
s naye ko point kare
```

WHY immutable — 3 reasons:

1. String Pool possible ho sake → same value = same object → memory save.
 2. Thread safe → multiple threads same String use karein → change nahi ho sakti.
- HashMap key safe → key change ho jaaye → hashCode alag → value milega hi nahi!
 - "String is immutable because it is a shared object"
 - if one reference changes it, all references would be affected.
 - Immutability enables the String Pool thread safety
 - and safe use as HashMap keys."
 - String Pool: s1 = "Arpan" → pool mein bana.
 - s2 = "Arpan" → same pool object.
 - new String("Arpan") → pool bypass, heap mein naya.
 - == = object check, .equals() = value check.

[TRAP] TRAP:

- == = same object check. .equals() = same value check.
- HAMESHA .equals() use karo.

| [OK] KEY: s1 == s2 → true (pool). s1 == s3 → false (heap). s1.equals(s3) → true.

- "String literals go to the String Pool"
- same value reuses the same object in memory.
- new String() forces a new heap object.
- Always use .equals() for value comparison, never ==."

[WARNING] WEAK SPOT (Quiz mein galat):

- String ke methods (concat, toUpperCase, replace)
- ORIGINAL string nahi badate
- hamesha NEW String return karte hain!
- s.concat(" World")
- karo aur result save nahi kiya toh original s unchanged rahega.
- Hamesha s = s.concat(...) likhna padta hai.

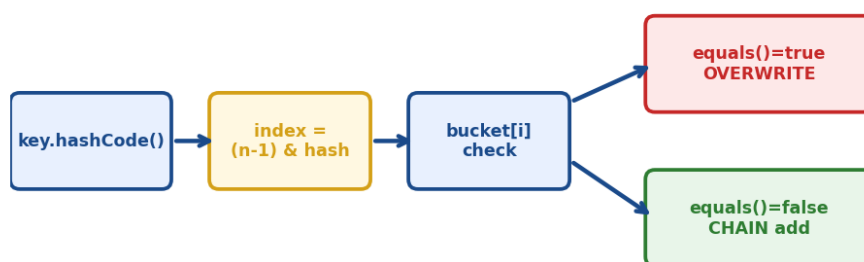
String	Immutable. Har change = naya object. Loop mein = memory waste. [X]
StringBuilder	Mutable. Thread safe NAHI. Fast — mostly yahi use karo.
StringBuffer	Mutable. Thread safe HAI. Thoda slow — sirf multithreading mein.

- "Use StringBuilder for string manipulation in loops
- it is mutable and faster.
- Use StringBuffer only in multithreaded code where thread safety is needed."
- String = shared → ek ne badla toh sabka badle → isliye immutable.
- Compare = .equals(). Loop = StringBuilder.

HashMap — Internal Working

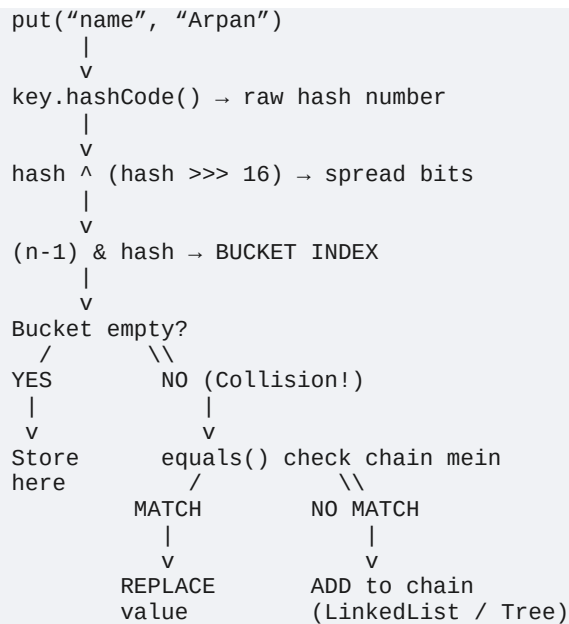
- WHY: HashMap O(1) KYUN? Array mein index se access = O(1). HashMap = key ko number mein convert (hashCode) → number se index nikalo ((n-1) & hash) → SEEDHA us index pe jao. 10 lakh entries mein bhi TURANT. Capacity power of 2 KYUN? Bitwise AND = 1 CPU instruction = modulo se FAST. Load factor 0.75 = Poisson distribution pe sweet spot.

HashMap put(key, value) Flow



▲ `put(key, value)`: key → `hashCode()` → index → bucket. Collision? Chain (LinkedList/Tree). `get()`: same path, `equals()` se match.

put(key, value) INTERNAL FLOW:



Java 8: chain 8+ nodes + table 64+ → Red-Black Tree
Average: $O(1)$ | Worst: $O(\log n)$ tree, $O(n)$ chain

- Soch tune ek library manage karni hai — 10,000 books hain.
- Koi aaya: "Harry Potter chahiye." Ek ek check karega? $O(n)$? Nahi.
- Library ne system banaya — book ka naam se shelf number calculate.
- "Harry Potter" → Shelf 42. Seedha wahan jaao.
- Exactly yehi HashMap karta hai — hashCode → index → seedha bucket → $O(1)$.
- Library: har book ka shelf number = hashCode().
- put("name", "Arpan") → hashCode → index → store.
- get("name") → same index → $O(1)$.

```

put("name", "Arpan") "name".hashCode()
index calculate us index pe store
get("name") → same hashCode → same index → seedha value lo →  $O(1)$ 
  
```

- Collision: do keys ka same index → LinkedList us bucket mein.
- Java 8: 8+ entries → LinkedList → Red-Black Tree → $O(n)$ se $O(\log n)$.

[TRAP] TRAP:

- HashMap null key ALLOWED.
- ConcurrentHashMap mein null key/value → NullPointerException!

WHY null banned? get(key) → null aaya. Key nahi hai ya value null?.

- Single thread mein containsKey() check karo.
- Multi thread mein get() aur containsKey()
- ke BEECH doosre thread ne data badal diya — answer galat!.
- Race condition.
- Null banned = ambiguity hi mat aane do.
- "HashMap uses hashCode() to find the bucket index and equals()
- to resolve collisions in the LinkedList.
- Java 8 converts the bucket to a Red-Black Tree when it has more than 8 entries
- $O(n)$ becomes $O(\log n)$."

hashCode → index → store. Collision = LinkedList. Java 8: 8+ entries → Tree.

HashMap vs ConcurrentHashMap

- Ek baar 2 background threads ek saath HashMap mein data daal rahe the.
- Kuch requests pe data corrupt ho raha tha — entries gayab, kuch overwrite.
- Root cause: HashMap thread-safe nahi hai.

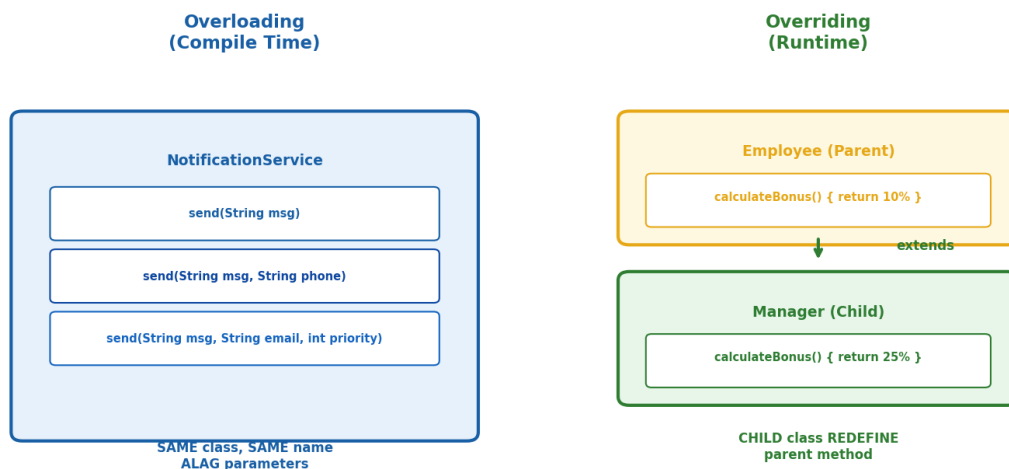
- Dono threads ne ek saath same bucket mein likh diya — ek ka data gaya.
- ConcurrentHashMap ne fix kiya — bucket-level locking.
- Alag buckets pe alag threads, bina poora map lock kiye.
- HashMap: 2 threads ek saath → race condition, data corrupt.
- ConcurrentHashMap → bucket-level locking → alag buckets pe alag threads → safe.

HashMap	NOT thread-safe. Single thread mein fast. Multiple threads = data corrupt.
ConcurrentHashMap	Thread-safe. Segment-level locking — poora map lock nahi hota.

[TRAP] TRAP: null key allowed in HashMap — ConcurrentHashMap mein NOT allowed.

- "HashMap is not thread-safe — use it in single-threaded code."
- ConcurrentHashMap uses segment-level locking so multiple threads can read and write concurrently without corrupting data."

Method Overloading vs Overriding



Overloading = SAME class, SAME naam, ALAG params (compile time). Overriding = CHILD class parent method REDEFINE (runtime).

- Notification service mein sawaal aaya — sendNotification ek rakhun ya alag alag?.
- Ek jagah sirf message, ek jagah message + phone, ek jagah message + email + priority.
- Same naam, alag parameters — Overloading, compile time pe decide hota hai.
- Aur jab Manager ne apna alag calculateBonus() likha parent ke upar se
- Overriding, runtime pe decide.
- Dono alag problems ke alag solutions.
- Overloading: same class, same naam, alag parameters → compile time.
- Overriding: child parent method redefine kare → runtime.
- @Override hamesha lagao.

Overloading	Same class, same naam, alag parameters. Compile time = Static Polymorphism.
Overriding	Child class parent method redefine kare. Runtime = Dynamic Polymorphism.

[TRAP] TRAP:

- @Override annotation HAMESHA lagao — typo pakdega compiler.
- Private/Static methods override nahi hote.
- "Overloading is compile-time polymorphism"
- same method name, different parameters in the same class.
- Overriding is runtime polymorphism — child class redefines a parent method.
- Always use @Override so the compiler catches mistakes."

Checked vs Unchecked Exceptions

- File padhne ka code likha. Compiler ne roka — "Agar file nahi mili toh?"
- Handle karo ya declare karo." Ye Checked Exception hai
- — compiler ko pehle se pata hai ye risky operation hai, force karta hai socho.
- NullPointerException pe compiler kabhi nahi roka
- kyunki runtime pe hota hai, pehle se predict nahi hota.
- Isliye do categories. Checked = compiler ka darwaan.
- Unchecked = runtime ka dhoka.
- Checked: compile time check → handle karo ya throws karo. Example: IOException.
- Unchecked: runtime fail → compiler nahi puchta. Example: NullPointerException.

Checked	Compile time check. try-catch ya throws declare karo. Example: IOException, FileNotFoundException.
Unchecked	Runtime fail. Compiler nahi puchta. Example: NullPointerException, ArrayIndexOutOfBoundsException.

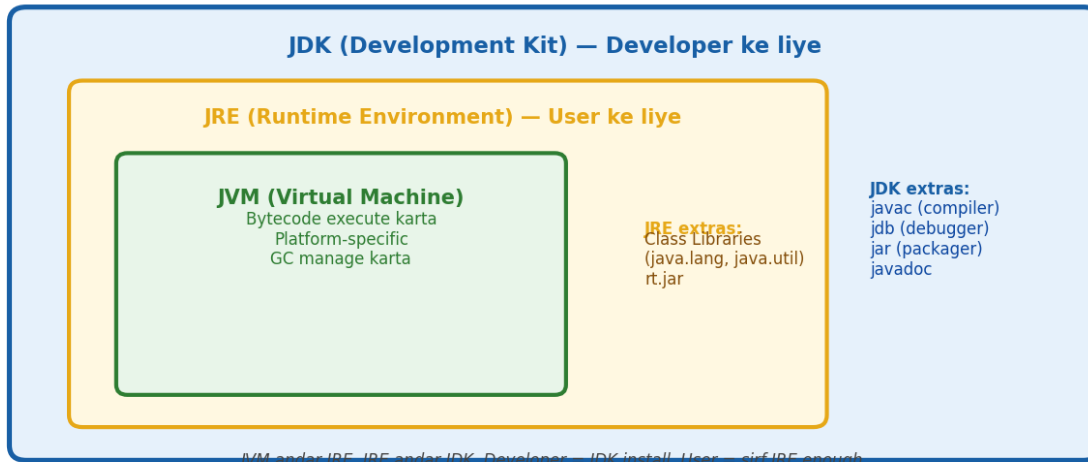
[TRAP] TRAP:

- @Transactional + Checked Exception = NO rollback by default!
- rollbackFor explicitly likhna MUST hai!
- "Checked exceptions are verified at compile time
- you must handle or declare them.
- Unchecked exceptions happen at runtime.
- Critical point: @Transactional only rolls back on RuntimeException by default
- — for IOException you must add rollbackFor = IOException.class."
- Checked = Compiler ka darwaan. Unchecked = Runtime ka dhoka.
- @Transactional sirf RuntimeException pe rollback.

Core Java Extras

- Ye sab 2-minute questions hain. Ghalat hua toh impression kharab hota hai.
- Ek ek padh lo.

JDK vs JRE vs JVM



JVM andar JRE, JRE andar JDK. Developer — JDK install. User — sirf JRE enough.

JVM andar JRE andar JDK. Developer = JDK. User = JRE. JVM = bytecode execute + GC.

→ WHY: JDK vs JRE vs JVM KYUN alag? JVM = bytecode execute kare (WORA ka foundation). JRE = JVM + libraries (run karne ke liye enough). JDK = JRE + compiler + tools (develop karne ke liye). Nested hain: JDK > JRE > JVM. Developer ko JDK chahiye, user ko sirf JRE.

JDK vs JRE vs JVM



▲ JDK > JRE > JVM: Developer = JDK (compiler+tools). User = JRE (libraries+JVM). JVM = bytecode execute. Nested structure.

- HelloWorld.java → compile → HelloWorld.class (bytecode) → JVM run karta hai.
- JDK = JRE + compiler + tools.
- JRE = JVM + libraries. $JDK \supset JRE \supset JVM$.
- JVM — bytecode ko machine pe chalata hai.
- Har OS pe alag JVM — isliye Java "Write Once, Run Anywhere."
- JRE — JVM + libraries. Sirf run karna hai? JRE kaafi hai.
- JDK — JRE + compiler + debugger + tools. Develop karna hai? JDK chahiye.
- | $JDK \supset JRE \supset JVM$
- Bada dabba → chota dabba → aur chota dabba.

[TRAP] TRAP: Server pe sirf run karna hai → JRE. Develop karna hai → JDK.

- "JVM executes bytecode. JRE is JVM plus libraries needed to run Java.
- JDK is JRE plus compiler and dev tools
- JDK contains JRE which contains JVM."

static keyword

- Tune Student class banayi.
- School ka naam — har student ka same hai "DPS".
- Agar har Student object apna alag school naam store kare
- 1000 students = 1000 baar same string memory mein.
- Bekar. static field = class ka property —
- sab share karte hain, ek baar memory.
- static method = object banaye bina call karo.
- Math.sqrt() yaad karo — naya Math() nahi kiya kabhi.
- static = class ka → sab share karte hain.
- Non-static = object ka → har object ka apna.
- Student.school → object ki zarurat nahi.

```
class Student {  
    static String school = "DPS" // class ka — sab share karte hain  
    String name; // student ka — har ek ka apna  
}  
Student.school // object ki zarurat nahi — directly access [OK]
```

[TRAP] TRAP:

- Static method mein this use nahi kar sakte — this = object ka reference.
- Static method tab exist karta hai jab object bana hi nahi hota!
- "Static members belong to the class, not to any instance —
- shared across all objects and accessible without creating an object."

final keyword — 3 Jagah Use Hota Hai

- Tune PI ka value store kiya — 3.14159.
- Koi aur developer ne accidentally PI = 3 kar diya. Bug.
- Tune config values define kiye
- koi service ne override kar diya production mein. final = lock laga do
- — ye nahi badlega.
- Variable pe, method pe, class pe
- teeno jagah same intent: ye change nahi hona chahiye.
- final variable → reassign nahi. final method → override nahi.
- final class → extend nahi.
- Teeno ka ek hi matlab: change nahi hoga.

```
final int x = 10;  
x = 20; // [TRAP] compile error — value change nahi hogi  
final class String { }  
class MyString extends String { } // [TRAP] compile error — extend nahi ho sakti  
final void show() { } // [TRAP] child override nahi kar sakta
```

[TRAP] TRAP:

- final variable ka matlab sirf reassign nahi
- object ke andar ke fields change ho sakte hain!

WHY andar badal sakta? final = LABEL lock hai, DABBA nahi.

- Jaise ghar ka address final
- ghar nahi badlega, lekin andar furniture rearrange kar sakte ho.
- list = new ArrayList() = ghar badalna = NAHI.
- list.add() = andar furniture = HAAN.

```
final List<String> list = new ArrayList<>();  
list.add("Arpan"); // [OK] allowed — andar change kiya  
list = new ArrayList<>(); // [TRAP] nahi — reference change nahi hoga
```

- "final on a variable prevents reassignment."

- final on a class prevents inheritance.
- final on a method prevents overriding —
- String class is final which is part of why it is immutable."

this vs super

- Tune Dog class banayi — extends Animal.
- Dog ke constructor mein Animal ko bhi initialize karna tha —
- naam, age. super(name, age) se parent ka constructor call kiya.
- Aur Dog ke andar apna naam field tha, Animal mein bhi
- this.name = Dog ka, super.name se Animal ka.
- Bina ye samjhe constructor chaining mein compile errors aate hain.
- this = current object ka reference. super = parent ka reference.
- super() → pehli line mein hona chahiye constructor mein.

```
class Animal {
    String name = "Animal"
    Animal() { System.out.println("Animal created"); }
}
class Dog extends Animal {
    String name = "Dog"
    Dog() {
        super(); // parent constructor call
        System.out.println(super.name); // "Animal" — parent ka field
        System.out.println(this.name); // "Dog" — apna field
    }
}
```

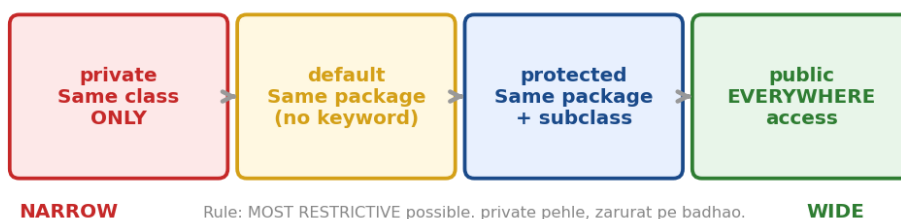
[TRAP] TRAP: super() hamesha constructor ki PEHLI line honi chahiye — warna compile error!

[TRAP] TRAP: Static method mein this aur super dono nahi — dono object se related hain.

- "this refers to the current object. super refers to the parent class —
- used to call the parent constructor or access parent fields hidden by the child."

Access Modifiers

Access Modifiers — Scope (Narrow → Wide)



▲ private (same class) → default (same package) → protected (+ subclass) → public (everywhere). Most restrictive first.

- Tune payment system banaya.
- Card number = koi bhi seedha access na kare — private.
- Transaction ID = same package mein share karo — default.
- Verification logic = subclasses mein bhi kaam aaye — protected.
- API response format = sab ke liye — public.
- Galat modifier lagaya toh ya security hole hai ya unnecessary restriction.
- private → same class only. default → same package.
- protected → same package + subclass. public → sab.

private → sirf apni class — bahar koi nahi
 default → same package ke sab — alag package nahi
 protected → same package + subclass (alag package mein bhi) [OK]
 public → sab — koi restriction nahi

Modifier	Same Class	Same Package / Subclass / Everywhere
private	[OK]	[X] [X] [X]
default	[OK]	[OK] [X] [X]
protected	[OK]	[OK] [OK] [X]
public	[OK]	[OK] [OK] [OK]

[TRAP] TRAP:

- default matlab koi modifier nahi likha — public nahi hota!
- Alag package ki subclass bhi access nahi kar sakti.

[TRAP] TRAP:

- protected = same package + alag package ki subclass bhi.
- Yehi default aur protected ka fark hai.
- "private is same class only. default is package-level.
- protected allows subclass access even across packages.
- public is accessible everywhere —
- always use the most restrictive modifier possible."

Wrapper Classes — Autoboxing & Unboxing

- Tune integers ki list banayi — List<int> list. Compile error.
- Collections mein sirf objects jaate hain, primitives nahi.
- Integer wrapper class use karo.
- Aur Java itna smart hai ki ye conversion khud karta hai
- Integer a = 10 likhte hi Java ne Integer.valueOf(10) kar diya.
- Ye autoboxing hai.
- Ek trap
- Integer cache -128 to 127 tak hai, uske baad == se compare karo toh false aayega.

WHY? -128 se 127 tak ke numbers BAHUT zyada use hote hain

- loop counters, array indexes, age, quantity.
- Java ne socha — har baar naya object banane se memory waste hogi.
- Toh ye common range pehle se cache kar li.
- Bahar ka number rare hai, uske liye naya object theek hai.
- Collections mein objects chahiye → int nahi, Integer chahiye.
- Autoboxing: Java khud Integer.valueOf(10) kar deta hai.
- Unboxing: Java khud intValue() kar deta hai.

int → Integer	double → Double
char → Character	boolean → Boolean

→ Autoboxing — Java khud suit pehna deta hai:

```
Integer a = 10; // tune int diya → Java ne Integer.valueOf(10) kiya [OK]
```

→ Unboxing — Java khud suit utaar deta hai:

```
int b = a; // tune Integer diya → Java ne a.intValue() kiya [OK]
```

→ Integer cache: -128 to 127 → same object. a=127, b=127 → a==b true.

→ a=128, b=128 → a==b false (naye objects).

→ Hamesha .equals() use karo.

```
Integer a = 127; Integer b = 127;  
a == b // true [OK] – same cached object  
Integer a = 128; Integer b = 128;  
a == b // false [X] – naye alag objects
```

[TRAP] TRAP:

→ Integer compare karne mein bhi hamesha .equals() use karo

→ == kabhi true kabhi false dega!

→ "Wrapper classes wrap primitives as objects — needed for Collections.

→ Autoboxing converts primitive to wrapper automatically via Integer.valueOf().

→ Unboxing does the reverse via intValue().

→ Integer caches -128 to 127 — always use .equals() for comparison."

abstract keyword

→ Tune Shape class banayi. area() method likhna chahta tha

→ lekin Shape abstract hai, kaunsa formula?.

→ Circle ka alag, Rectangle ka alag.

→ Shape khud area calculate nahi kar sakti

→ — sirf ye guarantee deti hai ki har shape ka area hoga.

→ abstract method = body nahi, sirf declaration.

→ Child implement karo — warna child bhi abstract ban jaayegi.

→ Shape → area() pata nahi. Circle → $3.14 * r * r$. Rectangle → $l * b$.

→ Shape abstract → body nahi, child implement kare. new Shape() → error.

```
abstract class Shape {  
    abstract double area(); // body nahi — sirf declaration  
    void display() { ... } // normal method — ye likh sakti hai  
}  
class Circle extends Shape {  
    double area() { return 3.14 * r * r; } // [OK] implement kiya  
}  
class Rectangle extends Shape {  
    double area() { return l * b; } // [OK] implement kiya  
}
```

[TRAP] TRAP: Abstract class ka object nahi ban sakta! new Shape() → [TRAP] error.

[TRAP] TRAP:

→ Child ne saare abstract methods implement nahi kiye

→ child bhi abstract ban jaayegi!

→ "Abstract class is an incomplete blueprint

→ it can have both abstract methods with no body and normal methods.

→ Subclasses must implement all abstract methods.

→ You cannot instantiate an abstract class directly."

| 3 Traps — Quiz Mein Galat Hue, Ab Yaad Rakhenge

[TRAP] TRAP 1 — static abstract = Illegal Combo

→ static = class ka hai, override nahi hota. abstract = child implement kare.

→ Dono contradictory hain — ek saath nahi chal sakte. Compile error aayega.

```
abstract class A {  
    static abstract void show(); // [X] COMPILE ERROR  
}
```

[TRAP] TRAP 2 — Interface mein private method Java 9 se

→ Java 9 se interface mein private methods allowed hain.

→ Sirf internal helper ke liye — default/static methods ke andar use hote hain.

→ Bahar se access nahi hota.

```
interface A {  
    private void helper() { ... } // [OK] Java 9+ valid  
    default void show() { helper(); } // sirf andar use karo  
}
```

[TRAP] TRAP 3 — Abstract class ka object KABHI nahi banta

- Chahe ek bhi abstract method na ho
- agar class abstract declare ki hai toh direct object nahi ban sakta.
- Hamesha compile error.

WHY?

- Abstract class adhoori hai
- ya toh methods ka body nahi
- ya tune khud bola "ye sirf extend karne ke liye hai."
- Soch: Vehicle ka start() abstract hai
- — new Vehicle() pe start() mein kya chalega?
- Kuch nahi. Isliye Java rokta hai — adhoori cheez se real object mat banao.

```
abstract class A { } // koi abstract method nahi  
new A(); // [X] COMPILE ERROR — abstract hai toh bas nahi banta
```

Immutable Class

→ WHY: Immutable class KYUN banate? Thread safety FREE (koi modify nahi kar sakta = no sync). HashMap key safe (hashCode kabhi nahi badle). Defensive programming — object pass karo, koi tamper nahi karega.
HOW: final class + final fields + no setters + defensive copy in constructor + defensive copy in getters.

- Tune BankAccount class banayi.
- Kisi function mein pass kiya — us function ne andar account.setBalance(0) kar diya.
- Bug.
- Agar account immutable hota — koi change hi nahi kar sakta tha.
- 4 rules: final class, private final fields, no setter, constructor se set karo.
- Aur andar List ho toh defensive copy lagao — warna reference se modify ho jaayega.
- BankAccount pass kiya, kisi ne balance 0 kar diya!
- Immutable: 4 rules
- final class + private final fields + no setter + constructor-only init.
- List ke liye defensive copy.

```
// 4 rules — bas itne
public final class BankAccount { // Rule 1 — final, extend nahi hogi
    private final String owner; // Rule 2 — private + final
    private final int balance;
    public BankAccount(String o, int b) { // Rule 3 — constructor se set karo
        this.owner = o;
        this.balance = b;
    }
    public String getOwner() { return owner; } // Rule 4 — sirf getter
    public int getBalance() { return balance; } // koi setter NAHI
}
// Badlana ho? Naya object banao — purana safe
BankAccount old = new BankAccount("Arpan", 5000);
BankAccount updated = new BankAccount("Arpan", 4000); // old wahi hai [OK]
// [TRAP] TRAP — andar List ho toh?
public BankAccount(List<String> txns) {
    this.txns = txns; // [X] bahar wali list ka reference!
} // koi bhi modify kar sakta hai!
// Fix — defensive copy
public BankAccount(List<String> txns) {
    this.txns = new ArrayList<>(txns); // [OK] apni copy banao
}
public List<String> getTxns() {
    return new ArrayList<>(this.txns); // [OK] copy return karo
}
```

[TRAP] TRAP —

- final sirf reassign rokta hai
- andar List ho toh us List ka content badal sakta hai!
- Defensive copy lagao constructor mein aur getter mein.
- "An immutable class uses final class private final fields no setters
- and constructor-only initialization.
- For mutable fields like List, always use defensive copies
- otherwise the reference can be modified externally."

instanceof operator

- Tune polymorphism use kiya
- Animal reference mein Dog, Cat, Bird sab aa sakte hain.
- Loop mein processing karni thi — Dog pe bark(), Cat pe meow(). Seedha call karo?
- Compile error
- — Animal reference pe Dog ka method nahi hota.
- instanceof se pehle confirm karo ki andar kya hai, phir safely cast karo.
- instanceof: object kis class ka hai?
- Animal a = new Dog() → a instanceof Dog = true.

→ Cast karne se pehle check karo → `ClassCastException` avoid hoga.

```
Animal a = new Dog(); // Andar Dog hai
a instanceof Dog // true [OK] – andar Dog hai
a instanceof Animal // true [OK] – Dog, Animal ka child hai
a instanceof Cat // false [X] – andar Cat nahi hai
// Real use – safely cast karne se pehle check karo
void makeSound(Animal a) {
    if (a instanceof Dog) {
        Dog d = (Dog) a;
        d.bark(); // [OK] safe – pehle confirm kiya
    } else if (a instanceof Cat) {
        Cat c = (Cat) a;
        c.meow(); // [OK] safe
    }
}
// [TRAP] TRAP – bina check ke cast kiya
Animal a = new Animal();
Dog d = (Dog) a; // [X] ClassCastException – andar Dog hai hi nahi!
// Safe tarika
if (a instanceof Dog) {
    Dog d = (Dog) a; // [OK] pehle check, phir cast
}
```

[TRAP] TRAP —

- `instanceof` sirf tab kaam karta hai jab object bana ho
- `extends` sirf blueprint hai.
- Bina object ke `instanceof` check karne ka koi matlab nahi.
- "instanceof checks if an object is an instance of a class or its subclass at runtime —
- always use it before downcasting to avoid `ClassCastException`."

Garbage Collection

- C++ mein tune object banaya, kaam hua, delete karna bhool gaya — memory leak.
- Java mein ye tension nahi.
- Koi reference nahi = GC eligible.
- JVM background mein check karta rehta hai — unreachable objects saaf kar deta hai.
- `System.gc()` sirf hint hai, guarantee nahi. `finalize()` deprecated hai
- use mat karo.
- C++: manually delete → bhool gaye → memory leak. Java: GC khud karta hai.
- Object ka koi reference nahi → GC eligible.
- `System.gc()` = sirf hint, guarantee nahi.

```
// C++ – manually
delete obj; // bhool gaye toh memory leak!
// Java – tu bhool ja, GC karega [OK]
// Object GC ke liye eligible kab hota hai?
// Case 1 – reference null kar diya
Student s = new Student("Arpan");
s = null; // koi point nahi kar raha → GC le jaayega
// Case 2 – reference reassign kiya
Student s = new Student("Arpan");
s = new Student("Rahul"); // "Arpan" wala orphan → GC
// Case 3 – method khatam hua
void method() {
    Student s = new Student("Arpan");
} // method khatam → s scope se bahar → GC
```

[TRAP] TRAP —

- `System.gc()` sirf request hai — guarantee nahi. JVM decide karta hai kab chalega.
- Production code mein `System.gc()` kabhi mat likho.
- `finalize()` deprecated hai Java 9+ mein.
- "Garbage Collector automatically frees memory of objects with no live references —
- you cannot force GC, `System.gc()` is just a hint to the JVM."

Constructor Chaining

- Tune Student class banayi
- teen constructors: naam only, naam+age, naam+age+course.
- Teeno mein same initialization code — validation, defaults.
- Kal validation badlegi toh teeno mein change karna padega.
- this() se ek constructor doosre ko call karo
- — code ek jagah, baaki delegate.
- this() = same class ka constructor call karo.
- super() = parent class ka constructor call karo.
- Hamesha pehli line mein hona chahiye. Dono ek saath ek constructor mein nahi.

```
// Same class mein — this() se
class Student {
    String name; int age; String course;
    Student(String name) {
        this(name, 18); // → doosra constructor call kiya
    }
    Student(String name, int age) {
        this(name, age, "Java"); // → teesra constructor call kiya
    }
    Student(String name, int age, String course) {
        this.name = name; // actual kaam yahan hota hai
        this.age = age;
        this.course = course;
    }
}
Student s = new Student("Arpan");
// "Arpan" → age default 18 → course default "Java" [OK]
// Parent class mein — super() se
class Dog extends Animal {
    String breed;
    Dog(String name, String breed) {
        super(name); // → Animal ka constructor call kiya
        this.breed = breed;
    }
}
// [TRAP] TRAP — this() ya super() hamesha PEHLI line honi chahiye
Dog(String name, String breed) {
    System.out.println("hello"); // [X] compile error!
    super(name);
}
```

[TRAP] TRAP —

- this() aur super() pehli line nahi hogi toh compile error.
- Ek constructor mein dono ek saath nahi aa sakte — ya this() ya super().
- "
- Constructor chaining allows one constructor to call another using this()
- for the same class or super() for the parent class —
- must always be the first statement in the constructor."

Enum — Fixed Set of Constants

- Tune payment system banaya. PaymentType int mein — 1=CASH, 2=CARD, 3=UPI.
- Koi developer ne 99 pass kar diya — invalid value, compiler nahi roka.
- Runtime crash.
- Enum banao — sirf allowed values. Bahar kuch diya toh compile error.
- Switch mein use karo — cleaner, type-safe.
- PaymentType = 1 (CASH), 2 (CARD), 3 (UPI)
- koi bhi 99 de sakta hai, compiler nahi rokta.
- Enum: sirf allowed values define karo. Bahar kuch diya → compile error.

→ Type safety.

→ Basic Enum

```
enum PaymentType {  
    CASH, CARD, UPI  
}  
PaymentType p = PaymentType.CASH; // [OK] valid  
PaymentType p = PaymentType.EMI; // [X] compile error – defined nahi
```

→ Enum with fields + constructor

```
enum Day {  
    MON("Weekday"), SAT("Weekend"), SUN("Weekend");  
    private String type;  
    Day(String type) { // constructor – private by default  
        this.type = type;  
    }  
    public String getType() { return type; }  
}  
Day.MON.getType(); // "Weekday"  
Day.SAT.getType(); // "Weekend"
```

→ Switch mein use

```
switch(p) {  
    case CASH: System.out.println("Cash liya"); break;  
    case CARD: System.out.println("Card swipe"); break;  
    case UPI: System.out.println("UPI scan"); break;  
}
```

→ Built-in methods

```
Day.MON.name() // "MON" – string naam  
Day.MON.ordinal() // 0 – index (0 se start)  
Day.valueOf("SAT") // Day.SAT – string se enum  
Day.values() // [MON, SAT, SUN] – sab values array
```

→ Enum = fixed constants ka type-safe set.

→ int use karo toh koi bhi invalid value de sakta hai.

→ Enum se compiler hi rok deta hai.

→ Har enum value internally ek object hai — isliye == se compare hota hai.

[TRAP] TRAP: Enum constructor public nahi ho sakta

→ private ya package-private only.

[TRAP] TRAP: ordinal() pe DB mein depend mat karo

→ order change kiya toh sab values shift ho jaayengi.

[TRAP] TRAP: Enum == se compare karo, .equals() nahi

→ enum values singletons hain, == safe hai.

[TRAP] TRAP: Singleton wala yaad hai? enum AppConfig { INSTANCE; }

→ BEST Singleton implementation.

→ JVM guarantee deta hai ek hi instance.

"Enum defines a fixed set of named constants with full type safety — the compiler rejects any value not in the enum. Each enum constant is a singleton object so == comparison is safe and preferred over .equals()."

Enum = type-safe constants. == se compare. ordinal() pe depend mat karo. Singleton ka best way bhi Enum hai.

Type Casting — Upcasting & Downcasting

→ Tune Animal reference mein Dog store kiya. Upcasting — automatic, safe.

- Ab Dog ke special methods chahiye — bark().
- Animal mein nahi. Downcast karna padega.
- Lekin bina check ke kiya toh ClassCastException
- runtime pe aata hai, compiler nahi pakdta.
- Isliye hamesha instanceof pehle, phir cast.
- Animal a = new Dog() → Dog andar hai, Animal reference.
- Upcasting: automatic, safe.
- Downcasting: manual, ClassCastException risk. instanceof se pehle check karo.

→ Upcasting — Child → Parent reference. Automatic. Always safe.

```
Animal a = new Dog(); // [OK] Upcasting — automatic, koi cast nahi
a.eat(); // [OK] Animal method — Animal reference se chalega
a.bark(); // [X] bark() Animal mein nahi hai!
```

→ Downcasting — Parent reference → Child type. Manual. Cast likhna padta hai.

```
Animal a = new Dog();
Dog d = (Dog) a; // [OK] Downcasting — andar Dog hai, safe
d.bark(); // [OK] Dog method ab accessible hai
// [TRAP] TRAP — andar Dog nahi hai
Animal a = new Animal();
Dog d = (Dog) a; // [X] ClassCastException! andar Animal hai, Dog nahi
// Safe tarika — instanceof pehle
if (a instanceof Dog) {
    Dog d = (Dog) a; // [OK] safe — confirm karke cast kiya
    d.bark();
}
```

- Upcasting = automatic (Dog → Animal).
- Downcasting = manual cast likhna padta hai (Animal → Dog).
- Bina instanceof check ke downcast karo → ClassCastException.
- Hamesha instanceof pehle.

[TRAP] TRAP: Upcasting mein child ke extra methods access nahi hote.

→ a.bark() → compile error even if Dog andar hai!

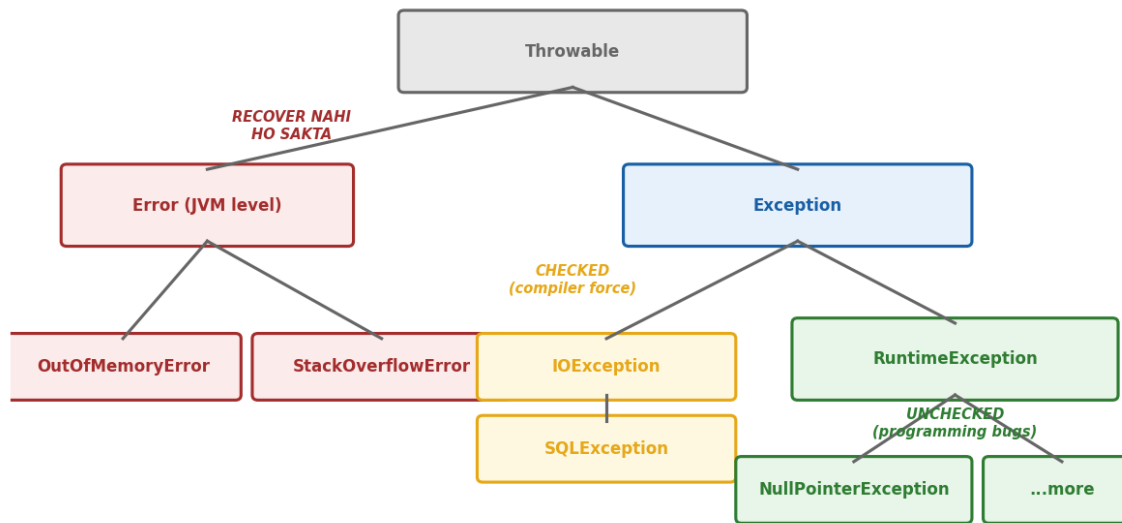
[TRAP] TRAP: ClassCastException runtime pe aata hai — compiler nahi pakdta.

→ Isliye instanceof check mandatory hai.

**"Upcasting is automatic and always safe
a child reference can always be assigned to a parent type.
Downcasting is manual and requires an instanceof check first
— without it you risk a ClassCastException at runtime."
Upcast = automatic. Downcast = manual + instanceof check.
Bina check → ClassCastException.**

Exception Handling — try-catch-finally

Exception Hierarchy — Throwable Tree



Checked = compiler pakde (handle ya declare). Unchecked = runtime bugs (fix karo). Error = JVM level (app nahi handle karti).

Throwable > Error (JVM, recover nahi) + Exception > Checked (compiler force) + RuntimeException (bugs, fix karo).

- Tune DB se data fetch karna tha.
- Kahin connection fail, kahin query fail, kahin result empty.
- Sab cases handle karne the.
- try mein risky code, catch mein handle, finally mein resources close
- — chahe exception aaye ya na aaye. finally = maa ki daant
- ghar se bhaag bhi jaaye toh bhi padegi.
- System.exit() ke alawa koi nahi rok sakta.
- try: risky code. catch: exception pakdo. finally: hamesha chalta hai —
- chahe exception aaye ya na aaye, chahe return bhi karo.
- Sirf System.exit() rokta hai.

→ Basic Flow

```
try {
    int result = 10 / 0; // [X] ArithmeticException
    System.out.println(result); // skip ho jaata hai
} catch (ArithmeticException e) {
    System.out.println("divide by zero!"); // [OK] pakda
} finally {
    System.out.println("finally chala"); // [OK] HAMESHA
}
```

→ Multi-catch — ek catch mein multiple exceptions

```
try {
    // risky code
} catch (IOException | SQLException e) { // [OK] multi-catch
    System.out.println(e.getMessage());
}
```

→ finally after return — TRAP

```

int test() {
    try {
        return 1; // return kiya
    } finally {
        System.out.println("finally!"); // YE BHI CHALTA HAI
    }
}
// Output: "finally!" print hoga, PHIR 1 return hoga
// finally return bhi override kar sakta hai
int test2() {
    try { return 1; }
    finally { return 2; } // [TRAP] 2 return hoga, 1 lost!
}

```

- try mein exception aaya → catch mein gaya → finally chala.
- try mein exception nahi aaya → catch skip → finally chala.
- finally HAMESHA chalta hai
- — chahe return karo, chahe exception aaye.
- Sirf System.exit(0) se rokta hai.

[TRAP] TRAP: finally mein return likhna = try ka return overwrite ho jaata hai.

- Kabhi finally mein return mat likho!

[TRAP] TRAP: Multi-catch mein: parent exception pehle likha toh child catch unreachable ban jaata hai

- compile error!

[TRAP] TRAP: catch(Exception e) sab pakdta hai

- lekin Error (OutOfMemoryError) nahi.
- Error = JVM level, recover nahi hota.

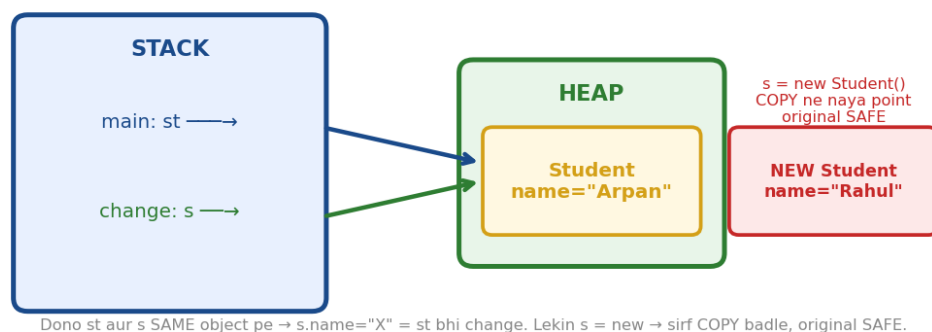
"The finally block always executes regardless of whether an exception was thrown or a return statement was hit — the only exception is System.exit(). Never put a return statement in finally as it will silently override the return from try." try → catch → finally (hamesha). Multi-catch: IOException | SQLException. finally overrides try return.

[WARNING] WEAK SPOT (2 baar galat):

- finally HAMESHA chalta hai — try mein return karo tab bhi!.
- Sequence: try code → return attempt → finally run → PHIR return.
- Sirf System.exit(0) rok sakta hai.
- Yaad rakh: finally = maa ki daant — ghar se bhaag bhi jaaye tab bhi padegi!

Pass by Value

Pass by Value — Reference ki COPY pass hoti



▲ Reference ki COPY pass hoti. Dono SAME object pe point = andar change dikhe. s=new kiya = COPY ne naya point, original SAFE.

WHY Java ne reference pass nahi kiya?. Safety.

- Agar actual reference pass hota toh method andar s = null kar deta
- caller ka variable BHI null!.
- Dangerous.
- Java socha — reference ki COPY bhejo. Copy pe kuch bhi karo, caller safe.
- State change hogi (s.name = "X")
- kyunki dono same object point karte, lekin reference khud nahi badlega.
- Tune method mein Student object pass kiya — method ne andar name badal diya.
- Wapas aaya — name badal gaya.
- Tune socha "Java pass by reference hai!" Galat.
- Java HAMESHA pass by value hai.
- Object pe reference ki copy pass hoti hai
- dono same object point karte hain isliye state change hoti hai.
- Lekin method andar s = new Student() karo
- original nahi badlega, sirf copy ne naya object point kiya.
- Kahani: Dost ko apni notebook ki photocopy di.
- Dost ne photocopy pe kuch likha — teri original notebook nahi badi.
- Lekin dost ne photocopy ke andar ki drawer ka lock khola aur drawer mein kuch rakha
- original drawer bhi badal gayi!.
- Yehi Java hai.
- Java ka ek hi rule — hamesha:

WHY? Java mein HAMESHA pass by value hota hai.
 Primitive pe actual value copy hoti hai.
 Object pe reference ki copy hoti hai
 lekin dono copy aur original same object point karte hain.
 Isliye andar ki state change hoti hai, reference nahi.

Case 1 — Primitive (int, boolean, double)

```
// x ka value COPY hota hai method mein
void change(int x) { x = 99; } // copy badli, original nahi
int num = 10; change(num); System.out.println(num); // 10 — nahi badla!
```

Case 2 — Object (reference ki copy pass hoti hai)

```
// reference ki COPY pass hoti hai — lekin dono same object point karte hain
void change(Student s) { s.name = "Rahul" } // andar ka data badla [OK]
Student st = new Student("Arpan") change(st)
System.out.println(st.name) // "Rahul"
badla!
```

Case 3 — TRAP: Naya object assign karna

```
// s = new Student() kiya — copy ne naya object point kiya — original same!
void change(Student s)
{ s = new Student("Rahul"); } // copy ne naya object point kiya
Student st = new Student("Arpan") change(st)
System.out.println(st.name) // "Arpan"
NAHI badla!
```

→ Teen lines mein yaad karo:

Primitive ⇒ actual value ki copy — method andar kuch bhi kare, original safe
Object ⇒ reference ki copy — andar ki state change hogi, naya object assign nahi hoga

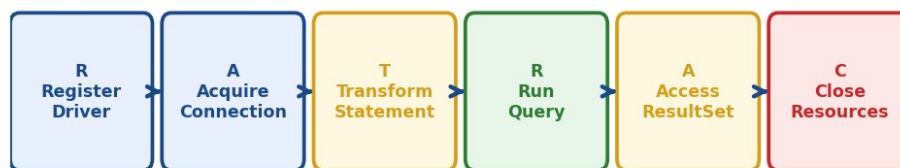
TRAP ⇒ s = new Student() method andar
 copy ne naya object point kiya, original untouched

[TRAP] TRAP:

- Java mein pass by reference exist NAHI karta
- log confuse hote hain kyunki object ke changes reflect hote hain.
- Lekin actually reference ki copy pass hoti hai.
- Ye pass by value hi hai — bas reference ka value.
- "Java is always pass by value. For primitives, the actual value is copied.
- For objects, the reference is copied
- both point to the same object, so state changes reflect.
- But reassigning the parameter inside the method does not affect the original reference."

JDBC — Java Database Connectivity

JDBC 6 Steps — RAT-RACE



Register → Acquire → Transform → Run → Access → Close. Ek bhi skip = error.

▲ Java 8 (Lambda, Streams) → Java 11 LTS (var, HttpClient) → Java 17 LTS (Records, Sealed, Text Blocks) → Java 21 LTS (Virtual Threads).

→ WHY: JDBC KYUN? Java application ko database se baat karni hai. JDBC = standard API jo HAR database ke saath kaam kare (MySQL, PostgreSQL, Oracle). Driver change karo = code same. PreparedStatement KYUN > Statement? SQL Injection prevent + query plan CACHE (1000x execute = 1x parse). Connection Pool KYUN? new connection = TCP + auth = 50ms. Pool = pehle se ready, REUSE.

- Spring Data JPA itna smooth hai ki kabhi sochna nahi padta andar kya hota hai.
- Lekin ek baar production mein connection pool exhaust ho gaya
- sab requests hang ho gayi.
- Tab JDBC khola andar se — 6 steps hain, ek bhi miss karo toh leak hoga.
- Ye 6 steps pata hain toh Hibernate aur JPA ke neech kya chal raha hai
- sab seedha samajh aata hai.
- JDBC = Java se DB seedha connect karo.
- Sab manually: driver, connection, query, result. Full SQL control.
- Hibernate = ORM → sab automatically.
- "JDBC is Java's low-level API to connect directly to a database —
- you manage the driver, connection, statement, and result set yourself.
- It gives full SQL control but requires manual work."

6 Steps — RAT-RACE

- 6 steps hamesha same order: Register → Acquire → Transform → Run → Access → Close.
- Ek bhi skip → error.
- Register → Acquire → Transform → Run → Access → Close

```

1.
Class.forName("com.mysql.cj.jdbc.Driver"); // Step 1: Driver register karo
2.
Connection con = DriverManager.getConnection(url user
pass); // Step 2: DB se connection lo
3.
PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE id = ?")
// Step 3: Query tayaar karo
4. ps.setInt(1, userId); // Input safely set karo
ResultSet rs = ps.executeQuery(); // Step 4: Query chalao
5.
while(rs.next())
{ System.out.println(rs.getString("name")); } // Step 5: Result padho
6. rs.close(); ps.close(); con.close(); // Step 6: Sab close karo – MUST!

```

Step 1 mein Driver load hua. Step 2 mein DB se handshake.

Step 3 mein query SQL injection safe. Step 4 mein execute.

Step 5 mein result loop mein padha.

Step 6 mein sab close — nahi kiya toh connection leak hoga!

→ "

→ The six JDBC steps are:

→ load the driver get a connection create a PreparedStatement

→ execute the query iterate the ResultSet and close all resources —

→ in that order."

Statement vs PreparedStatement

→ WHY: Statement mein user input SEEDHA SQL mein ghuse = SQL Injection. PreparedStatement mein ? placeholder = input HAMESHA data, kabhi SQL code nahi. Plus: DB query plan CACHE kare = repeated queries FAST. Production = ALWAYS PreparedStatement.

→ Login form mein ek junior ne Statement use kiya tha.

→ User ne email field mein likha: admin@gmail.com' OR '1'='1.

→ SQL ban gaya jo poora user table expose kar gaya. SQL Injection.

→ PreparedStatement mein ? placeholder use karo — user input DATA hai, SQL nahi.

→ Compiler nahi pakdta ye — sirf awareness se bachta hai.

→ Interview mein hamesha poochhte hain.

→ Statement: arp@gmail.com' OR '1'='1 → SQL ban jaata hai → sab users expose!

→ PreparedStatement: ? placeholder → input VALUE hai, SQL nahi.

→ ALWAYS PreparedStatement.

Statement	SQL Injection VULNERABLE. Har baar recompile. Static SQL only.
PreparedStatement	SAFE — input DATA hai, SQL nahi. Ek baar compile. Hamesha use karo.

[TRAP] TRAP: Statement + user input = SQL Injection. ALWAYS PreparedStatement.

→ "PreparedStatement pre-compiles the SQL and treats user input as data

→ not executable SQL —

→ this completely prevents SQL injection attacks.

→ Always use PreparedStatement with user input."

executeQuery()	ResultSet → SELECT ke liye
executeUpdate()	int (rows affected) → INSERT, UPDATE, DELETE
execute()	boolean → Any SQL

[OK] KEY: HikariCP = Spring Boot DEFAULT connection pool.
Connections pehle se bane — har request pe naya banana avoid.

→ "HikariCP is Spring Boot's default connection pool —

→ it maintains pre-created connections and reuses them per request,

→ avoiding the overhead of creating a new DB connection every time."

Hibernate & JPA — ORM Framework

- JDBC mein ek simple CRUD ke liye 50 lines likhne padte the —
- driver, connection PreparedStatement
- ResultSet iterate, object mein map, sab close.
- 100 fields wali table? 200 lines ka boilerplate.
- Aur agar column naam badla toh har SQL string dhundh ke update karo.
- Hibernate ne socha — ye mechanical kaam Java ke liye chhodo.
- Tu object do, SQL main likhta hoon.
- JDBC = manually sab karo.
- Hibernate = ORM tool → Java object dedo, SQL khud likhega.
- session.save(obj) → INSERT. session.get() → SELECT.
- "Hibernate is an ORM framework"
- it maps Java objects to database tables and handles SQL automatically.
- You work with objects; Hibernate translates them to DB operations."

Key Annotations

Annotation	Kya Karta Hai	Yaad Karo
@Entity	Ye class = ek DB table	Har model class pe required
@Id	Primary key field	Har @Entity pe chahiye
@GeneratedValue	Auto increment ID	DB khud ID generate karta hai
@Transient	Ye field DB mein SAVE NAHI hoga	Hibernate ignore karta hai
@OneToMany	Ek parent, kai children (LAZY default)	MANY = LAZY
@ManyToOne	Kai children, ek parent (EAGER default)	ONE = EAGER

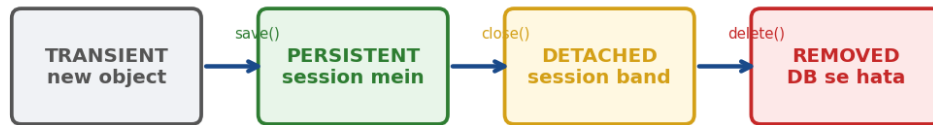
[TRAP] TRAP:

- @Transient ≠ @Transactional — MOST CONFUSED PAIR! @Transient = skip DB field.
- @Transactional = rollback.
- [WARNING] WEAK SPOT (past mein galat):
- @Transient = Hibernate field ignore karo (DB mein column nahi banega).
- @Transactional = sab operations ek saath succeed ya rollback.
- Dono bilkul alag kaam karte hain — interview mein mix mat karna!
- [OK] KEY: @Entity mein no-arg constructor MUST
Hibernate reflection se blank object banata hai.
- "@Entity marks a class as a DB table. @Id is the primary key.
- @Transient tells Hibernate to skip that field
- it has nothing to do with @Transactional.
- @Entity must have a no-arg constructor because Hibernate uses reflection to create objects."

Persistence Lifecycle — 4 States

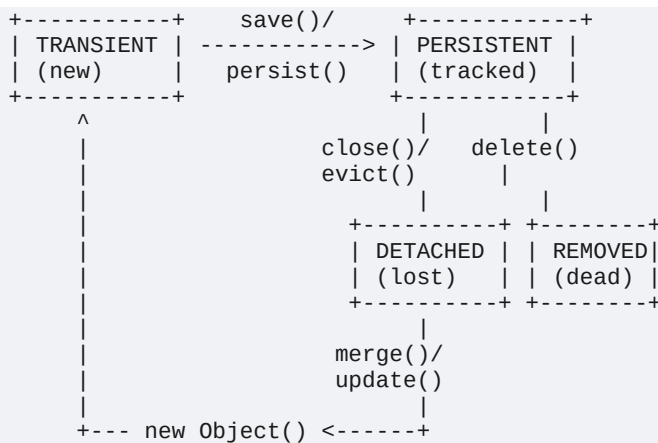
- WHY: 4 states KYUN? Transient = new object, Hibernate ko pata nahi. Persistent = session mein, Hibernate WATCH karta (dirty checking). Detached = session band, object hai lekin tracked nahi. Removed = delete marked. Ye states Hibernate ko batate KAB DB sync karna, KAB ignore karna. Bina states = har object har waqt sync = SLOW.

Hibernate Persistence Lifecycle



▲ 4 States: `new()` = Transient → `save()` = Persistent (tracked) → `session.close()` = Detached → `delete()` = Removed. Dirty checking sirf Persistent pe.

FLOW DIAGRAM:



TRANSIENT: new kiya, Hibernate ko pata nahi
 PERSISTENT: session track kar rahi, dirty check ON
 DETACHED: session band, object hai lekin tracked nahi
 REMOVED: delete marked, commit pe DB se hatega

- Socho Hibernate ek CCTV camera jaisa hai.
- Camera OFF = object kuch bhi kare, Hibernate ko khabar nahi → TRANSIENT.
- `session.save()` = Camera ON, har change track, commit pe auto-update → PERSISTENT.
- `session.close()` = Camera BAND, object DB mein hai lekin untracked → DETACHED.
- `session.delete()` = next commit pe delete scheduled → REMOVED.
- Ye 4 states samajh aaye toh dirty checking, auto-update
- DETACHED bugs sab seedhe samajh aate hain.
- `new Alien()` → TRANSIENT (Hibernate unaware, no ID).
- `session.save()` → PERSISTENT (tracking on, ID mili).
- `session.close()/evict()` → DETACHED. `session.delete()` → REMOVED.

```

Alien a = new Alien(); // TRANSIENT - camera OFF, koi ID nahi, Hibernate unaware
session.save(s); // PERSISTENT - camera ON! ID mili, har change watch
s.setName("Arpan Singh"); // Persistent hai dirty check
commit pe auto-update [OK]
session.evict(s); // DETACHED - camera band, changes saved nahi honge
s.setName("Something"); // DETACHED mein change kiya
DB mein nahi jaayega!
session.delete(s); // REMOVED - next commit pe DB se delete hoga
  
```

- TRANSIENT = camera off, koi tracking nahi.
- PERSISTENT = camera on, sab dekh raha hai.
- DETACHED = camera off phir se, DB mein object hai lekin untracked.
- REMOVED = delete scheduled.
- [REPEAT TRAP] CCTV Camera
- Baar Baar Galat! `new Alien()` → Camera OFF → TRANSIENT.
- `session.save()` → Camera ON → PERSISTENT.

→ session.close() → Camera OFF again → DETACHED.
 → session.delete() → Delete scheduled → REMOVED.
 → Sawaal mein "new" dikha?. Seedha TRANSIENT chun.

State	Kaise Pahuncho	Kya Hota Hai
TRANSIENT	new Alien()	No ID, no session. Hibernate unaware.
PERSISTENT	session.save() / session.get()	Har change watch. Auto-sync on commit.
DETACHED	session.close() / evict()	Tracking band. Changes NOT saved.
REMOVED	session.delete()	Next commit pe delete hoga.

[TRAP] TRAP:

→ DETACHED = session.close() OR evict(obj).
 → Object DB mein hai — bas untracked. evict() DB se delete NAHI karta!
 → "An entity goes through four states: Transient when just created with new,
 → Persistent when tracked by a session
 → Detached when the session is closed or evict() is called,
 → and Removed when marked for deletion."

get() vs load()

→ User ID se user fetch karna tha user exist nahi karta tha. load() use kiya tha
 → koi exception nahi aaya turant.
 → Phir user ka naam access kiya — tab ObjectNotFoundException.
 → Debug karna mushkil tha kyunki exception jagah se door aa raha tha.
 → get() use karo —
 → turant DB hit karta hai, nahi mila toh null return karta hai.
 → Safe. Predictable.
 → get() → immediately DB hit, nahi mila → null (safe).
 → load() → proxy return, access karo tab DB hit, nahi mila → ObjectNotFoundException.
 → get() use karo.

get()	Immediate DB hit. Nahi mila = null. SAFE. Default use karo.
load()	Proxy return. Non-ID access pe DB hit. Nahi mila = ObjectNotFoundException. RISKY.

→ "Use session.get() by default —
 → it hits the DB immediately and returns null if the record is not found
 → which is safe.
 → session.load()
 → returns a proxy and throws ObjectNotFoundException if the record doesn't exist."

save() vs persist() vs update() vs merge()

→ WHY: save() = Hibernate specific, ID return. persist() = JPA standard, void. update() = detached → persistent, exception agar session mein already. merge() = detached ka copy banake persistent, safe. Rule: JPA methods prefer (persist/merge) = vendor independent. Hibernate switch karo EclipseLink pe = code same.

→ Ek baar detached object tha — session close ho chuki thi. update() call kiya.
 → Crash — NonUniqueObjectException.
 → Same ID ka object already session mein tha.
 → Tab merge() seekha — safely copy banata hai, dono case handle karta hai.
 → Iske baad production mein kabhi update() nahi likha.

Method	Returns	Key Point
save()	Generated ID	Hibernate-specific. ID immediately chahiye toh.
persist()	void	JPA standard. Portability chahiye toh.
update()	void	RISKY — NonUniqueObjectException risk!
merge()	Managed copy	ALWAYS safe. update() ki jagah yahi use karo.
saveOrUpdate()	void	INSERT if new, UPDATE if exists.

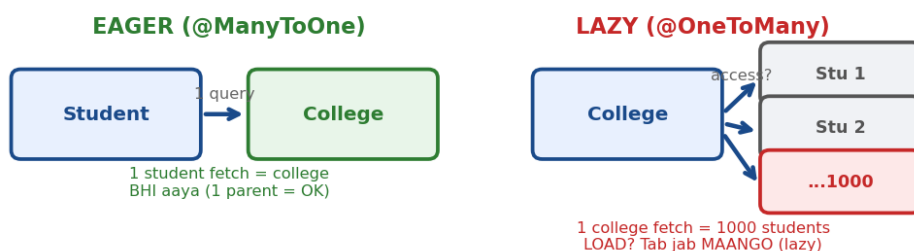
[TRAP] TRAP: update() = NonUniqueObjectException risk. HAMESHA merge() use karo!

[WARNING] WEAK SPOT (3 baar galat — ye ratta laga):

- update() = agar same ID ka object already session mein hai
- NonUniqueObjectException crash!
- merge() = dono case safely handle karta hai, copy banata hai.
- Production mein KABHI update() mat use karo — hamesha merge().
- [REPEAT TRAP] Detached Object Trap —
- Baar Baar Galat! update() = same ID already? → NonUniqueObjectException -- CRASH.
- merge() = copy banata, safe.
- Sawaal mein "detached"? merge() chun.
- "save() returns the generated ID immediately.
- persist() is the JPA standard.
- Always prefer merge() over update()
- merge() creates a managed copy safely and avoids NonUniqueObjectException
- when a same-ID object already exists in the session."

Eager vs Lazy — Golden Rule

Eager vs Lazy Loading — @ManyToOne vs @OneToMany



@ManyToOne = EAGER (1 parent = cheap). @OneToMany = LAZY (N children = expensive, jab maango tab lao).

▲ @ManyToOne = EAGER (1 parent = cheap). @OneToMany = LAZY (N children = expensive, jab maango tab lao).

→ WHY: Lazy KYUN default? Doctor fetch karo = 10,000 patients BHI load? Waste! Lazy = jab MAANGO tab lao. Eager = saath mein lao. @OneToMany = LAZY (bahut data). @ManyToOne = EAGER (ek hi parent). N+1 trap: lazy + loop = N alag queries. Fix: JOIN FETCH = bata do saath chahiye.

- API banaya — GET /doctor.
- Doctor fetch hoti, saath mein uske 10,000 patients bhi load ho jaate.
- Response 5 second lag raha tha. Root cause: @OneToMany pe EAGER tha.
- Fix: LAZY rakho — patients tab load hoon jab actually getPatients() call ho.
- Golden rule yaad kar lo: MANY = LAZY.
- ONE = EAGER.

- LAZY (default for @OneToMany) → sirf parent.
- getStudents() call karo tab children aayenge.
- EAGER (default for @ManyToOne) → parent ke saath child bhi.
- Golden rule: MANY=LAZY, ONE=EAGER.
- [REPEAT TRAP] Doctor-Patient Trap — Baar Baar Galat!
- GET /doctor → 10,000 patients load? Disaster! @OneToMany = LAZY.
- MANY = Memory bachao = LAZY. Quiz mein EAGER mat chun.

@OneToMany, @ManyToMany	LAZY (default) — heavy data, tab load karo jab maango.
@ManyToOne, @OneToOne	EAGER (default) — single object, lightweight, safe to load.

[OK] KEY: GOLDEN RULE: MANY = LAZY. ONE = EAGER.

- "The golden rule is MANY = LAZY, ONE = EAGER.
- @OneToMany is lazy by default because loading thousands of children every time you fetch a parent would be wasteful.
- @ManyToOne is eager because loading one parent alongside a child is lightweight."

Cascade Types

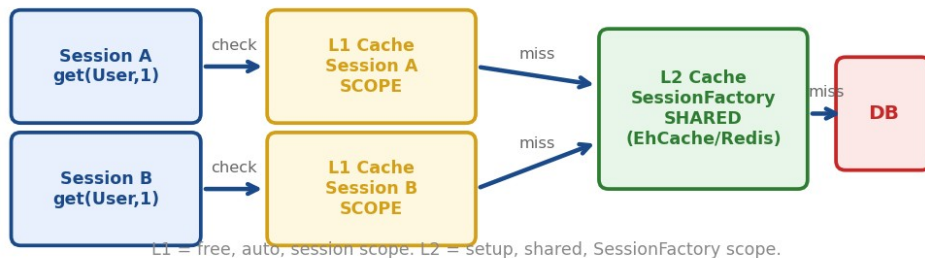
- Department delete kiya — Employees abhi bhi DB mein the. Orphan records.
- Alag alag manually delete karna padta tha.
- CascadeType.REMOVE lagaya — parent delete karo, children apne aap.
- CascadeType.ALL = sab operations cascade.
- Lekin careful — galat jagah ALL lagaya toh accidentally data delete ho sakta hai.

CascadeType.PERSIST	Parent save → child bhi save
CascadeType.REMOVE	Parent delete → child bhi delete
CascadeType.MERGE	Parent merge → child bhi merge
CascadeType.ALL	Sab operations cascade. Most commonly used.

- "Cascade type controls which operations propagate from parent to child.
- CascadeType.ALL is the most common
- save, update, and delete on the parent automatically apply to its children."

Caching — L1 & L2

Hibernate Cache — L1 (Session) vs L2 (SessionFactory)



L1 = free, auto, session scope. L2 = setup, shared, SessionFactory scope.

▲ Session A → L1 (session scope, auto) → miss → L2 (SessionFactory scope, shared, EhCache/Redis) → miss → DB. L1 free, L2 setup.

→ Ek request mein same user 5 baar fetch ho raha tha — 5 DB queries. Bekar.

→ L1 Cache = Hibernate ki session-level diary.

→ Pehli baar DB se lao, diary mein likho. Doosri baar same ID?

→ Diary mein dekho — DB nahi gaye. Session close = diary faad do.

→ L2 = sab sessions share karte hain

→ zyada powerful lekin manually enable karna padta hai.

→ Bina cache: har request → DB hit → slow.

→ Cache: pehli baar DB, baaki sab memory se. L1 = session-level.

→ L2 = application-level shared.

→ L1 = session-level cache → always ON.

→ Same session mein same ID → 2nd baar DB nahi jaata. Session close → L1 clear.

→ L2 = SessionFactory level → multiple sessions share.

→ Ab diary wala example code mein dekho:

```
session.get(Alien.class, 1); // Pehli baar — L1 mein nahi → DB HIT → L1 mein store
session.get(Alien.class, 1); // Doosri baar
L1 mein hai → CACHE HIT → DB nahi gaya!
session.evict(student); // Ye student diary se hata — DETACHED hua
session.clear(); // Poori diary saaf — sab students L1 se hate
session.close(); // Diary band — L1 completely CLEAR!
```

→ Pehli baar DB gaya → L1 mein rakha.

→ Doosri baar same ID → L1 mein milaa → DB nahi gaya.

→ `evict()` = ek page diary se faada.

→ `clear()` = poori diary faadi. `close()` = diary hi band.

Feature	L1 Cache	L2 Cache
Default?	Always ON	OFF — manually enable karo
Scope	Sirf ek session	Sab sessions (SessionFactory level)
Session close pe?	CLEAR ho jaata hai	SURVIVE karta hai
Config?	Kuch nahi	EhCache + @Cacheable entity pe
HQL ke saath?	Haan	NO! Alag Query Cache chahiye
<code>evict()</code> effect	Object → DETACHED	Affected nahi

[TRAP] TRAP:

→ L2 sirf `session.get()` ke saath kaam karta hai.

→ HQL ke liye `query.setCacheable(true)` alag se chahiye.

→ WHY: HQL KYUN SQL nahi? SQL = table/column names. DB badle = queries badle. HQL = Java class/field names. DB badle = mapping badle, queries SAME. DB INDEPENDENT. Plus: object return karta directly, manual mapping nahi.

- "L1 cache is per-session and always on
- the second call for the same ID skips the DB entirely.
- L2 cache is shared across all sessions and must be configured manually with EhCache.
- L2 does not work with HQL queries by default
- you need a separate Query Cache for that."

HQL — Hibernate Query Language

- JDBC mein SQL likhte the — table naam, column naam. Table rename hua?.
- Har SQL string dhundh ke change karo.
- Hibernate ke saath Java objects pe kaam karte the
- aur HQL ne socha why not queries bhi Java class names se likho?.
- FROM Student likhna hai, FROM tbl_student nahi. DB badla?
- HQL same rahega — DB independent.
- Ye small difference production mein bahut bada farak karta hai.
- HQL: Java class naam use karo, table naam nahi.
- FROM Alien (not alien, not tbl_alien). DB independent.
- Returns Java objects directly.

SQL	Table/column names. DB-specific.
HQL	Java CLASS/FIELD names. DB-independent. Java objects return karta hai.

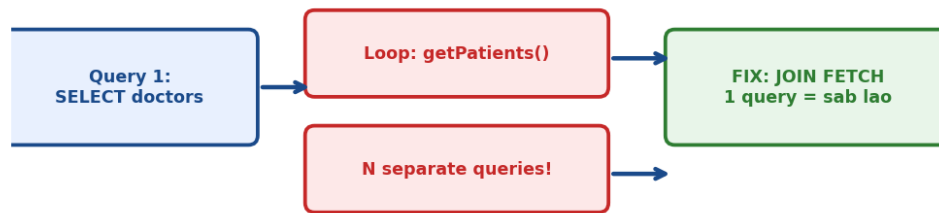
```
FROM Student // sab students
FROM Student WHERE age > 18 // filter
FROM Student WHERE id = :sid // named parameter
FROM Student s JOIN FETCH s.courses // N+1 FIX!
```

[TRAP] TRAP:

- HQL CASE SENSITIVE! "from student" fail hoga.
- "FROM Student" — capital S = Java class naam.
- | **[OK] KEY:** JOIN FETCH = N+1 ka fix. 11 queries → 1 query.
- "HQL uses Java class and field names instead of table and column names
- making it database-independent.
- It returns Java objects directly.
- JOIN FETCH in HQL is the standard fix for the N+1 problem
- it loads parent and child data in a single query."

N+1 Problem & LazyInitializationException

N+1 Problem



1 + N = 11 queries (10 doctors) → JOIN FETCH = 1 query

▲ N+1: 1 query (parent) + N queries (children). Fix = JOIN FETCH: 1 query mein sab. LazyInit = session band, proxy DB call fail.

- N+1 aur LazyInitializationException → alag problems hain.
- Dono confuse mat karna!

N+1 Problem — Kyun Hota Hai?

- 10 doctors fetch kiye.
- Loop mein getPatients() call kiya — LAZY tha.
- Har doctor ke liye alag DB query. 1 + 10 = 11 queries.
- Hibernate ko pata nahi tha tune loop mein call karega — ek ek fetch kiya.
- Fix: JOIN FETCH — ek query mein sab. 11 → 1.
- LAZY + loop = N+1.
- 10 students fetch → loop mein getCourses() → 10 alag queries → total 11.
- Hibernate ko pata nahi tune loop mein call karoge. Fix: JOIN FETCH → 1 query.
- 1 main query + N extra = N+1. 10 students → 10 separate getCourses()
- calls → 10 DB trips.
- JOIN FETCH → 1 trip mein sab.

```
// N+1 Problem
List<Student> students = session.createQuery("FROM Student").list() // 1 query
for(Student s : students) {
    s.getCourses(); // LAZY — har baar DB hit! 10 alag queries [X]
}
// FIX — JOIN FETCH
session.createQuery("FROM Student s JOIN FETCH s.courses").list()
// 1 query [OK]
```

[OK] KEY: N+1 = 1 + N queries. Fix = JOIN FETCH. 11 queries → 1 query.

- "N+1 happens when fetching N entities fires N additional queries for their lazy associations in a loop.
- The fix is JOIN FETCH in HQL
- — it loads the parent and all related children in a single SQL query
- reducing 11 queries to 1."

LazyInitializationException

- N+1 alag problem hai — session open hai, baar baar DB hit.
- LazyInit alag — session band ho gayi, baad mein lazy data access kiya.
- Ek baar service method mein doctor fetch ki, return kiya.
- Spring ne session close kar di.
- Controller mein getPatients() call kiya — session nahi thi. Crash.
- Fix: @Transactional service method pe — session poori method tak open rehti hai.
- N+1: session OPEN hai, baar baar DB hit (loop).
- LazyInit: session CLOSE ho gayi, baad mein lazy data access.

→ Completely alag!

N+1 vs LazyInitializationException — Fark yaad karo!

→ WHY: N+1 = PERFORMANCE problem (1+N queries). LazyInitException = SESSION problem (session band, proxy DB call nahi kar sakta). Dono ALAG problems. N+1 fix = JOIN FETCH. LazyInit fix = @Transactional (session open rakh). Dono ka ROOT = lazy loading lekin PROBLEM different.

N+1 → Session OPEN hai — lekin baar baar DB hit kar raha hai (loop mein).

LazyInit → Session CLOSE ho gayi — aur BAAD MEIN lazy data access kiya.

Completely alag!

→ @Service method return → Spring session close kar deta hai.

→ Controller mein getCourses() → session nahi milti → LazyInitializationException.

→ Fix: @Transactional on service → session method tak open.

// [TRAP] PROBLEM — @Transactional nahi lagaya

@Service

public class StudentService { // @Transactional nahi!

public Student getStudent(Long id) {

Student s = studentRepo.findById(id).get(); // DB se fetch kiya

return s; // Method return hua ↓

} // [X] Spring ne YAHAN session close kar di automatically!

}

// Controller mein — student wapas aaya, session already close

Student s = studentService.getStudent(1L);

s.getCourses(); // [TRAP] LazyInitializationException!

Session close ho chuki hai

// [OK] FIX — @Transactional lagao Service pe

@Transactional // Ab session method ke ANDAR tak open rahegi

public Student getStudent(Long id) {

Student s = studentRepo.findById(id).get();

s.getCourses(); // [OK] Session abhi OPEN hai — lazy load hoga safely

} // Yahan session close hogi — ab safe hai

Fix	Kaise	Kab Use Karo
@Transactional	Method ke poore duration tak session open	Best fix — most common
JOIN FETCH	HQL mein eager load	Jab related data hamesha chahiye
FetchType.EAGER	@OneToMany(fetch=EAGER)	Careful — hamesha load karta hai

[TRAP] TRAP:

→ LazyInitializationException = session close ke baad lazy data access.

→ Fix = @Transactional.

→ "LazyInitializationException is thrown when you access a lazy collection after the Hibernate session is closed.

→ The best fix is @Transactional on the service method

→ — it keeps the session open for the entire transaction so lazy loading can happen safely."

Spring Core & Dependency Injection

- Controller mein new EmailService() likha tha.
- Unit test likhne baitha — real emails ja rahe the test mein bhi.
- Mock nahi ho raha tha.
- Problem: controller khud bana raha tha EmailService — tightly coupled.
- Controller ka kaam request handle karna, objects banana nahi.
- Isliye Spring IoC aaya — tu new mat likh, Spring banayega aur inject karega.
- Bina Spring: new Service(), new Repository() — sab manually → tight coupling.
- Spring IoC container: khud objects banata hai, inject karta hai → loose coupling.
- "Spring's IoC container manages object creation and injects dependencies automatically."
- This removes tight coupling between classes and makes the code easy to unit test."

BeanFactory vs ApplicationContext

→ WHY: BeanFactory = basic container, LAZY loading (bean maango tab banaye). ApplicationContext = BeanFactory + EXTRAS (event publishing, i18n, AOP, annotation support). Production = HAMESHA ApplicationContext. BeanFactory sirf memory-constrained environment (rare). Interview mein: "ApplicationContext is the go-to, BeanFactory is bare minimum."

- BeanFactory = basic container, sirf beans banata hai.
- ApplicationContext = BeanFactory + bahut kuch
- events, AOP (@Transactional), i18n, eager loading.
- Spring Boot mein hamesha ApplicationContext use hota hai.
- Interview mein BeanFactory ka naam sun ke confuse mat hona
- production mein kabhi directly use nahi hota.
- BeanFactory = basic, lazy loading.
- ApplicationContext = BeanFactory + Events + AOP (@Transactional)
- + i18n + eager loading.
- Spring Boot = hamesha ApplicationContext.

Feature	BeanFactory	ApplicationContext
Bean Loading	LAZY — demand pe	EAGER — startup pe sab
Events	Nahi	Built-in ApplicationEvent [OK]
AOP	Nahi	@Transactional, @Aspect [OK]
i18n	Nahi	MessageSource [OK]
Use When	Legacy/low-memory (rare)	Spring Boot — HAMESHA [OK]

[OK] KEY: AppContext EXTENDS BeanFactory. Extra: Events, AOP, i18n, Eager loading.

- "
- ApplicationContext extends BeanFactory and adds AOP support for @Transactional,
- event publishing, i18n, and eager singleton loading at startup.
- Spring Boot always uses ApplicationContext — never BeanFactory directly."

@Component vs @Service vs @Repository

→ WHY: Teeno internally @Component hain. Phir 3 KYUN? SEMANTICS — code readability. Controller = web layer, Service = business, Repository = data. @Repository ka EK real fark: DB exceptions → DataAccessException translate. DB change karo = exception handling code SAME rahe.

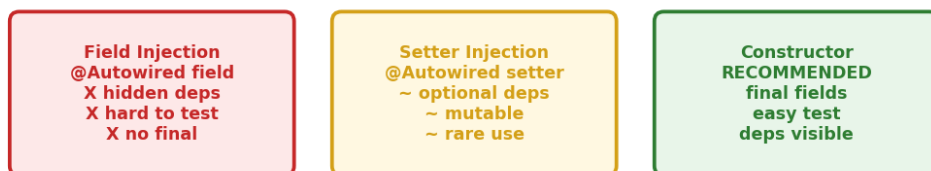
- Teeno andar se @Component hain → sab Spring-managed beans.
- Fark sirf: @Repository
- DB exceptions DataAccessException mein translate karta hai.

Annotation	Layer	Extra Feature?
@Component	Any layer	Kuch nahi — generic bean
@Service	Business logic	Kuch nahi — same as @Component
@Repository	DB layer	DB exceptions → DataAccessException [OK]

- "All three are @Component internally and create Spring-managed beans.
- The only real difference is @Repository
- it translates database-specific exceptions into Spring's DataAccessException
- keeping the service layer decoupled from the DB."

Dependency Injection — 3 Types

Dependency Injection — 3 Types



Constructor = BEST. final + testable + deps visible. Interview: "I always use constructor injection."

▲ Field (hidden, hard test) → Setter (optional, rare) → Constructor (BEST: final, testable, deps visible).

- Field injection (@Autowired field pe) → test mein Spring chahiye, hard to mock.
- Constructor injection → test mein mock directly pass karo
- field final bhi kar sakte ho.
- Best: constructor injection.

Type	Kaise	Recommended?
Constructor	@Autowired on constructor (implicit if single)	YES [OK] — Spring recommends
Setter	@Autowired on setter	Sirf optional dependencies
Field	@Autowired directly field pe	NO [TRAP] — hard to test

- "Constructor injection is recommended —
- dependencies are explicit, fields can be final for immutability,
- and you can inject mocks directly in tests without needing Spring at all."

@Transactional — All or Nothing

- Bank transfer — A se paisa nikalo, B mein daalo.
- Step 1 hua, Step 2 fail. A se paisa gaya, B mein nahi aaya. Beech mein gaayab.
- @Transactional = dono succeed ya dono rollback. Beech mein kuch nahi atakta.
- Trap — IOException pe rollback nahi hoga, sirf RuntimeException pe.
- Fix: Explicitly rollbackFor add karo.
- Bank transfer: A deduct AND B add → dono ek saath ya koi nahi.
- @Transactional = atomicity.
- Step 2 fail → Step 1 bhi rollback.

```
@Transactional // Transaction shuru – deduct aur add dono ek saath
public void transfer(Long fromId, Long toId, int amount) {
    accountRepo.deduct(fromId, amount); // Step 1: A se ₹1000 gaya
    accountRepo.add(toId, amount); // Step 2: B mein ₹1000 aaya
} // Dono success → commit. Koi fail → dono rollback!
```

→ Dono success → ek saath DB mein jaata hai (commit). Step 2 fail hua?

Step 1 bhi undo ho jaata hai (rollback).

→ Paisa kabhi bhi beech mein nahi atakta.

→ Ab TRAP dekho — IOException ke saath:

```
@Transactional // Sirf RuntimeException rollback karega
public void transfer() throws IOException { } // IOException aaya
NO ROLLBACK! [X]
@Transactional(rollbackFor = IOException.class)
// Ab IOException pe bhi rollback [OK]
public void safeTransfer() throws IOException { }
```

[TRAP] TRAP:

→ @Transactional default = sirf RuntimeException rollback.

→ IOException ke liye rollbackFor explicitly likhna MUST hai!

WHY sirf RuntimeException?.

→ Java philosophy: Checked exceptions EXPECTED hain — tune handle karna decide kiya.

→ Shayad IOException pe retry karna chahta hai, rollback nahi.

→ Runtime exceptions UNEXPECTED hain — NullPointerException, ArrayOutOfBoundsException

→ ye bugs hain, kuch bhi ho sakta tha, rollback safe hai.

→ Isliye default = sirf unexpected pe rollback.

→ "@Transactional wraps all DB operations in one transaction

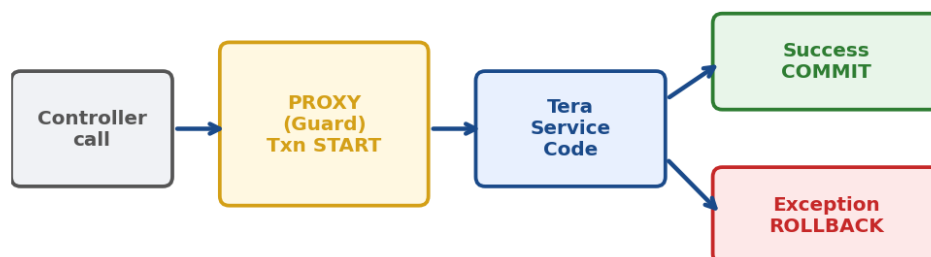
→ if anything fails, everything rolls back.

→ By default only RuntimeException triggers rollback.

→ For checked exceptions like IOException you must explicitly add rollbackFor = IOException.class."

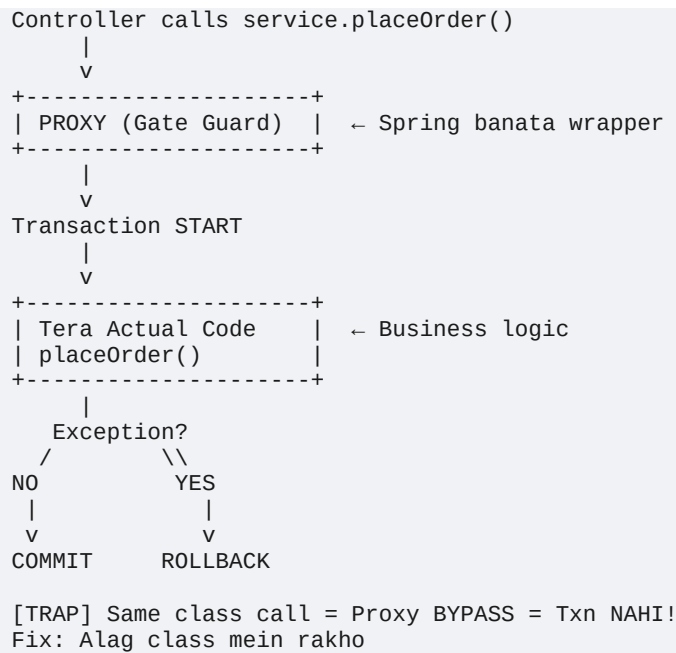
@Transactional Deep — Proxy + Propagation

@Transactional Proxy Flow



▲ Proxy intercept: txn.begin() → actual method (proceed) → success? commit. Exception? rollback. Same class call = proxy BYPASS.

PROXY FLOW DIAGRAM:



- PROXY = gate ka guard.
- Tu @Transactional likhta hai
- lekin tera code mein "transaction start" kaheen NAHI likha.
- Toh ye kaam KAUN karta hai? PROXY.
- Spring tera class ka ek WRAPPER (proxy) banata hai.
- Bahar se call aaye pehle PROXY pe aaye proxy transaction start kare
- teri method chale →
- proxy commit/rollback kare.

```

Controller call kare placeOrder()
  → PROXY pakde (guard)
  → Transaction START
  → Tera actual placeOrder() code chale
  → Sab theek? COMMIT. Exception? ROLLBACK.

```

WHY proxy? Tu business logic likh — transaction ka kaam Spring ka proxy kare.

- Separation of concerns.
- Tera code clean rahe, cross-cutting concerns (transaction, logging, security)
- proxy handle kare.

[TRAP] Proxy BYPASS:

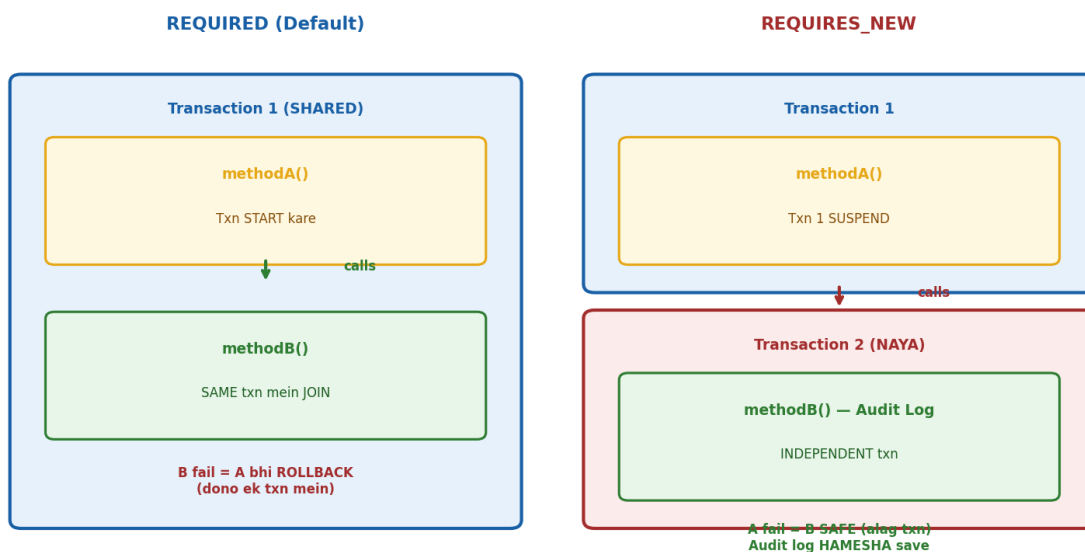
- Same class mein ek method doosre ko call kare = andar se andar = guard (proxy)
- ko pata hi nahi = @Transactional kaam NAHI karega!

```
// GALAT – same class, proxy bypass
@Service
class UserService {
    @Transactional
    public void methodA() {
        this.methodB(); // andar se andar = guard skip
    }
    @Transactional(propagation = REQUIRES_NEW)
    public void methodB() { } // Naya txn NAHI banega!
}
// SAHI – alag class, proxy se jayega
@Service
class OrderService {
    @Autowired AuditService auditService;
    @Transactional
    public void methodA() {
        auditService.methodB(); // BAHAR se call = proxy active
    }
}
@Service
class AuditService {
    @Transactional(propagation = REQUIRES_NEW)
    public void methodB() { } // Ab naya txn BANEGA
}
```

→ Same class = andar se andar = guard bypass = @Transactional dead.

→ Alag class = gate se = proxy active = transaction works.

Propagation — REQUIRED vs REQUIRES_NEW



REQUIRED = same txn JOIN (inner fail = dono rollback). REQUIRES_NEW = NAYA txn (inner fail = sirf inner rollback). Audit log = REQUIRES_NEW.

→ Propagation = 2 methods dono @Transactional

→ “naya transaction banau ya purane mein ghus jaau?”

→ REQUIRED (default) = same car mein baith ja. Pehle se transaction hai?

→ Usme ghus jao. Nahi hai? Naya banao.

→ Ek fail = sab rollback. 90% cases yehi.

```
@Transactional // REQUIRED = default
public void placeOrder() {
    saveOrder(); // T1 mein
    paymentService.pay(); // pay() bhi T1 mein ghus gaya
}
// pay() fail → POORA T1 rollback → order BHI rollback
```

→ REQUIRES_NEW = apni alag car le. Hamesha NAYA transaction. Purana PAUSE.

→ Independent commit/rollback.

```

@Transactional
public void placeOrder() {
    saveOrder(); // T1
    auditService.logAudit("placed"); // T2 NAYA - T1 pause
    // T2 commit (audit saved)
    // T1 resume. Agar yahan exception -> T1 rollback
    // Lekin audit log? SAFE - T2 already committed
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logAudit(String msg) { } // Alag transaction

```

WHY REQUIRES_NEW?

- Audit log, email notification
- ye HAMESHA save hone chahiye chahe main operation fail ho.
- "Order fail hua" ka LOG toh rehna chahiye na?
- Same transaction mein hota toh order rollback = log bhi rollback = record hi nahi.

[TRAP] try-catch TRAP:

- Method mein try-catch se exception pakad li → proxy ko pata hi nahi chali →
- proxy bola "sab theek" → COMMIT.
- Partial data save = DANGEROUS. Fix: catch mein throw karo.

[TRAP] readOnly hint:

- @Transactional(readOnly = true) — Hibernate dirty checking skip kare = faster.
- GET endpoints pe lagao.
- Write BLOCK nahi karta physically — sirf optimization hint.

FLASH:

- Proxy = guard gate pe. REQUIRED (default) = same car, sab saath.
- REQUIRES_NEW = alag car, audit/notification.
- Same class call = proxy bypass = txn dead.
- IOException = rollback NAHI by default.
- try-catch swallow = proxy ko pata nahi = commit.
- Quiz: 16/16.
- "Spring uses proxies for @Transactional
- the proxy intercepts external calls and manages transaction boundaries.
- Same-class calls bypass the proxy.
- For propagation
- REQUIRED joins the existing transaction while REQUIRES_NEW always creates a fresh one
- — useful for audit logging that must persist regardless of the parent transaction's outcome."

Spring Data JPA — Interface Likho, Query FREE

- Problem: Hibernate mein har entity ke liye 50 lines boilerplate
- findById, findAll, save, delete sab manually.
- Solution: Interface extend karo → implementation Spring generate kare.
- ZERO boilerplate.

REPOSITORY HIERARCHY:

```

CrudRepository      c PagingAndSortingRepository c JpaRepository
(basic CRUD)        (CRUD + pagination)           (sab + flush + batch)

Hamesha JpaRepository use karo:
→ findAll() returns List (CrudRepo = Iterable)
→ flush(), saveAndFlush(), deleteInBatch() extra

```

WHY JpaRepository? CrudRepository ka findAll()

- Iterable return karta — loop chahiye.
- JpaRepository List deta — seedha use.
- Plus batch operations production mein chahiye.

BASIC SETUP:

```
@Entity
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String city;
}

// Repository — BAS ITNA
public interface UserRepository
    extends JpaRepository<User, Long> {
    // save(), findById(), findAll(), delete()
    // sab milte hain FREE mein
}
```

- findById() returns Optional<User> — orElseThrow() use karo.
- Null nahi milega.

DERIVED QUERIES — METHOD NAAM = QUERY:

```
User findByEmail(String email);
    → SELECT * FROM user WHERE email = ?

List<User> findByNameAndCity(String name, String city);
    → SELECT * FROM user WHERE name = ? AND city = ?

List<User> findByAgeGreaterThan(int age);
    → SELECT * FROM user WHERE age > ?

List<User> findByNameContaining(String name);
    → SELECT * FROM user WHERE name LIKE '%name%'

boolean existsByEmail(String email);
long countByCity(String city);
void deleteByCity(String city);
```

KEYWORDS:

```
findBy / countBy / existsBy / deleteBy
And / Or / GreaterThan / LessThan / Between
Like / Containing / StartingWith
IsNull / IsNotNull / In / True / False
OrderBy___Asc / OrderBy___Desc
```

WHY kaam karta?

- Spring method naam PARSE karta — findBy + Email = WHERE email = ?.
- CamelCase se field naam nikale. ZERO SQL likha tune.

[TRAP]

- Field naam EXACT match chahiye.
- Entity mein “emailAddress” hai aur method mein findByEmail likha
- APPLICATION START PE ERROR.
- Spring boot pe validate karta.

@QUERY — COMPLEX QUERIES:

```
// JPQL – Java objects pe (DB independent)
@Query("SELECT u FROM User u WHERE u.city = :city AND u.age > :age")
List<User> findActiveByCityAndAge(@Param("city")
String city, @Param("age") int age);

// Native SQL – seedha DB query (DB dependent)
@Query(value = "SELECT * FROM users WHERE city = ?1"
nativeQuery = true)
List<User> findByCityNative(String city);

// UPDATE – @Modifying + @Transactional DONO chahiye
@Modifying
@Transactional
@Query("UPDATE User u SET u.active = false WHERE u.lastLogin <
:date")
int deactivateInactiveUsers(@Param("date") LocalDate date);
```

[TRAP]

- UPDATE/DELETE pe @Modifying + @Transactional DONO lagao.
- Bina @Modifying → error. Bina @Transactional → rollback nahi.
- Decision: Simple → Derived (method naam). Medium → JPQL @Query.
- Complex/performance → Native @Query.

PAGINATION FLOW:

```
Client: GET /users?page=0&size=10&sort=name
    |
    v
Controller: PageRequest.of(page, size, Sort.by(sort))
    |
    v
Repository: repo.findAll(pageable)
    |
    v
Response: {
  content: [user1, user2, ...],
  totalPages: 50,
  totalElements: 500,
  number: 0,          // current page
  size: 10,           // page size
  first: true,
  last: false
}
```

WHY pagination? 10 lakh rows bina pagination = OutOfMemoryError.

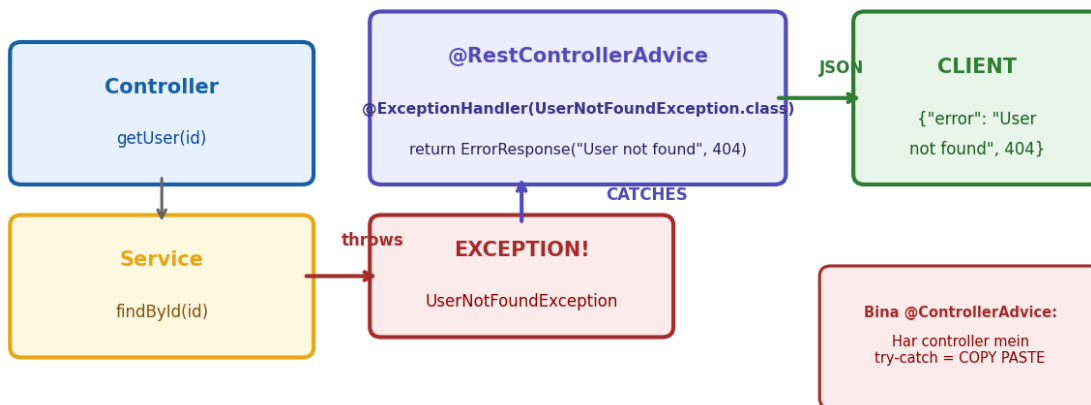
- Page 10-20 records dikhao. Production mein MUST.

FLASH:

- JpaRepository extend karo = CRUD free. Method naam = query auto.
- @Query = complex.
- Pagination = PageRequest.of(). findById = Optional. Quiz: 16/16.
- "I use Spring Data JPA with JpaRepository for all data access.
- Simple queries use derived methods — Spring generates SQL from the method name.
- For complex joins, I use @Query with JPQL.
- All list endpoints are paginated using PageRequest."

@ControllerAdvice — Global Exception Handling

@ControllerAdvice — Global Exception Handler



Exception kahi bhi ho — @ControllerAdvice EK jagah se handle. 50 controllers mein try-catch NAHI.
Global error handling = CLEAN code. DRY principle.

Controller > Service throws Exception > @ControllerAdvice CATCHES > JSON error response to client. Har controller mein try-catch NAHI = DRY.

→ Problem: Har controller mein try-catch = copy paste = DRY violation.

→ Solution: EK class mein sab exceptions handle karo — @ControllerAdvice.

EXCEPTION HANDLING FLOW:

```
Controller mein exception aaye
|
v
+-----+
| @RestControllerAdvice |
| GlobalExceptionHandler |
+-----+
|
v
@ExceptionHandler match kare
|       |       |
v       v       v
404     400     500
NotFound Validation Catch-all
|
v
Client ko CLEAN JSON:
{ status: 404, message: "...", timestamp: "..." }
```

GLOBAL EXCEPTION HANDLER:

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    // 404 – Resource not found
    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(ResourceNotFoundException ex) {
        return new ErrorResponse(404, ex.getMessage(), LocalDateTime.now());
    }

    // 400 – Validation error
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(e → e.getField() + ": " + e.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return new ErrorResponse(400, msg, LocalDateTime.now());
    }

    // 500 – Catch-all (LAST mein rakho)
    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleGeneral(Exception ex) {
        return new ErrorResponse(500, "Something went wrong",
            LocalDateTime.now());
    }
}

```

- **@RestControllerAdvice = @ControllerAdvice + @ResponseBody.**
- **REST API mein hamesha ye use karo.**

[TRAP]

- **Exception.class = catch-all. LAST mein rakho.**
- **Pehle specific (NotFound, Validation), phir generic.**
- **Nahi toh sab generic mein aa jayenge.**

CUSTOM EXCEPTIONS:

```

@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String resource, Long id) {
        super(resource + " not found with id: " + id);
    }
}

// Service mein use:
public User getUser(Long id) {
    return repo.findById(id)
        .orElseThrow(() → new ResourceNotFoundException("User", id));
    // → 404: "User not found with id: 5"
}

```

- **Custom exception = meaningful naam + auto HTTP status.**
- **Service throw kare → handler catch kare → client ko clean JSON.**

VALIDATION WITH @Valid:

```

public class CreateUserDto {
    @NotBlank(message = "must not be blank")
    private String name;

    @Email(message = "must be valid email")
    private String email;
}

@PostMapping("/users")
public User create(@Valid @RequestBody CreateUserDto dto) { }
// Invalid? → MethodArgumentNotValidException → handler → 400

```

[TRAP]

- **Production mein KABHI stack trace client ko mat bhejo!**
- **500 pe generic "Something went wrong".**
- **Stack trace sirf LOGS mein (CloudWatch/Papertrail). Security risk.**

WHY @ControllerAdvice? Controller CLEAN rahe — sirf business logic.

- Error handling EK jagah — consistent format.
- Naya developer try-catch bhool bhi jaye — global handler pakde.

FLASH:

- @RestControllerAdvice = global error handler.
- Custom exceptions throw karo service se.
- @ExceptionHandler se catch karo. Consistent JSON: status + message + timestamp.
- @Valid + DTO = 400 auto.
- Stack trace KABHI client ko nahi. Quiz: 16/16.
- "I use @RestControllerAdvice for global exception handling —
- custom exceptions with @ResponseStatus for specific errors
- and a catch-all for 500s that logs the stack trace but returns a generic message to the client.
- DTO validation uses @Valid with Bean Validation annotations."

All Annotations — Quick Reference

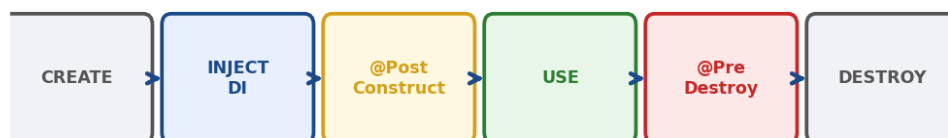
@Autowired	Inject dependency. BY TYPE first, phir @Qualifier se by name.
@Bean	Third-party classes jo annotate nahi kar sakte (e.g. RestTemplate).
@Configuration	Ye class beans define karti hai — @Bean methods ke saath.
@Primary	Multiple same type beans mein default bean.
@Qualifier("name")	Specific bean by name — @Primary ko override karta hai.
@PostConstruct	Bean creation + injection ke BAAD run hota hai.
@PreDestroy	Bean destroy hone se PEHLE run hota hai.
@Transient	Hibernate: ye field DB mein save mat karo. @Transactional se BILKUL ALAG!

[TRAP] TRAP:

- @Transient (Hibernate) ≠ @Transactional (Spring). Completely different!
- Most confused pair.
- "@Autowired injects by type. @Qualifier picks a specific bean by name.
- @PostConstruct runs after injection for init logic.
- @Transient skips a Hibernate field — do not confuse it with @Transactional."

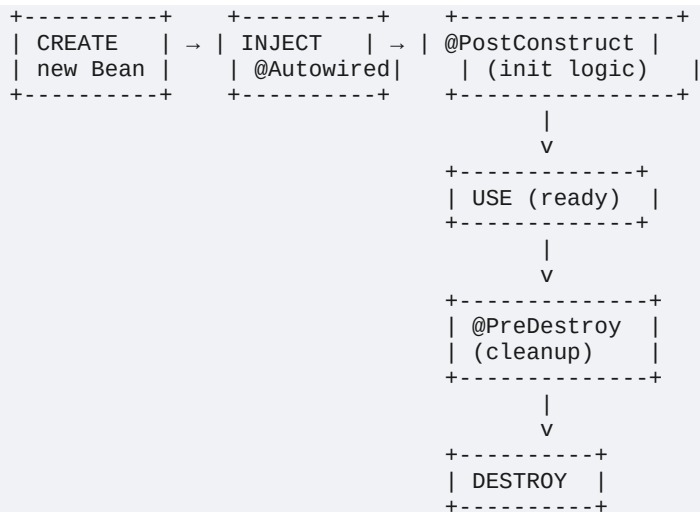
Bean Lifecycle & Scopes

Spring Bean Lifecycle



▲ Container start → Bean create → @PostConstruct → Ready → Use → @PreDestroy → Destroy. Singleton = 1x, Prototype = har request.

BEAN LIFECYCLE DIAGRAM:



Employee analogy: Hire → Laptop do → Training → Kaam → Exit
Create → Inject → @PostConstruct → USE → @PreDestroy → Destroy

WHY ye order?. Soch: Naya employee aaya (Create).

- Usko pehle laptop aur desk do (Inject) — bina tools ke kaam kaise karega?.
- PHIR orientation/training karo (@PostConstruct).
- Training mein laptop chahiye na?. Isliye Inject PEHLE, @PostConstruct BAAD.
- Ulta karo toh NullPointerException — dependency abhi aayi hi nahi.
- [REPEAT TRAP] Bean Lifecycle Trap — Baar Baar Galat!.
- Employee aaya (Create) → laptop diya (Inject) → PHIR training (@PostConstruct).
- Inject PEHLE, @PostConstruct BAAD.
- Sawaal mein Create→@PostConstruct→Inject dikhe?. GALAT.

Scope	Behavior	Analogy
Singleton (DEFAULT)	Ek shared object poori app	Office printer — sab share karte hain
Prototype	Har baar naya object	Disposable cup — fresh each time
Request	Ek HTTP request pe naya	Naya notepad per customer visit
Session	Ek user session pe naya	Naya notepad per user account

[TRAP] TRAP: @PostConstruct hai, @PreConstruct NAHI HOTA!

[TRAP] TRAP: Singleton + mutable state = thread-safety issue! Keep Singleton beans STATELESS.

- "Singleton is the default scope
- one shared instance for the entire application.
- Prototype creates a new instance every time.
- Singleton beans must be stateless
- shared mutable fields across threads cause race conditions."

Spring MVC — Web Framework

→ WHY: Spring MVC KYUN? Servlet API directly use karo = HttpServletRequest/Response handle karo manually = boilerplate. Spring MVC = DispatcherServlet sab route kare, tu sirf @Controller + @RequestMapping likho. request → DispatcherServlet → HandlerMapping → Controller → ViewResolver → response. Tu sirf Controller likho, baaki Spring.

- GET /users aayi. Kaun pakdega?.
- Spring mein ek front door hai — DispatcherServlet.
- Har request pehle yahan aati hai.
- Phir HandlerMapping se poochha
- "kaun handle karega?" UserController mila.
- Controller ne kaam kiya, response bheja.
- Ye flow interview mein hamesha poochhte hain
- seedha diagram draw karo dimaag mein.
- DispatcherServlet = front controller.
- Har HTTP request pehle yahan aati hai HandlerMapping Controller ViewResolver
- Response.
- "DispatcherServlet is the front controller
- every HTTP request passes through it first.
- It uses HandlerMapping to find the right controller invokes it
- and sends back the response."

Request Flow

Spring MVC Request Flow



▲ Client → DispatcherServlet → HandlerMapping → Controller → Service → Repository → DB. Response ulta path se wapas.

REQUEST FLOW DIAGRAM:



[OK] KEY: DispatcherServlet → HandlerMapping → Controller → ViewResolver.
HAMESHA poochha jaata hai!

- "The Spring MVC request flow is: DispatcherServlet receives the request,
- HandlerMapping finds the controller, the controller processes the business logic,
- and ViewResolver renders the response
- — skipped entirely for @RestController since JSON is returned directly."

/users/101 → @PathVariable	URL mein SLASH / ke baad = @PathVariable
/users?id=101 → @RequestParam	URL mein QUESTION MARK ? ke baad = @RequestParam

[TRAP] TRAP: / = @PathVariable. ? = @RequestParam. Simple but common trick question!

- "@PathVariable is used when the value is part of the URL path like /users/101.
- @RequestParam is used when the value is a query parameter like /users?id=101."

@Controller vs @RestController

→ WHY: @Controller = HTML pages return (ViewResolver). @RestController = JSON/XML data return (@Controller + @ResponseBody). REST API bana rahe = @RestController. Web app with Thymeleaf = @Controller. @RestController = har method pe @ResponseBody auto.

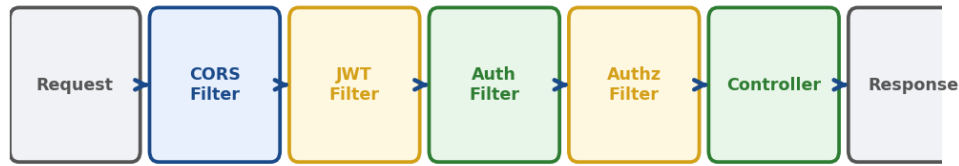
- @Controller → view naam return → ViewResolver → HTML.
- @RestController = @Controller + @ResponseBody → JSON seedha return.
- REST APIs mein @RestController use karo.

[TRAP] TRAP:

- @Controller use kiya, @ResponseBody bhool gaye
- JSON nahi milega, blank page milega!
- "@RestController is @Controller plus @ResponseBody combined —
- it returns JSON directly, skipping the ViewResolver entirely.
- Use @RestController for all REST APIs."

Spring Security

Spring Security — Filter Chain Flow



Request → CORS → JWT extract+validate → Authentication → Authorization → Controller

▲ Request → CORS Filter → JWT Filter (extract+validate) → Auth Filter → Authorization Filter → Controller → Response.

→ WHY: Spring Security KYUN? Authentication (KAUN hai) + Authorization (KYA kar sakta) = har app mein chahiye. Manual likho = har endpoint pe check = DRY violation. Spring Security = Filter Chain based — request aaye, filters check karein, authorized toh controller tak jaaye. EK jagah configure, SAB endpoints protected.

- API live hua. Koi bhi kisi bhi route pe ja sakta tha — admin panel bhi.
- Spring Security = filter chain.
- Request aayi JWT filter token validate authenticated? permission hai?
- controller.
- 401 = pehchaana nahi. 403 = pehchaana, permission nahi.
- Authentication: WHO are you? → 401 fail.
- Authorization: WHAT can you do? → 403 fail.
- Authentication hamesha pehle hoti hai.

Concept	Matlab	HTTP Code
Authentication	WHO are you? Identity verify.	401 Unauthorized
Authorization	WHAT can you do? Permissions check.	403 Forbidden

[OK] KEY: 401 = Authentication failed. 403 = Authorization failed. Auth HAMESHA pehle.

- "Authentication verifies who you are — it returns 401 if it fails.
- Authorization checks what you are allowed to access — it returns 403 if it fails.
- Authentication always comes first."

BCrypt — NOT SHA256!

- Passwords SHA256 mein hash kiye the.
- Hacker ne rainbow table attack kiya — pehle se computed hashes se match kar liya.
- SHA256 fast hai — ek second mein billions hashes.
- BCrypt = intentionally slow + random salt per hash.
- Same password ka alag alag hash. Brute force = saalon lagein.
- BCrypt ki slowness weakness nahi — feature hai.
- SHA256: fast + no salt → rainbow table attack.
- BCrypt: random salt per hash + intentionally slow → same password ka alag hash.
- Always BCrypt for passwords.

BCrypt [OK]	Intentionally SLOW. Random salt per hash. Same pass = alag hash. Passwords ke liye.
SHA256 [X]	Very fast. No built-in salt. General integrity — passwords ke liye NAHI.

[TRAP] TRAP: BCrypt ki slowness = intentional FEATURE. BCrypt slow hai isliye secure hai.

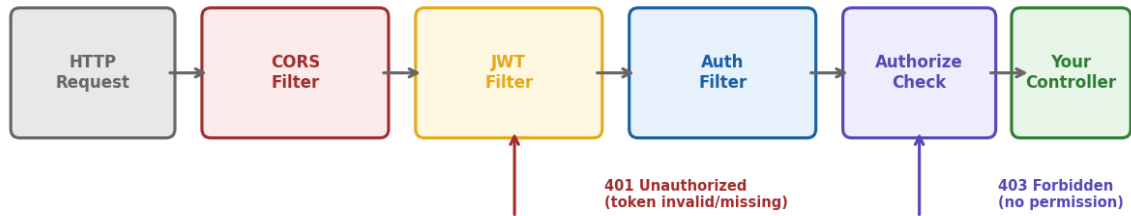
- "BCrypt is the correct choice for password hashing
- it is intentionally slow,
- uses a unique random salt per hash so the same password produces different hashes
- each time, making brute-force and rainbow table attacks completely impractical."

SecurityFilterChain

SecurityFilterChain — Request ka Safar

Har request SABSE pehle filter chain se guzre. Ek bhi fail = controller tak NAHI pahunche.

Spring Security Filter Chain



Request > CORS > JWT Filter > Auth > Authorize > Controller. Ek fail = 401/403. Controller tak nahi pahunche.

- CSRF: browser automatically cookie bhejta hai → evil site exploit kar sakta hai.
- JWT Authorization header mein hai
- browser kabhi automatically headers nahi bhejta CSRF threat nahi.
- JWT = CSRF disable karo.
- Ab ye config dekho — wohi cheezein jo upar boli:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) {
    http.csrf(c -> c.disable()) // JWT hai → CSRF threat nahi → disable
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/login").permitAll() // Login page → sab ke liye open
        .requestMatchers("/admin/**").hasRole("ADMIN") // Admin pages
        .satisfies((request) -> request.isAuthenticated()) // Baaki sab → login required
    );
    http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class); // JWT filter pehle chalao
    return http.build();
}
```

- csrf disable → kyunki JWT header mein.
- permitAll → /login pe koi bhi aa sakta hai. hasRole → /admin pe sirf ADMIN.
- addFilterBefore → request aate hi pehle JWT check ho.

[TRAP] TRAP: JWT ke saath CSRF disable karo — JWT header mein hai, cookie mein nahi.

- "CSRF protection is disabled for JWT-based APIs because JWT travels in the Authorization header —
- browsers never automatically send custom headers to other sites
- so CSRF attacks simply do not apply here."

JWT — JSON Web Token

→ WHY: JWT KYUN? Session-based = server store kare kaun login hai. 10L users = 10L sessions = memory FULL. Multiple servers = session sync issue. JWT = SELF-CONTAINED token. {userId, role, expiry} + signature. Server pe KUCH store nahi. Stateless = scale easy, koi bhi server handle kare.

- Session-based auth mein server state maintain karta hai — scale nahi hota.
- JWT mein sab token mein hota hai — server stateless.
- Lekin ek baat — payload encrypted nahi hai, sirf Base64 encoded hai.
- Browser console mein decode kar sakte ho.
- Isliye password kabhi payload mein mat daalo — sirf userId, role, expiry.
- JWT = Header.Payload.Signature.
- Payload: Base64 encoded → koi bhi decode kar sakta hai! Encrypted NAHI.
- Tamper karo → signature invalid → reject. Passwords kabhi JWT mein mat rakho.

Part	Contains	Secure?
Header	Algorithm (HS256) + type (JWT)	Base64 — koi bhi decode kar sakta hai!
Payload	username, role, expiry, user ID	Base64 — passwords KABHI mat daalo!
Signature	Hash(Header + Payload + SECRET_KEY)	Tamper-proof — secret bina forge nahi

[TRAP] TRAP:

- Payload = Base64 ENCODED, encrypted NAHI. Koi bhi decode kar sakta hai.
- Passwords KABHI JWT mein mat rakho!
- "JWT has three parts — header, payload, and signature.
- The payload is only Base64 encoded, not encrypted — anyone can decode and read it.
- The signature is a hash of the header and payload with a secret key
- any tampering invalidates it immediately."

Access Token vs Refresh Token

→ WHY: 2 tokens KYUN? Access token churaya = damage. Short access (15 min) = max 15 min damage. Refresh token = 7 din, HttpOnly cookie (JS access nahi). Access expire → refresh bhejo → naya access. Security + UX balance.

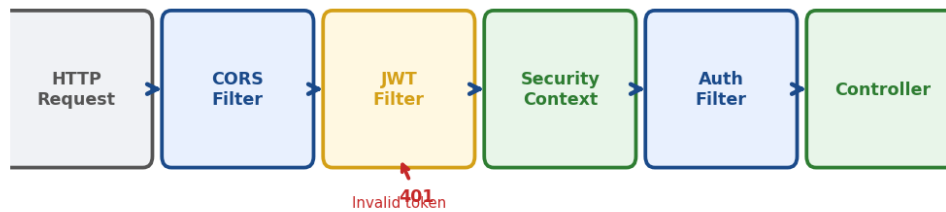
- Access Token: short-lived (15 min) → expire → limited damage.
- Refresh Token: long-lived (30 days)
- HttpOnly cookie → naya access token lene ke liye.
- Logout → DB se bhi revoke karo.

Feature	Access Token	Refresh Token
Lifespan	15 minutes	7–30 days
Storage	Client memory / Auth header	HttpOnly Cookie (JS access nahi!)
If stolen	Limited — 15 min max	Dangerous — DB se revoke karo!
Purpose	Har API call ke saath	Sirf naya Access Token lene ke liye

[TRAP] TRAP: Logout ≠ client se delete only. DB se bhi Refresh Token revoke karo server-side!

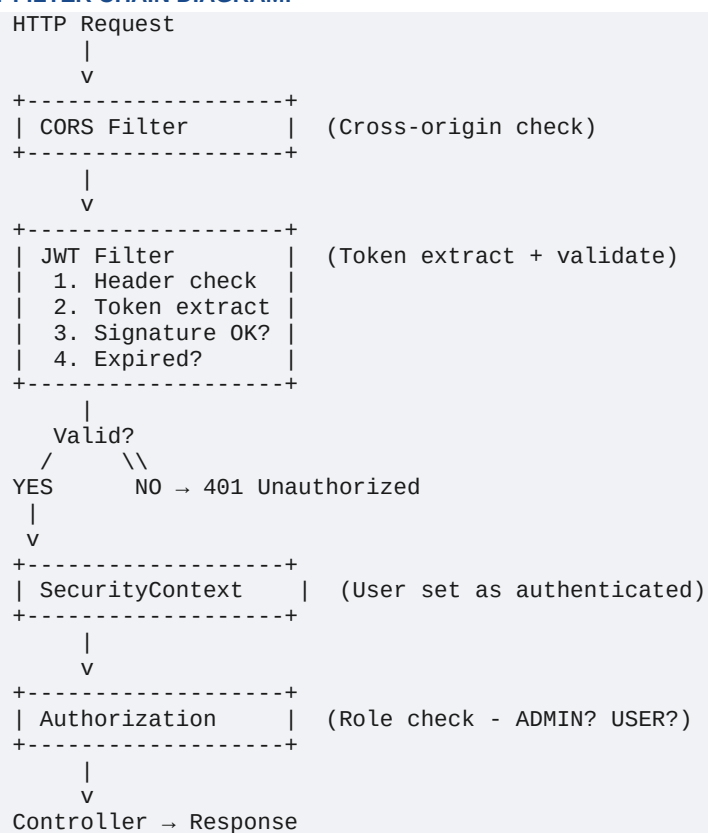
- "Access tokens are short-lived at 15 minutes
- if stolen, the damage is limited.
- Refresh tokens are long-lived and stored in HttpOnly cookies so JavaScript cannot read them.
- On logout, the refresh token must be revoked from the DB server-side
- — deleting it from the client alone is not enough."

JWT Authentication Filter Chain



▲ Request → JWT Filter (token extract + validate) → SecurityContext set → Controller. Invalid token = 401.

JWT FILTER CHAIN DIAGRAM:



→ JwtFilter: har request pe Token nikalo username extract signature validate

→ SecurityContext mein set karo.

→ OncePerRequestFilter = ek hi baar per request.

→ Ab steps dekho — guard kya kya karta hai har request pe:

```

// Step 1: Header mein Authorization dekho → "Bearer eyJhbG..."
String authHeader = request.getHeader("Authorization");
// Step 2: "Bearer " se start hota hai? Nahi toh skip – koi token hi nahi
if (authHeader == null || !authHeader.startsWith("Bearer "))
{ filterChain.doFilter(); return; }
// Step 3: "Bearer " hata ke sirf token nikalo
String token = authHeader.substring(7);
// Step 4: Token se username nikalo
String username = jwtUtil.extractUsername(token);
// Step 5: DB se us user ki details lo
UserDetails user = userDetailsService.loadUserByUsername(username);
// Step 6: Signature valid hai? Expiry nahi gayi? Dono check karo
if (jwtUtil.validateToken(token, user)) {
// Step 7+8: Authenticated object banao aur SecurityContext mein rakh do
UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(user,
null, user.getAuthorities());
SecurityContextHolder.getContext().setAuthentication(auth);
}
// Step 9: Controller ko pass karo – ab woh jaanta hai user authenticated hai
filterChain.doFilter(request, response);

```

Step 1-3: Token milega toh aage badho. Step 4-5: DB se user pehchano.

Step 6: Token pakka valid hai?.

Step 7-8: "Ye authenticated hai" SecurityContext mein likh do.

Step 9: Controller ko bhejo.

[OK] KEY: SecurityContextHolder = thread-local.
Ek baar set kiya = poori request ke liye authenticated.

→ "JwtFilter extends OncePerRequestFilter — it runs exactly once per request.

→ It extracts the Bearer token validates the signature and expiry

→ loads UserDetails from the DB,

→ and sets the authentication in SecurityContextHolder so the rest of the request knows the user is authenticated."

SOLID Principles

→ WHY: SOLID KYUN? Bina SOLID = code TIGHT coupled, modify karo toh 10 jagah todhe, test mushkil, extend mushkil. SOLID = loose coupling + high cohesion. S=ek reason to change. O=extend karo modify nahi. L=child parent ki jagah use ho. I=fat interface mat banao. D=abstraction pe depend karo.

SOLID Principles



▲ S=ek reason to change. O=extend karo, modify nahi. L=child parent ki jagah use ho. I=fat interface mat banao. D=abstraction pe depend.

- Ek baar OrderService mein sab kuch tha
- order logic, DB save, email, PDF generate.
- Marketing ne email format badla — OrderService toot gayi.
- DB team ne schema badla — phir toot gayi.
- Teen alag reasons to change = teen alag teams tod sakti hain.
- SOLID = ye pain avoid karne ke principles. Pehle se follow karo toh regret nahi.
- SOLID = good design principles.
- Spring Boot mein pehle se follow hota hai: @Service=SRP, interface=OCP, DI=DIP.

Letter	Principle	One-Line Rule
S	Single Responsibility	Ek class = ek hi kaam — ek hi reason to change
O	Open / Closed	Extension ke liye open. Modification ke liye closed.
L	Liskov Substitution	Child class ko parent ki jagah use karo — kuch break na ho
I	Interface Segregation	Client ko unneeded methods implement karne pe force mat karo
D	Dependency Inversion	High-level module abstraction pe depend kare, low-level pe nahi

S — Single Responsibility

- ShoppingCart: cart logic + DB save + email = 3 responsibilities = SRP violation.
- Split karo: CartService, CartRepository, EmailService.

[TRAP] TRAP: SRP ≠ one method per class. SRP = ONE REASON TO CHANGE.

- "Single Responsibility means a class should have only one reason to change.
- If ShoppingCart handles cart logic
- DB saving and email notifications
- three different teams can break it for three different reasons
- — split them."

O — Open / Closed

- PayPal hai, Stripe add karna hai. BAD: if-else chain → existing code touch.
- GOOD: PaymentProcessor interface → naya class banao.
- Existing code = ZERO change.

| [OK] KEY: Spring: @Repository interface + @Autowired = OCP. App never knows the DB impl.

- "Open/Closed means add new features by writing new classes
- not by editing existing tested code.
- Use interfaces so new implementations can plug in without touching the existing logic."

L — Liskov Substitution

- Account: withdraw() → FixedDeposit: withdraw() throws Exception.
- NotificationService crash.
- Child parent ki jagah use nahi ho sakta → LSP violation. Hierarchy fix karo.

[TRAP] TRAP:

- LSP violation = child throws exception on inherited method.
- Design fix karo, if-else nahi.
- "Liskov Substitution means a child class must be substitutable for its parent without breaking anything.
- If FixedDeposit throws an exception on withdraw()
- which Account promises to support, that is a design violation
- — fix the hierarchy, not the caller."

I — Interface Segregation

- Fat interface: save()+delete()+generatePDF()+sendEmail().
- ReadOnlyRepository ko delete() implement karna padega → useless + dangerous.
- Better: chhoti focused interfaces.
- "Interface Segregation means do not force a class to implement methods it does not need.
- Split fat interfaces into smaller focused ones
- — Spring's CrudRepository and JpaRepository are a perfect real-world example."

D — Dependency Inversion

- OrderService → MySQLOrderRepository seedha → tight coupling.
- DIP: OrderService → OrderRepository (interface)
- @Autowired inject → kaunsa DB pata nahi. Easy to swap.

[TRAP] TRAP:

- DIP ≠ Dependency Injection. DIP = PRINCIPLE. DI = TECHNIQUE to achieve it.
- Spring @Autowired implements DIP.
- "Dependency Inversion means high-level modules should depend on abstractions
- not on concrete implementations.
- Spring's @Autowired achieves this
- — the service depends on the repository interface
- not on a specific database class."

SOLID in Spring Boot

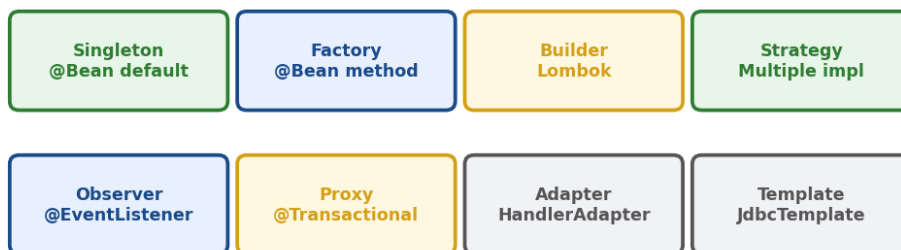
S — SRP	@Service = business logic, @Repository = DB, @Controller = HTTP. Har ek ka ek kaam.
O — OCP	Naya DB? Naya @Repository. Existing Service = untouched.
L — LSP	Koi bhi @Repository doosre ki jagah — same interface contract.
I — ISP	CrudRepository, JpaRepository, PagingAndSortingRepository alag focused interfaces.
D — DIP	@Autowired abstraction inject karta hai. Service kabhi new MySQLRepository() nahi karta.

Interview Traps — SOLID

Principle	Wrong Answer	Correct Answer
SRP	"One method per class"	"One REASON TO CHANGE"
OCP	"Never change existing code"	"Closed for modification, open for extension"
LSP	"Child extends parent"	"Child must honour parent contract fully"
ISP	"Use small interfaces"	"Client should not implement unused methods"
DIP	"DIP = Dependency Injection"	"DIP is the PRINCIPLE. DI is one way to achieve it."

Design Patterns

Design Patterns — 8 Patterns in Spring



Creational: Singleton, Factory, Builder. Behavioral: Strategy, Observer, Template. Structural: Adapter, Proxy

▲ Creational: Singleton(@Bean), Factory(@Bean method), Builder(Lombok). Behavioral: Strategy, Observer(@EventListener), Template. Structural: Adapter, Proxy(@Transactional).

→ WHY: Design Patterns KYUN? 50 saal mein developers ne SAME problems baar baar solve kiye. Patterns = proven solutions to recurring problems. Roz naya invent mat karo — pattern use karo. Interview mein: pattern jaano + Spring mein kahan use hota wo jaano. Factory=@Bean, Observer=@EventListener, Proxy=@Transactional, Strategy=multiple implementations.

- Ek baar team mein naya developer aaya.
- Code dekha
- object creation har jagah bikhri thi if-else chains thi
- same problem alag alag jagah solve ki gayi thi.
- Pattern naam = shared vocabulary.
- "Factory use karo yahan" bola — poori team ne seedha samjha.
- Patterns isliye nahi seekhte ki ratta lagao
- — isliye seekhte hain ki common problems ke common solutions hain
- aur naam se communicate karo.
- Pattern naam = shared vocabulary.
- "Strategy use kiya" → interviewer immediately samjha. Naam yaad karo.

Factory Pattern — Pizza Counter

→ WHY: Factory KYUN? new CreditCardProcessor() = client ko EXACT class pata = TIGHT coupling. Naya processor add = client code change. Factory: client bole "payment chahiye", factory decide kare kaunsa. Client ko implementation NAHI pata. @Bean = factory method.

- Notification system mein Email, SMS, Push — teeno banane ka kaam har jagah tha.
- Ek jagah new EmailNotification(), doosri jagah new SMSNotification().
- Kal naya type aaya toh har jagah dhundh ke change karo.
- Factory banaya — ek jagah decide hoga kaunsa object banega.
- Client sirf type bolta hai, Factory banayegi.
- Spring ka @Bean yehi karta hai.
- Client: "circle chahiye" ShapeFactory.create("circle")
- new Circle() return.
- Client ko class naam pata nahi. Object creation centralize.

```
// PROBLEM: Client ko nahi pata kaunsi Shape class use karein
// SOLUTION: Factory ek jagah decide karega – client sirf type boleگا
interface Shape {
    void draw(); // Contract – sab shapes yahi implement karein
}
class Circle implements Shape {
    public void draw() { System.out.println("Circle draw"); }
}
class Square implements Shape {
    public void draw() { System.out.println("Square draw"); }
}
// Factory – object creation ek jagah centralize
class ShapeFactory {
    static Shape create(String type) { // Client type dega
        if (type.equals("circle")) return new Circle(); // Factory decide karegi
        if (type.equals("square")) return new Square();
        return null;
    }
}
// Client – class naam jaanta hi nahi, bas type diya
Shape s = ShapeFactory.create("circle"); // Circle mila [OK]
s.draw(); // Client ko pata nahi andar kya bana
```

- Pizza counter jaisa — customer ne bas 'Margherita' bola.
- Counter ne chef se banwayi.
- Customer ko nahi pata kaunsa chef, kaunsa oven.
- ShapeFactory yehi karta hai
- client sirf 'circle' bolta hai, Factory new Circle() karti hai.

[OK] KEY: Spring @Bean annotation Factory pattern hai.

- [REPEAT TRAP] @Bean Pattern Trap — Baar Baar Galat!.
- @Bean method object banata hai = FACTORY. Singleton NAHI.
- @Scope("singleton") alag cheez.
- Sawaal: Spring @Bean = kaunsa pattern? → Factory chun.
- "Factory pattern centralises object creation —
- the client requests an object without knowing which concrete class is instantiated.
- Spring's @Bean annotation is a real-world example of this pattern."

Factory = Object creation ek jagah. Client ko class naam nahi pata.

Builder Pattern — Subway Sandwich

→ WHY: Builder KYUN? new User(name, null, 25, null, "MZN", null) = kaunsa param kya? Confusing. Builder: User.builder().name("Arpan").age(25).city("MZN").build() = READABLE. Step by step, naam ke saath. Lombok @Builder = free.

- User object banana tha — naam, email, age, address, phone, preferences.
- Constructor mein 8 parameters: new User("Arpan" null, 26 null
- null, "India" null true).
- Kaunsi position pe kya?. Bug dhundhna mushkil.
- Builder se: .name("Arpan").age(26).country("India").build()
- sirf jo chahiye, baaki skip.
- Lombok ka @Builder ye automatically generate karta hai.
- new Person("Arpan", null, 26, null) → confusing.
- Builder: .name("Arpan").age(26).build() → clear, sirf jo chahiye.
- Phone: setReceiver().setBrand().build().

```
// PROBLEM: new Person("Arpan", null, 26, null, "India")
- confusing!
// SOLUTION: Builder - step by step, sirf jo chahiye set karo
class Person {
    String name, email, country; int age;
    private Person(Builder b) { // Private - sirf Builder se banega
        this.name = b.name; this.age = b.age;
        this.email = b.email; this.country = b.country;
    }
    static class Builder {
        String name, email, country; int age;
        Builder name(String n) { this.name = n; return this; } // return this = chain
        Builder age(int a) { this.age = a; return this; }
        Builder email(String e) { this.email = e; return this; }
        Builder country(String c) { this.country = c; return this; }
        Person build() { return new Person(this); }
    }
}
// Client - sirf jo chahiye, null pass nahi karna
Person p = new Person.Builder()
    .name("Arpan")
    .age(26)
    .country("India")
    .build(); // email nahi diya - koi null nahi [OK]
```

- Subway sandwich jaisa — bread, filling, sauce ek ek step mein choose karo.
- Jo nahi chahiye simply skip.
- Builder mein bhi .name().age().country() — sirf jo zarurat, baaki skip.
- Constructor mein 10 null pass karne ki zarurat khatam.

[TRAP] TRAP:

- new Person("Arpan", null, 26, null, "India")
- kaunsi position pe kya?
- Builder mein .name().age() clearly dikhta hai.

[OK] KEY: Lombok @Builder is pattern ka shortcut. StringBuilder bhi iska real example.

- "Builder pattern constructs complex objects step by step using a fluent API —
- you only set what you need and skip the rest
- making the code readable and safe from positional null mistakes."

Builder = Complex object step-by-step readable banao. Null pass karna band!

Adapter Pattern — iPhone to Samsung Charger

→ WHY: Adapter KYUN? 2 incompatible interfaces ko connect karna. iPhone charger Android mein nahi lagta — adapter lagao. Legacy code ka interface naya code ke saath compatible nahi — adapter class banao. Wrapper pattern.

- Nayi logging library integrate karni thi.
- Purani library log(String msg) expect karti thi.
- Nayi library mein write(String msg, int level) tha. Seedha use karo?
- Compile error — alag interface. Poori codebase mein 500+ calls change karo?.
- Kal phir koi nayi library aaye toh phir 500 jagah?. Sustainable nahi.
- Isiye Adapter — purani interface implement karo, andar nayi library wrap karo.
- Dono side kuch nahi badla, beech mein translator laga diya.
- OldPayment → log(msg). NewPaymentSystem → write(msg, level).
- Direct compatible nahi.
- Adapter beech mein: OldPayment implement karo, andar NewPaymentSystem wrap karo.

```
// PROBLEM: OldLogger → log(String msg)
// NewLogger → write(String msg, int level) ← incompatible interface
// SOLUTION: Adapter — beech mein translator lagao, dono side kuch nahi badla
interface OldLogger {
    void log(String msg); // Expected interface — ye use karna hai
}
class NewLogger {
    void write(String msg, int level) { // Incompatible — alag method signature
        System.out.println "[" + level + " " + msg);
    }
}
// Adapter — OldLogger ka interface implement karo, andar NewLogger wrap karo
class LoggerAdapter implements OldLogger {
    private NewLogger newLogger; // Andar NewLogger hai
    LoggerAdapter(NewLogger nl) { this.newLogger = nl; }
    public void log(String msg) {
        newLogger.write(msg, 1); // Translate karke call karo [OK]
    }
}
// Client — OldLogger use karta rahega — kuch nahi badla dono side
OldLogger logger = new LoggerAdapter(new NewLogger());
logger.log("Server started"); // [OK] koi bhi class nahi badli
```

- iPhone-Samsung charger jaisa
- iPhone ke plug ko Samsung ke plug mein seedha nahi lagega.
- Adapter beech mein lagao — dono side kuch nahi badla.
- LoggerAdapter yehi hai — OldLogger ka face, andar NewLogger ka kaam.

[TRAP] TRAP:

- Adapter vs Decorator — Adapter INTERFACE BADALTA hai.
- Decorator same interface rakhta hai, sirf behavior add karta hai.
- "Adapter pattern acts as a translator between two incompatible interfaces —
- you write an adapter class in the middle without modifying either side.
- Adapter changes the interface; Decorator keeps it the same but adds behaviour."
- Adapter = Translator.
- Incompatible interfaces bina code change kiye compatible banao.

Composite Pattern — Computer Parts Tree

→ WHY: Composite KYUN? Tree structure represent karna — folder mein files + folders. Individual (leaf) aur group (composite) ko SAME treat karo. `getSize()` = file pe direct, folder pe = sab children ka sum. Recursive structure.

- Manager ne bola
- "computer ka total price calculate karo."
- Computer ke andar Cabinet hai Cabinet ke andar Motherboard
- Motherboard ke andar CPU aur RAM.
- Har level pe alag logic likhna pada
- `cabinet.price() + motherboard.price() + cpu.price()...`
- Jab naya component aaya toh poora logic todna padta tha.
- Tab Composite banaya — ek interface, `getPrice()` sab pe. `computer.getPrice()` bolo
- tree mein propagate ho gaya, sab ka sum aa gaya.
- Tu sirf root pe bolo, tree khud kaam karega.
- Computer → Cabinet → [CPU, RAM, Monitor, MB].
- `computer.getPrice()` → tree mein propagate → sab `getPrice()` → sum.
- Leaf aur Composite same interface.

```
// PROBLEM: Computer → Cabinet → Motherboard → CPU + RAM
// Har level pe alag price logic likhna padega
// SOLUTION: Composite — sabka ek interface, getPrice() sab pe kaam kare
interface Component {
    int getPrice(); // Leaf aur Composite dono same interface — uniform treat
}
// Leaf — akela, koi child nahi
class CPU implements Component {
    public int getPrice() { return 15000; } // Sirf apna price
}
class RAM implements Component {
    public int getPrice() { return 8000; }
}
// Composite — children hain, unka price jodo
class Motherboard implements Component {
    List<Component> parts = new ArrayList<>();
    void add(Component c) { parts.add(c); } // Children add karo
    public int getPrice() {
        return parts.stream()
            .mapToInt(Component::getPrice) // Sab children ka price jodo
            .sum();
    }
}
// Use — ek call, poora tree calculate
Motherboard mb = new Motherboard();
mb.add(new CPU()); // 15000
mb.add(new RAM()); // 8000
mb.getPrice(); // 23000 — manually kuch nahi joda [OK]
```

- computer.getPrice() bola —
- Computer apne andar Cabinet ko puchha Cabinet ne Motherboard ko
- Motherboard ne CPU aur RAM ko.
- Ek call mein poora tree calculate.
- Tu parent node pe kaam bolo — tree khud recursively kaam karega.

Leaf	Akela object — CPU, HDD — koi child nahi
Composite	Dabba — Cabinet, Computer — andar aur objects hain

- "Composite pattern treats individual objects and groups uniformly under a common interface.
- Calling getPrice()
- on a Computer automatically propagates through the tree and sums all leaf prices
- — you don't need to know the structure."

Composite = Ek node pe kaho kaam kar → woh khud aur saare bachhon se karwayega.

Prototype Pattern — Office Photocopy

→ WHY: Prototype KYUN? Object create karna EXPENSIVE hai (DB call, complex setup). Pehle se bana object CLONE karo = FAST. clone() method use. Shallow vs Deep copy samjho.

- BookShop object DB se fetch → 100 copies chahiye → 100 baar DB? Ya clone karo.
- BookShop implements Cloneable → clone() → naya object without DB hit.

```
// PROBLEM: BookShop object DB se fetch - 100 copies chahiye
// SOLUTION: clone() - scratch se mat banao, photocopy nikalo
class BookShop implements Cloneable {
    String name;
    List<String> courses; // [WARNING] Yahi problem hai shallow copy mein
    // Shallow Copy - Default clone()
    public Object clone() throws CloneNotSupportedException {
        return super.clone(); // name naya, lekin courses = SAME reference [X]
    }
    // Deep Copy - manually sab naya banao
    public Student deepCopy() {
        Student s = new Student();
        s.name = this.name; // String - naya ban jaata hai
        s.courses = new ArrayList<>(this.courses); // List - naya banao - KEY!
        return s;
    }
}
// Shallow Copy TRAP
Student s1 = new Student();
s1.courses.add("Java");
Student s2 = (Student) s1.clone(); // Shallow copy
s2.courses.add("Hibernate"); // s2 mein add kiya
System.out.println(s1.courses); // [Java, Hibernate] ← s1 mein bhi aa gaya! [X]
// Deep Copy - Safe
Student s3 = s1.deepCopy();
s3.courses.add("Docker");
System.out.println(s1.courses); // [Java] ← s1 safe! [OK]
```

- Photocopy nikali — form ka content copy hua.
- Lekin agar form mein staple lagi hai (List)
- toh shallow copy mein SAME staple hai dono ke paas.
- Ek ne form faada dono ka gaya. deepCopy() mein naya staple lagao
- alag alag independent.
- Shallow Copy — int, String naye ban jaate hain.
- Andar ke objects (List, Address) = SAME reference.
- Default clone() = Shallow.
- Deep Copy — Andar tak sab naya.
- Ek mein change = doosra same. clone() override karke khud likhna padta hai.

[TRAP] TRAP:

- Student clone kiya. Andar List<Course> hai.
- Copy mein course add kiya — ORIGINAL MEIN BHI aa gaya!
- Shallow ka trap.
- "Prototype pattern clones an existing object instead of building from scratch
- useful when creation is expensive.
- Be careful with shallow copy
- — nested objects like Lists share the same reference
- so changes in the clone affect the original."

Prototype = Photocopy. Clone karo, scratch se mat banao!

Observer Pattern — YouTube Subscription

→ WHY: Observer KYUN? Order create hua → email bhejo + inventory update + analytics log. OrderService mein teeno call? TIGHT coupled. Observer: event publish karo, subscribers sun lein. Naya subscriber add = ZERO change in publisher. Spring @EventListener = Observer.

- Order complete hone pe email bhejna tha — OrderService mein likha.
- Phir analytics chahiye — OrderService kholni padi.
- Phir push notification — phir kholni padi.
- Har naye feature pe same file touch karni padti thi.
- Ek baar kisi ne bug daala email code add karte waqt
- order save hi nahi ho raha tha.

- Problem: OrderService ko pata tha kaun sun raha hai.
- Fix: event fire karo, subscribers apne aap react karein.
- Naya feature?. Naya listener. OrderService ka code zero change.
- Channel = subject. Subscribers = observers.
- Creator upload kare → sab subscribers notify. Unsubscribe → list se hata.
- Creator ka code nahi badla.

```
// PROBLEM: Order complete → email, dashboard, analytics – sab OrderService mein?
// SRP + OCP violation! Naya system add = existing code touch
// SOLUTION: Event fire karo – subscribers apne aap react karein
class OrderCompletedEvent {
    private final Order order;
    OrderCompletedEvent(Order o) { this.order = o; }
    Order getOrder() { return order; }
}
@Service
class OrderService {
    @Autowired ApplicationEventPublisher publisher; // Spring ka broadcaster
    public void completeOrder(Order o) {
        orderRepo.save(o);
        publisher.publishEvent(new OrderCompletedEvent(o)); // Fire! Bas yahan tak
    } // OrderService ko nahi pata kaun sun raha hai [OK]
}
// Subscriber 1 – Email
@Component
class EmailListener {
    @EventListener
    public void onOrder(OrderCompletedEvent e) {
        emailService.send(e.getOrder().getUserEmail());
    }
}
// Subscriber 2 – Analytics (naya feature – OrderService ka ZERO change [OK])
@Component
class AnalyticsListener {
    @EventListener
    public void onOrder(OrderCompletedEvent e) { analytics.record(e.getOrder()); }
}
```

- YouTube jaisa — Creator ne video upload kiya.
- Usne individually kisi ko message nahi kiya.
- Jisne subscribe kiya tha sab ko apne aap notification.
- OrderService event fire karta hai
- EmailListener, AnalyticsListener apne aap react karte hain.
- Naya subscriber?.
- Sirf naya @EventListener banao.
- Order complete → OrderService event fire karo.
- EmailListener, AnalyticsListener @EventListener se react karein.
- Naya feature = naya listener. OrderService ka code ZERO change.
- "Observer pattern notifies all registered subscribers when an event occurs —
- the publisher doesn't know who is listening.
- Spring's ApplicationEvent and @EventListener implement this pattern directly."

Observer = YouTube subscription. Ek event fire → sab subscribers apne aap react!

- [REPEAT TRAP] Pattern Confusion — Baar Baar Galat!.
- YouTube notification = Observer. Google Maps mode swap = Strategy.
- Purana charger naya phone = Adapter. "event/listener" = Observer.
- "if-else hatao" = Strategy.

Singleton Pattern — Desh Ka PM

→ WHY: Singleton KYUN? DB connection pool, config, logger = EK instance enough. Multiple banao = resource waste + inconsistency. Private constructor + static getInstance(). Spring beans DEFAULT singleton hain. Enum singleton = BEST (JVM guarantee, serialization safe, reflection safe).

- App config — DB URL, API keys — baar baar load karna bekar hai.

- Ek baar load karo, poori app mein same object.
- Lekin 2 threads ek saath pehli baar access kiya dono ne null dekha
- dono ne naya object bana diya.
- 2 configs, alag alag values. Production bug.
- Isiye DCL + volatile
- pehla check bina lock ke null mila toh synchronized block mein jao
- andar phir check karo.
- Best way: Enum Singleton — JVM khud guarantee deta hai ek hi instance.
- class ABC: private constructor + private static instance + getInstance().
- Poori app mein ek hi object.
- Spring @Service/@Repository = Singleton by default.

- Eager → class load hote hi banta hai. Thread safe automatically.
- Memory waste ho sakti hai.
- Lazy → pehli baar maango tabhi banta hai. Thread safe NAHI by default.
- 2 threads → dono null check → dono banate hain → 2 objects!
- DCL: check → sync block → check again → create.
- volatile: CPU cache se stale value na dikhe.
- Best: Enum Singleton.

```
// SINGLETON — sirf ek instance poori app mein
// Option 1: Eager Loading — class load hote hi banta hai (simple + thread safe)
class Config {
    private static final Config INSTANCE = new Config(); // Startup pe banta hai
    private Config() {} // Private — bahar new nahi hoga
    public static Config getInstance() { return INSTANCE; }
}

// Option 2: DCL + volatile — Lazy (maango tabhi banao)
class Config {
    private static volatile Config instance = null; // volatile — visibility guarantee
    private Config() {}
    public static Config getInstance() {
        if (instance == null) { // Check 1 — lock lene se pehle (fast path)
            synchronized (Config.class) { // Ek hi thread andar
                if (instance == null) { // Check 2 — doosra thread pehle bana chuka ho?
                    instance = new Config();
                }
            }
        }
        return instance;
    }
}

// Option 3: ENUM — BEST WAY (interview mein yahi bolo!)
enum AppConfig {
    INSTANCE; // JVM ek hi banata hai — reflection safe, serialization safe
    public String getDbUrl() { return "jdbc:mysql://..." }
}

// Use karo
AppConfig.INSTANCE.getDbUrl(); // Har jagah SAME instance [OK]
// TV Test — Singleton proof
AppConfig obj1 = AppConfig.INSTANCE;
obj1.setChannel(5);
AppConfig obj2 = AppConfig.INSTANCE;
obj2.setChannel(6);
System.out.println(obj1.getChannel()); // 6 print hoga
SAME object [OK] Singleton PASS
```

- DCL: pehla check bina lock ke (fast path — 99% requests yahan se return).
- Null mila?
- Toh synchronized block mein jao
- doosra check karo (thread A aur B dono null check kar chuke the).
- Ab ek hi banata hai. volatile isiye
- CPU caching se instance ka stale value dikhe toh buggy hoga.
- Enum = best: JVM seedha guarantee deta hai.
- TV proof
- Papa ne channel 5, Bhai ne 6. obj1.show() → 6 print hoga kyunki SAME object!

[TRAP] TRAP:

- 5 5 print hua = alag objects = Singleton FAIL.
- 5 6 print hua = same object = Singleton PASS [OK]
- "Singleton ensures only one instance exists in the entire application.
- The best implementation is Enum Singleton
- — JVM guarantees exactly one instance and it is safe from reflection and serialization attacks.
- Use DCL plus volatile for lazy initialization when Enum is not possible."

Singleton = Sirf ek instance. Poori app same object. Best way = Enum.

Strategy Pattern — Google Maps Route Modes

→ WHY: Strategy KYUN? if(type=="UPI") processUPI(); else if(type=="CARD") processCard(); = har naya type pe if-else add = OCP violation. Strategy: PaymentStrategy interface, UpiStrategy, CardStrategy classes. Runtime pe swap karo. Naya type = naya class, existing ZERO change.

- Payment ke liye if-else chain thi — PayPal, Razorpay, UPI.
- Naya payment type aaya? if-else mein ghusna pado.
- Existing tested code touch karna pada.
- Ek baar Razorpay ka bug fix karte waqt PayPal ka flow toot gaya
- same function tha.
- Strategy: har payment type ek alag class.
- Checkout ko pata hi nahi andar kaunsa hai — sirf pay() call karta hai.
- Naya type?. Naya class.
- Existing code zero change.
- if-else for payment types → OCP violation.
- Strategy: PaymentStrategy interface → PayPal class, Razorpay class.
- Checkout(new PayPal()) → runtime pe swap. Naya payment = naya class only.

```
// PROBLEM: Payment ke liye if-else chain — naya payment = existing code touch
// SOLUTION: Strategy — har payment ek class, interface pe depend karo
interface PaymentStrategy {
    void pay(int amount); // Contract — sab payment methods yahi follow karen
}
class PayPal implements PaymentStrategy {
    public void pay(int amount) { System.out.println("PayPal: " + amount); }
}
class Razorpay implements PaymentStrategy {
    public void pay(int amount) { System.out.println("Razorpay: " + amount); }
}
// Context — runtime pe kaunsi strategy use karni hai, ye inject hogi
class Checkout {
    private PaymentStrategy strategy; // Interface pe depend — class pe nahi
    Checkout(PaymentStrategy s) { this.strategy = s; } // DI se inject
    void processPayment(int amount) {
        strategy.pay(amount); // Strategy khud decide karegi — Checkout ko nahi pata
    }
}
// Runtime pe swap karo
Checkout c = new Checkout(new PayPal()); // PayPal use karo
c.processPayment(500);
c = new Checkout(new Razorpay()); // Razorpay pe switch
c.processPayment(500); // Koi bhi class nahi badli [OK]
// Naya payment? Sirf naya class banao — Checkout ka ZERO change [OK] (OCP!)
```

- Google Maps jaisa — destination same, sirf mode alag: Driving, Walking, Transit.
- Strategy = mode.
- Maps ko nahi pata andar kaunsa algorithm chal raha — bas call karo.
- Payment mein same — Checkout ko nahi pata andar PayPal hai ya Razorpay.
- "Strategy pattern lets you swap algorithms at runtime by depending on an interface —
- instead of if-else chains for payment types or sorting algorithms,
- each option is a separate class implementing the same strategy interface."

Strategy = Runtime pe algorithm badlo. if-else ki jagah interface!

Quick Revision — Sab Patterns

Pattern	Ek Line Mein	Real Example
Factory	Object creation centralize karo	Spring @Bean
Builder	Step-by-step readable object banao	Lombok @Builder, StringBuilder
Adapter	Incompatible interface translator	iPhone-Samsung charger
Composite	Tree — ek call sab se karwao	Computer price tree
Prototype	Clone karo, scratch se mat banao	Game enemy clone
Observer	Event fire → sab subscribers react	YouTube, Spring @EventListener
Singleton	Sirf ek instance poori app	Spring beans, Logger
Strategy	Runtime pe algorithm swap karo	Google Maps, Payment types

Interview Traps

→ Ye table print karke rakh lo — ek baar ghoomna kaafi hai.

Topic	Trap Question	Correct Answer
Core Java	String s1="Java" s2="Java" s1==s2?	TRUE — same String pool object
Core Java	HashMap duplicate key?	Old value REPLACED. No exception!
Core Java	finally after return?	HAMESHA runs. Sirf System.exit() rokta hai.
Core Java	Integer a=128; b=128; a==b?	FALSE — -128 to 127 cache ke bahar.
JDBC	Statement security?	Injection risk. Always PreparedStatement.
Hibernate	get() not found?	NULL return karta hai (safe).
Hibernate	load() not found?	ObjectNotFoundException throw karta hai.
Hibernate	DETACHED kaise banao?	session.close() OR evict(). Commit se nahi!
Hibernate	@OneToMany default fetch?	LAZY. @ManyToOne = EAGER.
Hibernate	@Entity requirement?	No-arg constructor MUST.
Hibernate	update() vs merge()?	merge() HAMESHA safe. update() = risk.
Hibernate	evict(obj) DB se delete karta hai?	NO! Sirf L1 se → DETACHED.
N+1	10 students + lazy courses = queries?	11 (1+10). Fix: JOIN FETCH → 1.
Caching	L2 with HQL?	NO! Alag Query Cache chahiye.
Spring	AppCtx vs BeanFactory?	AppCtx adds Events+AOP+i18n+Eager.
Spring	@Transient vs @Transactional?	COMPLETELY different! Transient=skip DB.
Spring	@Transactional on IOException?	NO rollback! rollbackFor add karo.
Spring	@Repository vs @Component?	@Repository ka extra = exception translation.
Spring	Init annotation naam?	@PostConstruct. @PreConstruct exist NAHI karta!
MVC	/users/101 annotation?	@PathVariable (/ = path var)
MVC	/users?id=101 annotation?	@RequestParam (? = query param)
Security	401 vs 403?	401 = not authenticated. 403 = no permission.
Security	BCrypt vs SHA256?	BCrypt = slow+salt = passwords. SHA256 = NOT.
Security	CSRF for JWT?	DISABLE. JWT header mein hai, cookie mein nahi.
JWT	Payload encrypted hai?	NO! Base64 only. Passwords NEVER!
JWT	Logout = client delete only?	NO! DB se bhi Refresh Token revoke karo.

Code Snippets

JDBC PreparedStatement

```
String sql = "SELECT * FROM users WHERE email = ?"
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, userEmail); // safe – query mein nahi jaata
ResultSet rs = ps.executeQuery();
while(rs.next()) { System.out.println(rs.getString("name")); }
```

Hibernate CRUD + N+1 Fix

```
Long id = (Long) session.save(student); // ID return karta hai
Student st = session.get(Student.class, 1); // null if not found
st.setName("Updated"); tx.commit(); // dirty check auto-syncs
Student m = (Student) session.merge(det); // safe reattach
"FROM Student s JOIN FETCH s.courses" // N+1 FIX
```

Constructor Injection

```
@Service
public class StudentService {
    private final StudentRepository repo;
    public StudentService(StudentRepository repo) { // @Autowired implicit
        this.repo = repo;
    }
}
```

REST Controller

```
@RestController @RequestMapping("/api/students")
public class StudentController {
    @GetMapping public List<Student> getAll() { ... }
    @GetMapping("/{id}") public Student getById(@PathVariable Long id) { ... }
    @PostMapping @ResponseStatus(HttpStatus.CREATED)
    public Student create(@RequestBody Student s) { ... }
    @DeleteMapping("/{id}") public void delete(@PathVariable Long id) { ... }
}
```

Power Phrases

- Concepts tere dimaag mein hain. Phrases = packaging.
- Padh, samjh, apne words mein bol.

Q: BeanFactory vs ApplicationContext

- BeanFactory basic IoC container hai — beans lazily banata hai.
 - ApplicationContext extends BeanFactory karta hai aur event publishing
 - AOP (@Transactional), i18n, aur eager singleton loading add karta hai.
 - Spring Boot hamesha ApplicationContext use karta hai.
- Power words: extends BeanFactory | events | AOP | i18n | eager loading | Spring Boot = ApplicationContext

Q: N+1 Problem

- N+1 lazy loading se hota hai. 10 students fetch karo, loop mein courses access karo
 - 1+10 = 11 queries.
 - Fix = JOIN FETCH jo sab ek query mein load karta hai.
- Power words: 1+N queries | lazy loading | JOIN FETCH = fix | 11 → 1

Q: LazyInitializationException

- Session close ke baad lazy collection access karo toh aata hai.
 - Student fetch karo, session close, getCourses() call — exception.
 - Fix = @Transactional service method pe
 - session poori transaction tak open rehti hai.
- Power words: lazy after session closed | @Transactional keeps session open | JOIN FETCH alternative

Q: Dependency Injection

- new keyword ki jagah Spring bahar se dependencies inject karta hai.
 - Loose coupling — classes specific implementations pe depend nahi karte.
 - Unit testing easy — mocks inject kar sakte hain.
- Power words: inject from outside | loose coupling | no new keyword | easy to mock

Q: @Component vs @Service vs @Repository

- Teeno @Component hain internally — sab Spring-managed beans banate hain.
 - Real fark sirf ek: @Repository DB exceptions DataAccessException mein translate karta hai.
- Power words: all three = @Component | @Repository has exception translation

Q: BCrypt vs SHA256

- BCrypt passwords ke liye design kiya gaya hai.
 - Random salt per hash — same password ka alag hash.
 - Intentionally slow — brute-force expensive.
 - SHA256 fast hai, no salt — passwords ke liye nahi.
- Power words: one-way | random salt | intentionally slow | NOT SHA256

Q: @Transactional

- Sab DB operations succeed ya fail together — all or nothing.
- RuntimeException → rollback.
- Default mein sirf unchecked RuntimeExceptions.
- IOException ke liye rollbackFor = IOException.class explicitly add karo.

| Power words: all or nothing | RuntimeException only by default | rollbackFor for checked

Q: Authentication vs Authorization

- Authentication WHO verify karta hai — credentials se identity check. Fail → 401.
- Authorization WHAT check karta hai — resource ki permission. Fail → 403.
- Authentication hamesha pehle.

| Power words: verify identity | check permissions | 401 | 403 | authentication first

Tera Introduction

→ Mock interview mein weak tha. First impression = everything.

→ Strong intro → confidence dono sides ko.

Hi, I am Arpan Maheshwari.

I have 4 years of experience as a Java Backend Developer, primarily working with Java Spring Boot, and Microservices in the healthcare domain.

Currently I work at Konovo on the Product Support Team.

My day-to-day work involves building and debugging REST APIs

AWS Lambda and API Gateway, and managing data in MySQL and MongoDB.

One thing I am proud of

— I identified a production bug where our system was capturing AWS IPs instead of client IPs for survey responses.

The team dismissed it initially,

but I proved it with CloudWatch and Papertrail logs fixed it and received recognition from our VP and CTO.

I have a strong foundation in Core Java Spring ecosystem

Hibernate, and JWT-based security.

Looking for a mid-level backend role to work on complex distributed systems and grow technically.

Part	Kya Bolna	Kyun
Line 1	Naam + Years + Stack + Domain	Who you are. Immediate context.
Line 2	Company + Role + Daily work	Real production experience.
Line 3	IP bug + VP/CTO recognition	Concrete achievement. Trust builder.
Line 4	Skills + What you want	Growth mindset, not desperation.

Practice Rules

1. Aaj hi 5 baar padho — ZORSE. Mouth ko practice chahiye, brain ko nahi.
2. Phone pe record karo. Sunna bore lag raha hai? Wahi gap hai jo fix karna hai.
3. **Word by word mat yaad karo. Structure samjho → apne flow mein bol.**
4. Target: 2 minute se kam, koi "umm" nahi, koi filler nahi.
5. Strong start: "Hi, I am Arpan Maheshwari" — pehla word = apna naam.

Multithreading

- API ke 3 DB calls the — user fetch, orders fetch, recommendations fetch.
- Ek ke baad ek — 300ms + 300ms + 300ms = 900ms response time.
- Manager ne bola "too slow." Teen threads banaye, teeno parallel chale
- 300ms total.
- Multithreading ke bina modern backend slow hai.
- Lekin shared state pe careful rehna padta hai
- race condition aur deadlock ka darr hamesha rehta hai.

Thread kya hai?

- Restaurant mein ek waiter = ek kaam ek time pe.
- Teen waiters = teeno alag customers ek saath.
- Thread = alag alag kaam parallel mein.

```
// Without Thread:  
Task1 → Task2 → Task3 (ek ke baad ek)  
  
// With Thread:  
Task1  
Task2 } → sab ek saath  
Task3 }
```

Thread banane ke 2 tarike

- Tarika 1: Thread extend karo.
- Tarika 2: Runnable implement karo (prefer karo
- kyunki Java mein ek hi extend ho sakta hai).

```
// Tarika 1 — Thread extend karo  
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread chal raha hai!");  
    }  
}  
MyThread t = new MyThread();  
t.start(); // ← start() likho, run() NAHI!  
  
// Tarika 2 — Runnable implement karo (PREFER KARO)  
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Task chal raha hai!");  
    }  
}  
Thread t = new Thread(new MyTask());  
t.start();
```

[TRAP] TRAP:

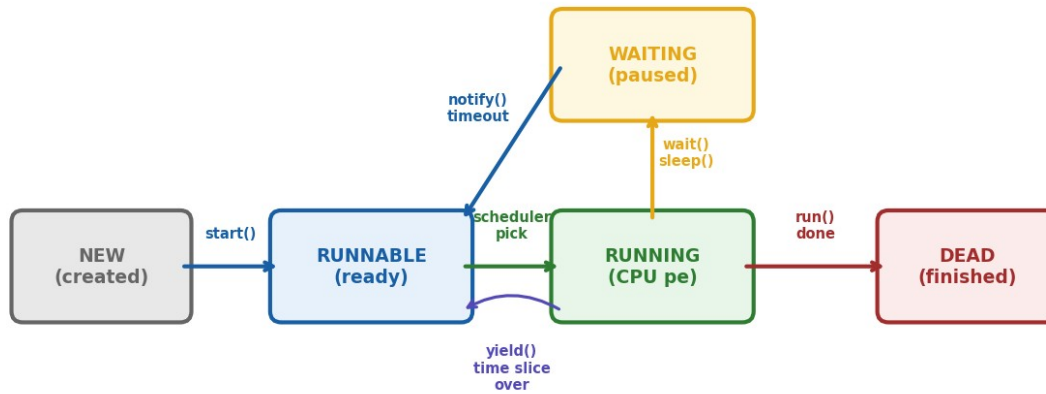
- t.run() likha toh new thread NAHI banega — normal method call hoga same thread mein.
- HAMESHA t.start() likho.

[TRAP] TRAP:

- Runnable prefer karo Thread extend karne se
- — kyunki agar Thread extend kiya toh koi aur class extend nahi kar sakte (Java single inheritance).
- "Thread can be created by extending Thread class or implementing Runnable.
- Runnable is preferred because Java doesn't support multiple inheritance
- — implementing Runnable keeps the class free to extend others."

Thread Lifecycle

Thread Lifecycle — 5 States



NEW → start() → RUNNABLE → scheduler → RUNNING → done → DEAD
 ↓ wait()/sleep() ↔ WAITING ↓ notify()/timeout

NEW > start() > RUNNABLE > scheduler > RUNNING > wait()/sleep() > WAITING > notify() > back. Done > DEAD.

→ WHY: Thread states KYUN? OS ko thread manage karna hai — kab CPU de, kab wait karwaye. NEW = abhi bana. RUNNABLE = ready, CPU ka wait. RUNNING = CPU mil gaya. BLOCKED/WAITING = kisi resource ka wait (lock, sleep, I/O). TERMINATED = kaam khatam. State transitions = OS scheduler decide kare.

- new Thread() banaya
- thread exist karta hai lekin CPU nahi mila abhi (NEW). start() call kiya
- — CPU milne ke liye ready (RUNNABLE).
- JVM ne CPU diya
- kaam chal raha hai (RUNNING). synchronized block wait kar raha hai
- (BLOCKED). run() complete
- — (TERMINATED).
- Ek baar TERMINATED hua toh dobara start() nahi kar sakte
- IllegalStateException.
- Thread ka life = 5 states.

→ new Thread() → start() → JVM schedule kare → run() → khatam.

NEW → RUNNABLE → RUNNING → BLOCKED/WAITING → TERMINATED

NEW → new Thread() banaya, start() abhi nahi kiya
 RUNNABLE → start() call kiya, CPU ke liye ready hai
 RUNNING → JVM ne CPU diya, run() chal raha hai
 BLOCKED → kisi lock ka wait, ya sleep(), ya wait()
 TERMINATED → run() khatam ho gaya

```

Runnable task = new MyTask();
Thread t = new Thread(task); // NEW
t.start(); // RUNNABLE
// JVM decide karta kab RUNNING
// run() complete → TERMINATED
  
```

[TRAP] TRAP:

- start() ke baad thread turant RUNNING nahi hota — pehle RUNNABLE hota hai.
- JVM/OS scheduler decide karta hai kab CPU milega.

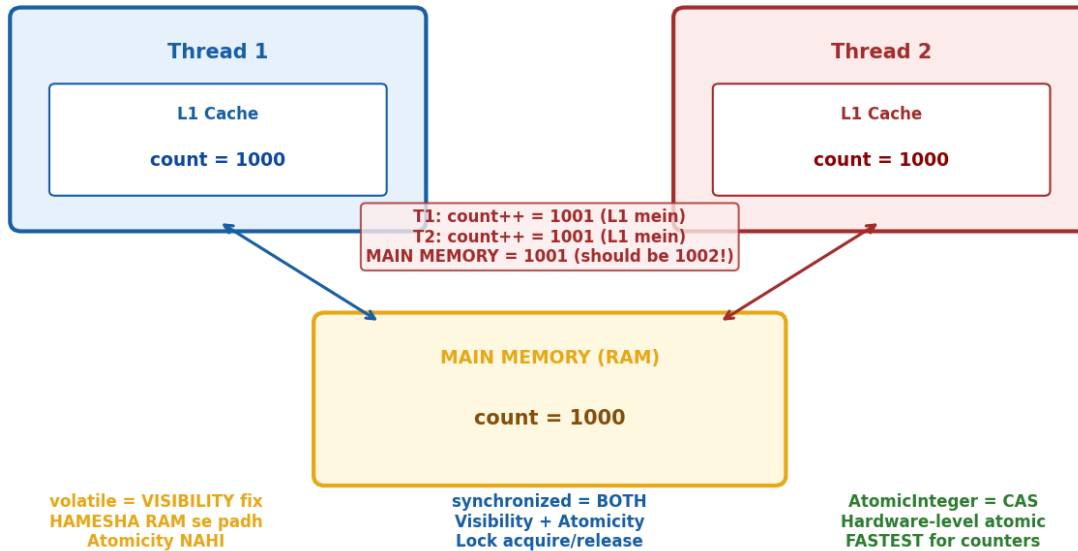
[TRAP] TRAP:

- ek baar TERMINATED thread dobara start() nahi ho sakta
- IllegalStateException.
- "A thread goes through 5 states: NEW when created, RUNNABLE after start(),
- RUNNING when the scheduler gives it CPU time
- BLOCKED or WAITING when it needs a lock or waits,

→ and TERMINATED when run() completes."

synchronized — Race Condition se Bachao

synchronized vs volatile — CPU Cache Problem



CPU L1 cache = visibility problem. volatile = RAM se padh (visibility). synchronized = lock (visibility+atomicity). AtomicInteger = CAS (fastest counter).

- 2 threads ek saath bank account update kar rahe the.
- Thread 1 ne balance=1000 padha, Thread 2 ne bhi 1000 padha.
- Thread 1 ne 1500 likha, Thread 2 ne 800 likha.
- Thread 1 ka update lost. Final balance 800 — galat.
- Ye race condition hai.
- synchronized = ek time pe sirf ek thread critical section mein.
- Lock lega, kaam karega, release karega.
- Tab doosra aayega.
- Bank mein 2 threads ek saath balance update kar rahe hain
- dono ne ₹1000 padha, dono ne alag alag likha.
- Ek ka update lost ho gaya. Ye Race Condition hai.

```
// Thread 1: balance + 500 karna hai
// Thread 2: balance - 200 karna hai

// Bina sync ke:
// Thread 1 reads: 1000
// Thread 2 reads: 1000 // Thread 1 ne abhi write nahi kiya
// Thread 1 writes: 1500
// Thread 2 writes: 800 // Thread 1 ka update LOST!
// Final: 800 – GALAT! 1300 hona chahiye tha

// FIX – synchronized
class BankAccount {
    private int balance = 1000;

    synchronized void deposit(int amount) {
        balance = balance + amount; // ek time pe sirf ek thread
    }
    synchronized void withdraw(int amount) {
        balance = balance - amount;
    }
}

// Method vs Block
synchronized void deposit(int amount) { ... } // poori method lock – slow

void deposit(int amount) {
    synchronized(this) { // sirf critical part lock – faster
        balance += amount;
    }
}
```

[TRAP] TRAP:

- synchronized slow karta hai
- poori method mat lock karo, sirf jahan zaroori ho utha block-level use karo.

[TRAP] TRAP:

- synchronized = visibility + atomicity dono fix karta hai.
- volatile sirf visibility.
- "synchronized ensures only one thread executes the critical section at a time —
- it acquires a lock on the object, preventing race conditions.
- Use block-level sync for better performance."

volatile — Visibility Problem Fix

- Ek background thread tha jo flag check karke loop chal raha tha.
- Doosre thread ne flag false kar diya — lekin pehla thread band nahi hua.
- Ghante baad pata chala
- CPU caching ki wajah se pehle thread ko naya value dikha hi nahi.
- Apni purani cache se padh raha tha. volatile lagaya
- ab har thread seedha main memory se padhta hai, cache se nahi.
- CPU fast kaam kare isliye har thread apni local cache banata hai
- main memory ki copy.
- Problem: ek thread update kare, doosre ko pata nahi chalta
- wo apni purani cache dekh raha hota hai.

```
// Bina volatile:
boolean running = true;

// Main Memory: running = true
// Thread 1 cache: running = true // apni copy
// Thread 2 cache: running = true // apni copy

// Thread 2 ne false kiya:
// Main Memory: running = false [OK]
// Thread 1 cache: running = true [X] // abhi bhi purana!
// Thread 1 ko pata hi nahi chala – loop band nahi hoga

// volatile lagao – cache band, seedha main memory:
volatile boolean running = true;

// Thread 2 ne false kiya → main memory update
// Thread 1 seedha main memory dekh raha hai → false dikh gaya [OK]
```

[TRAP] TRAP:

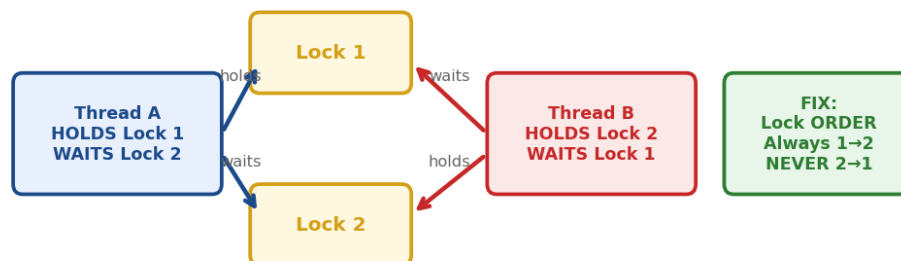
- volatile = sirf visibility fix.
- count++ jaise operations ke liye synchronized ya AtomicInteger use karo
- – volatile se atomic nahi hoga.

WHY atomicity nahi? count++ = 3 steps: READ count, ADD 1, WRITE count.

- volatile sirf guarantee karta ki READ latest hogi.
- Lekin 2 threads dono ne count=5 padha, dono ne 6 likha — ek increment LOST.
- Poora operation ek saath hona chahiye = synchronized ya AtomicInteger.
- "volatile ensures every thread reads the latest value from main memory instead of CPU cache.
- It solves visibility but not atomicity
- — for compound operations like increment, use synchronized or AtomicInteger."
- [REPEAT TRAP] volatile Trap — Baar Baar Galat!
- Thread 2 purani value padh raha (CPU cache).
- volatile = MAIN MEMORY se padho = VISIBILITY fix.
- count++ ke liye volatile NAHI
- — wo Atomicity hai.
- Sawaal: volatile kya fix? → Visibility chun.

Deadlock — Circular Wait

Deadlock — Circular Wait (4 Conditions)



4 conditions: Mutual Exclusion + Hold & Wait + No Preemption + Circular Wait. Break ANY = no deadlock.

▲ Thread A holds Lock 1, waits Lock 2. Thread B holds Lock 2, waits Lock 1 = CIRCULAR WAIT. Fix: always same lock order.

- 2 log ek doosre ko raasta dene ka wait kar rahe hain — dono rukhe hain.
- Yahi deadlock hai.
- Thread 1 ke paas Lock A hai, Lock B chahiye.

→ Thread 2 ke paas Lock B hai, Lock A chahiye. Koi release nahi karega.

```
// Deadlock kaise hota hai:
// Thread 1:
synchronized(lockA) { // Lock A liya [OK]
    synchronized(lockB) { // Lock B chahiye – Thread 2 ke paas hai ⌚
    }
}

// Thread 2:
synchronized(lockB) { // Lock B liya [OK]
    synchronized(lockA) { // Lock A chahiye – Thread 1 ke paas hai ⌚
    }
}

// Thread 1 wait kar raha hai Lock B ke liye
// Thread 2 wait kar raha hai Lock A ke liye
// Koi release nahi karega → DEADLOCK

// FIX – same order mein lock lo:
// Thread 1 aur Thread 2 dono mein A pehle, B baad
synchronized(lockA) {
    synchronized(lockB) { // dono same order [OK]
        // kaam karo
    }
}

// Thread 2 – same order (A pehle, B baad):
synchronized(lockA) {
    synchronized(lockB) {
        // kaam karo
    }
}

// Ab circular wait IMPOSSIBLE – dono A ke liye compete karenge,
// jo pehle lega wo B bhi lega. Deadlock nahi.
```

[TRAP] TRAP:

→ Deadlock koi exception nahi throw karta — program silently hang ho jaata hai.

→ Diagnose karna mushkil hota hai.

[TRAP] TRAP: Nested synchronized blocks = deadlock ka risk. Avoid karo jahan tak ho sake.

→ "Deadlock occurs when two threads hold one lock each and wait for the other's lock

→ creating a circular wait.

→ Prevention: always acquire locks in the same order across all threads."

wait() vs sleep() — Bahut Poochha Jaata Hai

→ WHY: wait() = LOCK RELEASE kare + doosre thread ko chance. Inter-thread communication (producer-consumer). sleep() = LOCK HOLD kare + thread pause. Sirf delay. Biggest fark: wait() = Object class (lock pe call), sleep() = Thread class (thread pe call).

→ Dono thread ko rokne ke liye hain

→ but reason alag.

→ sleep() = main thak gaya, thodi der rest karunga, lock nahi chhodunga.

→ wait() = main doosre ka wait kar raha hoon, tab tak lock chhod deta hoon.

```
// sleep() – lock nahi chhodta
Thread.sleep(1000); // 1 second ruko, lock apne paas rakho

// wait() – lock release kar deta hai
synchronized(obj) {
    obj.wait(); // lock chhodo + wait karo
} // notify() aane pe wapas aao

// Producer-Consumer example:
// Consumer:
synchronized(queue) {
    while(queue.isEmpty()) {
        queue.wait(); // lock chhodo, producer aane do
    }
    queue.remove();
}

// Producer:
synchronized(queue) {
    queue.add(item);
    queue.notify(); // consumer ko jagao
}

// sleep() wait()
// Lock release? [X] Nahi [OK] Haan
// Kahan define? Thread class Object class
// Kab wapas? timer khatam notify() pe
// sync zaroori? [X] [OK] Must
```

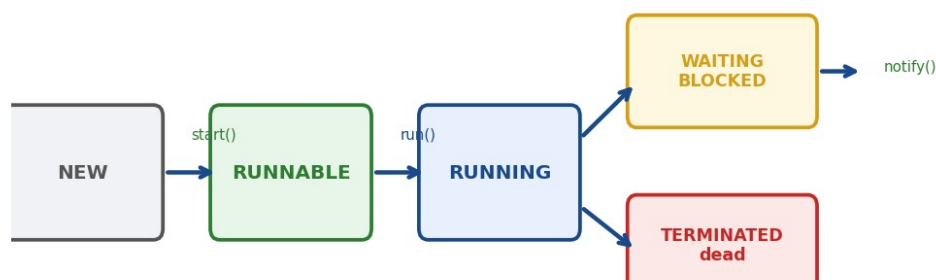
[TRAP] TRAP: wait() synchronized block ke bahar call kiya → IllegalMonitorStateException.

[TRAP] TRAP:

- sleep() lock hold karke rakhta hai
- agar koi doosra thread lock chahiye toh usko bhi rukna padega.
- "sleep() pauses the thread for a fixed time without releasing the lock.
- wait() releases the lock and waits until notify() is called —
- used for inter-thread communication in producer-consumer pattern."

ExecutorService — Thread Pool

Thread Lifecycle



▲ NEW → start() → RUNNABLE → CPU mile → RUNNING → wait/sleep → WAITING/BLOCKED → notify/done → TERMINATED.

- Har request pe new Thread() bana ke kaam karo
- thread create hoga, kaam karega, destroy hoga.
- 1000 requests = 1000 threads create destroy. Expensive.
- Thread pool = pehle se 10 threads ready.
- Request aayi — available thread utha, kaam kiya, wapas pool mein.
- Hospital emergency room jaisa
- har patient ke liye naya doctor hire nahi karte

- permanent doctors pool mein hote hain.
- Kahani: Hospital Emergency Room mein har patient ke liye naya doctor hire karo
- expensive!.
- Thread Pool = pehle se 5 doctors ready.
- Patient aaya → available doctor utha → kaam kiya → wapas pool mein ready.
- Queue mein wait karo agar sab busy hain. `new Thread()`
- har baar = naya doctor hire karo
- overhead aur slow.

```
// GALAT [X] - har baar naya thread banao, destroy karo
new Thread(() -> doTask()).start(); // overhead + no reuse
// SAHI [OK] - thread pool reuse karo
ExecutorService pool = Executors.newFixedThreadPool(5); // 5 threads ready
pool.submit(() -> doTask()); // task submit - available thread utha [OK]
pool.shutdown(); // kaam khatam - pool gracefully band karo
```

→ 4 Types — ek baar lock karo:

- `newFixedThreadPool(5)` ⇒ exactly 5 threads, extra tasks queue mein wait karein
- `newSingleThreadExecutor()` ⇒ 1 thread — tasks ek ke baad ek (sequential guaranteed)
- `newCachedThreadPool()` ⇒ zarorat pe naya thread, 60s baad idle thread destroy
- `newScheduledThreadPool(3)` ⇒ delayed ya repeating tasks (cron jaisa)

[TRAP] TRAP: shutdown() karo kaam ke baad

- — nahi kiya toh app kabhi terminate nahi hogi! `shutdownNow()`
- = forceful stop, pending tasks cancel.
- "ExecutorService manages a thread pool"
- threads are reused instead of created per task.
- `newFixedThreadPool(n)` keeps n threads ready.
- Always call `shutdown()` after use."

CompletableFuture — Async Programming

- 3 API calls parallel chalaani thi.
- `Thread.sleep()` wala approach blocking tha — ek thread rok ke baitha rehta tha.
- `CompletableFuture` = non-blocking.
- Pizza order kiya — phone rakh diya, apna kaam karo.
- Pizza ready hone pe bell bajegi.
- `supplyAsync` background mein chalaao `thenApply` result aane pe transform karo
- `thenCombine` 2 futures combine karo.
- Kahani: Pizza order kiya — phone rakh diya, apna kaam karo.
- Pizza ready hone pe bell bajegi.
- `CompletableFuture` = same
- thread block nahi hoga, result aane pe callback fire hoga.

```
// supplyAsync - background mein kaam, result return karega
CompletableFuture.supplyAsync(() -> fetchFromDB()) // background thread
    .thenApply(data -> data.toUpperCase()) // result transform karo
    .thenAccept(result -> System.out.println(result)) // consume - kuch return nahi
    .exceptionally(e -> "Error: " + e.getMessage()); // fallback
```

→ Key methods — interview mein yaad rahein:

- `supplyAsync()` ⇒ background task, result return karega
- `thenApply()` ⇒ result aaya — transform karo (`String` → `String`)
- `thenAccept()` ⇒ consume karo, return kuch nahi
- `thenCombine()` ⇒ 2 futures combine — dono complete hone pe kaam karo
- `exceptionally()` ⇒ exception aaya — fallback return karo

[TRAP] TRAP: thenApply() vs thenCompose()

- `thenApply` = simple transform.
- `thenCompose` = result se naya `CompletableFuture` banao (`flatMap` jaisa)
- nesting avoid karta hai).

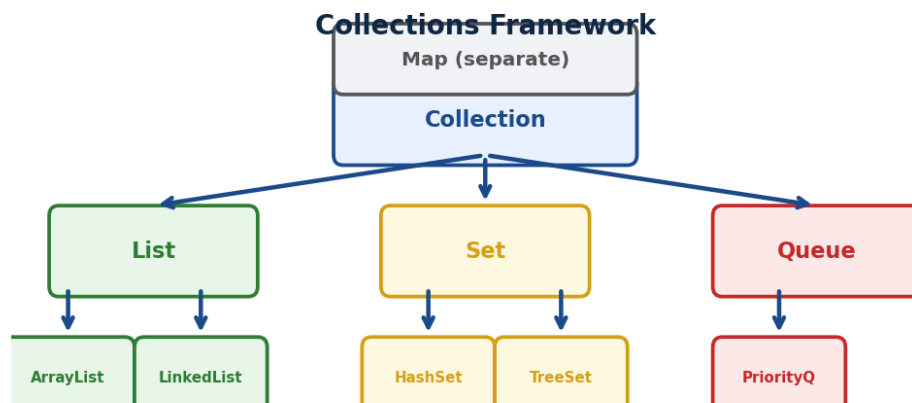
- "CompletableFuture enables non-blocking async code —
- supplyAsync runs in background thenApply transforms the result
- thenCombine merges two futures exceptionally handles errors.
- Thread never blocks."

Collections Framework

- Office mein pehla din.
- Manager bola
- "Saare active user IDs store karo." Tune likha: int id1 = 101
- int id2 = 102;...
- Ab manager bola — "1000 users hain." 1000 variables?. Ye bakwaas hai.
- Isliye Array aaya.
- Lekin Array ki problem — size FIXED. Beech se remove karo toh khud shift karo.
- Ye pain solve karne ke liye Collections Framework aaya.

WHY? Array = fixed size dabba.
Collections = rubber ka dabba — khud stretch hota hai, built-in tools bhi hain.
Dynamic size, add/remove/search built-in, alag data structures — List, Set, Map.

Collections Framework ka Map



▲ Collection: List (ordered, duplicates) → ArrayList/LinkedList. Set (unique) → HashSet/TreeSet. Map (key-value) → HashMap/TreeMap.

```
Collection (interface)
├── List → ORDER maintain, DUPLICATE allowed
│   ├── ArrayList
│   └── LinkedList
├── Set → NO DUPLICATE, order nahi
│   └── HashSet
├── Map → KEY-VALUE pairs
│   ├── HashMap
│   ├── LinkedHashMap
│   └── TreeMap
```

List kab? Order chahiye, duplicates allowed — survey IDs, user list

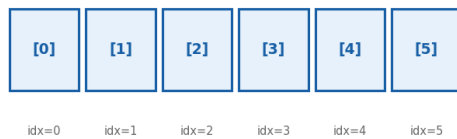
Set kab? Duplicates nahi chahiye — unique emails, unique tags

Map kab? Key se value dhundhna — userId → userName

ArrayList — Andar Se Samjho

ArrayList (Array Inside)

Continuous Memory Block



Random Access = $O(1)$ FAST
Middle Insert = $O(n)$ SLOW (shift all)

LinkedList (Nodes + Pointers)

Scattered Memory — Pointer Links



Insert/Remove = $O(1)$ FAST (pointer change)
Random Access = $O(n)$ SLOW (traverse)

ArrayList = continuous array $O(1)$ access. LinkedList = nodes + pointers $O(1)$ insert. 95% cases ArrayList better (cache-friendly).

→ WHY: ArrayList = internally Object[] ARRAY. Index se access = $O(1)$ FAST. Add at end = $O(1)$ amortized. Add at start/middle = $O(n)$ shift. Full hone pe 1.5x grow + copy. LinkedList = nodes with pointers. Add/remove = $O(1)$ agar reference hai. Index access = $O(n)$ traverse. Use case decide karo: random access = ArrayList, frequent insert/remove = LinkedList.

- Manager ne bola — active survey IDs maintain karo.
- Array liya, size 100 diya. 101st ID aayi — crash.
- Tab ArrayList — internally dynamic array hai, khud grow karta hai.
- Default size 10, full hua toh 1.5x naya array. Index se access = $O(1)$ fast.
- Beech mein insert/remove = shift karna padta hai = $O(n)$ slow.
- Read-heavy kaam ke liye best.
- Manager bola — "Active users ki list maintain karo.
- Naye add hote hain, purane remove hote hain." ArrayList andar se ek Array hi hai
- bas smart wrapper hai uske upar.

```
// Jab tu likhta hai: new ArrayList<>()
// Andar ye hota hai:
Object[] elementData = new Object[10]; // default size 10
size = 0; // abhi koi element nahi
WHY size 10? 10 elements ke baad 11th aaya → naya array banao 10 × 1.5 = 15.
Purana copy karo → 11th add karo.
Costly operation isliye agar pata ho: new ArrayList<>(500)
resizing avoid karo.
```

Operations — Time Complexity:

Operation	Time	Kyun?
get(index)	$O(1)$	Direct index access — instant
add(end)	$O(1)$	Last position pe daalo
add(middle)	$O(n)$	Shift karna padega
remove(middle)	$O(n)$	Shift karna padega
search(value)	$O(n)$	Ek ek check karo

ArrayList kab? [OK] Zyada READ karna ho. [OK] End pe add/remove. [OK] Index se access.
[X] Beech mein baar baar insert/delete slow.

- "ArrayList internally uses a dynamic array with default capacity 10,
- growing by 1.5x when full. get() is $O(1)$ due to index-based access,
- but add/remove in the middle is $O(n)$ due to shifting.
- Best for read-heavy operations."

LinkedList — Andar Se Samjho

- Support ticket queue thi — urgent tickets beech mein insert karne the.
- ArrayList try kiya

- beech mein insert karo toh saare elements shift hote hain. 10,000 tickets?
- 10,000 shift operations.
- Slow.
- LinkedList beech mein insert ke liye bana hi hai
- sirf pointers badlo, koi shifting nahi.
- $O(1)$.
- Lekin tradeoff — index se seedha access nahi hota, HEAD se traverse karna padta hai.
- Rule: zyada read = ArrayList.
- Zyada insert/delete beech mein = LinkedList.
- Hospital Queue
- naye patients aate hain purane process hote hain
- beech mein priority tickets bhi aate hain.
- ArrayList mein beech mein insert = shifting = slow.
- LinkedList iske liye bana hai. ArrayList = Array tha andar.
- LinkedList = chain of nodes hai andar.

Har node mein 3 cheezein:

```
prev | data | next |
```

```
null ← [101] ↔ [102] ↔ [103] → null
HEAD  TAIL
```

Koi continuous memory nahi — har node alag jagah RAM mein, pointers se connected.
WHY? add/remove = sirf pointers badle $\Rightarrow O(1)$. get = HEAD se traverse karo $\Rightarrow O(n)$.
 Memory zyada — har node mein prev + next extra hai.

ArrayList vs LinkedList — Ek Saath:

Operation	ArrayList	LinkedList	Kyun?
get(index)	$O(1)$	$O(n)$	Array = direct, Node = traverse
add(end)	$O(1)$	$O(1)$	Dono fast
add(middle)	$O(n)$	$O(1)$	Array = shift, Node = pointers
remove(middle)	$O(n)$	$O(1)$	Array = shift, Node = pointers
Memory	Less	More	Node mein prev + next extra

[TRAP] TRAP:

- "LinkedList is always faster" — GALAT! get() mein ArrayList wins.
- Real world mein READ zyada hota hai — isliye 90% cases mein ArrayList better hai.

[TRAP] TRAP:

- LinkedList insert = 2 steps: traverse $O(n)$ + pointer change $O(1)$ = net $O(n)$.
- ArrayList ka shift bhi $O(n)$.
- Practice mein ArrayList cache-friendly hai — continuous memory.
- LinkedList = scattered nodes = cache miss.

```
// Example:
// Support ticket queue — priority insert beech mein
LinkedList<Ticket> ticketQueue = new LinkedList<>();
ticketQueue.addFirst(urgentTicket); //  $O(1)$  ✓
ticketQueue.addLast(normalTicket); //  $O(1)$  ✓
```

- "LinkedList uses doubly linked nodes
- each node holds data, prev and next pointers.
- Insert and delete are $O(1)$ since only pointers change, but get is $O(n)$
- since traversal is needed.
- Use when frequent insertion and deletion in the middle is required."

ArrayList → Zyada READ, index access, end pe add/remove.
LinkedList → Zyada INSERT/DELETE beech mein, queue/stack banani ho.

Iterator vs ListIterator

- Loop mein expired surveys remove karne the. `forEach` mein `list.remove()` call kiya
- `ConcurrentModificationException`.
- Crash. Kyun?
- Loop chal raha hai aur saath mein structure modify kar raha hai
- Java allow nahi karta.
- Iterator se karo
- `it.remove()` safe hai kyunki Iterator internally `modCount` sync karta hai.
- `ListIterator` = Iterator ka bada bhai — aage bhi, peeche bhi, add bhi, set bhi.
- Survey Results. Tujhe ek ek karke saare results process karne hain.
- Seedha for loop laga sakte ho — but real problem: loop mein remove karna ho toh?
- Java exception dega.
- Iterator iske liye hai.

```
// Ye mat karo [X]
for(String s : surveys) {
    if(s.equals("S102")) {
        surveys.remove(s); // ConcurrentModificationException
    }
}
// Ye karo [OK]
Iterator<String> it = surveys.iterator();
while(it.hasNext()) {
    String s = it.next();
    if(s.equals("S102")) {
        it.remove(); // SAFE [OK] — Iterator ka remove, list ka nahi
    }
}
```

WHY? Loop ke saath safely traverse karo + remove karo — bina exception ke.
Iterator ka `remove()` = loop ke saath safe.
List ka `remove()` = exception.

Iterator ke 3 Methods — bas itne:

Method	Kya karta hai?
<code>hasNext()</code>	Aage kuch hai? → true/false
<code>next()</code>	Agla element lo + pointer aage badho
<code>remove()</code>	Abhi jo <code>next()</code> se liya, usse hata do

Direction: Sirf FORWARD — ek hi taraf.

ListIterator — Iterator ka Bada Bhai:

- Iterator sirf forward ja sakta hai.
- Suppose karo S103 pe ho, wapas S102 pe jaana hai — Iterator se nahi hoga.
- `ListIterator` = dono direction — aage bhi, peeche bhi. Extra methods:

Method	Kya karta hai?
<code>hasPrevious()</code>	Peeche kuch hai? → true/false
<code>previous()</code>	Pichla element lo + peeche jao
<code>add(e)</code>	Current position pe naya element daalo
<code>set(e)</code>	Current element replace karo

Iterator vs ListIterator — Table:

Feature	Iterator	ListIterator
Direction	Forward only	Forward + Backward
<code>add()</code>	Not available	Available

set()	Not available	Available
Works with	Any Collection	Only List

[TRAP] TRAP:

- forEach loop mein list.remove() = ConcurrentModificationException.
- it.remove() safe hai.
- Ye classic interview question hai — dono ka difference yaad rakho.

[TRAP] TRAP:

- it.next() bina hasNext() check kiye = NoSuchElementException.
- HAMESHA while(it.hasNext()) pehle check karo.
- | **Shortcut:** Remove only ⇒ Iterator kaafi. Add/Set bhi chahiye ⇒ ListIterator must.
- "Iterator allows safe traversal and removal during iteration —
- calling list.remove()
- inside a loop throws ConcurrentModificationException, but it.remove()
- is safe.
- ListIterator extends Iterator with bidirectional traversal, add(), and set()
- but only works with List."

HashMap vs LinkedHashMap vs TreeMap

→ WHY: HashMap = $O(1)$ lookup, NO order. LinkedHashMap = $O(1)$ + INSERTION order maintain. TreeMap = $O(\log n)$, SORTED by key (Red-Black tree). Order chahiye? LinkedHashMap. Sorting chahiye? TreeMap. Speed chahiye? HashMap. 95% cases HashMap.

- User database — userId se naam fetch karna hai.
- HashMap = fastest, $O(1)$, no order.
- Ek baar LRU cache banana tha — insertion order maintain karni thi.
- HashMap mein order nahi — LinkedHashMap liya.
- Aur ek baar sorted users chahiye the by ID
- TreeMap liya, keys automatically sorted.
- Teen alag problems, teen alag Maps.
- User Database. Tere paas users hain — har user ka ek ID hai aur naam hai.
- 101→"Arpan", 102→"Rahul", 103→"Priya".
- Teen alag Map implementations hain — andar se bilkul alag kaam karte hain.

1. HashMap – Sabse Fast, No Order:

```
Map<Integer, String> map = new HashMap<>();
map.put(103 "Priya"); map.put(101 "Arpan"); map.put(102
"Rahul");
System.out.println(map);
// {102=Rahul, 101=Arpan, 103=Priya} – ORDER KUCH BHI HO SAKTA HAI
// Wahi HashMap hai jo pehle padha – hashCode → bucket → store
// get() aur put() dono O(1)
```

HashMap kab? [OK] Sirf key-value dhundhna ho, order matter nahi. [OK] Maximum speed chahiye.

90% cases mein HashMap hi best hai.

2. LinkedHashMap – Insertion Order Maintain:

```
Map<Integer, String> map = new LinkedHashMap<>();
map.put(103 "Priya"); map.put(101 "Arpan"); map.put(102
"Rahul");
System.out.println(map);
// {103=Priya, 101=Arpan, 102=Rahul} – INSERTION ORDER SAME [OK]
// HashMap ke upar ek doubly linked list extra hai – insertion order track karta hai
// get() / put() → O(1). Extra memory – thodi zyada (linked list pointers)
```

LinkedHashMap kab? [OK] Insertion order maintain karni ho. [OK] LRU Cache banani ho. [X] Sorted order chahiye

TreeMap use karo.

3. TreeMap – Sorted Order, Hamesha:

```
Map<Integer, String> map = new TreeMap<>();
map.put(103 "Priya"); map.put(101 "Arpan"); map.put(102
"Rahul");
System.out.println(map);
// {101=Arpan, 102=Rahul, 103=Priya} – KEY PE SORTED [OK]
// Andar Red-Black Tree hai – har put() pe tree balanced rehta hai
// get() / put() → O(log n). HashMap se slow – lekin sorted guaranteed.
```

TreeMap kab? [OK] Keys sorted order mein chahiye. [OK] Range queries firstKey(), lastKey(), subMap(). [X] Sirf speed chahiye – HashMap use karo.

Teeno Ek Saath — Table:

Feature	HashMap	LinkedHashMap	TreeMap
Order	No order	Insertion order	Sorted by key
Performance	O(1)	O(1)	O(log n)
Andar kya?	Hash table	Hash + LinkedList	Red-Black Tree
null key?	Allowed (1 only)	Allowed (1 only)	Not allowed
Memory	Less	Thoda zyada	Zyada

[TRAP] TRAP:

→ TreeMap null key nahi leta — NullPointerException. Kyun?

→ Sort karta hai — null.compareTo() = crash.

→ HashMap null key allowed hai.

[TRAP] TRAP:

→ TreeMap = KEYS sort hoti hain, values nahi.

→ map.put("Rahul",2); map.put("Arpan",1);

→ Arpan pehle aayega (A<R).

→ Values (1,2,3) se koi matlab nahi — sirf keys dictionary order mein.

→ "HashMap gives O(1) performance with no order guarantee.

→ LinkedHashMap maintains insertion order using an additional linked list

→ same O(1) but slightly more memory.

→ TreeMap keeps keys sorted using a Red-Black Tree — O(log n).

→ TreeMap does not allow null keys since it needs to compare them."

HashMap = fast, no order. LinkedHashMap = fast + insertion order.
TreeMap = sorted + slow.
Keys ki dictionary – values se koi sorting nahi.

HashSet — Andar Se Samjho

- 10,000 panelists — kuch ne duplicate emails register kiye.
- ArrayList mein add karo — duplicate aa jaata.
- HashSet liya — duplicate automatically reject. Kyun?
- Andar se HashMap hi hai — tera element key ban jaata hai, value ek dummy object.
- Same key dobara?.
- Replace hoti hai, naya entry nahi banta. Isliye duplicate impossible.
- 10,000 users hain — kuch ne ek se zyada baar register kiya.
- Duplicate emails store nahi karni.
- ArrayList mein duplicate aa jaata hai. HashSet = duplicate automatically reject.

```
Set<String> emails = new HashSet<>();
emails.add("arpan@gmail.com");
emails.add("rahul@gmail.com");
emails.add("arpan@gmail.com"); // duplicate!
// [arpan@gmail.com, rahul@gmail.com] — duplicate gone [OK]
WHY? HashSet andar se HashMap hi hai. Tera element = HashMap ki KEY ban jaata hai.
Value = ek dummy Object (PRESENT) — koi matlab nahi iska.
Same key dobara put karo replace hogi, naya entry nahi banega
isliye duplicate impossible.
```

WHY HashMap reuse kiya?.

- Duplicate check ke liye hashCode + equals chahiye
- HashMap already ye karta hai keys ke liye!.
- Naya data structure banane ki zarurat hi nahi.
- Clever reuse — set.add("X") = map.put("X", PRESENT).
- Same key dobara?.
- Replace — naya entry nahi = duplicate rejected.

```
// HashSet ka source code andar:
private HashMap<E, Object> map;
private static final Object PRESENT = new Object(); // dummy value

public boolean add(E e) {
    return map.put(e, PRESENT) == null; // element = key, value = dummy
}
```

Operations:

Method	Time	Kaise?
add(e)	O(1)	hashCode → bucket → store
contains(e)	O(1)	hashCode → bucket → check
remove(e)	O(1)	hashCode → bucket → delete
Order	No order	HashMap jaisa — koi guarantee nahi

Custom Class — hashCode() + equals() dono zaroori:

```
// hashCode() + equals() nahi likhe [X]
set.add(new Panelist("arpan@gmail.com"));
set.add(new Panelist("arpan@gmail.com"));
// set.size() = 2 — duplicate aaya! Alag objects, alag hashCode, alag bucket.
// FIX — dono override karo [OK]
@Override
public int hashCode() { return email.hashCode(); } // same email = same bucket

@Override
public boolean equals(Object o) {
    Panelist p = (Panelist) o;
    return this.email.equals(p.email); // same email = equal
}
// ab set.size() = 1 [OK]
```

[TRAP] TRAP:

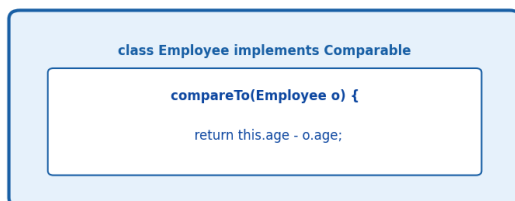
- Sirf equals() override kiya, hashCode() nahi —
- alag hashCode → alag bucket → equals()
- kabhi call hi nahi hoga → duplicate aa jaayega.
- RULE: equals() override karo toh hashCode()
- ZAROOR override karo. Java ka contract hai.

[TRAP] TRAP:

- HashSet order guaranteed nahi. Order chahiye → LinkedHashSet.
- Sorted chahiye → TreeSet.
- "HashSet internally uses a HashMap
- each element becomes a key with a dummy value.
- Duplicates are prevented because HashMap replaces existing keys.
- For custom objects, both hashCode() and equals() must be overridden
- — without hashCode()
- two equal objects may land in different buckets and both get stored."
- HashSet = HashMap ki keys. Duplicate nahi kyunki same key replace hoti hai.
- Custom class mein hashCode() + equals() dono override karo.

Comparable vs Comparator — Sorting

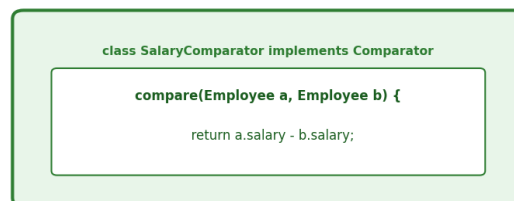
Comparable (Natural Order)



Class ke ANDAR define
java.lang.Comparable
EK default sorting
Collections.sort(list)

Tera KHUD ka order
"Main age se sort hounga"

Comparator (Custom Order)



ALAG class mein define
java.util.Comparator
MULTIPLE sorting options
list.sort(new SalaryComp())

BAHAR se bolo
"Isko salary se sort karo"

Comparable = class ke ANDAR (natural order, EK). Comparator = BAHAR se (custom, MULTIPLE). Comparable=compareTo(). Comparator=compare().

- Panelists ki list age se sort karni thi. Collections.sort() call kiya — error.
- Panelist class mein compareTo() nahi tha.
- Comparable = class ke andar ek natural ordering define karo.
- Lekin kal naam se sort karna tha, parso rating se

- ek hi compareTo() nahi chalega.
- Comparator = bahar se alag alag rules.
- Class touch nahi karni, lambda se ek line mein naya rule.
- Dono alag situations ke liye hain.
- Scene: 3 employees hain — Rahul (age 30), Arpan (age 25), Priya (age 28).
- Tu inhe sort karna chahta hai.
- Collections.sort(list) likhega
- lekin Java boleگا: "Bhai, mujhe kya pata inhe kaise sort karna hai?"
- Ye teri custom class hai!" Java ko sorting ka rule batana padega
- 2 tarike hain.

WHY? Java ko sort karte waqt har 2 objects compare karne hote hain kaun pehle jaayega?.

Ye rule Comparable se class ke andar dete hain, ya Comparator se bahar se.

Comparable = Arpan ka apna rule.

Comparator = Manager ka rule — aaj name se, kal age se.

1. Comparable — Class Ke Andar

- Class khud implement kare Comparable interface. Method: compareTo(T other).
- Return: negative = this pehle, zero = equal, positive = other pehle.

```
class Panelist implements Comparable<Panelist> {
    String name;
    int age;

    @Override
    public int compareTo(Panelist other) {
        return Integer.compare(this.age, other.age); // best practice
        // Arpan(25) vs Rahul(30) → 25 < 30 → negative → Arpan pehle [OK]
        // Rahul(30) vs Arpan(25) → 30 > 25 → positive → Arpan pehle [OK]
    }
}

// Use:
List<Panelist> list = new ArrayList<>();
Collections.sort(list); // compareTo() khud use hoga
```

- compareTo() ke 3 styles — teeno valid hain:

```
return this.age - other.age; // [X] Avoid — overflow risk
return Integer.compare(this.age, other.age); // [OK] Best practice — 1 line, safe
if (this.age < other.age) return -1; // [OK] Valid — readable
if (this.age > other.age) return 1;
return 0;
```

[TRAP] TRAP:

- this.age - other.age likhna — overflow ho sakta hai!.
 - Agar age = Integer.MAX_VALUE aur doosra = -1, toh subtraction overflow.
 - HAMESHA Integer.compare(this.age, other.age) use karo.
 - Interview mein yahi best practice hai.
- ## 2. Comparator — Class Ke Bahar
- Comparator = class touch nahi karna, bahar se alag alag rules banao.
 - Method: compare(T a, T b).
 - Jitne rules chahiye — utne Comparators bana sakte ho.

```
// Rule 1 – Age se sort karo
Comparator<Panelist> byAge = new Comparator<Panelist>() {
    @Override
    public int compare(Panelist a, Panelist b) {
        return Integer.compare(a.age, b.age);
    }
};

// Rule 2 – Name se sort karo
Comparator<Panelist> byName = new Comparator<Panelist>() {
    @Override
    public int compare(Panelist a, Panelist b) {
        return a.name.compareTo(b.name);
    }
};

// Use karna:
Collections.sort(list, byAge); // age se
Collections.sort(list, byName); // name se

// Java 8 lambda shorthand:
Collections.sort(list, (a, b) -> Integer.compare(a.age, b.age));
```

→ Comparable vs Comparator — Table:

Feature	Comparable	Comparator
Kahan? Class ke andar Class ke bahar	Class ke andar	Class ke bahar
Method compareTo(other) compare(a, b)	compareTo(other)	compare(a, b)
Kitne rules? Sirf EK – default Jitne chahiye	1 rule (default)	Multiple rules
Class change? Haan – implement karo Nahi – untouched	Haan	Nahi
Sort kaise? Collections.sort(list) Collections.sort(list, rule)	Collections.sort(list)	Collections.sort(list, rule)

Kahan? Class ke andar Class ke bahar
 Method compareTo(other) compare(a, b)
 Kitne rules? Sirf EK – default Jitne chahiye
 Class change? Haan – implement karo Nahi – untouched
 Sort kaise? Collections.sort(list) Collections.sort(list, rule)

"Comparable defines natural ordering within the class via compareTo()."

Comparator defines external, flexible ordering via compare()

you can have multiple Comparators for the same class.

Use Integer.compare() instead of subtraction to avoid overflow."

Comparable = class andar, ek rule, compareTo().
Comparator = class bahar, multiple rules, compare().
Integer.compare() = overflow safe, best practice.

thenComparing() — Chained Sorting

→ Scene: Employees ki list — age se sort karna hai.

→ 2 panelists ki age same ho gayi.

→ Ab manager bola: pehle age se sort karo, tie ho toh name se.

→ Comparator ke andar condition lagana nahi padega — thenComparing() hai!

WHY? Ek Comparator sirf ek rule deta hai.
 Multiple rules chahiye toh chain karo
 comparing() = pehla rule, thenComparing() = tiebreaker.
 Jitne chahiye utne lagao.

```
// Pehle age se, tie ho toh name se
list.sort(
    Comparator.comparing((Panelist p) -> p.age)
    .thenComparing(p -> p.name)
);

// Teen rules:
Comparator.comparing(p -> p.age)
    .thenComparing(p -> p.name)
    .thenComparing(p -> p.salary);
```

[TRAP] TRAP:

→ && operator int pe kaam nahi karta

→ compile error. list.sort((a,b) -> Integer.compare(a.age, b.age))

→ && a.name.compareTo(b.name)) = GALAT. thenComparing() use karo.

"Comparator.comparing() sets the primary sort rule."

thenComparing() adds a tiebreaker

applied only when the previous comparison returns zero.

Chain as many as needed."

```
comparing() = pehla rule. thenComparing() = tiebreaker.  
&& operator = compile error.
```

fail-fast vs fail-safe — ConcurrentModificationException

WHY exception throw karta?.

- Soch: tu list padh raha hai (iterator) aur koi beech mein list badal raha hai.
- Iterator ko consistent view chahiye
- agar data change ho gaya toh galat result aayega ya `ArrayIndexOutOfBoundsException`.
- Safe rehne ke liye iterator `modCount` track karta hai — badla?
- Turant roko, galat data dene se acha exception de do.
- Scene: List iterate kar raha hai
- aur saath mein remove bhi kar raha hai. `list.remove()` andar loop mein BOOM!
- `ConcurrentModificationException`. Kyun aaya ye?

WHY? `ArrayList` ka iterator ek internal counter rakhta hai — `modCount`. Jab bhi list structurally modify hoti hai (add ya remove dono), `modCount` badhta hai. Iterator start hote waqt `expectedModCount` set hota hai. Har step pe check karta hai — `modCount` badla?. EXCEPTION!. Ye fail-fast hai — turant pakad lo, galat result mat do.

→ fail-fast — Galat Code:

```
List<String> list = new ArrayList<>();  
list.add("Rahul");  
list.add("Arpan");  
  
for (String s : list) {  
    list.remove(s); // [X] ConcurrentModificationException!  
    // modCount badh gaya — iterator ne pakad liya  
}
```

→ fail-fast — Sahi Fix:

```
Iterator<String> it = list.iterator();  
while (it.hasNext()) {  
    String s = it.next();  
    it.remove(); // [OK] Safe — modCount + expectedModCount dono sync  
}
```

[TRAP] TRAP:

- `list.remove()` aur `it.remove()` alag hain! `list.remove()` bahar se modify karta hai
- — `modCount` badha, `expectedModCount` nahi badha = CME. `it.remove()`
- dono ek saath update karta hai
- safe.
- fail-safe — `CopyOnWriteArrayList`:
- fail-safe collections iterate karte waqt copy banate hain.
- Original list alag, iterator ki copy alag.
- Modify karo — exception nahi aayega.

```
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
list.add("Rahul");
list.add("Arpan");

for (String s : list) {
    list.remove(s); // [OK] No exception – copy pe iterate ho raha hai
}
// Tradeoff: extra memory lagti hai copy banane mein
// Stale data: loop mein add hua element us iteration mein nahi dikhega
fail-fast fail-safe
Exception? Haan – CME Nahi
Collections ArrayList, HashMap CopyOnWriteArrayList, ConcurrentHashMap
Iterate over Original list Copy of list
Memory Less Zyada (copy banata hai)
```

"fail-fast iterators throw ConcurrentModificationException as soon as they detect a structural modification during iteration.

fail-safe iterators work on a copy of the collection — no exception, but may iterate over stale data."

```
fail-fast = ArrayList, HashMap
CME on modify. fail-safe = CopyOnWriteArrayList, ConcurrentHashMap
– copy banata hai, no CME. it.remove() = safe. list.remove() inside loop = CME.
```

PriorityQueue — Min-Heap by Default

- Scene: 1000 survey responses aaye — top 5 highest-rated dikhane hain.
- List sort karein? Zyada slow.
- PriorityQueue seedha min/max efficiently deta hai — sorting ki zaroorat nahi.

WHY:

- PriorityQueue andar se min-heap hai
- poll() hamesha sabse CHHOTA element deta hai.
- Max-heap chahiye toh Comparator.reverseOrder() pass karo.
- Normal Queue nahi hai ye — FIFO nahi, PRIORITY se nikalta hai.
- Min-Heap (default) — chhota pehle:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(30);
pq.add(10);
pq.add(20);

pq.poll(); // 10 – sabse chhota pehle! [OK]
pq.poll(); // 20
pq.poll(); // 30
```

- Max-Heap — bada pehle:

```
PriorityQueue<Integer>
maxPQ = new PriorityQueue<>(Comparator.reverseOrder());
maxPQ.add(30);
maxPQ.add(10);
maxPQ.add(20);

maxPQ.poll(); // 30 – sabse bada pehle! [OK]
```

- 3 Important Methods:

```
pq.add(x) → element daalo
pq.poll() → sabse priority wala nikalo + REMOVE karo ← most used
pq.peek() → sabse priority wala DEKHO – remove mat karo
C++ Mapping: C++ top() = Java peek() (sirf dekho) | C++ pop() = Java poll()
(nikalo + hatao) | C++ push() = Java add()
```

- Konovo Real Use Case — Top-K pattern:

```
// Top 5 highest-rated survey responses dhundho
PriorityQueue<Integer> top5 = new PriorityQueue<>(); // min-heap

for (int rating : allRatings) {
    top5.add(rating);
    if (top5.size() > 5) top5.poll(); // sabse chhota hata do
    // Sirf 5 largest hamesha queue mein rahenge
}
// top5 mein ab 5 highest ratings hain [0K]
```

[TRAP] TRAP:

- for-each loop PriorityQueue pe sorted order guarantee NAHI karta.
- Sirf poll() sahi order mein deta hai.
- print(pq) ya for(int x : pq) karo toh random order dikhega
- — internal heap structure dikhta hai, sorted nahi.
- [REPEAT TRAP] PriorityQueue Trap Baar Baar Galat! for(int x : pq) = 10 30 20
- SORTED NAHI!
- Heap order. poll() loop = 10 20 30 — YE sorted. C++: top()=peek(), pop()=poll().

[TRAP] TRAP:

- Custom objects ke liye Comparator pass karo
- — warna ClassCastException. e.g.
- new PriorityQueue<>((a,b) -> Integer.compare(a.rating, b.rating))

*"PriorityQueue is a min-heap by default
poll() always returns the smallest element.
Pass Comparator.reverseOrder() for max-heap.
poll() removes and returns, peek() only reads.
Sorted iteration requires poll() — for-each does not guarantee order."*

Default = min-heap (chhota pehle).
Comparator.reverseOrder() = max-heap. poll() = nikalo+hatao.
peek() = sirf dekho. for-each = sorted nahi.

Collections Utility Class — Built-in Helper Methods

- Scene: Survey responses ki list bani —
- ab sort karna hai reverse karna hai
- aur kuch lists ko readonly banana hai taaki koi accidentally modify na kare.
- Collections class = sab kuch ready-made.

WHY:

- Collections = static utility class
- object nahi banate, seedha Collections.methodName() call karo.
- Sirf List/Set/Map ke saath kaam karta hai — primitives nahi.
- sort() — ascending, ya Comparator pass karo:

```
List<Integer> list = Arrays.asList(30, 10, 20);
Collections.sort(list); // [10, 20, 30] ascending
Collections.sort(list, Comparator.reverseOrder()); // [30, 20, 10] descending
```

- reverse(), shuffle(), min(), max(), frequency():

```
Collections.reverse(list); // [3,2,1] — order seedha ulta
Collections.shuffle(list); // [2,3,1]
random order (survey questions ko shuffle karo)
Collections.min(list); // sabse chhota element
Collections.max(list); // sabse bada element
Collections.frequency(list, "A"); // "A" kitni baar hai list mein
```

- unmodifiableList() — Readonly banana:

```
List<String> original = new ArrayList<>();
original.add("Question 1");
original.add("Question 2");

List<String> readOnly = Collections.unmodifiableList(original);
readOnly.add("Question 3"); // [X] UnsupportedOperationException!
// Template readonly list – koi accidentally modify na kare
```

→ Quick Reference Table:

```
sort(list) → ascending sort (Comparator optional)
sort(list, comparator) → custom order
reverse(list) → order seedha ulta
shuffle(list) → random order
min(list) / max(list) → sabse chhota / bada
frequency(list, x) → x kitni baar hai
unmodifiableList(list) → readonly wrapper – add/remove = exception
```

[TRAP] TRAP:

- `unmodifiableList()` sirf wrapper hai —
- original list abhi bhi mutable hai.
- `original.add()` karo toh `readOnly` mein bhi change dikhega!
- Immutable copy chahiye toh `List.of()` use karo (Java 9+).

[TRAP] TRAP:

- `Collections.sort()`.
- `Comparable` use karta hai — agar class mein `compareTo()`
- nahi hai toh `ClassCastException`.
- `String` aur `Integer` already `Comparable` implement karte hain
- custom class mein tujhe karna padega.

"Collections is a utility class with static methods.

sort() uses Comparable or takes a Comparator.

unmodifiableList() returns a read-only view

—
modifying via the original reference still works.

shuffle() randomizes, reverse() flips, frequency() counts occurrences."

```
sort/reverse/shuffle/min/max/frequency/unmodifiableList
– sab static.
```

```
unmodifiableList = wrapper only, original modify hoga toh view bhi change hoga.
```

Java 8 — Lambda, FI, Streams, Optional

- Ek din manager ne bola
- — "100 patients ki list age se sort karo."
- Purana style mein `Comparator` likhne baitha
- — anonymous inner class, `@Override`, compare method, 10 lines sirf sorting ke liye.
- Sochne laga — itna code sirf ek kaam ke liye?. Tab Lambda aaya.
- Ek line mein kaam ho gaya.
- Lambda ke bina Streams nahi chalti — isliye pehle ye.

Lambda Expressions — Anonymous Function

- Purana style — `Comparator` likhne mein 10 lines boilerplate.
- Anonymous inner class, `@Override`, return statement — sab ek kaam ke liye.
- Lambda ne ye sab hata diya.
- Naam nahi, class nahi, seedha logic.
- Lambda = anonymous function. Naam nahi, class nahi, seedha logic.
- (parameters) -> { body }.
- Boilerplate hatao ⇒ code readable banao ⇒ Streams API use kar pao.

Streams API Pipeline



▲ Source → filter() → map() → sorted() → collect(). LAZY: terminal op ke bina execute NAHI. Pipeline pattern.

```
// Java 7 – purana tarika (10 lines boilerplate)
Collections.sort(patients, new Comparator<Patient>() {
    @Override
    public int compare(Patient a, Patient b) {
        return a.getName().compareTo(b.getName());
    }
});

// Java 8 Lambda – ek line mein same kaam
Collections.sort(patients, (a, b) -> a.getName().compareTo(b.getName()));
// Lambda syntax:
() -> System.out.println("Hello") // zero parameter
name -> System.out.println(name) // ek parameter – brackets optional
(a, b) -> a + b // do parameters – return implicit
(a, b) -> { // multiple lines – curly braces
    int sum = a + b;
    return sum;
}
```

[TRAP] TRAP: Lambda ke bina Streams nahi chalti — isliye pehle Lambda, phir Streams.

→ "

→ Lambda expressions are anonymous functions that reduce boilerplate by replacing anonymous inner classes with a concise syntax —

→ (parameters) -> body.

→ They enable functional programming in Java and are the foundation for Streams API."

Method Reference (::) — Lambda ka Shortcut

→ Jab Lambda mein sirf ek existing method call ho raha ho — toh :: use karo.

→ Aur short.

```
// Lambda
list.forEach(name -> System.out.println(name));
// Method Reference – same kaam, aur short
list.forEach(System.out::println);

// 3 common types:
(x) -> Math.abs(x) ⇒ Math::abs // Static method
(s) -> s.toUpperCase() ⇒ String::toUpperCase // Instance method
() -> new Patient() ⇒ Patient::new // Constructor
```

→ "Method references are shorthand for lambdas that only call an existing method —

→ ClassName::methodName for static instance::method for instance

→ and ClassName::new for constructors."

Functional Interface — Lambda Kahan Store Hota Hai?

→ Lambda ko store karne ke liye ek special interface chahiye

→ jisme sirf ek abstract method ho.

→ Isko Functional Interface bolte hain.

→ @FunctionalInterface annotation lagao

→ compiler check karega, agar 2 abstract methods likhe toh compile error.

```
@FunctionalInterface
interface MathOperation {
    int operate(int a, int b); // sirf ek abstract method
}

MathOperation add = (a, b) -> a + b;
MathOperation multiply = (a, b) -> a * b;

System.out.println(add.operate(5, 3)); // 8
System.out.println(multiply.operate(5, 3)); // 15
```

- Lambda —store— Functional Interface (sirf 1 abstract method).
- @FunctionalInterface = compiler check.
- 2 abstract methods = compile error.
- "A Functional Interface has exactly one abstract method"
- it is the target type for lambda expressions.
- @FunctionalInterface is a compiler check that ensures only one abstract method exists."

4 Built-in Functional Interfaces — PFCS

- Java 8 ne 4 ready-made functional interfaces diye.
- Yaad karne ka trick — kya le raha hai, kya de raha hai.
- PFCS: Predicate=Filter, Function=Convert, Consumer=Action, Supplier=Factory.

```
// 1. Predicate<T> — le: T, de: boolean (test karo)
Predicate<String> isEmpty = s -> s.isEmpty();
isEmpty.test(""); // true
isEmpty.test("Arpan"); // false

// 2. Function<T, R> — le: T, de: R (transform karo)
Function<String, Integer> length = s -> s.length();
length.apply("Arpan"); // 5

// 3. Consumer<T> — le: T, de: kuch nahi (use karo, return nahi)
Consumer<String> print = s -> System.out.println(s);
print.accept("Arpan"); // Arpan

// 4. Supplier<T> — le: kuch nahi, de: T (produce karo)
Supplier<String> greet = () -> "Hello"
greet.get(); // Hello
```

Memory Trick
—
PFCS:
Predicate = Filter (haan ya nahi)
Function = Convert (A se B)
Consumer = Action (karo, return nahi) | Supplier = Factory (banao, input nahi)

- [REPEAT TRAP] PFCS Method Trap — Baar Baar Galat!. P=test(bool).
- F=apply(transform). C=accept(void).
- S=get(produce). filter()=Predicate. map()=Function. forEach()=Consumer.
- Sawaal: filter ke andar kya? → Predicate chun.

[TRAP] TRAP: Predicate ka method test() hai, Function ka apply(), Consumer ka accept(), Supplier ka get(). Naam mix mat karna — interview mein specifically poochhte hain.

- "Java 8 provides four core functional interfaces: Predicate for boolean test,
- Function for transformation, Consumer for side-effect actions with no return,
- and Supplier for object creation with no input.
- These are the building blocks used heavily in Streams API."

Lambda = anonymous function, boilerplate killer. :: = Lambda shortcut.
Functional Interface = sirf 1 abstract method.
PFCS = Predicate/Function/Consumer/Supplier.
Ye charon Streams mein har jagah use honge.

- [REPEAT TRAP] Arrow Trap — Baar Baar Galat! Java => (dash arrow).
- JavaScript ==> (fat arrow).
- Java mein ==> EXIST NAHI.
- Sawaal: Lambda syntax? → -> wala chun. ==> dikhe = GALAT.

Streams API — Pipeline Banao, Loop Hatao

Streams API — Pipeline Pattern (Lazy Evaluation)



Data Flow:

[Arpan 25, Rahul 17, Priya 22, Amit 16] → [Arpan 25, Priya 22] → ["Arpan", "Priya"] → ["Arpan", "Priya"] → List<String>

*LAZY: filter/map/sorted kuch NAHI karte jab tak collect() na call ho. Terminal = trigger.
Bina terminal operation = stream EXECUTE nahi hoga. Ye interview TRAP hai.*

Source > filter > map > sorted (LAZY) > collect (TERMINAL = trigger). Bina terminal = kuch nahi hota.

- Ek din manager ne bola
- “
- saare panelists mein se sirf active wale dhundho unke naam nikalo
- alphabetical sort karo aur ek list bana do.”
- Purana style mein for loop, if condition, naya list, Collections.sort()
- — 5 lines, 3 alag operations, loop mein sab ghusa hua.
- Streams ne socha — ye sab operations chain karo. Ek pipeline banao.
- Data ek side se jaaye, operations beech mein hon, result doosri side se aaye.

```
// Purana style — 5 lines
List<String> result = new ArrayList<>();
for (Panelist p : panelists) {
    if (p.isActive()) { result.add(p.getName()); }
}
Collections.sort(result);

// Streams — ek pipeline
List<String> result = panelists.stream()
    .filter(p -> p.isActive()) // sirf active
    .map(p -> p.getName()) // naam nikalo
    .sorted() // alphabetical
    .collect(Collectors.toList()); // list banao
```

- Pipeline: Source (list.stream()) → Intermediate (filter, map, sorted —
- LAZY) → Terminal (collect, forEach, count TRIGGER).
- Intermediate tab tak kuch nahi karte jab tak terminal na aaye.

PFCS Connection:

**filter() = Predicate (test) | map() = Function (apply) | forEach()
= Consumer (accept) | collect() = final result pakdo**

```
// filter() – chhaan ke nikalo (Predicate → boolean)
panelists.stream()
.filter(p -> p.isActive()) // true = rehne do, false = hatao
.collect(Collectors.toList());

// map() – transform karo (Function → ek type se doosra)
panelists.stream()
.map(p -> p.getName()) // Panelist → String
.collect(Collectors.toList());

// sorted() – sort karo (Comparable ya Comparator)
names.stream()
.sorted() // natural order A-Z
.collect(Collectors.toList());
panelists.stream()
.sorted(Comparator.comparing(Panelist::getAge).reversed()) // custom

// forEach() – har pe kaam karo (Consumer → void)
names.stream()
.forEach(System.out::println); // method reference

// reduce() – sab ko ek value mein milao
List<Integer> nums = List.of(10, 20, 30);
int total = nums.stream()
.reduce(0, (a, b) -> a + b); // 0+10=10, 10+20=30, 30+30=60

// distinct() – duplicates hatao (equals + hashCode internally)
names.stream()
.distinct() // duplicate gone
.collect(Collectors.toList());

// limit(n) + skip(n) – pagination jaisa
items.stream().skip(5).limit(5) // page 2, size 5
.collect(Collectors.toList());

// count() – kitne hain
long active = panelists.stream()
.filter(p -> p.isActive())
.count();
```

→ Collectors — Pipeline ka result kahan jaaye?

```
.collect(Collectors.toList()) // List banao
.collect(Collectors.toSet()) // Set banao (duplicates gone)
.collect(Collectors.joining(", ")) // String: "Arpan Rahul Priya"
.collect(Collectors.toMap(
p -> p.getId(), p -> p.getName() // Map: {101=Arpan, 102=Rahul}
))
.collect(Collectors.groupingBy( // Group: {Delhi=[...], Mumbai=[...]}
p -> p.getCity()
))
```

→ Real Interview Combinations:

```
// Active panelists ke naam, sorted, top 5
panelists.stream()
    .filter(p -> p.isActive())
    .map(p -> p.getName())
    .sorted()
    .limit(5)
    .collect(Collectors.toList());
```

```
// Total salary of all employees
employees.stream()
    .map(e -> e.getSalary())
    .reduce(0, Integer::sum);
```

```
// Unique cities, sorted
employees.stream()
    .map(e -> e.getCity())
    .distinct().sorted()
    .collect(Collectors.toList());
```

[TRAP] TRAP: Stream ek baar use ho sakta hai — dobara .filter() ya .map() call kiya toh IllegalStateException. Naya stream banao.

[TRAP] TRAP: Intermediate operations (filter, map) LAZY hain — jab tak terminal (collect, forEach) nahi aata, kuch nahi hota.

→ "Streams API provides a declarative pipeline for data processing"

→ filter, map, sort, collect in a chain.

→ Intermediate operations are lazy and only execute when a terminal operation like collect()

→ or forEach() is called.

→ Collectors determine the output format

→ toList(), toSet(), toMap(), groupingBy(), or joining()."

filter=chhaan, map=transform, sorted=sort, forEach=action,
reduce=combine, distinct=unique, limit/skip=pagination, collect=result pakdo.
Stream ek baar use, dobara nahi.

Optional — NullPointerException se Bachao

→ Ek din user profile fetch karna tha — getUserById(101).

→ User nahi mila — null aaya.

→ Tune seedha user.getName() call kiya — NullPointerException. Production crash.

→ Har jagah if (user != null) likhne laga — code ugly, phir bhi kahin miss ho jaata.

→ Java 8 ne socha — null ki jagah ek dabba do.

→ Dabba khali bhi ho sakta, bhara bhi.

→ Lekin kholne se pehle check karna padega.

→ Optional = dabba. Andar value ho bhi sakti, na bhi.

→ Kholne se pehle check karo. null ka safe replacement.

→ orElse() = khali toh default. orElseThrow() = khali toh exception.

```
// Purana style — null check bhool gaye = crash
User user = getUserById(101); // null aa sakta hai
user.getName(); // -- CRASH NullPointerException
```

```
// Optional style — dabba diya, safe access
Optional<User> user = findById(101);
String name = user
    .map(u -> u.getName()) // andar hai toh naam nikalo
    .orElse("Unknown"); // khali toh default
```

```
User u = user
    .orElseThrow(() -> new RuntimeException("User nahi mila!"));
```

→ 3 tarike se dabba banao:

```
Optional.of("Arpan") // GUARANTEE: andar value hai
// null diya toh NullPointerException -- CRASH

Optional.ofNullable(name) // SHAYAD: null bhi ho sakta, safe
// 90% time YAH! use karo

Optional.empty() // KHALI: jaanta hoon kuch nahi hai
of() = 100% sure value hai.
ofNullable() = pata nahi, null bhi ho sakta (90% yahi use karo).
empty() = jaanta hoon khali hai.
```

→ Important Methods:

```
optional.isPresent() // andar kuch hai? true/false
optional.orElse("default") // khali toh default do
optional.orElseThrow(...) // khali toh exception
optional.map(u -> u.getName()) // transform – khali toh skip
optional.filter(u -> u.isActive()) // condition – false toh empty
optional.get() // [TRAP] DANGEROUS – khali toh crash
```

[TRAP] TRAP: optional.get() directly kabhi mat karo — khali ho toh NoSuchElementException. Hamesha orElse() ya orElseThrow() use karo.

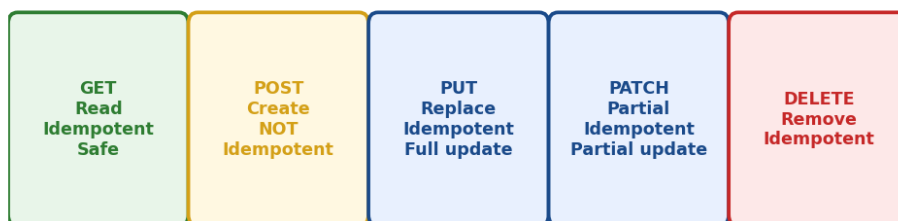
[TRAP] TRAP: Optional ko method parameter mein mat do — sirf return type mein use karo. void process(Optional<User> u) = bad practice.

- "Optional is a container that may or may not hold a value"
- it replaces null checks with a fluent API.
- Use orElse() for defaults, orElseThrow() for exceptions, and map()
- to transform safely.
- Never call get() directly.
- Use ofNullable() when the value might be null, of() when guaranteed non-null."

```
Optional = null ka safe replacement. of()=guaranteed.
ofNullable()=maybe null (90% yahi).
orElse/orElseThrow=safe access. get() seedha=dangerous.
Sirf return type mein use karo, parameter mein nahi.
```

REST API

REST API — HTTP Methods + Idempotency



Idempotent = 10 baar call = same result. POST = har baar NAYA resource.

▲ GET (read, safe), POST (create, NOT idempotent), PUT (replace, idempotent), PATCH (partial update), DELETE (remove, idempotent).

- Razorpay pe payment kar raha tha. Button dabaya — network slow.
- Dobara dabaya. 3 baar request gayi.
- Agar 3 baar paisa kata — disaster.
- Agar sirf 1 baar cuta chahe 100 baar button dabo — ye idempotent hai.
- REST API mein har method ka ek behaviour hota hai
- aur har response mein ek status code
- server bata raha hai kya hua.

HTTP Methods — Kaunsa Kab

GET → data padho → safe + idempotent
POST → naya banao → NOT idempotent (10x = 10 entries)
PUT → poora replace → idempotent
PATCH → partial update → idempotent
DELETE → hatao → idempotent
Idempotent = 10 baar bhejo, result same. POST kyun nahi?
Har request pe NAYA record banega.
Fix: Idempotency key bhejo — server check kare "ye key pehle aaya?
Ignore karo."

→ PUT vs PATCH: PUT = poora object bhejo, replace ho jayega.

→ PATCH = sirf jo badla wo bhejo, baaki untouched.

→ Safe method = data modify nahi karta — sirf GET safe hai.

Status Codes — Server ka Jawab

2xx = SUCCESS
200 OK → general success, data aa gaya
201 Created → naya resource bana (POST ke baad)
204 No Content → success, kuch return nahi (DELETE ke baad)

4xx = CLIENT KI GALT
400 Bad Request → validation fail, galat data bheja
401 Unauthorized → "Tu KAUN hai?" — login nahi kiya
403 Forbidden → "Pehchaana, but NAHI milega" — permission nahi
404 Not Found → resource exist nahi karta
409 Conflict → duplicate entry ya state conflict

5xx = SERVER KI GALT
500 Internal → server crash, unhandled exception
503 Unavailable → server overloaded ya maintenance

→ Interview mein aise poochhte hain:

POST /users ke baad? → 201 Created (200 nahi)
DELETE /users/101 ke baad? → 204 No Content
User already exist, POST? → 409 Conflict
Invalid email bheja? → 400 Bad Request
Token expired? → 401 Unauthorized
Admin endpoint, normal user? → 403 Forbidden

TRAP: 401 = "Tu KAUN hai?" (authentication fail). 403 = "Pehchaana, but allowed nahi" (authorization fail). Pehle 401, phir 403. Ye dono MIX mat karna.

TRAP: POST ke baad 200 nahi 201 use karo. DELETE ke baad 204 use karo. Ye specific codes interview mein seedha poochhte hain.

POWER PHRASE: "REST APIs use standard HTTP methods where GET is safe and idempotent, POST creates new resources and is not idempotent, PUT replaces the entire resource, and PATCH updates partially. Status codes communicate outcomes

*—
201 for creation, 204 for deletion,
401 for authentication failure, 403 for authorization failure
and 409 for conflicts like duplicate entries."*

FLASH: Idempotent = same result 10 baar. POST = nahi (10 entries).

PUT vs PATCH = poora vs partial. 401 = kaun ho?

403 = allowed nahi. 201 = created. 204 = deleted. 409 = duplicate. Quiz: 10/10.

Brain Ticklers

→ Ye wo concepts hain jo brainstorming mein nikle

→ WHAT pata tha, lekin andar ki trick nahi pata thi.

→ Interview mein aise hi poochhte hain — seedha nahi, ghuma ke.

String intern() — Pool Mein Force Karo

- new String("Hello") heap mein bana — pool se alag.
- Lekin intern() call karo toh pool mein jaake dekhta hai — "Hello" hai?.
- Haan.
- Toh pool wala object return karta hai. Ab == bhi true aayega.

```
String s1 = "Hello" // pool mein
String s2 = new String("Hello"); // heap mein — alag object
String s3 = s2.intern(); // pool mein jaake uthaya

s1 == s2 → false // alag objects
s1 == s3 → true // dono pool ka SAME object
```

WHY intern()?. Performance.

- Agar bahut saari duplicate strings hain (XML parsing
- CSV reading) toh intern() se sab pool mein ek hi object share karein —
- memory save.
- Lekin overuse mat karo — pool PermGen/Metaspace mein hai, bhar gaya toh problem.

HashMap mein equals() Override Nahi Kiya — Duplicate Ghus Gayi

- Custom class ko HashMap key banaya. hashCode() override kiya, equals() nahi.
- Same data ke 2 objects daale — duplicate entry ban gayi!
- HashMap ne hashCode se same bucket mein daala, phir equals() check kiya
- Object.equals() = reference check
- — alag objects — dono store ho gaye.

```
// hashCode() override kiya, equals() NAHI
Employee e1 = new Employee("Arpan", 101);
Employee e2 = new Employee("Arpan", 101);

map.put(e1, "Dev"); // bucket 5
map.put(e2, "Dev"); // bucket 5 — equals() false — DONO store!

// Fix: hashCode + equals DONO override karo
// hashCode same → same bucket
// equals true → REPLACE (duplicate rejected)
```

WHY dono chahiye? hashCode = kaunsa bucket (address).

- equals = same hai ya nahi (identity).
- Sirf hashCode diya toh address same but identity check fail — dono alag maan liya.
- Dono do toh: same bucket + same identity = replace = duplicate rejected.

Immutable Class Toot Gayi — Defensive Copy Nahi Kiya

```
final class Student {
    private final List<String> courses;
    Student(List<String> c) { this.courses = c; } // SAME reference!
    List<String> getCourses() { return courses; } // SAME reference!
}

List<String> list = new ArrayList<>();
list.add("Java");
Student s = new Student(list);
list.add("Python"); // bahar se andar badal gaya!
s.getCourses().size(); // 2 — "Immutable" TOOT gayi!

// Fix — Defensive Copy:
Student(List<String> c) { this.courses = new ArrayList<>(c); }
List<String> getCourses() { return new ArrayList<>(courses); }
```

WHY defensive copy?.

- String field mein ye problem nahi — String khud immutable hai.
- Lekin List mutable hai — same reference bahar bhi hai.
- Bahar se add kiya toh andar bhi badla.
- Nayi copy banao — connection tod do. Constructor mein bhi, getter mein bhi.

Static Method Override? — NAHI, Ye Method Hiding Hai

```
class A { static void show() { print("A"); } }
class B extends A { static void show() { print("B"); } }

A obj = new B();
obj.show(); // "A" — NOT "B"!
```

WHY "A"? Static = class se bind hota hai, object se nahi.

- Compiler reference type dekhta hai = A.
- A.show() call — "A".
- Agar non-static hota toh runtime pe object dekhta = B
- "B" (true polymorphism).
- Static mein polymorphism NAHI hota — ye method hiding hai, overriding nahi.

finally Mein Return — Sab Override

```
try { return 1; }
catch(Exception e) { return 2; }
finally { return 3; }

// Result: 3 — finally ka return sab OVERRIDE kar deta hai!
```

WHY override? try mein return 1 "ready" hua

- — lekin finally HAMESHA chalta hai return se pehle.
- finally mein bhi return hai toh purana return discard, naya set.
- Ye BAD PRACTICE hai
- finally mein return mat likho. finally = cleanup (close connections), return nahi.

sleep() Mein Lock Release Nahi Hota

- synchronized method mein Thread.sleep() call kiya.

- Doosra thread us method mein ja sakta hai?.
- NAHI. sleep() = lock apne paas rakhta hai. So raha hai but lock nahi chhoda.
- Doosra thread wait karega.
- Agar wait() hota toh lock release hota — doosra andar ja sakta tha.

WHY sleep lock rakhta? sleep = "main thak gaya, rest chahiye"

- kaam abhi baaki hai, lock chhodu toh doosra beech mein data badal dega.
- wait = "main doosre ka intezaar kar raha"
- — mera kaam ruka, doosre ko kaam karne do isliye lock chhoda.
- sleep = selfish rest. wait = generous wait.

TreeMap = Hamesha Sorted Order

```
Map<String, Integer> map = new TreeMap<>();
map.put("Banana", 2);
map.put("Apple", 5);
map.put("Cherry", 1);

map.keySet() → [Apple, Banana, Cherry] // SORTED!

// HashMap → koi order guarantee nahi
// LinkedHashMap → insertion order
// TreeMap → sorted (natural) order — Red-Black Tree
```

WHY 3 alag Maps?. HashMap = fastest (O(1)), order nahi chahiye tab.

- LinkedHashMap = insertion order chahiye (LRU cache jaisa).
- TreeMap = sorted order chahiye (leaderboard, dictionary).
- Interview mein "HashMap vs TreeMap vs LinkedHashMap" = ORDER ka fark batao.

Diamond Problem — 2 Interfaces, Same Default Method

```
interface A { default void show() { print("A"); } }
interface B { default void show() { print("B"); } }

class C implements A, B { } // COMPILE ERROR!

// Fix — explicitly override karo:
class C implements A, B {
    public void show() {
        A.super.show(); // A wala choose kiya
    }
}
```

WHY error?. Java confuse hai — A wala show() use karun ya B wala?.

- Dono same signature.
- Ye Diamond Problem hai
- Java 8 se default methods aaye toh ye naya problem bhi aaya.
- Fix = class mein explicitly bata do kaunsa chahiye.
- A.super.show() = A interface ka default method call karo.

FLASH: intern() = pool force. hashCode>equals dono override karo.
Immutable = defensive copy (List).
Static = method hiding (override nahi). finally return = sab override.
sleep = lock rakhta.
TreeMap = sorted.
Diamond = 2 default same, explicitly override karo.

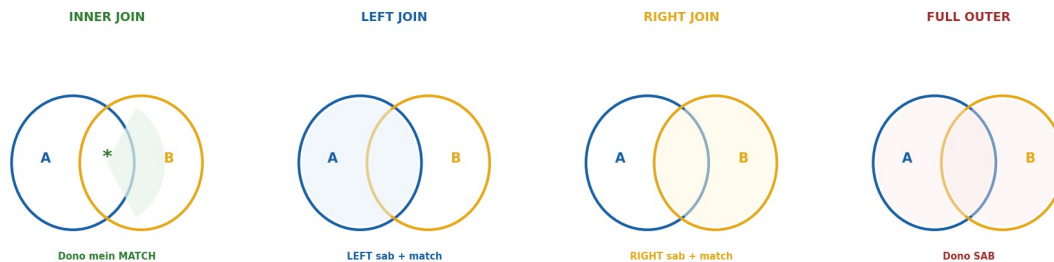
MySQL — Database Interview Essentials

→ Tu daily MySQL use karta hai IRIS mein.

→ Ye naya nahi hai — sirf interview language mein polish karna hai.

JOINS — 2 Tables ko Jodo

SQL JOINS — Venn Diagram



INNER = dono match. LEFT = left sab + match. RIGHT = right sab + match. FULL = dono sab.

→ 2 tables: employees (Arpan dept_id=101 Rahul dept_id=102

→ Priya dept_id=NULL) aur departments (101=Engineering, 102=Marketing, 103=HR).

INNER JOIN → sirf MATCH wale rows
Arpan-Engineering, Rahul-Marketing
Priya OUT (dept NULL), HR OUT (koi employee nahi)

LEFT JOIN → LEFT table ke SAB + right se match ya NULL
Arpan-Engineering, Rahul-Marketing, Priya-NULL

RIGHT JOIN → RIGHT table ke SAB + left se match ya NULL
Arpan-Engineering, Rahul-Marketing, NULL-HR

FULL OUTER → dono ke SAB (MySQL mein LEFT UNION RIGHT)
Arpan-Engineering, Rahul-Marketing, Priya-NULL, NULL-HR

WHY alag alag JOINS?. INNER = sirf confirmed data chahiye.

→ LEFT = "mujhe SAARE employees chahiye

→ department mila toh acha nahi toh NULL."

→ RIGHT = rare, mostly LEFT use hota.

→ FULL = dono side ka sab chahiye — MySQL mein UNION se karo.

WHERE vs HAVING

WHERE = rows filter PEHLE group karo
HAVING = group ke BAAD filter (aggregate pe)

```
-- WHERE: salary > 50000 wale chhaan, PHIR group
SELECT dept, COUNT(*) FROM employees
WHERE salary > 50000 GROUP BY dept;
```

```
-- HAVING: group karo, PHIR 5+ wale chhaan
SELECT dept, COUNT(*) FROM employees
GROUP BY dept HAVING COUNT(*) > 5;
```

TRAP: WHERE mein COUNT() nahi likh sakte — compile error. Aggregate = HAVING mein likho.

INDEX — Query Fast Kyun Hoti Hai

- 10 lakh rows. WHERE email = 'arpan@gmail.com'.
- Bina index? 10 lakh rows ek ek check → $O(n)$ → SLOW.
- Index lagaya? B+ Tree ban gayi → 20 steps mein mila → $O(\log n)$ → FAST.

```
CREATE INDEX idx_email ON users(email);

-- Kya hota andar:
-- 1. MySQL POORI table padhta hai
-- 2. Emails SORT karta + row ka address note karta
-- 3. Sorted data se B+ Tree banata (disk pe save)
-- 4. Search = tree navigate → pointer se row pe jao
-- Original data SAME rehta — ye EXTRA structure hai
```

- **Clustered vs Non-Clustered:**
- Clustered = data KHUD sorted. Dictionary jaisa — words sorted HAIN.
- PK = automatic. SIRF EK.
- Non-Clustered = ALAG tree with pointer. Book ki INDEX page jaisa.
- MULTIPLE ho sakte.
- Kab index lagao:
- WHERE/JOIN/ORDER BY mein frequently aata → lagao.
- Unique values zyada (email, phone) → lagao.
- Kam unique (gender = M/F) → mat lagao. Aadhi table scan hogi anyway.
- Bahut INSERT/UPDATE → soch ke lagao. Har write pe saare indexes update honge.

EXPLAIN — Slow Query ka X-Ray

- EXPLAIN = query chalao mat, plan batao.
- Swiggy jaisa — khana mat lao, batao KAISE laoge.

```
EXPLAIN SELECT * FROM users WHERE email = 'arpan@gmail.com';

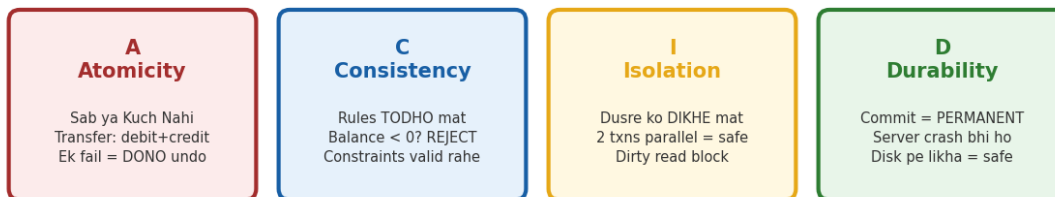
-- 3 columns dekho:
-- type: const/ref = FAST | ALL = FULL SCAN = SLOW
-- key: idx_name = GOOD | NULL = NO INDEX = FIX
-- rows: 1-10 = FAST | 100000+ = SLOW = FIX
```

WHY EXPLAIN?. Query slow — but kyun?. Guess mat karo.

- EXPLAIN lagao — MySQL khud batayega. type=ALL dikha?.
- Index lagao. key=NULL dikha?. Index lagao. rows bahut zyada?
- Query optimize karo.
- Yehi tera "30% query optimization" tha — EXPLAIN + Index + Pagination.

ACID — Transaction ke 4 Rules

ACID — Transaction ke 4 Rules



Bank Transfer: A = sab ya kuch nahi | C = balance rules valid | I = parallel safe | D = commit = permanent

A=sab ya kuch nahi. C=rules valid. I=parallel safe. D=commit permanent. Bank transfer = perfect example.

- A = Atomicity — sab ho ya kuch nahi.
- Bank transfer: Step 1 (A se nikalo) + Step 2 (B mein daalo). Step 2 fail?
- Step 1 bhi UNDO.
- @Transactional = Java mein Atomicity.
- C = Consistency — DB rules hamesha valid.
- Balance negative nahi. Total match kare. FK valid. NOT NULL constraints check.
- I = Isolation — ek transaction doosre ko disturb na kare.
- 2 users same account — bina isolation = race condition = paisa extra nikla.
- Java mein synchronized jaisa concept.
- MySQL default = REPEATABLE READ. PostgreSQL = READ COMMITTED.
- D = Durability — commit = PERMANENT.
- Server crash, light gayi — committed data safe.
- WAL (Write Ahead Log) se recover.

POWER PHRASE: "ACID guarantees reliable transactions —
Atomicity ensures all-or-nothing,
Consistency maintains database rules,
Isolation prevents concurrent transactions from interfering
and Durability ensures committed data survives crashes.
In Spring
@Transactional provides Atomicity with automatic rollback on RuntimeException."

Quick-Fire Differences

- SQL vs MySQL:
- SQL = LANGUAGE (jaise Hindi). SELECT, INSERT, UPDATE, DELETE.
- MySQL = SOFTWARE (jaise Chrome). SQL language ko samajhta hai.
- Baaki: PostgreSQL, Oracle, SQL Server.
- DELETE vs TRUNCATE vs DROP:
- DELETE = kuch rows hatao. WHERE lagta. Rollback ho sakta. Slow (row by row).
- Table REHTI.
- TRUNCATE = SAB rows hatao. WHERE NAHI lagta. Rollback NAHI. Fast.
- Table REHTI (khaali).
- DROP = table hi uda do. Data + structure sab gone. Table GAYAB.

TRAP: TRUNCATE mein WHERE nahi lagta! Sirf DELETE mein WHERE.

- Primary Key vs Unique Key:
- PK = NULL NAHI. SIRF EK per table. Auto Clustered Index.
- Row ki identity. (Aadhaar Card)
- Unique = NULL ok (ek baar). MULTIPLE per table.
- Auto Non-Clustered Index. (Phone Number)

- Subquery vs JOIN:
- Subquery = query ke andar query. Readable, lekin SLOW (nested loop).
- JOIN = tables jodo. Optimizer ek baar mein plan banata = FAST.
- Result SAME, speed ALAG.
- UNION vs UNION ALL:
- UNION = merge + duplicates HATAO. Slow (sort + compare).
- UNION ALL = merge + duplicates RAKHO. Fast (seedha jod do).
- FULL OUTER JOIN bhi LEFT UNION RIGHT tha.

Normalization — 1NF, 2NF, 3NF

Normalization — 1NF → 2NF → 3NF



1NF=atomic (no comma-separated). 2NF=full key dependency. 3NF=no $X \rightarrow Y \rightarrow Z$ (remove transitive).

▲ Unnormalized → 1NF (atomic, no repeating) → 2NF (no partial dependency) → 3NF (no transitive $X \rightarrow Y \rightarrow Z$).

- Normalization = data DUPLICATE hatao, tables TOD DO, sab ek jagah rakho.
- Update ek jagah karo, sab jagah reflect ho.
- 1NF — Har cell mein EK value:

GALAT: id=1, courses="Java, DBMS" (ek cell mein 2 values)
 SAHI: id=1, course=Java (Row 1) id=1, course=DBMS (Row 2)

Rule: Atomic values. Comma-separated hatao. Alag rows mein todo.

- 2NF — Partial dependency hatao:
- Composite PK = (student_id + course).
- student_name sirf student_id pe depend = PARTIAL.

→ Fix: students table alag (student_id, name).

→ marks table alag (student_id, course, marks).

Rule: Har non-key column POORI PK pe depend kare, aadhi pe nahi.

- 3NF — Transitive dependency hatao:

→ students table: student_id, name, dept, dept_head.

→ dept_head → dept pe depend. dept → student_id pe depend.

→ Chain: student_id → dept → dept_head = TRANSITIVE.

→ Fix: departments table alag (dept, dept_head). students mein sirf dept FK rakho.

Rule: Non-key column DOOSRE non-key pe depend na kare. Sirf PK pe.

WHY Normalize?.

- Bina normalize = duplicate data ek jagah update karo 10 jagah bhool jao
- delete karo toh related data bhi gayab.
- Trick yaad rakho: "The key the whole key and nothing but the key." 1NF = key exists.
- 2NF = WHOLE key.
- 3NF = NOTHING BUT the key.

[TRAP] Denormalization kab?

- Jab READ performance chahiye
- reporting/analytics mein intentionally duplicate rakho, JOINS avoid karo.
- Trade-off: fast reads but storage zyada + update complex.

FLASH: INNER=sirf match. LEFT=left ke sab. WHERE=rows pe, HAVING=aggregate pe.
Index=B+ Tree, PK=auto clustered, Non-clustered=pointer.
EXPLAIN: type=ALL/key=NULL=fix karo.
ACID: A=all or nothing, C=rules valid, I=no interference, D=permanent.
SQL=language, MySQL=software.
DELETE=rows+WHERE+rollback, TRUNCATE=sab+fast+no rollback, DROP=table gayab.
PK=ek+no NULL, Unique=multiple+NULL ok. JOIN>Subquery (speed).
UNION=unique, UNION ALL=fast+duplicates.
1NF=ek cell ek value, 2NF=poori key pe depend, 3NF=sirf key pe depend.
Denormalization=read fast, write complex. Quiz: 12/12+12/12+10/10.

Basic Differences — Interview Favorites

→ WHY: Ye 12 differences GUARANTEED interview mein poochhe jaate. Sirf KYA fark nahi — KYUN fark samjho. Spring vs Boot: Spring = framework (manual config), Boot = opinionated (auto-config + embedded server). JPA vs Hibernate: JPA = SPECIFICATION (interface), Hibernate = IMPLEMENTATION. == vs equals(): == reference compare, equals() content compare.

Spring vs Spring Boot

Spring = FRAMEWORK (DI, AOP, MVC)
XML config manually likhni padti
Server (Tomcat) ALAG install karo
Setup = 2-3 ghante

Spring Boot = Spring ka WRAPPER
Auto-configuration — annotations, XML nahi
Embedded Tomcat — jar mein server
Starters — ek dependency, sab aa jaye
Setup = 5 minute

→ Spring Boot = Spring hi hai. Sirf setup easy kiya. Andar Spring chal raha.

→ Jaise Android = Linux hai — user-friendly bana diya.

Spring MVC vs Spring Boot

→ MVC = web LAYER — sirf HTTP handle (DispatcherServlet → Controller → View).

→ Boot = POORA app setup — MVC + Security + JPA + Server sab included.

→ MVC = car ka steering. Boot = poori car ready-to-drive.

→ Compare nahi hote — Boot USE karta hai MVC ko.

JPA vs Hibernate

→ JPA = SPECIFICATION (rules, interfaces). Khud kuch NAHI karta.

→ Hibernate = IMPLEMENTATION (actual code). JPA ke rules follow karke kaam karta.

→ JPA = driving license ka rule book.

→ Hibernate = actual driver jo rules follow karta.

→ Baaki implementations: EclipseLink, OpenJPA.

@Component vs @Bean

→ @Component = CLASS pe lagao. Spring auto-detect kare. TERI class.

→ @Service, @Repository, @Controller = @Component ki specializations.

→ Internally SAME.

→ @Bean = METHOD pe lagao.

→ Tu manually object bana. 3rd party class (RestTemplate, ObjectMapper).

```
// @Component – teri class
@Service
public class UserService { }

// @Bean – 3rd party ka object
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

[TRAP] @Service/@Component internally SAME — Spring ko fark nahi.

→ Label TERE liye hai (code readability).

@Controller vs @RestController

@Controller → return "home" → home.html page (VIEW)
 @RestController → return user → {"name":"Arpan"} JSON (DATA)

@RestController = @Controller + @ResponseBody
 REST API bana rahe = @RestController

== vs equals()

→ == = REFERENCE check — "kya ye SAME object hai?"

→ equals() = CONTENT check — "kya andar ka data SAME hai?"

```
String a = new String("Hello");
String b = new String("Hello");

a == b // false – alag objects (heap mein alag)
a.equals(b) // true – content same "Hello"

String c = "Hello"
String d = "Hello"
c == d // true – SAME pool object
```

[TRAP] String compare = HAMESHA equals(). == sirf reference check.

→ Interview mein ZAROOR poochhenge.

Abstract Class vs Interface

Abstract Class:
 Shared CODE de sakta (methods with body)
 Constructor HAI. Variables koi bhi type.
 SIRF EK extend. IS-A relationship.

Interface:
 CONTRACT define karta (Java 8+: default methods)
 Constructor NAHI. Variables = public static final.
 MULTIPLE implement. CAN-DO capability.

Shared code chahiye = Abstract Class. Capability define karni = Interface.

Comparable vs Comparator

→ Comparable = class ke ANDAR, compareTo(), EK fixed order.

→ Comparator = class ke BAHAR, compare(), MULTIPLE orders.

→ Dono mein TU decide karta kaise sort karna.

```
// Comparable – class mein (EK order)
public int compareTo(Student s) {
    return this.name.compareTo(s.name); // name se sort
}

// Comparator – bahar se (MULTIPLE orders)
list.sort(Comparator.comparing(Student::getName)); // name
list.sort(Comparator.comparing(Student::getAge)); // age
```

@Autowired — 3 Types of Injection

→ Field Injection = @Autowired field pe. Easy but BAD (final nahi, test mushkil).

→ Constructor Injection = BEST. final lagao, test = new Service(mockRepo).

→ Setter Injection = rare. Optional dependencies ke liye.

```
// BEST – Constructor Injection
@Service
public class UserService {
    private final UserRepo repo; // final!

    public UserService(UserRepo repo) {
        this.repo = repo;
    }
}
// Ek constructor ho toh @Autowired bhi nahi chahiye
```

[TRAP] Interview mein poochhe "kaunsa best?" = Constructor.

→ Kyun? final lagta, test easy, clearly dikhta kya chahiye.

Checked vs Unchecked Exception

→ Checked = compiler FORCE kare handle (IOException, SQLException).

→ EXPECTED problems.

→ Unchecked = runtime pe aaye (NullPointerException, ArrayIndex). PROGRAMMING BUGS.

→ @Transactional default = sirf Unchecked (RuntimeException) pe rollback.

→ Checked pe NAHI.

→ Fix: rollbackFor = IOException.class.

final vs finally vs finalize

final = value/method/class CHANGE nahi ho sakta
final int x = 10; x = 20 → COMPILE ERROR

finally = try ke baad HAMESHA chale (cleanup)
try { risky(); } finally { conn.close(); }

finalize = DEPRECATED (Java 9+). Bhool jao.
try-with-resources use karo instead.

GET vs POST

GET:

Data LENE ke liye. URL mein params.
Idempotent – 10 baar karo, same result. Cacheable.

POST:

Data BHEJNE / CREATE ke liye. Body mein data.
NOT idempotent – 3 baar = 3 records. Not cacheable.

GET /users/123 → user #123 ka data lao
POST /users {body} → naya user banao
PUT /users/123 {body} → user #123 poora replace
PATCH /users/123 {age:28} → sirf age update
DELETE /users/123 → user #123 hatao

Extra Interview Topics — Quick Reference

OOP — Extra Concepts

→ Covariant Return Type — child override mein parent ka SUBTYPE return kar sakta.

```
class Animal { Animal create() { } }  
class Dog extends Animal { Dog create() { } } // VALID — Dog IS-A Animal
```

→ Access Modifier Override Rule

→ child mein access SAME ya WIDER hona chahiye, NARROW nahi.

Parent: public show() → Child: private show() = COMPILER ERROR (narrower)
Parent: protected show() → Child: public show() = VALID (wider)

→ super() aur this() — dono constructor mein FIRST statement hone chahiye.

→ Ek hi likh sakte — dono nahi.

→ Interface mein private methods — Java 9 se allowed.

→ Default methods mein common code reuse ke liye helper methods.

String — Extra Traps

→ Compile Time Constant Folding

→ "Hel" + "lo" = dono literals →

→ compiler COMPILER time pe "Hello" bana de → pool mein same object.

```
String s1 = "Hello";  
String s2 = "Hel" + "lo"  
// compile time = "Hello"  
s1 == s2 → TRUE (same pool object)
```

```
String x = "Hel";  
String s3 = x + "lo"; // x VARIABLE hai → runtime = NAYA object  
s1 == s3 → FALSE
```

[TRAP]

→ Literals + literals = compile time fold = same pool.

→ Variable + literal = runtime = naya object.

→ String.valueOf(null) → "null" string (safe).

→ null.toString() → NullPointerException (crash).

→ valueOf prefer karo.

→ String class FINAL kyun?

→ Agar extend karke equals() override karein toh HashMap key tamper ho sakti.

→ Security + immutability guarantee.

HashMap — Extra Internals

→ Treeify — bucket mein LinkedList → Red-Black Tree kab? DONO conditions chahiye:

```
1. Bucket mein nodes >= 8 (TREEIFY_THRESHOLD)
2. Table size >= 64 (MIN_TREEIFY_CAPACITY)
Agar 8+ nodes but table < 64 → RESIZE hota, tree nahi.
Tree wapas LinkedList → nodes <= 6 (UNTREEIFY_THRESHOLD)
```

WHY 8? Poisson distribution — 8 collisions ki probability 0.00000006.

- Bahut rare — tab tree bana worth hai.
- hashCode() aur equals() CONTRACT — equals() override kiya toh hashCode()
- BHI karo.
- Sirf hashCode() kiya toh same bucket mein jayenge lekin equals()
- default reference check = duplicate ghus jayegi.

[TRAP]

- HashMap multi-thread mein: Java 7 mein infinite loop (rehashing).
- Java 8 mein data corruption, lost updates.
- Fix = ConcurrentHashMap.

Collections — Extra Topics

- ArrayList add(index, element) = $O(n)$
- index ke baad sab RIGHT shift. add(end) = $O(1)$ amortized.
- EnumSet — Enum ke saath SABSE FAST Set. Internally bit vector.
- Har enum value = 1 bit.
- Contains/add/remove = $O(1)$ bit operation.
 - EnumSet.of(MON, WED, FRI) → internally 1010100 (7 bits)
- Collections.sort() = TimSort (Java 7+) — Merge Sort + Insertion Sort hybrid.
- Stable. $O(n \log n)$.
 - Arrays.sort(primitives) = Dual-Pivot QuickSort (unstable)
 - Arrays.sort(objects) = TimSort (stable)
- WeakHashMap — keys = weak references.
- Key ka strong reference nahi bachta → GC collect kare → entry auto remove.
- Self-cleaning cache.
- Iterator.forEachRemaining() — Java 8+.
- Remaining elements pe Consumer action ek line mein.

```
it.next(); // pehla skip
it.forEachRemaining(System.out::println); // baaki sab print
```

Java 8 — Extra Concepts

- Stream.peek() — DEBUGGING purpose. Pipeline mein intermediate results dekho.
- Production mein avoid — JVM optimize karke skip kar sakta.
- Parallel Stream
- AVOID jab: chhota data (<10000) order matters
- shared mutable state, I/O operations.
- USE jab: CPU-intensive + large data + stateless.
- Optional.map() vs flatMap() — function returns Optional?
- flatMap use karo (double wrap avoid).
- Otherwise map.

```
Optional.of(user).map(u → u.getName()) → Optional<String>
Optional.of(user).flatMap(u → u.getAddress()) → Optional<Address>
// getAddress() already returns Optional — flatMap = no double wrap
```

[TRAP]

- Collectors.toMap() mein duplicate key → IllegalStateException CRASH.

- Fix: merge function do (existing, newVal) → existing.
- IntStream vs Stream<Integer>
- IntStream = primitive, no autoboxing, sum()/average() direct.
- Stream<Integer> = wrapper, boxing overhead.
- Performance matter kare → IntStream.

Hibernate — Extra Concepts

- save() vs persist()
- save() returns ID (Hibernate-specific). persist() returns void (JPA standard).
- Best practice: persist() use karo.
- CascadeType.ALL — DANGER! Parent delete → ALL children delete.
- Safe approach: CascadeType.PERSIST + MERGE alag se.
- ALL sirf jab child CANNOT exist without parent.
- Dirty Checking
- Persistent object mein change kiya → koi save() nahi call kiya →
- transaction commit pe Hibernate AUTOMATICALLY detect kare → UPDATE SQL fire.

WHY automatic? Session SNAPSHOT rakhta load time ka. Flush pe compare kare.

- Different? UPDATE generate.
- @Transactional(readOnly=true) → dirty checking SKIP = faster.
- @GeneratedValue — IDENTITY = DB auto_increment (batch slow, INSERT ke baad ID).
- SEQUENCE = DB sequence (batch fast, pehle ID). MySQL = IDENTITY.
- PostgreSQL = SEQUENCE.
- L2 Cache setup
- 3 cheezein: cache provider (EhCache/Redis) entity pe @Cacheable + @Cache
- config enable.
- L1 = free auto. L2 = setup chahiye.

Spring — Extra Concepts

- @Configuration vs @Component
- @Configuration mein @Bean method doosre ko call kare → SAME singleton.
- @Component mein → NAYA object har baar!
- @Bean methods = @Configuration class mein rakho.
- Auto-Configuration
- @EnableAutoConfiguration + spring.factories + @ConditionalOnClass.
- Jar classpath mein hai?
- Auto-config. Nahi hai? Skip. Convention over configuration.
- @Value vs @ConfigurationProperties — @Value = single property quick.
- @ConfigurationProperties(prefix) = group of related properties, type-safe, validated.
- 5+ properties → @ConfigurationProperties.
- Circular Dependency — A needs B, B needs A.
- Field/setter injection: Spring 3-phase creation se handle.
- Constructor injection: CRASH. Fix: @Lazy ya redesign. Best: avoid circular.

[TRAP]

- @Autowired field injection BAD
- final nahi, test mein mock mushkil, dependencies hidden.
- Constructor injection BEST
- final lagao, test mein new karo, dependencies visible.

Security — Extra Concepts

- JWT Logout Problem
- JWT stateless hai, server "invalidate" nahi kar sakta.
- Workarounds: short expiry (15-30 min)
- + refresh token, token blacklist (Redis), client-side delete.
- CORS — Cross-Origin Resource Sharing.
- Browser BLOCK karta different origin se API call.
- Server bole "is origin se aane do" via @CrossOrigin ya WebConfig.
Origin = protocol + domain + port. Ek bhi alag = different origin.
localhost:3000 (React) → localhost:8080 (Spring) = BLOCKED by default
Fix: @CrossOrigin(origins = "http://localhost:3000")
- BCrypt Random Salt — har baar ALAG hash same password ka.
- Salt hash ke ANDAR stored.
- BCrypt.matches() salt extract karke compare karta.
- Rainbow table attack impossible.

Multithreading — Extra Concepts

- synchronized method vs block —
- method = POORA lock (coarse).
- block = SPECIFIC code lock (fine-grained, better performance).
- Block prefer karo.
- ReentrantLock — synchronized + extra features: tryLock()
- (non-blocking), timeout, fairness (longest waiting pehle), interruptible.
- Advanced control chahiye → ReentrantLock.
- ThreadLocal — har thread ka APNA private variable.
- Doosre thread access nahi kar sakte.
- Use: SimpleDateFormat (thread-unsafe), user session, DB connection per thread.

```
ThreadLocal<SimpleDateFormat> df = ThreadLocal.withInitial(
    () → new SimpleDateFormat("yyyy-MM-dd")
);
df.get() → har thread ki apni copy
```
- supplyAsync() vs runAsync()
- supplyAsync = value RETURN karta (chain mein use). runAsync = void (fire and forget).
- Thread Pool — core size = hamesha alive. max size = peak load pe temporary.
- Load gira → extra die → wapas core.

REST — Extra Concepts

- Stateless — har request SELF-CONTAINED.
- Server previous request yaad NAHI rakhta. Client har baar token/auth bheje.
- Kyun? Scalability — server kuch store nahi → easily scale.
- Rate Limiting — ek client bahut requests bheje → 429 Too Many Requests.
- API abuse roko.
- Token bucket / sliding window algorithm. AWS API Gateway mein built-in.

MySQL — Extra Concepts

- Covering Index — index mein SAB columns jo query chahiye.

→ Table pe jaana hi nahi padta.

→ EXPLAIN mein "Using index" = covering index.

```
Query: SELECT name, email FROM users WHERE city="Delhi"  
Normal index on city → index se match → TABLE jaake name, email nikalo  
Covering index on (city, name, email) → index mein hi sab → TABLE SKIP!
```

→ SQL Injection — user input mein SQL code ghusao.

→ Fix: PreparedStatement (parameterized queries).

→ ORM (Hibernate/JPA) = auto parameterized = safe.

```
BAD: "SELECT * FROM users WHERE name = '"  
+ input // injection possible  
GOOD: ps = conn.prepareStatement("SELECT * WHERE name = ?"); ps.setString(1  
input);
```

[TRAP] NEVER use string concatenation for SQL. Hamesha PreparedStatement ya ORM.

→ MySQL Deadlock — T1 lock row1, want row2. T2 lock row2, want row1.

→ Circular wait. MySQL detect kare → ek rollback.

→ Fix: same ORDER mein rows access karo.

→ VARCHAR(255) — 1 byte mein length store (0-255).

→ VARCHAR(256) → 2 bytes. 255 = sweet spot, memory efficient.

Java Core — Extra Concepts

→ try-with-resources (Java 7+) — AutoCloseable resources AUTOMATICALLY close.

→ Finally likhne ki zarurat nahi.

→ Bholna impossible.

```
// PURANA — manually close  
try { Connection c = get(); } finally { c.close(); }  
  
// NAYA — auto close  
try (Connection c = get()) { } // auto close!
```

→ Marker Interface — EMPTY interface, koi method nahi, sirf "mark" karta.

→ Example: Serializable, Cloneable.

→ JVM check: instanceof Serializable? → serialize allowed.

→ Enum mein constructor — PRIVATE constructor allowed.

→ Har constant ke liye call hota. new Enum() kabhi nahi — hamesha private.

```
enum Planet { EARTH(5.97e24, 6.37e6), MARS(6.42e23, 3.39e6);  
    private final double mass, radius;  
    Planet(double m, double r) { mass=m; radius=r; } // private!  
}
```

→ ClassLoader — .class files load karta JVM mein.

→ 3 types: Bootstrap (core Java), Extension (ext), Application (tera code).

→ Parent-first delegation.

FLASH:

→ Extra topics: Covariant return, constant folding, treeify 8+64

→ EnumSet bit vector TimSort WeakHashMap,

→ peek=debug, parallel stream dangers, dirty checking

→ CascadeType.ALL danger, @Configuration vs @Component,

→ circular dependency JWT logout CORS ReentrantLock ThreadLocal, covering index

→ SQL injection,

→ try-with-resources, marker interface.

→ Quiz V4 DEEP: 63/65 (97%).

Extra Concepts — Quick Reference

try-with-resources

→ Java 7+ — resources AUTOMATICALLY close. finally likhne ki zarurat nahi.

```
// PURANA – manually close (bhool gaye toh LEAK)
try { Connection con = getConnection(); }
finally { con.close(); }

// NAYA – auto close (bhoolna impossible)
try (Connection con = getConnection()) {
    // use con
} // AUTOMATICALLY close
```

→ Resource must implement AutoCloseable.

→ Connection, InputStream, BufferedReader — sab AutoCloseable hain.

WHY? finally mein close bhoolna = connection leak = HikariCP exhaustion jaisa bug.

→ try-with-resources = bhoolna IMPOSSIBLE.

SQL Injection — PreparedStatement

→ User input mein SQL code ghusao → database hack.

```
// GALAT – String concatenation
String sql = "SELECT * FROM users WHERE name = '"
+ userInput + "'"
// userInput = ' OR 1=1 -- → SAARI rows return! Data leak.

// SAHI – PreparedStatement (parameterized)
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM users WHERE name = ?");
ps.setString(1, userInput); // input = data, SQL nahi
```

→ ? = placeholder. Input kabhi SQL mein mix nahi hoga.

→ ORM (Hibernate/JPA) = automatically parameterized = safe.

[TRAP] SQL Injection:

→ NEVER use string concatenation for SQL queries.

→ Hamesha PreparedStatement ya ORM use karo.

equals() + hashCode() Contract

→ HashMap/HashSet ke liye DONO override karna MUST hai.

→ Sirf equals() override → same bucket mein nahi jayenge → duplicate add ho jayegi.

→ Sirf hashCode() override

→ same bucket mein jayenge lekin equals() false (reference) duplicate.

→ DONO override = same bucket + equals true = correctly handled.

```
Contract:
    equals() true → hashCode() MUST same
    hashCode() same → equals() ZARURI nahi same (collision)
```

WHY dono?

→ HashMap pehle hashCode() se bucket dhundhe, phir equals() se exact match.

→ Ek bhi missing = duplicate entries.

→ Rule: equals() override kiya? hashCode() BHI karo. HAMESHA.

Marker Interface

→ EMPTY interface — koi method nahi. Sirf TYPE mark karta.

```
public interface Serializable { } // EMPTY!

class User implements Serializable { }
// JVM check: instanceof Serializable? Haan → serialize allowed.
```

→ Examples: `Serializable`, `Cloneable`, `Remote`.

→ Modern alternative: annotations (`@Entity`, `@Cacheable`).

→ Lekin `Serializable` abhi bhi marker interface hai.

Covariant Return Type

→ Override mein child method parent ka SUBTYPE return kar sakta.

```
class Animal { Animal create() { return new Animal(); } }
class Dog extends Animal { Dog create() { return new Dog(); } } // VALID
// Dog IS-A Animal → covariant return allowed.
```

→ Java 5 se support hai. Return type SAME ya SUBTYPE hona chahiye, kuch aur nahi.

Override mein Access Modifier Rule

→ Override mein access SAME ya WIDER — NARROWER nahi.

```
Parent: public → Child: public (SAME) → OK
Parent: protected → Child: public (WIDER) → OK
Parent: public → Child: private (NARROWER) → COMPILE ERROR
```

WHY? Child parent ki jagah baith sake (Liskov).

→ Agar child zyada restrictive ho toh substitution break hoga.

ThreadLocal

→ Har thread ka APNA private variable — doosre thread access nahi kar sakte.

```
ThreadLocal<SimpleDateFormat> df = ThreadLocal.withInitial(
    () → new SimpleDateFormat("yyyy-MM-dd")
);
// Thread A: df.get() → A ki apni copy
// Thread B: df.get() → B ki apni — A se ALAG
```

WHY? SimpleDateFormat thread-safe NAHI. Share karo → corrupt.

→ `ThreadLocal` = har thread ki apni copy = safe.

→ Use cases: user session, DB connection per thread, date formatters.

ReentrantLock vs synchronized

→ `ReentrantLock` = `synchronized` + EXTRA features.

`synchronized`: lock ya wait forever — koi control nahi.

```
ReentrantLock:
tryLock() → mila toh lo, nahi toh move on
tryLock(timeout) → itni der wait, phir chhodo
fairness → longest waiting pehle
lockInterruptibly() → waiting thread interrupt
```

→ Simple case? `synchronized`. Advanced control (timeout, try, fair)? `ReentrantLock`.

synchronized method vs block

→ Method = POORA method lock. Block = SPECIFIC code lock.

```
// Method — poor lock (coarse)
synchronized void transfer() { ... }

// Block — sirf critical section (fine-grained, BETTER)
void transfer() {
    // ye code unlocked — parallel chale
    synchronized(this) {
        // sirf ye locked
    }
}
```

Rule: jitna kam lock utna acha. Block prefer karo — performance better.

Covering Index

→ Index mein SAB columns hain jo query chahiye — table access hi nahi karna padta.

Query: SELECT name, email FROM users WHERE city='Delhi'

Normal index on city: index se city match → table pe jaake name, email nikalo.

Covering index on (city, name, email): index mein hi sab hai → table SKIP!

EXPLAIN mein Extra: 'Using index' = covering index use hua.

WHY fast? Disk I/O kam.

→ Index ALONE enough — table access = expensive disk read skip.

Circular Dependency — Spring

→ A depends on B, B depends on A — chicken-egg problem.

→ Field/setter injection mein Spring handle karta (3-phase creation).

→ Constructor injection mein CRASH — BeanCurrentlyInCreationException.

Fix:

1. @Lazy on one side → lazy proxy inject
2. Redesign → extract common logic (BEST)
3. Event-based communication

[TRAP] Circular Dependency: Usually bad design ka sign. Fix = extract shared logic into a separate service.

CORS — Cross-Origin Resource Sharing

→ Browser BLOCK karta different origin se API call. Server ALLOW kare.

→ Origin = protocol + domain + port. Ek bhi alag = different origin.

```
// Controller level
@CrossOrigin(origins = "http://localhost:3000")

// Global level
@Configuration
class WebConfig implements WebMvcConfigurer {
    cors.addMapping("/**").allowedOrigins("...")
}
```

→ Bina CORS config → frontend ka API call FAIL hoga browser mein.

JWT Logout Problem

→ JWT stateless hai — server pe kuch STORE nahi. Token valid hai toh valid hai.

→ Server 'invalidate' nahi kar sakta

→ perfect logout NAHI hota JWT mein.

Workarounds:

1. Short expiry (15-30 min) + refresh token
2. Token blacklist (Redis mein) – lekin ye stateful ho gaya
3. Client side delete (localStorage se hatao)

WHY problem? JWT = stateless by design. Logout = state chahiye. Conflict.

→ Short expiry + refresh token = practical solution.

Dirty Checking — Hibernate

→ Persistent object mein change kiya Hibernate AUTOMATICALLY detect kare

→ UPDATE SQL fire.

```
User u = session.get(User.class, 1); // PERSISTENT
u.setName('Rahul'); // change kiya – koi save() NAHI call kiya!
// Transaction commit pe → Hibernate detect → UPDATE fire!
```

→ Session SNAPSHOT rakhta load time ka. Flush pe compare kare. Different? UPDATE.

→ @Transactional(readOnly=true) → dirty checking SKIP = faster for read operations.

CascadeType.ALL — Danger

→ CascadeType.ALL = ALL operations cascade (persist, merge, REMOVE sab).

[TRAP] CascadeType.ALL: Department delete kiya → ALL employees bhi delete! Kya ye chahiye? Mostly NAHI.

Safe approach:

CascadeType.PERSIST + CascadeType.MERGE – create/update cascade.

Remove SEPARATELY handle karo.

ALL sirf jab child CANNOT exist without parent (Order → OrderItems).

@GeneratedValue — IDENTITY vs SEQUENCE

→ IDENTITY: DB ka auto_increment. INSERT ke BAAD hi ID pata. Batch insert SLOW.

→ SEQUENCE: DB mein sequence object. INSERT se PEHLE ID mil jaata.

→ Batch insert FAST.

```
MySQL = IDENTITY (auto_increment default)
PostgreSQL/Oracle = SEQUENCE (native support)
```

→ Performance matter kare → SEQUENCE prefer. MySQL mein IDENTITY hi chalega.

@Configuration vs @Component — CGLIB Proxy

→ @Configuration mein @Bean methods = CGLIB proxy se SINGLETON guarantee.

→ @Component mein same @Bean method call = NAYA object har baar! Singleton BREAK.

```
@Configuration
class Config {
    @Bean A a() { return new A(); }
    @Bean B b() { return new B(a()); } // a() = SAME singleton
}
// @Component mein b() call kare a() → NAYA A object!
```

Rule: @Bean methods = @Configuration class mein hi rakho.

Parallel Stream — Kab NAHI use karo

→ 4 cases mein AVOID:

1. Chhota data (< 10000) → thread overhead > benefit
2. Order matters → parallel = order guarantee nahi
3. Shared mutable state → race condition
4. I/O operations (DB, file) → thread block = worse

→ Kab use? CPU-intensive + large data + stateless operation.

Collectors.toMap() — Duplicate Key Trap

→ Duplicate key aaye toh `IllegalStateException` — crash by default!

```
// CRASH — same dept ke 2 employees
.collect(Collectors.toMap(e→e.getDept(), e→e.getName()))

// FIX — merge function do
.collect(Collectors.toMap(
    e→e.getDept(),
    e→e.getName(),
    (existing, newVal) → existing + "," + newVal // merge
))
```

[TRAP] toMap() trap: Duplicate key = crash. Hamesha merge function do agar duplicates possible.

Rate Limiting — 429 Too Many Requests

→ Ek client bahut requests bheje → server DOWN. Rate limit = roko.

```
100 requests/minute per client. Exceed? 429 Too Many Requests.
Response headers:
  X-RateLimit-Limit: 100
  X-RateLimit-Remaining: 23
  Retry-After: 60
```

→ Implementation: Token bucket

→ Sliding window, ya API Gateway level (AWS built-in).

→ try-with-resources = auto close.

→ SQL injection = PreparedStatement se roko. equals+hashCode = DONO override.

→ Marker interface = empty (Serializable).

→ ThreadLocal = thread ki apni copy.

→ ReentrantLock = synchronized + tryLock/timeout/fair.

→ Covering index = table access skip. CORS = server allow kare cross-origin.

→ JWT logout = short expiry + refresh. Dirty checking = auto UPDATE detect.

→ CascadeType.ALL = danger, prefer PERSIST+MERGE.

→ @Configuration = CGLIB singleton guarantee.

→ Parallel stream = chhota data/IO/shared state pe AVOID.

→ toMap duplicate = crash, merge function do.

Quiz Concepts — Naye Topics Jo Seekhe

Aggregation vs Composition:

→ Aggregation = weak HAS-A. University HAS Students.

→ University band → students EXIST karte.

→ Composition = strong HAS-A. House HAS Rooms. House gira → rooms GONE.

```
Aggregation: class Team { List<Player> players; } // players exist without team
Composition: class House { List<Room> rooms; } // rooms die with house
```

Template Method Pattern:

→ Parent class mein algorithm SKELETON — child specific steps override kare.

```

abstract class DataProcessor {
    final void process() { // order FIXED
        readData();        // child override
        validateData();    // child override
        saveData();        // common – parent mein
    }
}
class CSVProcessor extends DataProcessor {
    void readData() { /* CSV read */ }
    void validateData() { /* CSV validate */ }
}

```

WHY? Algorithm order FIXED rahega — steps FLEXIBLE.

→ Spring ka JdbcTemplate = Template Method pattern.

Decorator Pattern — Java I/O:

→ Wrapper layers — har layer extra functionality ADD kare. Original untouched.

```

new BufferedReader( // layer 3: buffering
    new InputStreamReader( // layer 2: byte-char
        new FileInputStream(file) // core: file read
    )
)

```

→ Collections mein bhi: Collections.synchronizedList(list) = sync wrapper.

→ Collections.unmodifiableList(list) = read-only wrapper.

HashMap Capacity = Power of 2 WHY:

→ Normal: hash % n = bucket index (modulo = SLOW, division).

→ Power of 2: (n-1)

→ & hash = bucket index (bitwise AND = FAST, 1 CPU instruction).

```

n=16: n-1 = 15 = 0000 1111
hash & 0000 1111 = last 4 bits = index 0-15
Capacity: 16 → 32 → 64 → 128 (always 2^n)

```

→ Custom capacity diya? HashMap next power of 2 pe round up karta.

Optimistic Locking — @Version:

→ 2 users ek saath same record update — kaise handle?

```

@Entity
class Product {
    @Version
    private int version; // auto-increment on update
}

```

→ User A: read (v=1), edit, save → v=1→2 OK.

→ User B: read (v=1), edit, save → DB has v=2, expected v=1 → OptimisticLockException!

WHY Optimistic? Pessimistic = DB row lock = SLOW + deadlock risk.

→ Optimistic = version check = FAST, retry on conflict.

Page vs Slice — Spring Data JPA:

→ Page = extra COUNT query fire karta → totalElements, totalPages.

→ SLOW on large tables.

→ Slice = sirf hasNext() batata → total count NAHI. FAST.

```

Infinite scroll (mobile) → Slice (next hai? load karo)
Page numbers dikhane → Page (total chahiye)

```

Singleton + Prototype TRAP:

→ Singleton mein prototype inject karo → prototype bhi EK baar bane.

→ Naya KABHI nahi.

```

Fix 1: ObjectFactory<B> factory; factory.getObject() → naya har baar
Fix 2: @Lookup method → Spring override kare → naya return
Fix 3: ApplicationContext.getBean() → manual

```

[TRAP]

→ Singleton ek baar bane = prototype bhi ek baar inject.

→ ObjectFactory/@Lookup se fix karo.

@ManyToMany — Join Table:

→ DB mein automatically JOIN TABLE banta — dono entities ka ID store.

```
@ManyToMany List<Course> courses;

DB tables:
student:      id, name
course:       id, title
student_course: student_id, course_id (AUTO)

Customize: @JoinTable(name="enrollment",
    joinColumns = @JoinColumn(name="sid"),
    inverseJoinColumns = @JoinColumn(name="cid"))
```

Java Memory — Stack vs Heap vs Metaspace:

Stack	Heap	Metaspace
Local variables	Objects (new)	Class metadata
Method calls	Arrays	Static variables
References	Strings (pool)	Method info
Thread-specific	GC managed	Java 8+ (was PermGen)
LIFO, fast	Shared	

→ int x = 5 → stack. new User() → heap. Class definition → metaspace.

→ Stack mein reference → heap mein object point kare.

Generics — PECS Rule:

→ Producer Extends, Consumer Super.

```
List<? extends Number> list;
list.get(0) → Number return OK (READ safe)
list.add(5) → COMPILER ERROR (kaunsa type?)

List<? super Integer> list;
list.add(5) → OK (Integer ya parent accept)
list.get(0) → Object return (specific nahi pata)
```

→ extends = safely READ. super = safely WRITE.

serialVersionUID:

→ Serialization ka VERSION CONTROL.

→ Class change hui + UID same → backward compatible (naye fields = null).

→ Class change hui + UID alag → InvalidClassException CRASH.

```
private static final long serialVersionUID = 1L; // manual = safe
```

[TRAP]

→ Bina UID: JVM auto generate. Class change = naya auto UID = CRASH.

→ Manual UID = control tere haath.

try-with-resources — Close Order:

→ REVERSE order — LIFO. Last opened = first closed.

```
try (FileReader fr = new FileReader(f); // opened 1st
    BufferedReader br = new BufferedReader(fr)) { // opened 2nd
    // use br
} // br closed FIRST, then fr
```

WHY reverse? br depends on fr. br pehle close = safe. Dependency order respect.

findFirst() vs findAny():

→ findFirst() = PEHLA element. Order maintain. Sequential mein use.

→ findAny() = KOI BHI. Parallel mein fast (whichever thread finds first).

→ Sequential mein dono same. Parallel mein findAny() faster.

Optional.of() vs ofNullable():

→ of(null) → NullPointerException CRASH!

→ ofNullable(null) → Optional.empty() SAFE.

```
Value PAKKA nahi null → Optional.of(value)
Value null HO SAKTA → Optional.ofNullable(value)
```

[TRAP] Service se data aaye → ofNullable. Constant → of.

Stream REUSE nahi kar sakte:

→ Stream ek baar use. Dobara → IllegalStateException.

```
Stream s = list.stream();
s.filter(...).collect(...); // OK
s.filter(...).collect(...); // IllegalStateException!
```

→ Dobara chahiye → `list.stream()` se NAYA banao.

`ExecutorService shutdown()` vs `shutdownNow()`:

→ `shutdown()` = naye reject, purane COMPLETE ho jaayein. Graceful.

→ `shutdownNow()` = sab TURANT roko. Running INTERRUPT. Emergency.

```
executor.shutdown();
if(!executor.awaitTermination(60, SECONDS)) {
    executor.shutdownNow(); // force after timeout
}
```

`CompletableFuture.exceptionally()`:

→ Async pipeline ka CATCH block. Exception aaye → fallback value do.

```
CompletableFuture.supplyAsync(() → getUser(1))
    .thenApply(user → user.getName())
    .exceptionally(ex → {
        log.error(ex.getMessage());
        return "Unknown"; // fallback
    });
```

401 vs 403 — Dono Access Deny:

→ 401 Unauthorized = KAUN HO TUM? Token nahi/expired/invalid. Login karo.

→ 403 Forbidden = JAANTA HU KAUN HO — lekin ALLOWED NAHI. Permission nahi.

```
401: No token / expired → "Login karo"
403: Valid token, role=VIEWER, tried DELETE → "Permission nahi"
```

Spring Boot Starter — Kya Karta:

→ CURATED dependency bundle. Ek starter = related 10 jars auto aayein.

```
spring-boot-starter-web → MVC + Tomcat + Jackson + Validation
spring-boot-starter-data-jpa → Hibernate + JPA + HikariCP + JDBC
```

→ Bina starter: 10 dependencies manually + version conflict.

→ Starter = tested, compatible, ek line.

`ClassCastException` — Kab Aata:

→ Runtime pe galat downcast. Compile time nahi roke.

```
Animal a = new Cat();
Dog d = (Dog) a; // RUNTIME: Cat is not Dog → ClassCastException!

Fix: instanceof check pehle
if(a instanceof Dog d) { d.bark(); } // Java 16+ pattern matching
```

`ResponseExceptionHandler` extend karo:

→ `@RestControllerAdvice` mein extend karo

→ Spring built-in exceptions (404, 405, 415) auto handle.

→ Bina extend: ugly default HTML error. Extend: consistent JSON sab pe.

```
@RestControllerAdvice
class GlobalHandler extends ResponseExceptionHandler {
    // Built-in: 404, 405, 415 auto handle with proper status
    // Custom: @ExceptionHandler(ResourceNotFoundException.class)
}
```

Enum Utility Methods:

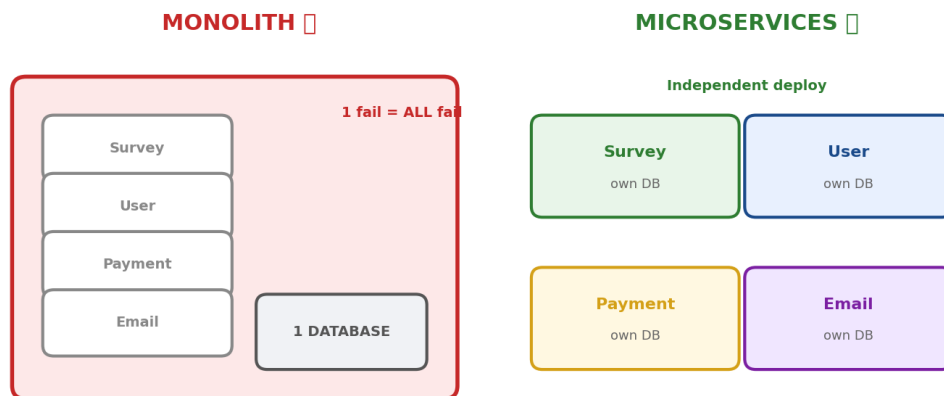
```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

Day.values() → [MON, TUE, WED, ...] (sab constants array)
Day.valueOf("MON") → Day.MON (string → enum)
Day.MON.name() → "MON" (enum → string)
Day.MON.ordinal() → 0 (index position)
```

[TRAP] `valueOf()` mein galat string → `IllegalArgumentException`. Case sensitive hai.

Microservices — Distributed Architecture

Monolith vs Microservices — WHY Split?



▲ Monolith: 1 WAR, 1 DB, 1 deploy = ek change sab affect. Microservices: alag services, alag DB, independent deploy.

- Monolith = ek WAR file, ek database, ek deploy. Sab modules EK application mein.
- Problem 1 — Ek fail = sab fail. Payment module mein bug? Poora app down. Survey bhi band. Email bhi band.
- Problem 2 — Scale nahi hota. Survey pe 10,000 users ka load, Payment pe 100. Lekin POORA app scale karna padega — resources waste.
- Problem 3 — Deploy = nightmare. Chhota email template fix? Poora app redeploy. 30 min downtime.

→ WHY Microservices? Monolith CHHOTI team ke liye BEST — simple, ek jagah sab. Lekin jab app BADA ho (50+ modules, 10+ developers) — monolith TODHNA padta. Ye evolution hai, revolution nahi.

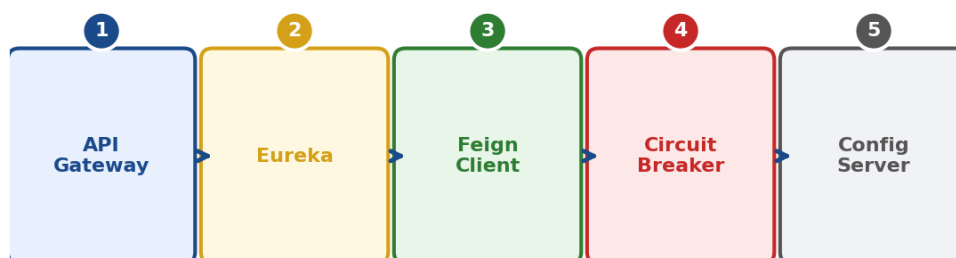
- Microservices = har module ALAG Spring Boot app, ALAG database, ALAG deploy, ALAG port.
- Services ek doosre se HTTP/REST ya messaging se baat karte hain.
- Payment down? Sirf payment band. Survey chalta. Email chalta. Fault isolation.
- Survey pe load? Sirf Survey scale karo (3 instances). Payment ko chhedo mat.
- Email fix? Sirf email service deploy. 0 downtime. Baaki UNTOUCHED.

→ WHY alag database? Har service ka data USKA hai. Survey ka schema badle toh Payment ko PATA nahi chalna chahiye. Database per service = loose coupling ka foundation.

→ [TRAP] "Microservices always better than monolith" = GALAT. Chhoti team, naya product = monolith BEST. Micro add karte: network latency, deployment complexity, debugging difficulty. "It depends on scale, team size, complexity."

5 Building Blocks — Microservices Kaise Kaam Karte

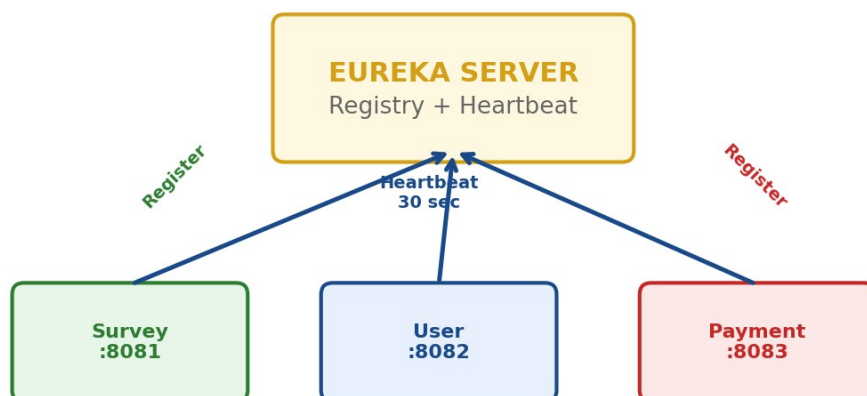
5 Building Blocks



▲ 5 blocks: Gateway (entry), Eureka (discover), FeignClient (call), Circuit Breaker (protect), Config Server (central config).

- Sirf services banane se nahi hota — 5 cheezein chahiye saath mein:

- 1. API Gateway — single entry point. Routing, Auth, Rate limit, Logging.
- 2. Service Discovery (Eureka) — address book. Kaun kahan chal raha, dynamically register/discover.

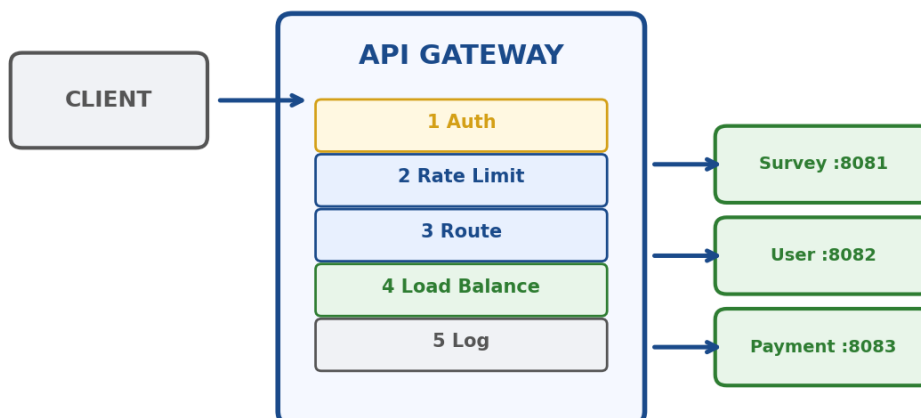


▲ 5 blocks: Gateway (entry), Eureka (discover), FeignClient (call), Circuit Breaker (protect), Config Server (central config).

- 3. FeignClient — service-to-service call. Interface likho, Spring call kare.
- 4. Circuit Breaker (Resilience4j) — fuse protection. Down hai? Fallback do.
- 5. Config Server — central config. 15 services ki settings ek jagah.

→ **WHY ye 5?** Bina Gateway = security hole. Bina Discovery = hardcoded URLs. Bina Circuit Breaker = cascading failure. Bina Config = 15 jagah properties change. Ye **PROBLEMS** solve karte hain, fancy nahi hain.

API Gateway — Single Entry Point



▲ Client → Gateway (auth + rate limit + route) → lb://SERVICE → Eureka se IP → Service. Single entry point, cross-cutting.

- Building ka MAIN GATE jaisa. Sab request pehle Gate pe aaye. Guard check kare (auth), phir bataye kahan jaana.
- Client ko sirf EK URL pata: `https://api.iris.com`. Andar 15 services — client ko kuch nahi pata.
- Gateway ke 5 kaam:
- 1. ROUTING — `/api/surveys/**` → Survey Service, `/api/users/**` → User Service.
- 2. AUTHENTICATION — JWT token check EK jagah. Token valid? Aage bhejo. Invalid? 401.

- 3. RATE LIMITING — ek user max 100 req/min. Zyada? 429 Too Many Requests.
- 4. LOAD BALANCING — Survey ke 3 instances. Round Robin baante — 1→2→3→1.
- 5. LOGGING — har request log. Kab aayi, kahan gayi, kitna time laga.

→ WHY ek jagah? Cross-cutting concerns (auth, logging, rate limit) har service mein chahiye. 15 jagah copy-paste = DRY violation. Gateway pe ek baar = clean. Spring Security filter chain yaad hai? Gateway = POORE system ka filter chain.

→ Request flow order: Auth → Rate Limit → Route → Load Balance → Log.

→ WHY ye order? Auth PEHLE — invalid token toh route hi mat karo (resource waste). Rate limit DOOSRA — authenticated user bhi spam kar sakta. Route TEESRA — ab pata kahan jaana.

```
spring:
  cloud:
    gateway:
      routes:
        - id: survey-service
          uri: lb://SURVEY-SERVICE
          predicates:
            - Path=/api/surveys/**
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/api/users/**
```

→ lb:// = Load Balanced. Eureka se service dhundho, instances mein distribute karo.

→ predicates = condition. Path match kare toh us service pe route karo.

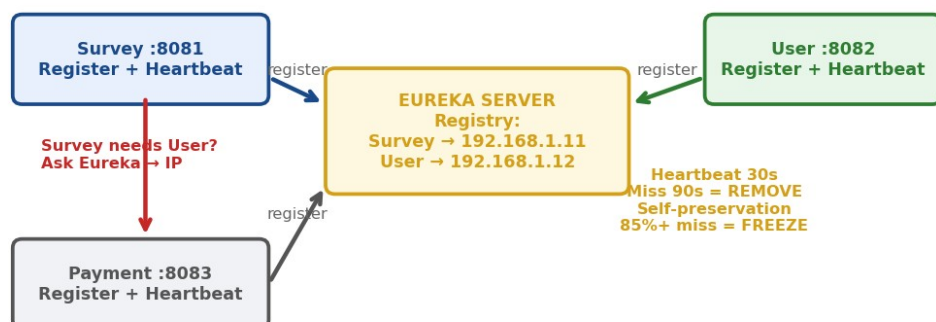
→ [TRAP] Spring Cloud Gateway = WebFlux based (reactive, non-blocking). Spring MVC (servlet) nahi. Isliye Mono/Flux return karta. Interview mein poochhein "Gateway WebFlux kyun?" — Non-blocking = 1000s concurrent connections bina thread block kiye. Gateway pe SABSE zyada traffic aata.

→ [TRAP] Gateway single point of failure? Fix: multiple gateway instances behind cloud load balancer (AWS ALB). Gateway STATELESS — JWT validate, kuch store nahi. 3 instances = ek down toh baaki handle.

→ FLASH: "API Gateway is the single entry point that handles routing, authentication, rate limiting, and logging for all microservices. At IRIS, we use AWS API Gateway to route requests across 15+ services serving 10 countries."

Service Discovery (Eureka) — Address Book

Eureka — Service Discovery Flow



▲ Services register at Eureka. Need IP? Ask Eureka. Heartbeat 30s, miss 90s = remove. Self-preservation 85%+ miss = freeze.

→ Contacts app jaisa — services NAAM se dhundho, IP yaad mat rakho.

→ Bina Discovery: IP hardcode. Service move ho = sab todh. Raat 2 baje call aaye.

→ 3-step process:

- REGISTER — Service start hone pe Eureka ko bole "main SURVEY-SERVICE hoon, mera address ye hai." Automatic.
- DISCOVER — Gateway poochhe "SURVEY-SERVICE kahan?" Eureka list de (3 instances). Gateway ek choose kare.
- HEARTBEAT — Har 30 sec service Eureka ko bole "main alive hoon." 90 sec no heartbeat = DEAD mark, registry se remove.

→ **WHY automatic? Instance add/remove = AUTOMATICALLY register/deregister. Koi manual config nahi. Dynamic scaling — yehi microservices ka power hai.**

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApp { }
// Port: 8761 (default)

// Client — har service mein:
spring:
  application:
    name: SURVEY-SERVICE
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

→ Server: @EnableEurekaServer + port 8761. Client: dependency + application.name + eureka URL. Bas.

→ **[TRAP] spring.application.name = EXACTLY same naam har jagah — Gateway routing (lb://SURVEY-SERVICE), FeignClient (@FeignClient("SURVEY-SERVICE")). Naam galat = service nahi milegi.**

- Self-preservation mode: 85%+ heartbeat achanak miss = NETWORK issue, services dead nahi.
- Action: Registry FREEZE — kisi ko mat hatao. Stale data better than empty registry.
- Development mein OFF karo (testing ke liye). Production mein ALWAYS ON.
- Eureka HA: Peer-to-peer replication — 2-3 instances pointing to each other. Ek down toh doosra serve.
- Clients LOCALLY registry cache rakhte — Eureka down bhi ho toh cached addresses se communicate.

→ **WHY self-preservation? Wrong info (sab dead) se ZYADA damage compared to stale info (purane addresses). Network theek hote hi heartbeats wapas aayengi. Conservative approach.**

→ **FLASH: "We use service discovery so microservices register by name. When we scale survey to 3 instances, gateway automatically discovers all via Eureka and load balances. No manual config — services come and go dynamically."**

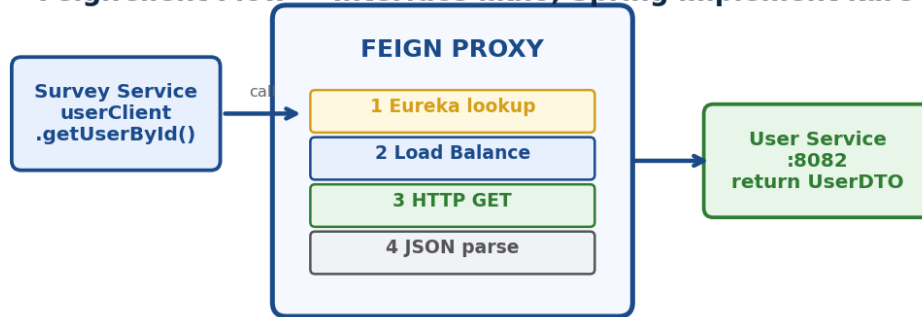
IRIS Platform — Tera Real Experience

- Tu ALREADY microservices mein kaam karta hai — bas NAAM nahi jaanta tha:
- SightX API call = inter-service communication (FeignClient concept).
- CloudWatch logs across services = Distributed Tracing.
- HikariCP bug — connection pool exhaust = Circuit Breaker hota toh cascade nahi hota.
- 15+ microservices IRIS mein deploy. 10+ countries. 500+ cross-service incidents debug.
- Survey re-invitation mein 2 databases = database per service pattern.
- AWS API Gateway = API Gateway. AWS service URLs = Service Discovery concept.

→ **WHY important? Interview mein theory nahi bolna — "IRIS mein 15+ services, SightX integration, 2 databases, CloudWatch monitoring, 500+ incidents debug kiye across services." REAL experience beats textbook.**

FeignClient — Declarative REST Client

FeignClient Flow — Interface likho, Spring implement kare



Interface likho → Spring proxy banaye → Eureka se dhundhe → Load balance → HTTP call → Auto parse

JPA Repository jaisa — DECLARATIVE. Tu KYA bata, Spring HOW kare.

▲ Interface likho → Spring PROXY banaye → method call → Eureka lookup → load balance → HTTP call → JSON parse → return object.

- FeignClient = Interface likho, Spring implement kare. JPA Repository jaisa — DECLARATIVE.
- RestTemplate = tu URL banao, call karo, parse karo, error handle — sab manually. IMPERATIVE.
- FeignClient = tu sirf bata KYA chahiye (interface + annotations). Spring kaise kare wo decide kare.

→ WHY: FeignClient KYUN? RestTemplate mein 10 services call = 10 jagah SAME boilerplate. FeignClient mein 10 clean interfaces — consistent, less bugs. Eureka integration built-in, load balancing FREE.

- Step 1: Dependency — spring-cloud-starter-openfeign
- Step 2: @EnableFeignClients main class pe lagao
- Step 3: Interface banao —

```

@FeignClient(name = "USER-SERVICE")
public interface UserClient {
    @GetMapping("/api/users/{id}")
    UserDTO getUserById(@PathVariable Long id);
}
  
```

- Step 4: Inject + use — userClient.getUserById(123) — bas EK line.
- name = "USER-SERVICE" = Eureka mein registered naam. Feign Eureka se IP dhundhe.
- @GetMapping = WAHI annotations jo @RestController mein. Same syntax!
- Return type = auto deserialize. JSON → Java object. Jackson handle kare.
- Internally: @EnableFeignClients scan kare → har @FeignClient ka DYNAMIC PROXY banaye → method call pe proxy HTTP call kare → Eureka se IP → load balance → response parse.

→ WHY: Proxy pattern PHIR SE! @Transactional = proxy (txn). @FeignClient = proxy (HTTP). @Cacheable = proxy (cache). Spring LOVES proxy — tera code CLEAN rahe, cross-cutting concerns proxy handle kare.

```

feign:
  client:
    config:
      USER-SERVICE:
        connectTimeout: 5000
        readTimeout: 10000
  
```

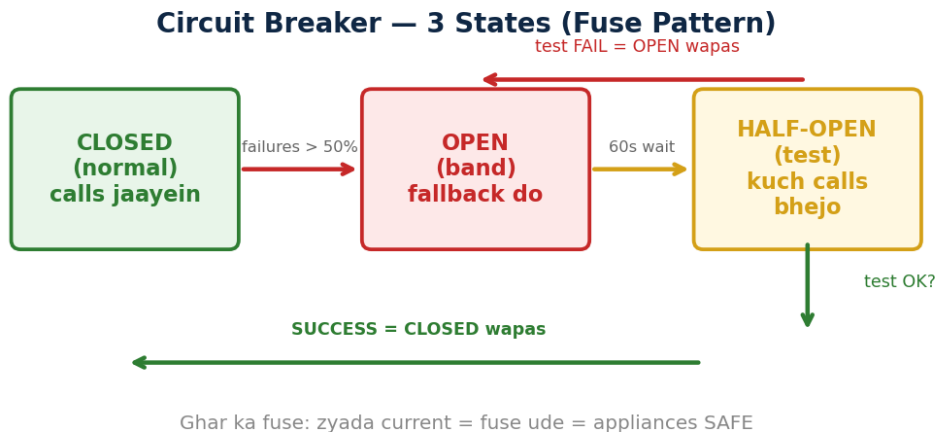
→ [TRAP] @EnableFeignClients BHOOL gaye = beans nahi banenge = "No qualifying bean" error. YAAD se lagao.

→ [TRAP] Default timeout BAHUT lamba (60 sec). Production mein SET karo — warna slow service pe thread block = pool exhaust = cascading failure. Tera HikariCP bug jaisa.

→ [TRAP] RestTemplate deprecated NAHI lekin maintenance mode mein. Interview: "FeignClient for sync calls, WebClient for reactive."

→ **FLASH: "FeignClient is a declarative HTTP client — interface with Spring MVC annotations, Spring creates proxy at runtime. Auto-discovers via Eureka, load balances, handles serialization. At IRIS, 15+ services communicate via REST — FeignClient makes these calls clean and maintainable."**

Circuit Breaker — Fault Tolerance (Fuse Pattern)



▲ CLOSED (normal, calls jaayein) → failures > 50% → OPEN (band, fallback) → 60s wait → HALF-OPEN (test) → pass? CLOSED. Fail? OPEN.

- Circuit Breaker = software ka FUSE. Service fail ho rahi? Call BAND karo, fallback do. Baaki system SAFE.
- Bina CB: User Service down → Survey 60 sec wait → threads block → Survey bhi down → Payment bhi down = CASCADING FAILURE.
- Tera HikariCP bug = EXACTLY ye pattern. Pool exhaust → dependent services hang → domino effect.

→ WHY: Circuit Breaker KYUN? Ek service fail pe POORA system mat girao. Early fail + fallback = resources safe. Ghar ka fuse — zyada current = fuse ude = appliances SAFE.

- 3 States:
- CLOSED (normal) — calls jaayein. Har call ka result track (success/fail). Failures < threshold = sab theek.
- OPEN (band) — failures > 50%. Call hi NAHI karega. Seedha FALLBACK return. Thread block NAHI = fast fail.
- HALF-OPEN (test) — 60 sec wait ke baad. 3 test calls bhejo. Pass? → CLOSED wapas. Fail? → OPEN wapas.

→ WHY: WHY HALF-OPEN? Seedha CLOSED karo toh service abhi bhi down = phir fail = phir OPEN. HALF-OPEN = slowly test. Gradual recovery, shock nahi.

- Resilience4j — Spring Cloud default library (Hystrix DEPRECATED):

```
resilience4j:
  circuitbreaker:
    instances:
      userService:
        failureRateThreshold: 50
        slidingWindowSize: 10
        waitDurationInOpenState: 60s
        permittedNumberOfCallsInHalfOpenState: 3
```

- Code — @CircuitBreaker annotation:

```
@CircuitBreaker(name = "userService",
                fallbackMethod = "getUserFallback")
public UserDTO getUser(Long userId) {
    return userClient.getUserById(userId);
}

public UserDTO getUserFallback(Long userId, Throwable ex) {
    return new UserDTO(userId, "Unavailable", "Down");
}
```

- FeignClient + CB combo = BEST practice:
- CLOSED → Feign call normal. OPEN → call hi nahi, seedha fallback. HALF-OPEN → test calls.
- FallbackFactory > simple fallback — exception milti, log karo, specific error handle karo.

```
@FeignClient(name = "USER-SERVICE",
            fallbackFactory = UserFallbackFactory.class)
public interface UserClient { ... }
```

→ [TRAP] Hystrix DEPRECATED (2018 se maintenance mode). Resilience4j = modern replacement. Interview mein: "Resilience4j is Spring Cloud default, Hystrix is legacy."

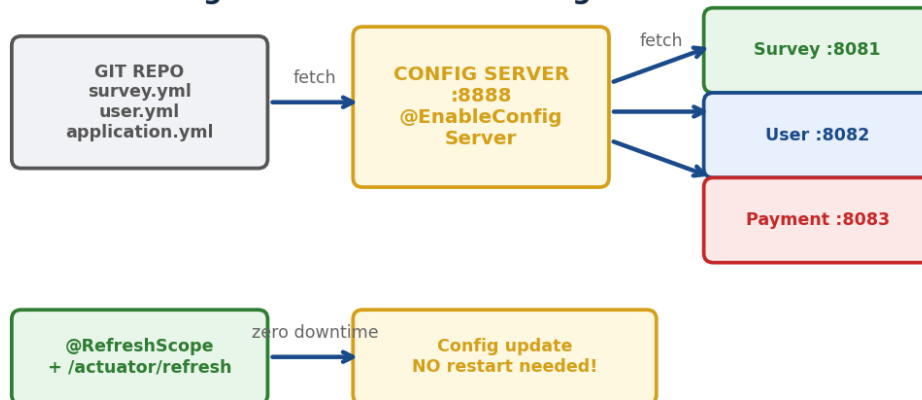
→ [TRAP] fallbackMethod signature EXACT match hona chahiye — same parameters + Throwable last. Mismatch = NoSuchMethodException.

→ [TRAP] CB vs Retry: Retry = temporary failure (2-3 baar try). CB = sustained failure (band karo). COMBINE karo: Retry 3x → still fail → CB threshold cross → OPEN.

→ **FLASH: Circuit Breaker = fault tolerance. Service fail → call band, fallback do. 3 states: CLOSED, OPEN, HALF-OPEN. Resilience4j + FeignClient + FallbackFactory. HikariCP bug mein ye chahiye tha — cascade rokta.**

Config Server — Central Config Management

Config Server — Central Config for All Services



Git mein change + refresh call = LIVE update. 15 services, 0 redeploy.

▲ Git repo → Config Server (:8888) fetch → Services startup pe fetch. @RefreshScope + /refresh = zero downtime update.

→ Config Server = HR department jaisa. 500 employees ki info HR ke paas CENTRALIZED. Policy badle? HR update, sab ko apply.

→ 15 services × 4 environments = 60 config files. DB URL badle = 15 jagah edit + 15 redeploy. Config Server = EK jagah.

→ **WHY: Config Server KYUN? Config SCATTERED = maintenance nightmare. Central repo = ek jagah badle, sab services ko apply. Git-backed = version control + audit trail + PR review. History mile — kaun badla, kab, kyun.**

→ Flow: Git repo mein config files → Config Server fetch kare → Services startup pe Config Server se fetch.

→ URL pattern: GET /{application}/{profile} — /survey-service/prod = survey ka prod config.

→ application.yml = SAB services common (JWT secret, Redis URL). survey-service.yml = service-specific.

→ Config Server setup:

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApp { }

// application.yml
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/team/config-repo
```

→ Client setup (har service mein):

```
spring:
  application:
    name: survey-service
```

```
config:
  import: optional:configserver:http://localhost:8888
profiles:
  active: prod
```

- spring.application.name = PHIR WAHI naam! Config Server isi naam se file dhundhe. Eureka, Gateway, FeignClient — sab jagah SAME naam.
- Dynamic Refresh — bina restart config update:
- @RefreshScope annotation bean pe lagao + POST /actuator/refresh call = bean RE-CREATE naye config ke saath. ZERO downtime.
- Spring Cloud Bus + RabbitMQ = 1 call, SAB instances update. 15 instances pe ek ek refresh nahi — broadcast.
- Environments: dev = local DB, debug ON. staging = staging DB. prod = production DB, real services. SAME code, DIFFERENT config.

→ WHY: WHY no restart? Production mein 15 services restart = DOWNTIME. @RefreshScope + /refresh = Git mein change, refresh call, LIVE update. CI/CD ka part.

→ [TRAP] Config Server DOWN at startup = service START NAHI HOGI. Fix: optional: prefix lagao + local fallback config rakh.

→ [TRAP] @RefreshScope ke bina refresh pe PURANI value rahegi. @RefreshScope = re-create bean with new config. Bhoolna = stale config.

→ [TRAP] Secrets Git mein PLAINTEXT mat rakho. Spring Cloud Vault, AWS Secrets Manager, ya encrypted properties use karo. Git history mein password = SECURITY breach.

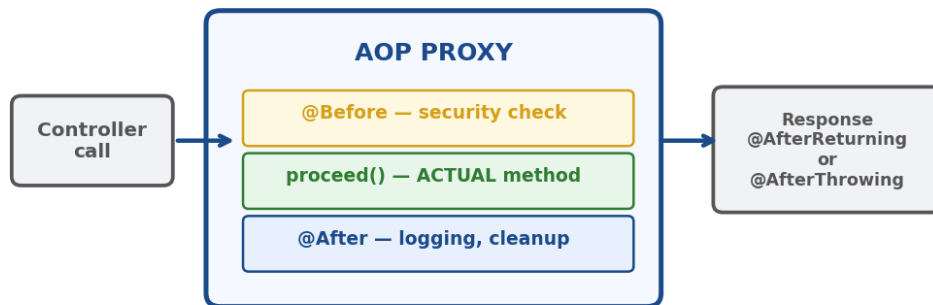
→ Alternatives: Consul (config + discovery dono), AWS Parameter Store + Secrets Manager (IRIS ke liye fit — AWS pe ho), K8s ConfigMaps.

→ **FLASH: Config Server = central config, Git-backed, version control + audit. @RefreshScope = zero downtime update. At IRIS, AWS Parameter Store serves similar purpose for 15+ services.**

Spring Extras — Interview Must-Know

Spring AOP — Cross-Cutting Concerns

Spring AOP — Proxy Intercept Flow



@Transactional, @Cacheable, @PreAuthorize, @CircuitBreaker = sab AOP internally

▲ Controller call → AOP PROXY intercept → @Before (security) → proceed() (actual method) → @After (logging) → Response.

→ AOP = Aspect-Oriented Programming. Cross-cutting concerns (logging, security, txn) ko business code se ALAG karo.

→ Problem: 50 methods mein logging + security + txn = 300 lines boilerplate copy-paste. DRY violation.

→ Solution: @Aspect class mein likho. Spring PROXY se automatically har matching method pe apply kare.

→ WHY: AOP KYUN? Business method mein sirf BUSINESS likho. Logging, security, transaction = AOP handle kare. Separation of Concerns ka ULTIMATE form.

→ 5 Key Terms:

→ Aspect = WO class jisme cross-cutting logic hai. @Aspect + @Component annotation.

→ Advice = KAB execute karna. @Before (pehle), @After (baad hamesha), @AfterReturning (success pe), @AfterThrowing (exception pe), @Around (dono = MOST POWERFUL).

→ Pointcut = KAHAN lagao. Expression se define. execution(* com.iris.service.*(..)) = service ke sab methods.

→ JoinPoint = WO specific point jahan advice execute ho. Method call = joinpoint.

→ Weaving = Aspect code ko actual code mein merge karna. Spring = RUNTIME weaving (proxy).

→ Code — Logging Aspect:

```
@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* com.iris.service.*(..))")
    public Object logTime(ProceedingJoinPoint pjp)
        throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed(); // ACTUAL method
        long time = System.currentTimeMillis() - start;
        log.info("{} took {}ms", pjp.getSignature(), time);
        return result;
    }
}
```

→ @Around = pjp.proceed() se PEHLE + BAAD dono mein code. Most powerful advice.

→ Pointcut expression: execution(* com.iris.service.*(..)) = any return, any class, any method, any args.

→ @annotation(Transactional) = jis bhi method pe @Transactional hai wahan apply karo.

→ Tu ALREADY AOP use karta hai:

→ @Transactional = @Around advice. Proxy: txn begin → proceed → success? commit. Exception? rollback.

→ @Cacheable = AOP. Method call se pehle cache check. Hit? Return cached. Miss? Proceed, cache result.

→ @PreAuthorize = AOP. Method call se pehle role check. Allowed? Proceed. Denied?

AccessDeniedException.

→ @CircuitBreaker = AOP. Method call pe failure track. Threshold cross? Fallback.

→ WHY: AOP = Spring ka BACKBONE. @Transactional, @Cacheable, @Async, @PreAuthorize, @CircuitBreaker — sab internally AOP. Ek concept samjho = 10 features clear.

→ Internally: JDK Dynamic Proxy (interface implement) ya CGLIB (class extend = subclass). Runtime pe proxy create, method call intercept, advice execute.

→ [TRAP] @Aspect lagaya lekin @Component nahi = bean NAHI banega = AOP kaam NAHI karega. Dono ZARURI.

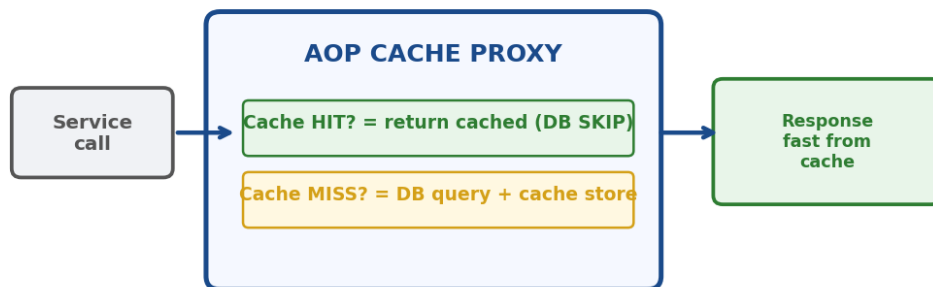
→ [TRAP] @Around mein proceed() call bhool gaye = actual method EXECUTE hi NAHI hoga. Silently skip.

→ [TRAP] Same class internal call = proxy BYPASS = AOP dead. @Transactional wahi problem — alag class mein rakho.

→ **KEY: AOP = Spring ka backbone. Cross-cutting concerns alag, business code clean. @Transactional, @Cacheable, @PreAuthorize = sab AOP. Proxy based — internal call = bypass.**

@Cacheable — Spring Cache Abstraction

@Cacheable Flow — AOP Proxy Cache Check



@Cacheable = read. @CacheEvict = delete. @CachePut = update. Redis = production.

▲ Service call → Cache PROXY → HIT? Return cached (DB skip). MISS? DB query + cache store. @CacheEvict = delete. @CachePut = update.

→ **WHY: Cache KYUN?** Same data 1000 baar DB se = 1000 queries = waste. DB query = 5-50ms. Memory = nanoseconds. Cache = pehli baar DB, baad mein memory. 1000x FASTER. DB load kam, response fast.

→ Cache kab USE: Data frequently READ lekin rarely CHANGE (user profile, config, country list). DB query EXPENSIVE (complex joins).

→ Cache kab NAHI: Data har request pe CHANGE (stock price). Data SENSITIVE aur stale nahi hona chahiye (bank balance).

→ @EnableCaching main class pe lagao + @Cacheable method pe:

```
@Cacheable(value = "users", key = "#id")
public UserDTO getUserById(Long id) {
    return userRepo.findById(id);
}
// Pehli baar: DB query + cache store
// Doosri baar: seedha cache se, DB hit ZERO
```

→ value = "users" = cache naam. key = "#id" = cache key. Internally: AOP PROXY — cache check pehle, miss pe execute + store.

→ @CacheEvict — cache HATAO (delete/update pe):

```
@CacheEvict(value = "users", key = "#id")
public void deleteUser(Long id) { ... }
```

→ @CachePut — cache UPDATE karo (write pe). Method HAMESHA execute + result cache mein UPDATE:

```
@CachePut(value = "users", key = "#user.id")
public UserDTO updateUser(UserDTO user) { ... }
```

→ Fark: @Cacheable = READ pe (hit? return, miss? execute+store). @CachePut = WRITE pe (hamesha execute+update). @CacheEvict = DELETE pe (cache se hatao).

→ Redis = production cache. Local ConcurrentHashMap = single JVM only. Redis = shared (sab instances), persistent, TTL (auto expire), sub-millisecond.

→ IRIS mein: survey config cache, user session Redis mein, OTP store (5 min TTL), rate limiting.

→ **WHY: Redis KYUN local nahi?** Microservices = multiple instances. Local cache = har instance ka ALAG = inconsistency. Redis = CENTRAL cache, sab share = single source of truth.

→ [TRAP] @EnableCaching bhool gaye = @Cacheable silently ignore. No error, no cache.

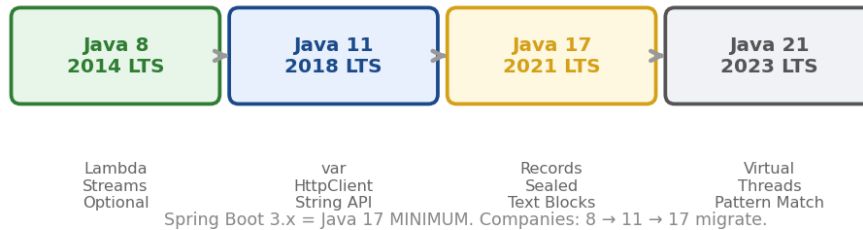
→ [TRAP] Same class internal call = proxy BYPASS = cache DEAD. @Transactional wahi problem — alag class mein rakho.

→ [TRAP] Cache invalidation = hardest CS problem. Data badle toh cache bhi badle. Bhool gaye = STALE data. TTL + explicit evict = safe strategy.

→ **KEY: @Cacheable = AOP proxy, DB hit bachao. @CacheEvict = stale hatao. Redis = production (shared, TTL). Cache invalidation = TTL + explicit evict.**

Java 11/17 Features — Modern Java

Java Version Timeline — LTS Versions



▲ Java 8 (Lambda, Streams) → Java 11 LTS (var, HttpClient) → Java 17 LTS (Records, Sealed, Text Blocks) → Java 21 LTS (Virtual Threads).

→ **WHY: Java 11/17 KYUN?** Spring Boot 3.x = Java 17 MINIMUM. Companies 8 → 11 → 17 migrate. Interview: "Java 8 ke baad kya naya?" = ye 6 features batao.

→ Java 11 (LTS): var (local type inference), String methods (strip, isBlank, repeat, lines), Files.readString, HttpClient built-in.

```
var names = new ArrayList<String>(); // compiler INFER
"hello".strip() // "hello" (Unicode-aware)
"hello".repeat(3) // "hellohellohello"
"".isBlank() // true
Files.readString(Path.of("file.txt")) // 1 line read
```

→ Java 17 (LTS) — 6 KEY features:

→ 1. Records — 1 line DTO, immutable. Auto constructor + getters (name() NOT getName()) + equals + hashCode + toString.

```
public record UserDTO(String name, int age) {}
// = 40+ lines boilerplate gone. Lombok @Data ka alternative.
```

→ 2. Sealed Classes — kaun extend kare TU decide. permits ke baad jo listed = sirf wahi.

```
public sealed class Shape permits Circle, Rectangle {}
// Square extends Shape = COMPILER ERROR!
```

→ 3. Text Blocks — multi-line strings. JSON, SQL, HTML readable. Triple quotes """".

```
String json = """
    { "name": "Arpan", "age": 25 }
    """;
```

→ 4. Pattern Matching instanceof — cast BUILT-IN. if(obj instanceof String s) { s.length(); }

→ 5. Switch Expressions — arrow syntax, no break. String result = switch(day) { case MON -> "Work"; };

→ 6. Helpful NullPointerException — exact field/method naam bataye jo null tha.

→ [TRAP] var = LOCAL variables only. Fields, params, return mein NAHI. var x = null INVALID. Readability bigde toh explicit type likho.

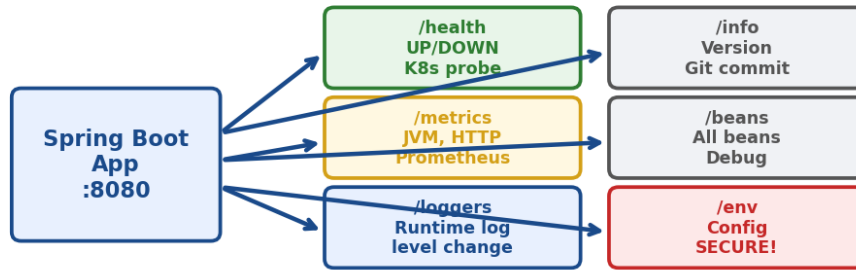
→ [TRAP] Record mein getter = name() hai, getName() NAHI. Record extend nahi kar sakte (implicitly final). Mutable fields NAHI.

→ IRIS currently Java 11. Migration plan 17 pe. Records for DTOs, Text Blocks for SQL, Spring Boot 3 = 17 minimum.

→ **KEY: Java 11 = var, String API, HttpClient. Java 17 = Records, Sealed, Text Blocks, Pattern Matching, Switch Expressions. Spring Boot 3 = 17 minimum.**

Spring Boot Actuator — Production Monitoring

Spring Boot Actuator — Production Monitoring



/health = Load Balancer + K8s. /metrics = Prometheus + Grafana. Secure karo production mein!

▲ /health (UP/DOWN, K8s probe), /metrics (JVM, Prometheus), /loggers (runtime change), /info, /beans, /env. Secure karo production mein!

→ WHY: Actuator KYUN? Production mein app health kaise check? Manual SSH = pain. Actuator = READY-MADE endpoints. K8s liveness probe, Prometheus, Load balancer — sab /health use karte.

→ /health = app UP ya DOWN. DB, Redis, disk sab check. K8s liveness/readiness probe ye use karta.
 → /metrics = JVM memory, thread count, HTTP request count, response times. Prometheus + Grafana integrate.
 → /info = app version, git commit. /beans = sab Spring beans. /loggers = RUNTIME pe log level change bina restart!
 → /env = environment properties. /threaddump = thread snapshot (deadlock debug). /mappings = URL → controller mapping.
 → Setup: spring-boot-starter-actuator dependency + yml mein expose:

```

management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics, loggers
    endpoint:
      health:
        show-details: always
  
```

→ Custom Health Indicator: HealthIndicator implement karo — SightX API check, payment gateway check. /health mein auto dikhe.

→ [TRAP] /actuator/* endpoints PUBLIC mat karo production mein! /env mein DB password, /beans mein internal structure. Security lagao.

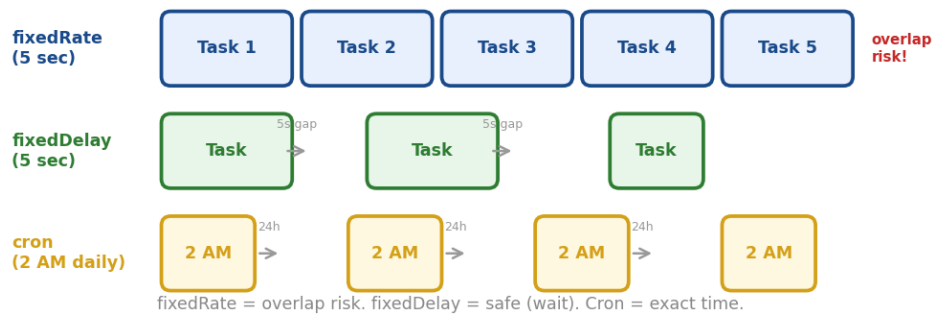
→ [TRAP] include: "*" SIRF development mein. /shutdown enable karo toh koi bhi POST se app BAND kar sakta!

→ IRIS mein: CloudWatch health checks = Actuator /health jaisa. Logs CloudWatch. Metrics New Relic/Datadog.

→ KEY: Actuator = production monitoring built-in. /health = K8s + LB. /metrics = Prometheus. /loggers = runtime debug. Secure karo!

@Scheduled — Cron Jobs in Spring

@Scheduled — fixedRate vs fixedDelay vs Cron



▲ fixedRate = har 5 sec (overlap risk). fixedDelay = pichla complete ke baad (safe). Cron = exact time (roz 2 AM).

→ WHY: @Scheduled KYUN? Report roz raat 2 baje, token cleanup har ghante, inactive users check — MANUAL nahi, automatic. Spring ka built-in scheduler.

```
@SpringBootApplication
@EnableScheduling // ye lagao!
public class App { }
```

→ fixedRate = 5000 — har 5 sec pe chale (overlap risk). fixedDelay = 5000 — pichla COMPLETE ke 5 sec baad (safe for heavy tasks).

→ Cron = "0 0 2 * * *" — roz raat 2 AM. Format: sec min hr day month weekday. "0 */15 * * * *" = har 15 min.

```
@Scheduled(fixedRate = 5000) // har 5 sec
public void checkHealth() { }

@Scheduled(fixedDelay = 5000) // complete ke 5 sec baad
public void processQueue() { }

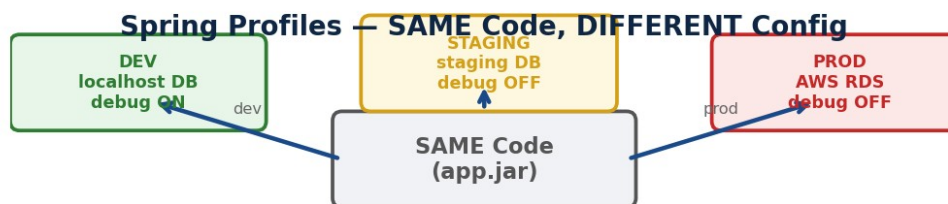
@Scheduled(cron = "0 0 2 * * *") // roz 2 AM
public void generateReport() { }
```

→ [TRAP] @EnableScheduling bhool gaye = kuch nahi chalega. Silently ignore.

→ [TRAP] Default SINGLE thread scheduler. 2 methods same time = ek wait. Fix: ThreadPoolTaskScheduler configure (pool size > 1).

→ KEY: @EnableScheduling + @Scheduled. fixedRate = light. fixedDelay = heavy. Cron = exact time. Single thread default.

Spring Profiles — Environment Config



spring.profiles.active = dev/staging/prod. Config switch, code ZERO change.

▲ SAME code (app.jar) → DEV (localhost, debug ON) / STAGING / PROD (AWS RDS, debug OFF). spring.profiles.active switch karo.

→ WHY: Profiles KYUN? Dev mein localhost DB, prod mein AWS RDS. SAME code, DIFFERENT config. Code change NAHI, config switch.

```
// application-dev.yml
spring.datasource.url: jdbc:mysql://localhost:3306/iris_dev
logging.level.root: DEBUG
```

```
// application-prod.yml
spring.datasource.url: jdbc:mysql://prod-rds:3306/iris
logging.level.root: WARN

// Activate: spring.profiles.active=prod
// Ya: java -jar app.jar --spring.profiles.active=prod
```

→ @Profile("dev") annotation — bean level control. Dev mein FakeEmailService, Prod mein RealEmailService. SAME interface, DIFFERENT implementation.

→ Priority: application-prod.yml > application.yml. Profile-specific OVERRIDE kare common config.

→ [TRAP] Profile set nahi kiya = default chale. Production mein HAMESHA explicitly set karo. Galti se dev prod pe = debug logs + fake services!

→ [TRAP] Secrets yml mein + Git push = EXPOSED. Environment variables ya Vault use karo.

→ **KEY:** application-{profile}.yml. @Profile = bean control. spring.profiles.active = switch. Secrets env vars mein, yml mein nahi.

Logging — SLF4J + Logback

SLF4J Log Levels — LOW to HIGH severity



Level=WARN set karo → sirf WARN + ERROR dikhe. Production = INFO/WARN. Dev = DEBUG.

▲ TRACE < DEBUG < INFO < WARN < ERROR. Level=WARN set = sirf WARN+ERROR dikhe. Production=INFO. Dev=DEBUG.

→ WHY: Logging KYUN? System.out.println() = no levels, no file, no rotation, no pattern, synchronized (slow). SLF4J = facade, Logback = implementation. Built-in Spring Boot.

```
private static final Logger log =
    LoggerFactory.getLogger(UserService.class);
// Ya Lombok: @Slf4j class pe

log.debug("Fetching user {}", id);    // {} placeholder
log.info("User found: {}", user);
log.error("DB error for user {}", id, ex);
```

→ 5 Levels: TRACE (very detail) < DEBUG (dev) < INFO (business) < WARN (potential) < ERROR (failure). Set level = us se upar sab dikhe.

→ MDC (Mapped Diagnostic Context) — har request ka userId/traceld log mein. 1 user ke sab logs filter karo. Debugging POWERFUL.

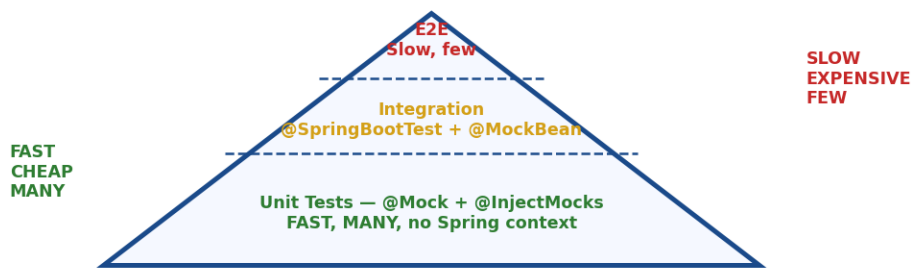
→ [TRAP] log.debug("User: " + user) = string concat HAMESHA even if debug OFF. log.debug("User: {}", user) = placeholder, sirf jab ON. Performance fark.

→ [TRAP] Sensitive data (passwords, tokens, PII) log NAHI karo. GDPR violation.

→ **KEY:** SLF4J facade + Logback impl. {} placeholder. 5 levels. MDC = request context. Sensitive data log NAHI. println() production mein NAHI.

Testing — JUnit + Mockito + SpringBootTest

Test Pyramid — Unit > Integration > E2E



▲ Unit (FAST, many, @Mock) > Integration (@SpringBootTest, medium) > E2E (slow, few). Pyramid: base = unit tests.

→ WHY: Testing KYUN? Code change kiya, kuch aur TODH diya — pata nahi. Tests = safety net. Production bug = expensive. Test = early catch.

→ Unit Test = ek class test, dependencies MOCK. FAST. @Mock + @InjectMocks + JUnit.

→ Integration Test = multiple layers, REAL beans. @SpringBootTest + @MockBean. Spring context load = SLOW.

→ @WebMvcTest = SIRF controller layer. Service/Repo NAHI load. MockMvc se test. Fastest integration.

```
// Unit Test
@Mock
private UserRepository userRepo;    // FAKE
@InjectMocks
private UserService userService;    // REAL

// RULE set: jab findById(1L) call ho → fakeUser de do
when(userRepo.findById(1L))
    .thenReturn(Optional.of(fakeUser));

// REAL service call — internally mock trigger ho
UserDTO result = userService.getUser(1L);

// CHECK — sahi kiya?
assertEquals("Arpan", result.getName());
verify(userRepo).findById(1L);
```

→ when().thenReturn() = RULE set (mock ko sikhao). Service call kare → internally mock trigger → fake data mile. assertEquals() = result check.

→ @Mock = plain Mockito (unit, no Spring). @MockBean = Spring context mein bean REPLACE (integration). @Mock FAST, @MockBean SLOW.

→ [TRAP] @SpringBootTest = FULL context = SLOW. Har test mein mat lagao. Unit = @Mock. Controller = @WebMvcTest. Full = @SpringBootTest.

→ KEY: Unit = @Mock (fast). Integration = @SpringBootTest (slow). AAA: Arrange-Act-Assert. when()=rule, service()=trigger, assertEquals()=check.

Spring Boot DevTools — Hot Reload

→ WHY: DevTools KYUN? Code change → 30 sec restart → test. 100 changes = 50 min WAIT. DevTools = 2 sec auto restart. Development FAST.

```
// pom.xml mein add — BAS!
spring-boot-devtools
scope = runtime, optional = true
```

→ Auto Restart — code save = 2 sec restart (classloader trick, full restart NAHI). LiveReload — browser auto refresh. Dev Defaults — caching OFF, show-sql ON.

→ Production safe: runtime scope + optional=true = production JAR mein include NAHI hota.

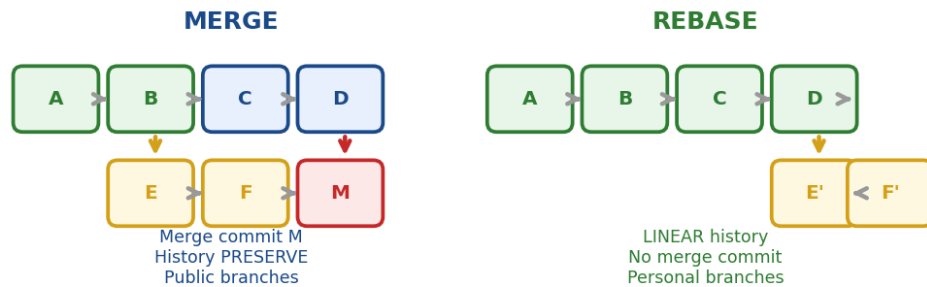
→ [TRAP] IDE mein auto-build ON karo (IntelliJ: Build → Compiler → Build project automatically). Warna detect nahi karega.

→ KEY: DevTools = dev speed up. Auto restart 2 sec. LiveReload browser. Runtime scope = production safe.

Git — Version Control for Teams

Merge vs Rebase

Git Merge vs Rebase



MERGE = safe, public. REBASE = clean, personal.

Push ke baad REBASE = NEVER. History rewrite = team break.

▲ MERGE = merge commit, history preserve, public branches. REBASE = linear replay, clean history, personal branches.
Push ke baad rebase = NEVER.

→ WHY: Git KYUN? 5 developers same codebase pe kaam karein = code combine kaise? Purana code wapas kaise? Kaun ne kya badla? Git = history, branching, collaboration. Interview mein daily use poochhte.

→ MERGE = combine branches with merge commit. SAB commits PRESERVE. Non-destructive. Public/shared branches pe use (main, develop).

```
git checkout feature
git merge main
// Result: E --- F --- M (merge commit)
// History: sab dikhe, merge point clear
```

→ REBASE = replay your commits ON TOP of target. Linear history, no merge commit. Personal feature branches pe use.

```
git checkout feature
git rebase main
// Result: C --- D --- E' --- F' (replayed)
// History: clean straight line
```

→ [TRAP] PUBLIC branch pe KABHI rebase nahi. Push ke baad rebase = commits ki NAYE hash bante = team ke paas purani hash = SAB BREAK. Force push chahiye = DANGEROUS.

Cherry-pick + Stash

→ Cherry-pick = EK specific commit doosri branch se le aao. Poora merge nahi — sirf wo ek commit. Hotfix scenario: bug fix main mein, release branch mein bhi chahiye.

```
git cherry-pick abc123 // sirf ye ek commit apply
```

→ Stash = temporary save bina commit kiye. Branch switch karna, aadha kaam hua. Commit nahi karna aadhe ka.

```
git stash // changes save + dir clean
git checkout main // doosri branch pe kaam
git checkout feature // wapas aao
git stash pop // saved changes wapas
```

→ [TRAP] stash pop = apply + DELETE. stash apply = apply lekin RAKH (safe). Pehle apply karo, sahi lage toh stash drop.

Reset vs Revert

→ Reset = commit HATAO (history rewrite). LOCAL mein use. Push se PEHLE.

```
git reset --soft HEAD~1 // commit undo, changes staged
git reset --mixed HEAD~1 // commit undo, changes unstaged
git reset --hard HEAD~1 // commit + changes DELETE (DANGER!)
```

→ Revert = NAYA commit banao jo purane ko UNDO kare. History SAFE. Push ke BAAD use. Team safe.

```
git revert abc123 // abc123 ka opposite commit
```

→ [TRAP] Push ke baad reset kiya + force push = team ka code BREAK. Push ke baad = HAMESHA revert use karo. Interview mein: "I use revert for pushed commits to preserve history."

Daily Workflow

```
git checkout -b feature/user-auth // branch banao
git add . && git commit -m "Add JWT" // commit
git fetch origin && git rebase origin/main // latest lao
git push origin feature/user-auth // push
// PR banao -> code review -> CI pass -> merge
```

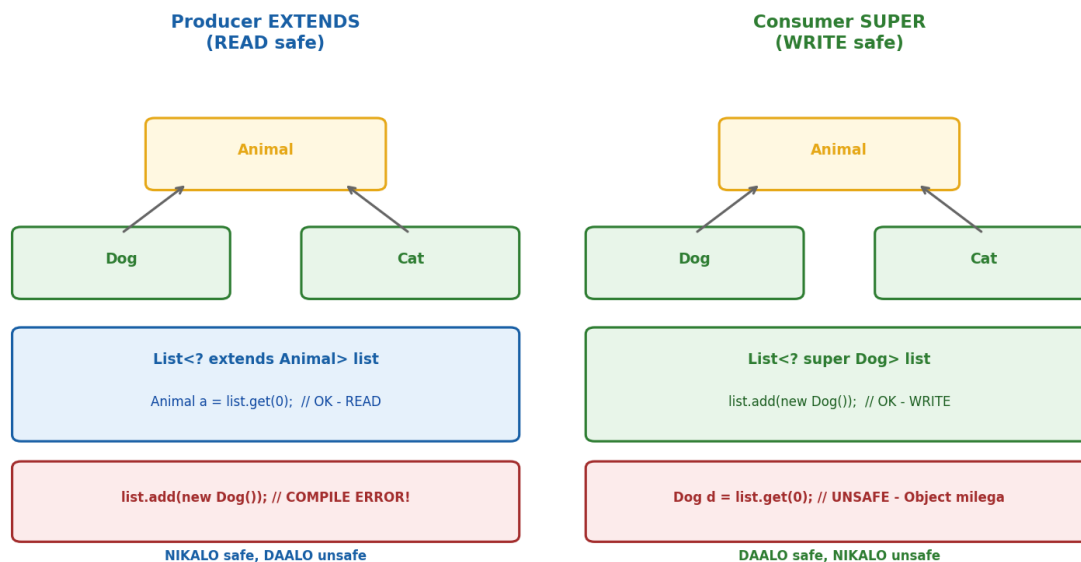
→ Branch naming: feature/xyz, bugfix/xyz, hotfix/xyz. Commit: "Add/Fix/Update [what]" — clear, short.

→ IRIS mein: Jira ticket commit mein (IRIS-1234: Fix survey API). AWS CI/CD. PR mandatory, sab reviewed.

```
git log --oneline // short history
git diff // unstaged changes
git blame file.java // kaun ne kaunsi line likhi
git branch -a // sab branches
```

→ **KEY: Merge = safe, public. Rebase = clean, personal. Cherry-pick = targeted. Stash = temp save. Reset = local undo. Revert = public undo. Push ke baad = REVERT only.**

Generics Wildcards — PECS Rule



Producer EXTENDS = READ safe (nikalo). Consumer SUPER = WRITE safe (daalo). ? extends = get() OK. ? super = add() OK.

→ **WHY: Generics KYUN?** Bina generics: List mein String bhi daalo, Integer bhi. Runtime pe `ClassCastException` = CRASH. Generics = COMPILE TIME type check. `List<String>` = sirf String. Bug runtime se compile time pe shift.

→ **Wildcards KYUN?** `List<Integer>` ko `List<Number>` mein assign NAHI kar sakte. Generics INVARIANT hain. Agar allow ho toh Double ghus jaaye Integer list mein = type safety BREAK.

```
// YE KAAM NAHI KAREGA:
void print(List<Number> list) { }
print(new ArrayList<Integer>()); // COMPILE ERROR!
// Integer IS-A Number lekin List<Integer> IS-NOT List<Number>

// FIX = Wildcard:
void print(List<? extends Number> list) { } // ab chalega!
```

→ PECS = Producer Extends, Consumer Super. Joshua Bloch (Effective Java).

→ ? extends T (UPPER bound) = data NIKALO (read). Producer. Number n = list.get(0) = SAFE. list.add(123) = BANNED (compiler ko exact type nahi pata).

→ ? super T (LOWER bound) = data DAALO (write). Consumer. list.add(123) = SAFE. list.get(0) = Object milega (exact nahi pata).

```
// Producer Extends – data nikalo
List<? extends Number> nums = integerList;
Number n = nums.get(0); // READ = SAFE
nums.add(123); // WRITE = COMPILE ERROR!

// Consumer Super – data daalo
List<? super Integer> nums = numberList;
nums.add(123); // WRITE = SAFE
```

```
Object o = nums.get(0);    // READ = Object (exact nahi pata)

// REAL example - Collections.copy()
public static <T> void copy(
    List<? super T> dest,    // CONSUMER - data daalenge
    List<? extends T> src   // PRODUCER - data nikalenge
) { for (T item : src) dest.add(item); }
```

→ [TRAP] ? extends = READ only. Add karo = COMPILE ERROR. ? super = WRITE ok lekin READ pe sirf Object milega. Dono mein limitation = intentional (type safety).

→ [TRAP] List<?> = unbounded wildcard. Read = Object. Write = NOTHING (null ke alawa). Sirf jab type matter nahi (list.size(), list.clear()).

→ **KEY: Generics = compile-time type safety. PECS = Producer Extends (read), Consumer Super (write). Invariant = type safety reason. ? = unbounded, read=Object, write=nothing.**

Serialization — Object to Bytes

→ WHY: Serialization KYUN? Java object MEMORY mein hai. Server band = object GONE. Network pe bhejni = bytes chahiye. File mein save = bytes. Redis cache = bytes. Kafka message = bytes. Serialization = Java object → byte stream. Deserialization = byte stream → Java object.

→ Serializable = MARKER interface (koi method nahi). JVM ko signal: "is class ko serialize ALLOWED." Nahi lagaya = NotSerializableException.

```
public class User implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private int age;
    private transient String password; // SKIP!
}
```

→ transient = serialize mein SKIP karo. Password, Connection, Thread — ye serialize NAHI hona chahiye. Deserialize pe null/default milega.

→ transient KAB: (1) Sensitive data — password, token. (2) Non-serializable fields — Connection, Socket. (3) Derived data — totalPrice = price * quantity, recalculate kar lena.

→ serialVersionUID = version control for serialization:

```
// V1: User { name, age } → serialize → file save
// V2: email field ADD kiya
// V2 se V1 ka file deserialize = InvalidClassException!
// JVM auto-generated UID BADAL gaya (class structure badli)

// FIX: explicit serialVersionUID set karo
private static final long serialVersionUID = 1L;
// V1 aur V2 mein SAME UID = deserialize ho jayega
// Naya field (email) = null milega (graceful)
```

→ WHY: serialVersionUID KYUN? Class badle = auto UID badle = purani files BREAK. Explicit UID = TU control karo backward compatibility. HAMESHA set karo.

→ [TRAP] serialVersionUID nahi diya = class mein KUCH BHI badle = purani serialized objects BREAK. IDE warning bhi deta.

→ [TRAP] static fields serialize NAHI hote (class level). transient = null. Constructor CALL NAHI hota deserialize mein — JVM reflection se directly fields set.

→ [TRAP] Modern approach: JSON serialization (Jackson) mostly replace karta binary serialization ko. REST APIs = JSON. lekin Redis/Kafka mein binary serialization abhi bhi use hota.

→ **KEY: Serialization = object → bytes. Serializable = marker. transient = skip sensitive. serialVersionUID = version control, HAMESHA set karo. Modern = Jackson JSON, lekin concept jaanna MUST.**

Spring Deep — AOP, Proxy, @Transactional Advanced

AOP Extra — Spring Ka Backbone (Recap)

WHY: 50 methods mein logging + security + txn copy-paste = 300 lines boilerplate. AOP = ye sab ALAG likho (Aspect mein), Spring PROXY ke through har method pe lagao. Business code CLEAN rahe.

→ Tu ALREADY use karta: @Transactional = AOP. @Cacheable = AOP. @PreAuthorize = AOP. @CircuitBreaker = AOP. @Async = AOP. Ek concept = SAB ka backbone.

→ Spring PROXY banata har annotated bean ka. Method call aaye → proxy PEHLE intercept kare (logging, txn start, cache check) → PHIR actual method chale → proxy BAAD mein kaam kare (txn commit, cache store).

[TRAP] Same class ke andar call = PROXY BYPASS. @Transactional, @Async, @Cacheable — TEENO mein ye TRAP hai. Alag class se call karo HAMESHA.

WHY Spring Loves Proxy — Spring Ka Dil

WHY: Bina proxy = 2 lines ka business logic, 20 lines ka boilerplate (logging, txn, security, cache). 50 methods = 50 jagah SAME copy-paste. Proxy = annotations lagao, Spring handle kare.

Bina Proxy (Purana Zamana — J2EE/EJB)

```
public void transferMoney(Long from, Long to, int amount) {
    log.info("Transfer started");           // Logging
    if (!isAuthorized(user)) throw ...;    // Security
    Transaction tx = beginTransaction();    // Transaction
    try {
        Object cached = cache.get(key);     // Cache check
        debit(from, amount);               // ACTUAL LOGIC
        credit(to, amount);               // ACTUAL LOGIC
        tx.commit();                       // Transaction
        cache.put(key, result);            // Cache store
        log.info("Transfer done");          // Logging
    } catch (Exception e) {
        tx.rollback();                     // Transaction
        log.error("Transfer failed");       // Logging
    }
}
```

→ 2 lines ACTUAL logic (debit + credit). Baaki 15 lines = boilerplate. HAR method mein repeat = maintenance nightmare.

Spring Proxy Se (Modern Way)

```
@Transactional
@PreAuthorize("hasRole('ADMIN')")
@Cacheable("transfers")
public void transferMoney(Long from, Long to, int amount) {
    debit(from, amount);    // BAS ITNA!
    credit(to, amount);     // BUSINESS LOGIC ONLY!
}
```

→ Sirf 2 lines business logic. Annotations lagao = proxy AUTOMATICALLY handle kare logging, txn, security, cache.

Proxy Internally Kya Karta

→ Client call kare → PROXY intercept (client ko pata nahi)

→ PEHLE: Security check (@PreAuthorize) → Cache check (@Cacheable) → Transaction start (@Transactional) → Log start (@Before)

→ ACTUAL method chale (debit + credit)

→ BAAD MEIN: Transaction commit/rollback → Cache store → Log end (@After)

→ Response client ko — client ko lagta seedha method call hua. Beech mein proxy ne 6 kaam kiye.

Spring Proxy = 1 Concept, 10 Features

→ @Transactional = proxy transaction manage kare

→ @Cacheable = proxy cache check kare

→ @Async = proxy alag thread mein bheje

→ @PreAuthorize = proxy security check kare

→ @CircuitBreaker = proxy failure handle kare

→ @FeignClient = proxy HTTP call kare

→ SAB ek hi pattern — Proxy + AOP. Ek samjho = sab samjho.

Agar Spring Na Hota?

→ J2EE/EJB time (2000-2005): 500 lines XML configuration, manually wire, boilerplate itna ki logic dhundho na mile, heavy servers (JBoss, WebSphere). Developers ROTE the.

→ Spring ne kiya: PROXY + Annotations = magic. Developer sirf BUSINESS LOGIC likhe. Infrastructure = Spring ka kaam. Isliye Spring JEETA aur J2EE/EJB HAARA.

[TRAP] Same class internal call = PROXY BYPASS. @Transactional, @Cacheable, @Async, @PreAuthorize — SAB mein ye TRAP. Alag bean se call karo HAMESHA.

INTERVIEW: Spring uses Proxy pattern as backbone for cross-cutting concerns. Instead of mixing infrastructure code with business logic, Spring creates proxies that handle transactions, caching, security, async transparently. Business code stays CLEAN = Single Responsibility Principle.

@Transactional — Checked vs Unchecked Rollback (REPEAT TRAP)

WHY: Ye interview mein ZAROOR poochhte — galat bola toh RED FLAG. Bahut developers confused rehte.

→ DEFAULT: RuntimeException (UNCHECKED) → ROLLBACK hota. Checked Exception → ROLLBACK NAHI hota.

→ Unchecked = compiler ne CHECK nahi ki. Runtime pe surprise BLAST. NullPointerException, ArrayIndex, Arithmetic. Programmer ki GALT.

→ Checked = compiler ne PAKAD liya pehle se. IOException, SQLException. External problem. Code COMPILE nahi hoga bina handle kiye.

NullPointerException (unchecked)	→ ROLLBACK hota	✓
ArithmeticException (unchecked)	→ ROLLBACK hota	✓
IOException (checked)	→ ROLLBACK NAHI	✗
SQLException (checked)	→ ROLLBACK NAHI	✗

// Force rollback on checked:

@Transactional(rollbackFor = Exception.class)	// SAB pe
@Transactional(rollbackFor = IOException.class)	// specific

WHY: Checked = expected problem (file nahi mili) → recoverable → tu handle kar. Unchecked = unexpected bug (NullPointerException) → recover nahi ho sakta → ROLLBACK kar do sab.

YAAD RAKH: UN-checked = UN-do (rollback). Checked = CHECK kar khud handle kar. rollbackFor = Exception.class lagao agar SAB pe rollback chahiye.

Spring Boot Extras — Interview Topics (Part 2)

Maven vs Gradle

WHY: Build tools — code compile, test, package, deploy. Bina inke JAR/WAR kaise banega?

→ Maven = XML config (pom.xml). Fixed lifecycle: clean → compile → test → package → install → deploy. Convention over configuration. Simple projects ke liye BEST. ZYADA popular Java mein.

→ Gradle = Groovy/Kotlin DSL (build.gradle). Flexible task graph. Incremental builds = FASTER. Complex/custom builds ke liye. Android default.

Maven:	mvn clean package	→ target/app.jar
Gradle:	./gradlew bootRun	→ build/libs/app.jar

→ IRIS mein Maven use karte. Spring Initializr mein choose karo — dono work equally.

Spring Boot Starters

WHY: 10 dependencies manually add karne ki jagah 1 STARTER add karo = sab aa jaye. Pre-configured dependency bundles.

→ spring-boot-starter-web = Spring MVC + Tomcat + Jackson (JSON)

→ spring-boot-starter-data-jpa = Hibernate + Spring Data JPA + HikariCP

→ spring-boot-starter-security = Spring Security + Filter chain

→ spring-boot-starter-test = JUnit + Mockito + Hamcrest

→ spring-boot-starter-actuator = Health + Metrics endpoints

→ spring-boot-starter-data-redis = Redis + Lettuce client

→ spring-boot-starter-parent = parent POM. Sab dependencies ka version EK jagah. Upgrade Spring Boot = sab auto-compatible.

[TRAP] Starter add kiya = auto-configured. Lekin tu OVERRIDE kar sakta — apna @Bean banao = auto-config SKIP.

@Async — Alag Thread Mein Run

WHY: Email bhejna, report generate = 5 sec lagta. User ko WAIT kyun karwana? Alag thread mein karo, user ko turant response do.

```
@Configuration
@EnableAsync // Step 1: Enable
public class AsyncConfig { }

@Service
public class EmailService {
    @Async // Step 2: Method pe lagao
    public void sendEmail(String to) {
        // 5 sec lage — alag thread mein chalega
    }
}
```

→ Internal: Spring PROXY banata (AOP — same concept!). Proxy method ko ThreadPool mein submit karta.

→ Return type: void ya CompletableFuture<T> (result chahiye toh Future use karo).

[TRAP] Same class mein call = proxy BYPASS = async NAHI chalega! SAME trap as @Transactional! Alag class se call karo.

[TRAP] Default = SimpleAsyncTaskExecutor = har call pe NAYA thread = DANGEROUS. Custom ThreadPoolTaskExecutor configure karo.

application.properties vs application.yml

→ DONO SAME kaam karte — Spring Boot configuration. Sirf FORMAT alag.

→ .properties = flat key=value. server.port=8080. Simple.

→ .yml = hierarchical YAML. server: (newline) port: 8080. Nested config readable.

→ Dono SIMULTANEOUSLY support. Priority: properties > yml (same level pe).

@SpringBootApplication = 3 Ka Combo

→ @SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan

→ @Configuration — ye class config source hai, @Bean methods yahan.

→ @EnableAutoConfiguration — classpath dekh ke auto-configure (HikariCP jar hai? DataSource ban jaye).

→ @ComponentScan — current package + sub-packages scan, @Component/@Service/@Repository dhundho.

→ Main class pe lagta. Pehle teeno ALAG likhte the — ab ek mein sab. SHORTCUT hai.

Embedded Server — jar vs war

WHY: External Tomcat install + WAR deploy = pain. Embedded = java -jar app.jar = server + app dono start. DONE.

→ JAR (DEFAULT) = Tomcat ANDAR (embedded). Self-contained. Docker friendly. Microservices = JAR. IRIS = JAR.

→ WAR (Legacy) = external Tomcat/JBoss mein deploy. pom.xml mein <packaging>war</packaging>. Company force kare tab.

→ Embedded servers: Tomcat (default), Jetty, Undertow. spring-boot-starter-web = Tomcat auto included.

@RestControllerAdvice vs @ControllerAdvice

→ @ControllerAdvice = global exception handler. @ResponseBody LAGANA padta manually.

→ @RestControllerAdvice = @ControllerAdvice + @ResponseBody = automatic JSON response.

→ REST API? → @RestControllerAdvice. MVC HTML? → @ControllerAdvice. SAME pattern as @RestController vs @Controller.

Pagination & Sorting — Spring Data JPA

WHY: 10,000 rows ek baar load = DB slow + RAM full + browser hang. Pagination = sirf 20 laao. DB level pe LIMIT/OFFSET. Baaki 9,980 ko TOUCH nahi.

```
@GetMapping("/users")
public Page<User> getUsers(Pageable pageable) {
    return userRepo.findAll(pageable);
}
// Client: GET /users?page=0&size=20&sort=name,asc
// Response: { content:[...], totalPages:50, totalElements:1000 }
```

→ JpaRepository extend karo → findAll(Pageable) FREE. PageRequest.of(0, 20, Sort.by("name")) = programmatic.

→ IRIS mein survey results paginated — 10,000+ responses, 20 per page.

Spring Boot Auto-configuration — Internally Kaise

→ @EnableAutoConfiguration trigger karta. META-INF/spring.factories mein listed auto-config classes LOAD.

@ConditionalOnClass(HikariDataSource.class) // Jar hai? Configure karo

@ConditionalOnMissingBean(DataSource.class) // User ne nahi banaya? Auto banao

```
@ConditionalOnProperty(name="spring.cache") // Property set? Enable karo
```

→ Override: apna @Bean banao = auto-config SKIP. 'Convention over configuration' = YE hai.

CommandLineRunner / ApplicationRunner

WHY: App start ke BAAD ek baar code run karna — DB mein data load, cache warm-up, startup validation.

```
@Component
public class DataLoader implements CommandLineRunner {
    @Override
    public void run(String... args) {
        // App start ke BAAD ek baar chale
    }
}
```

→ CommandLineRunner = String[] args. ApplicationRunner = ApplicationArguments (parsed). Multiple? @Order(1), @Order(2).

Concept Level Topics — 1-2 Line Interview Answers

Virtual Threads (Project Loom, Java 21)

→ Lightweight threads, JVM managed. Platform thread = OS thread (heavy, ~2000 limit). Virtual thread = JVM thread (millions possible). Spring Boot 3.2+ support. spring.threads.virtual.enabled=true. Non-blocking concurrency WITHOUT reactive code.

Spring Boot Starter Parent

→ Parent POM — version management. Sab Spring dependencies ka version ek jagah define. Upgrade Spring Boot = sab dependencies auto-compatible. pom.xml mein <parent> mein set karo.

Flyway / Liquibase

→ DB migration tools. Schema changes version control mein. Flyway = SQL files (V1__create_users.sql). Liquibase = XML/YAML/JSON. App start pe automatically pending migrations run. Production MUST.

@ConfigurationProperties

→ Type-safe config binding. @Value = single property. @ConfigurationProperties(prefix='app') = group of related properties into a class. Validated, testable. 5+ related properties? Use this.

Spring Initializr

→ start.spring.io — web UI se project generate. Maven/Gradle choose, dependencies select, download. IDE bhi internally ye use karta.

Swagger / OpenAPI

→ API documentation auto-generate. springdoc-openapi dependency. /swagger-ui.html = visual API explorer. Annotations: @Operation, @ApiResponse.

Embedded H2 Database

→ In-memory DB for testing. spring-boot-starter-data-jpa + h2 dependency = auto configure. Production = MySQL. H2 console: /h2-console.

WebClient vs RestTemplate vs FeignClient

→ RestTemplate = synchronous, blocking, maintenance mode (LEGACY). WebClient = reactive, non-blocking (Spring WebFlux). FeignClient = declarative, interface-based (humne padha). New projects = WebClient ya FeignClient.

GraalVM / Native Image

→ Java code → native executable (ahead-of-time compile). Startup = milliseconds (vs seconds). Memory = 50% less. Spring Boot 3 support. Microservices + serverless = perfect. Trade-off: build time slow, reflection limitations.

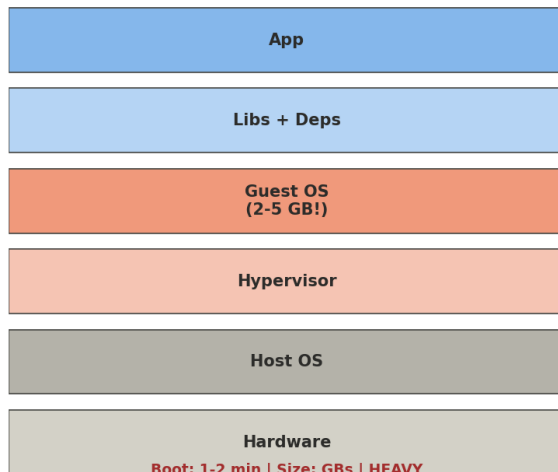
Infrastructure — Docker, Redis

Docker — Containerization

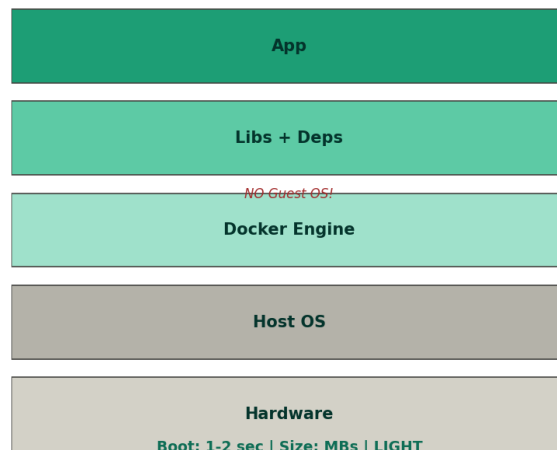
WHY: "Mere machine pe chal raha tha!" problem. Dev mein Java 17, server pe Java 11 = CRASH. Docker = code + Java + MySQL + config SAB ek container mein pack karo. Kahi bhi run karo = SAME chalega. Server pe sirf Docker Engine chahiye.

VM vs Container

Virtual Machine



Docker Container



▲ VM = full Guest OS (GB, minutes boot). Container = shares host kernel (MB, seconds boot). Container LIGHT hai, VM HEAVY.

- VM = har VM mein POORA OS install (Windows/Linux). 2-5 GB size. Boot = 1-2 min. Strong isolation (separate OS).
- Container = Guest OS NAHI. Host ka kernel SHARE karta. MB size. Boot = 1-2 sec. Process level isolation.
- Analogy: VM = alag ghar banao (foundation se). Container = ek building mein alag flat (shared foundation).
- Washing Machine analogy: Docker Engine = washing machine. Container = tera kapde + detergent + softener sab APNA leke aaya. Machine ko fark nahi kya andar hai.

Dockerfile + docker-compose

```
FROM openjdk:17-jdk-slim    # Base image (Java 17 ANDAR hai!)
WORKDIR /app                # Container ke andar directory
COPY target/app.jar app.jar  # Tera JAR container mein
EXPOSE 8080                 # Port document (actual mapping alag)
ENTRYPOINT ["java", "-jar", "app.jar"] # Start pe kya run
```

→ docker-compose = multiple containers ek saath. Survey + Payment + MySQL teeno START ek command: docker-compose up

→ Image vs Container: Image = recipe (read-only). Container = running instance. Class vs Object jaisa. Ek image se 10 containers.

Docker Commands

```
docker build -t myapp .      # Image banao
docker run -p 8080:8080 myapp # Container start
docker ps                    # Running containers dekho
docker stop <id>             # Band karo
docker-compose up -d         # Sab services start (background)
docker-compose down          # Sab band
docker logs <id>             # Logs dekho
```

[TRAP] EXPOSE sirf documentation hai — actual port mapping -p 8080:8080 se hota. EXPOSE likhna = port open NAHI hota.

[TRAP] Container = lightweight lekin isolation VM se WEAK. Security-critical = VM better. Normal microservices = Container better.

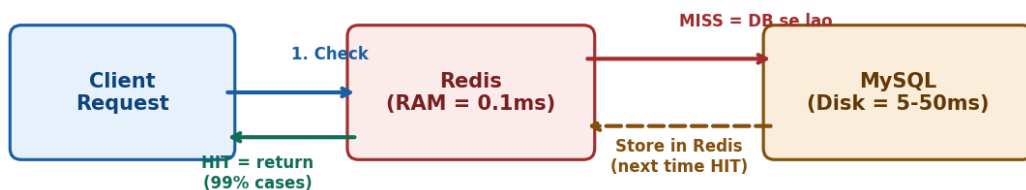
INTERVIEW: Docker containers package app + all dependencies. Host needs only Docker Engine. Eliminates environment bugs. IRIS = 15 services, each in own container, AWS ECR + ECS deploy.

Redis — In-Memory Cache

WHY: User profile 100 baar khole = 100 DB queries SAME data ke liye. DB pe load, slow. Redis = pehli baar DB se lao, RAM mein RAKH do. Baaki 99 baar Redis se = 0.1ms. DB = almari (door kholna padta). Redis = table pe rakha (haath badhao le lo). 50x-1000x faster.

How Redis Works

Redis Cache Flow — @Cacheable internally ye karta



▲ Client → Redis check. HIT = return (0.1ms). MISS = DB se lao (5-50ms) + Redis mein store. @Cacheable internally ye karta.

→ In-memory = RAM mein rehta, disk pe NAHI. RAM access = nanoseconds, disk = milliseconds. Isliye FAST.

→ Key-value store = har data ka ek naam (key) aur ek value. HashMap jaisa! SET user:1 "Arpan" → GET user:1 → "Arpan"

→ TTL (Time To Live) = kitni der tak cache rakho. EXPIRE key 3600 = 1 hour baad AUTO delete. Stale data problem solve.

Redis Commands

```
SET user:1 "Arpan"          # Store
GET user:1                  # Retrieve = "Arpan" (0.1ms!)
EXPIRE user:1 3600          # 1 hour baad auto delete
DEL user:1                  # Manually delete
HSET user:1 name "Arpan" age 25 # Hash (object cache)
HGETALL user:1              # Full object return
```

5 Data Types

→ String — simple key-value. Use: session, OTP, config cache.

→ Hash — object (field-value pairs). Use: user profile cache (name, age, email ek key mein).

→ List — ordered collection. Use: recent activity feed, message queue.

→ Set — unique values (no duplicates). Use: online users track, unique visitors.

→ Sorted Set — set with SCORE (auto sorted). Use: leaderboard, ranking, top products.

IRIS mein Redis Use Cases

→ Session/JWT Cache — login pe token Redis mein. Har request pe Redis se verify. Logout = Redis se delete = INSTANT.

→ OTP Storage — SET otp:phone:9876 "4521" EXPIRE 300 (5 min). Auto delete. DB zarurat nahi.

→ Rate Limiting — INCR api:user:1. Counter > 100? 429 Too Many Requests. TTL 60s = auto reset.

→ Survey Config Cache — config DB se ek baar, Redis mein cache. 10,000 respondents = Redis se serve, DB ZERO load.

→ Spring Boot mein: spring-boot-starter-data-redis + @Cacheable = Redis integration DONE. Humne padha hai!

[TRAP] Redis = RAM mein. Server restart = data GONE (by default). Persistence chahiye toh RDB/AOF enable karo. Primary use = CACHE, permanent storage NAHI = MySQL.

[TRAP] Cache Invalidation = hardest CS problem. DB update hua lekin cache purana. Fix: TTL + @CacheEvict on write operations.

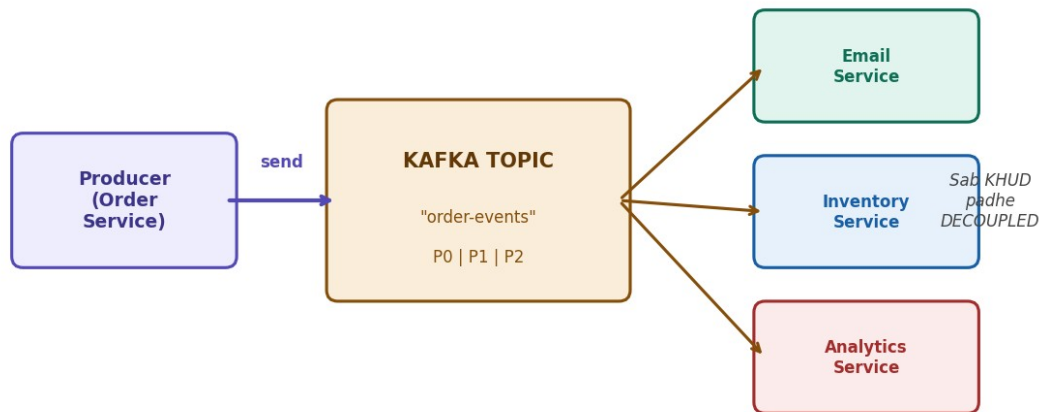
INTERVIEW: Redis = in-memory key-value store, 0.1ms response. MySQL = permanent storage, Redis = speed layer. IRIS mein session, OTP, rate limiting, survey config cache. Spring Boot @Cacheable + Redis = production cache.

Kafka — Distributed Event Streaming

WHY: Order place → Email, Inventory, Analytics sab ko batana. Direct API call = TIGHT coupling + ek down = fail. Kafka = WhatsApp GROUP — message daalo, sab KHUD padhe. LOOSE coupling + fast + resilient.

Kafka Flow — 4 Key Concepts

Kafka — WhatsApp Group for Microservices



▲ Producer (Order Service) → Topic → Consumers (Email, Inventory, Analytics) KHUD padhe. DECOUPLED.

- Producer = message BHEJNE wala. `KafkaTemplate.send('topic', message)`.
- Consumer = message PADHNE wala. `@KafkaListener(topics='order-events')` = auto called.
- Topic = category. 'order-events', 'payment-events'. WhatsApp GROUP jaisa.
- Broker = Kafka SERVER. Multiple brokers = cluster. Ek down? Doosra handle.
- REST = synchronous (WAIT). Kafka = asynchronous (fire-and-forget).

Partitions + Consumer Groups

- Partition = topic ko TODHO. 3 partitions = 3 consumers PARALLEL = 3x FAST.
- Consumer Group = same service ke instances. Kafka auto partition assign. Duplicate nahi.
- Offset = bookmark. Crash? Restart pe WAHI se continue.
- Retention = 7 days default. Consume ke baad DELETE nahi. Re-read possible.
- Replication = partition copy. Broker down? Replica take over. Data loss ZERO.

Spring Boot + Kafka Code

```
// Producer
@Service
public class OrderService {
    private final KafkaTemplate<String,String> kafka;
    public void createOrder(Order order) {
        orderRepo.save(order);
        kafka.send("order-events", order.toJson());
    }
}

// Consumer
@Service
public class EmailConsumer {
    @KafkaListener(topics = "order-events")
    public void handleOrder(String msg) { sendEmail(msg); }
}
```

Kafka Ki Birth — LinkedIn Ka Dard (2011)

WHY: LinkedIn mein MILLIONS events per second — profile dekha, job search, connection request, post like, recruiter activity. Ye data 10+ teams ko chahiye tha (Analytics, Search, Recommendations, Ads, Notifications). Har team ke liye ALAG pipeline = nightmare. Jay Kreps ne socha EK jagah daalo, jisko chahiye padh le.

- PROBLEM: 10+ teams × alag alag data pipelines = spaghetti architecture. Naya team add karo = naya pipeline banao. Maintain karna IMPOSSIBLE.
- SOLUTION: Kafka = CENTRAL event bus. Sab events EK jagah. Analytics team padhe → dashboards. Search team padhe → index update. Recommendation team padhe → 'People you may know'. Ads team padhe → targeted ads.
- 2011 mein Jay Kreps + LinkedIn team ne banaya. Baad mein Apache Foundation ko de diya = open source. Naam Franz Kafka (famous writer) ke naam pe — kyunki ye system WRITING/LOGGING ke liye bana.
- Ab KAUN use karta: LinkedIn (original), Netflix (recommendations), Uber (ride tracking), Spotify (song plays), Twitter/X (tweets), Swiggy/Zomato (order tracking). YouTube Live comments jaisa system bhi SAME concept (Google apna Pub/Sub use karta).
- INTERVIEW mein bol: 'Kafka was built by LinkedIn in 2011 to handle their internal data pipeline problem — millions of events per second needed by multiple teams. It's now the industry standard for real-time event streaming.'

Kafka vs RabbitMQ

→ Kafka = distributed LOG. Retain messages. Re-read possible. 1M+ msg/sec. Event streaming.
→ RabbitMQ = QUEUE. Consume = delete. Smart routing. Lower throughput. Task queue.
→ LinkedIn ne Kafka banaya DATA PIPELINE ke liye — millions events/sec. Naam Franz Kafka (writer) se.

[TRAP] Kafka = queue NAHI. Distributed EVENT LOG. Messages delete nahi hote (retention based).

[TRAP] Message order sirf SAME partition mein. Different partitions = NO order. Same KEY = same partition = ordered.

[TRAP] Partitions > consumers = idle waste. Consumers > partitions = extra idle. Best: equal.

INTERVIEW: Kafka = distributed event streaming. Producers → topics → consumers independently. Partitions = parallelism. Async microservice communication. LinkedIn built it for data pipelines.

Quick-Fire Interview Questions — Aaltu-Faaltu Poochhe Jaate

→ WHY: Ye chhote questions hain jo 30 sec mein pooch lete. Answer nahi aaya = "basics weak" impression. 1 min mein padh lo, yaad rahega.

→ throw vs throws

throw = method ke ANDAR exception FENKO manually. throw new IllegalArgumentException("invalid"). throws = method SIGNATURE mein declare karo ki ye method exception throw KAR SAKTA hai. void read() throws IOException. throw = action (fenko). throws = declaration (batao).

→ shallow copy vs deep copy

Shallow = object copy lekin internal references SAME point karte. Original ka List change = copy mein bhi change. Deep = object + internal objects sab NAYE copy. Original change = copy SAFE. Immutable objects mein fark nahi. Mutable fields = deep copy MUST (defensive copy). clone() default = shallow.

→ transient vs volatile

transient = Serialization mein SKIP karo is field ko. Password save nahi karna file mein = transient. volatile = har read/write MAIN MEMORY se. CPU cache bypass. Thread visibility fix. Dono ALAG concepts: transient = persistence. volatile = concurrency. Dono fields pe lagao.

→ Serializable vs Externalizable

Serializable = marker interface, JVM automatically serialize SARE fields. Easy lekin slow (reflection). Externalizable = TU khud likho readExternal() writeExternal() mein KYA serialize karna. Fast lekin manual effort. 99% cases Serializable enough. Performance critical = Externalizable.

→ Runnable vs Callable

Runnable = run() method, return NAHI karta, exception throw NAHI kar sakta. Callable = call() method, return VALUE deta (Future mein), exception THROW kar sakta. ExecutorService.submit(callable) = Future milta. Result chahiye thread se = Callable. Sirf fire-and-forget = Runnable.

→ notify() vs notifyAll()

notify() = EK random waiting thread jagao. notifyAll() = SAB waiting threads jagao. Problem: notify() se galat thread jag sakta (jo condition handle nahi kar sakta) = missed signal. Best practice: notifyAll() use karo, har thread apni condition check kare (while loop mein). notifyAll() safer hai.

→ ClassNotFoundException vs NoClassDefFoundError

ClassNotFoundException = CHECKED exception. Class.forName("com.xyz.MyClass") pe agar class nahi mili classpath mein. Runtime pe dynamically load karne pe. NoClassDefFoundError = ERROR. Compile time pe class thi, RUNTIME pe nahi mili (JAR missing, classpath wrong). JVM level error. Pehla = tera code galat. Doosra = environment galat.

→ Error vs Exception

Error = JVM level, RECOVER nahi ho sakta. OutOfMemoryError, StackOverflowError. Catch mat karo (kyunki kuch kar nahi sakte). Exception = application level, RECOVER ho sakta. IOException, NullPointerException. Catch karo, handle karo. Dono Throwable ke child. Error = JVM ki galti. Exception = teri galti.

→ StackOverflowError vs OutOfMemoryError

StackOverflowError = stack memory full. Infinite recursion (method apne aap ko call kare bina base case). Fix: recursion check, base case add. OutOfMemoryError = heap memory full. Bahut objects bane, GC clean nahi kar paya. Fix: memory leak dhundho, heap size badhao (-Xmx). Stack = method calls. Heap = objects.

→ @Autowired vs @Inject vs @Resource

@Autowired = Spring annotation. By TYPE inject. required=false possible. @Inject = Java standard (javax.inject). By TYPE. Spring + other DI frameworks mein kaam. @Resource = Java standard. By NAME inject (field naam se match). Spring team recommend: @Autowired ya better = constructor injection (no annotation needed).

→ @RequestParam vs @PathVariable

@RequestParam = query parameter. /users?city=Delhi. @RequestParam("city") String city. Optional ho sakta (defaultValue). @PathVariable = URL path mein embedded. /users/{id}. @PathVariable Long id. Required. REST convention: resource identify = PathVariable. Filter/search = RequestParam.

→ @ResponseBody vs @RequestBody

@ResponseBody = method ka return value HTTP response BODY mein bhejo (JSON). @RestController mein auto lagta. @RequestBody = HTTP request BODY se data leke Java object mein convert (deserialize). POST/PUT mein use. Jackson internally handle karta dono mein (JSON <-> Java).

→ CORS vs CSRF

CORS = Cross-Origin Resource Sharing. Browser BLOCK karta different domain se API call. Server Access-Control-Allow-Origin header se ALLOW kare. Backend fix. CSRF = Cross-Site Request Forgery. Attacker user ke naam se request bheje (user logged in hai). Fix: CSRF token har form mein. REST API (stateless JWT) = CSRF risk kam. CORS = browser restriction. CSRF = attack type.

→ @Entity vs @Table

@Entity = JPA annotation. "Ye class ek DB table represent karti." MUST hai entity class pe. @Table = OPTIONAL. Table ka naam specify karo agar class naam se ALAG hai. @Table(name="tbl_users"). @Entity nahi lagaya = Hibernate is class ko IGNORE karega. @Table nahi lagaya = class naam = table naam (default).

→ VARCHAR vs CHAR

CHAR(10) = FIXED 10 characters. "Hi" store = "Hi" (8 spaces padded). HAMESHA 10 bytes. VARCHAR(10) = VARIABLE length. "Hi" store = "Hi" (2 bytes + 1 length byte). Space SAVE. CHAR = fixed length data (phone code, state code). VARCHAR = variable (name, email). VARCHAR(255) = 1 byte length. VARCHAR(256) = 2 bytes. 255 sweet spot.

→ Array vs ArrayList

Array = FIXED size, declare pe set. Primitive + object dono. int[] arr = new int[10]. Direct index access. ArrayList = DYNAMIC size, grow hota. Sirf objects (wrapper). ArrayList<Integer>. add/remove easy. Internally array hai lekin auto-resize (1.5x). Array = performance (primitive, no boxing). ArrayList = flexibility (dynamic, Collections API).

→ break vs continue vs return

break = loop/switch se BAHAR niklo. Pehla match mile = break, baaki skip. continue = is iteration SKIP karo, next iteration pe jao. Odd numbers skip karo = continue. return = method se BAHAR niklo + value return. Method khatam. break = loop tod do. continue = is baar skip. return = method chhor do.

→ **Ye sab 30-sec questions hain. Ek baar padh lo = yaad rahega. Interview mein confidence dikhta jab chhote questions pe bhi TURANT answer aaye.**

Interview Preparation — Rounds, Questions, Scenarios

Interview Rounds — Kya Poochhte Hain Kab

Round 1: Screening (30 min)

- Tell me about yourself — 30 sec power intro
- Java experience — IRIS platform, 10+ countries, 500+ incidents
- Spring Boot use? — microservices, REST APIs, production support
- Why switch? — growth, international exposure, remote work

Round 2: Core Java + Spring (45-60 min)

- OOP pillars, String immutability, HashMap internals, == vs equals()
- Collections: ArrayList vs LinkedList, HashMap vs TreeMap
- Java 8: Streams, Optional, Lambda, Functional Interfaces
- Multithreading: volatile vs synchronized, deadlock
- Spring: DI types, Bean lifecycle, @Transactional traps, AOP
- Hibernate: N+1, Lazy vs Eager, lifecycle states
- SOLID + Patterns: Singleton (thread-safe), Factory, Strategy

Round 3: Coding (45 min)

- LeetCode medium — Array, String, HashMap, DP. 1000+ LC done = STRONG.

Round 4: SQL (15-30 min)

- Joins (HAMESHA), Index, ACID, GROUP BY/HAVING queries, EXPLAIN

Round 5: Behavioral (30 min)

→ HikariCP bug STAR story, team conflict, naya seekhna, why switch

Reverse Questions — Interviewer Ko Impress Karo

→ RULE: Har interview mein 2-3 pooch. Generic NAHI. Specific pooch.

Technical Depth

→ "What does your deployment pipeline look like — from code push to production?" → CI/CD samajhta hai
→ "What's the biggest technical debt the team is dealing with?" → Real codebases mein problems — maturity
→ "How do you handle production incidents? On-call rotation?" → 500+ incidents = tera STRENGTH
→ "What monitoring tools? Datadog, New Relic, CloudWatch?" → Specific tools = hands-on experience
→ "How does the team handle DB migrations? Flyway/Liquibase?" → Production risk sochta

Team + Culture

→ "How does a feature go from idea to production? Review process?"
→ "What does code review culture look like? Seniors bhi reviewed?"
→ "How much autonomy does a mid-level dev have?"

Growth + Killer Questions

→ "What does successful first 90 days look like?"
→ "If you could mass-fix one thing in the codebase, what would it be?"
→ "How does the team balance shipping fast vs code quality?"

Scenario Questions — Recruiters Ke Favourites (Full Answers)

Q: Monolith se Microservices migrate kaise karoge?

Strangler Fig pattern. EK baar mein poora convert NAHI — risk bahut zyada. Step 1: Identify boundaries — kaunsa module independent hai (User, Payment, Notification). Step 2: Naya feature microservice mein banao. Step 3: Ek ek module extract — pehle simple (Notification), phir complex (Payment). Step 4: API Gateway lagao — client ko fark nahi padna chahiye. Step 5: Database split per service — shared DB = coupling wapas. Step 6: Event-driven communication (Kafka/RabbitMQ). Timeframe: months/years, not weeks. IRIS mein 15+ services = gradually evolved.

Q: Naya technology jaldi seekhni ho toh kaise?

Step 1: Official docs — getting started guide (30 min overview). Step 2: Quick tutorial follow karo — hands-on, copy-paste se start. Step 3: Chhota project banao — real use case, CRUD enough. Step 4: Source code padho existing projects ka — GitHub pe production code. Step 5: Common pitfalls jaano — Stack Overflow top questions. Meri approach: self-taught, koi mentor nahi. Internet + documentation + hands-on se Java, Spring, Hibernate sab seekha. 1000+ LeetCode self-driven.

Q: Code review mein senior ka code galat lage toh?

Respectfully point out with logic. 'I noticed this section might cause X issue because of Y. What do you think about Z approach?' Data/logic se baat karo, ego se nahi. Example do, test case dikhao. KABHI personal mat karo ('your code is wrong' NAHI — 'this code might cause' bolo). Achhe teams mein ye ENCOURAGED hai. Senior disagree kare toh reasoning suno. Agree to disagree bhi OK — document karo.

Q: Session vs JWT — kab kya use karein?

Session: server pe store (memory/Redis). Session ID cookie mein. Logout = session delete = INSTANT. Problem: multiple servers = session sync chahiye. Stateful. JWT: self-contained token. Server sirf signature verify (kuch STORE nahi). Logout = MUSHKIL (token valid jab tak expire na ho). Fix: short expiry + blacklist. Stateless = scale easy. Decision: Monolith + single server = Session fine. Microservices + multiple instances = JWT better. IRIS = JWT (15+ services, stateless).

Q: REST API design karo library system ke liye

Resources: /books, /members, /loans. GET /api/books (list, paginated). GET /api/books/{id} (single). POST /api/books (create, 201). PUT /api/books/{id} (update). DELETE /api/books/{id} (delete, 204). GET /api/members/{id}/loans (member ke loans). POST /api/loans (issue book). PUT /api/loans/{id}/return. Status: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found, 409 Conflict. Pagination: ?page=0&size=20&sort=title,asc. Versioning: /api/v1/books.

Q: SQL query 30 sec le rahi. Optimize kaise?

Step 1: EXPLAIN lagao — type=ALL = full table scan (WORST). Key=NULL = index nahi use hua. Step 2: Index add karo WHERE columns pe. Composite index leftmost prefix rule. Step 3: SELECT * hatao, specific columns. Covering index = table access ZERO. Step 4: N+1 check — loop mein query? JOIN FETCH. Step 5: Subquery → JOIN convert. Step 6: Pagination (LIMIT/OFFSET). Step 7: Cache (Redis). Step 8: Connection pool check — slow query lock hold = pool exhaust. Ye mera daily kaam — IRIS mein 500+ slow query fixes.

Q: Code mein bug hai aur deadline kal hai — kya karoge?

Step 1: Manager ko TURANT batao — 'bug mila, ye impact, ye time lagega.' Surprise mat do deadline pe. Step 2: Severity assess — critical (payment fail)? Sab chhodo fix karo. Minor? Hotfix later. Step 3: Critical = hotfix deploy (temporary). Proper fix baad mein with test. Step 4: Regression test likho. Step 5: Post-mortem — bug KYUN aaya? Code review miss? Test coverage kam? Honesty BEST policy.

Q: NULL return ya Optional?

Optional ALWAYS prefer method return mein. null = caller ko PATA nahi check karna, bhool gaya = NullPointerException. Optional = FORCE karta handle karo (isPresent, orElse, orElseThrow). Lekin SAHI jagah: method return = Optional achha. Method parameter = Optional mat do (ugly). Class field = Optional mat do (serialization). Collection return = empty list, Optional nahi.

Q: GDPR — customer bola data delete karo?

Right to Erasure = legal requirement. Step 1: Soft delete (is_deleted=true) — immediate effect. Step 2: Hard delete scheduled job se (30 din baad). Step 3: Backups se bhi hatao. Step 4: Audit log maintain (kab request, kab process). Step 5: Third party notify — analytics, email service, logs se PII hatao. Deletion confirmation user ko bhejo. Cascade delete handle (user delete = orders, payments affected).

Q: 10x traffic suddenly aaye toh?

Rate limiting pehle (429). Auto-scaling (AWS Auto Scaling Group). Caching (Redis = DB load 90% kam). Load balancer distribute. CDN static content. Database read replicas + connection pool optimize. Circuit Breaker downstream fail pe. Monitoring = CloudWatch/Grafana alerts PEHLE se — spike detect early.

Q: Microservice A→B→C, C slow — kya hoga?

Cascading failure. C slow = B threads block. B block = A bhi block. Domino effect. Fix: Circuit Breaker (Resilience4j) — C failures > threshold = OPEN = seedha fallback. Timeout har service pe (connect 5s, read 10s). Retry with backoff (1s, 2s, 4s). Bulkhead = alag thread pools per service. EXACTLY mera HikariCP bug — ek slow component ne cascading failure kiya.

Q: 2 developers same file Git mein?

Merge conflict. Git auto merge agar DIFFERENT lines. SAME lines = conflict, manually resolve. Best practice: feature branches (feature/user-auth). Main mein direct push NAHI. PR banao, code review, CI tests pass, tab merge. Conflict avoid: chhoti branches (2-3 din), frequently pull main se.

Q: DB connection pool exhaust?

Detect: HikariCP metrics — active connections = maxPoolSize. Root cause: slow queries (EXPLAIN), connection leak (try-with-resources check), N+1 (JOIN FETCH), pool size chhota. Immediate: pool size temporary increase. Long term: slow queries optimize, leak fix, caching (Redis). Mera REAL experience — IRIS mein HikariCP exhaustion, root cause = slow query 30 sec lock hold.

Q: Deployment ke beech crash?

Rolling deployment — ek ek instance update. Fail? Rollback automatic. Blue-Green: purana (blue) live, naya (green) ready. Health check pass? Switch. Fail? Blue pe wapas. K8s mein readinessProbe fail = traffic nahi. IRIS = AWS CloudFormation + health checks. 500+ incidents mein deployment failures dekhi — rollback MUST.

Q: API call ke beech token expire?

Client-side interceptor handle karta. 401 Unauthorized aaye → interceptor automatically refresh token bheje → server naya access token de → original request RETRY. User ko pata nahi chalta. Refresh bhi expire? Login redirect. Access = 15 min, Refresh = 7 din. Spring Security mein OncePerRequestFilter mein handle.

YE SAB 'aaltu-faaltu' questions hain — koi sar pair nahi lekin POOCHHTE hain. Interview se 1 din pehle ek baar skim kar lena.

Phase 2: Code Visualization — Real Spring Boot Code

1. Project Structure — Layered Architecture

→ Spring Boot project = package by LAYER. Controller/Service/Repository/Entity/DTO/Config/Filter.

```
survey-service/
├── src/main/java/com/iris/survey/
│   ├── SurveyApplication.java      ← @SpringBootApplication (entry point)
│   ├── controller/
│   │   └── SurveyController.java   ← REST endpoints (@GetMapping, @PostMapping)
│   ├── service/
│   │   └── SurveyService.java      ← Business logic (@Transactional, @Cacheable)
│   ├── repository/
│   │   └── SurveyRepository.java   ← DB access (interface, Spring generates impl)
│   ├── entity/
│   │   └── Survey.java            ← DB table mapping (@Entity, @Id, @Column)
│   ├── dto/
│   │   ├── SurveyRequest.java     ← Input DTO (@NotBlank, @Size validation)
│   │   └── SurveyResponse.java    ← Output DTO (filtered fields, password NAHI)
│   ├── exception/
│   │   ├── ResourceNotFoundException.java ← Custom 404
│   │   └── GlobalExceptionHandler.java   ← @RestControllerAdvice
│   ├── config/
│   │   └── SecurityConfig.java     ← JWT + BCrypt + CORS
│   ├── filter/
│   │   └── JwtAuthFilter.java      ← OncePerRequestFilter
│   └── src/main/resources/
│       ├── application.yml        ← Config (DB, Redis, JWT, Actuator)
│       └── application-dev.yml    ← Dev profile override
├── pom.xml                       ← Dependencies (starters)
└── Dockerfile                    ← Container config
```

INTERVIEW: "My project follows layered architecture — Controller handles HTTP, Service has business logic, Repository does data access. Each layer depends only on the layer below."

2. Entity — Har Line Ka KYUN

→ Entity = DATABASE TABLE ka Java representation. Har field = column.

```
@Entity                                // Hibernate: ye class = DB table
@Table(name = "surveys")                // Table naam specify
public class Survey {

    @Id                                  // Primary Key — ZARURI har entity mein
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-increment
    private Long id;                     // Long KYUN? null ho sakta (naya object)
                                         // int = default 0 = ambiguous

    @Column(nullable = false, length = 200) // DB constraint: NOT NULL + VARCHAR(200)
    private String title;

    @Enumerated(EnumType.STRING)         // "DRAFT" save, ORDINAL mat (order badle = corrupt)
    private SurveyStatus status;

    @ManyToOne(fetch = FetchType.LAZY)   // LAZY = jab access karo tab fetch
    @JoinColumn(name = "created_by")     // FK column naam
    private User createdBy;

    @OneToMany(mappedBy = "survey", cascade = CascadeType.ALL)
    private List<Response> responses;    // N+1 TRAP yahan!

    @CreationTimestamp                   // Auto set current time on INSERT
    private LocalDateTime createdAt;
}
```

WHY: @Entity bina = Hibernate ignore. @Id bina = startup error. Long id bina = null detect nahi. @Enumerated STRING bina = order change pe data corrupt.

[TRAP] @OneToMany + loop = N+1 Problem. 10 surveys = 11 queries. FIX: JOIN FETCH.

[TRAP] Entity DIRECTLY return mat karo API se — LazyInitException + password expose. DTO use karo.

3. DTO — Entity vs DTO, Kya Fark, KYUN Chahiye

→ Entity = DB ka mirror (ANDAR ka kaam). DTO = client ka dabba (BAHAR ka kaam). Aadhaar card vs photocopy.

```
// == REQUEST DTO — client se data RECEIVE + VALIDATE ==
public class SurveyRequest {

    @NotBlank(message = "Title is required")    // null + empty + whitespace = REJECT
    @Size(min = 3, max = 200)                    // 3 se kam ya 200 se zyada = REJECT
    private String title;

    @NotNull(message = "Max panelists required")
    @Min(1) @Max(100000)
    private Integer maxPanelists;    // Integer KYUN int nahi?
                                    // int default=0, null detect NAHI hoga
                                    // Integer = null ho sakta = @NotNull KAAM kare

    @NotNull
    private Long userId;
}

// == RESPONSE DTO — client ko FILTERED data bhejo ==
public class SurveyResponse {

    private Long id;
    private String title;
    private String status;    // String KYUN enum nahi? Client ko Java enum NAHI pata
    private LocalDateTime createdAt;
    private String createdByName;    // User OBJECT nahi — sirf NAAM
    // password? NAHI. internal IDs? NAHI. lazy objects? NAHI.
}
}
```

WHY: DTO KYUN? 3 reasons: (1) Security — password expose nahi. (2) LazyInit — Entity return = session close = CRASH. (3) Decoupling — DB schema badle, API SAME rahe.

[TRAP] @Valid bhool gaye @RequestBody ke saath = validation CHALTA HI NAHI. Blank title accept ho jayega.

[TRAP] int maxPanelists likha Integer ki jagah = default 0 = @NotNull BEKAAR = client ne nahi bheja phir bhi 0 accept.

4. Repository — Interface Likho, Implementation Spring Dega

```
public interface SurveyRepository
    extends JpaRepository<Survey, Long> {
    // JpaRepository<ENTITY, ID_TYPE> extend = FREE mein:
    // save(), findById(), findAll(), deleteById(), count(), existsById()

    // Derived Query — method naam se SQL auto-generate
    List<Survey> findByStatus(SurveyStatus status);
    // → SELECT * FROM surveys WHERE status = ?
    // Spring method naam PARSE kare → SQL auto-build

    List<Survey> findByStatusAndCreatedByEmail(SurveyStatus s, String email);
    // → WHERE status=? AND created_by.email=?

    // Custom Query — complex logic ke liye
    @Query("SELECT s FROM Survey s JOIN FETCH s.responses WHERE s.id = :id")
    Optional<Survey> findByIdWithResponses(@Param("id") Long id);
    // → N+1 FIX! Ek query mein survey + responses
    // HQL mein ENTITY naam (Survey), table naam (surveys) NAHI

    // Pagination
    Page<Survey> findByStatus(SurveyStatus status, Pageable pageable);
    // → LIMIT 20 OFFSET 0 (DB level pe)
}
```

WHY: Interface KYUN class nahi? Spring PROXY runtime pe KHUD banata. Tu sirf method naam likho — implementation Spring ka kaam.

[TRAP] Repository mein @Transactional mat lagao (generally). Service layer manage kare transactions — ek business operation mein MULTIPLE repository calls.

[TRAP] HQL mein table naam likhna (surveys) = ERROR. Entity naam likho (Survey). HQL = entity based, SQL nahi.

5. Service — Business Logic + Transaction + Cache

```

@Service // Spring bean + naming convention
public class SurveyService {

    private final SurveyRepository surveyRepo; // final = IMMUTABLE
    private final UserRepository userRepo;

    // Constructor injection (BEST practice – immutable + testable)
    public SurveyService(SurveyRepository surveyRepo, UserRepository userRepo) {
        this.surveyRepo = surveyRepo;
        this.userRepo = userRepo;
    }

    @Transactional // Proxy: BEGIN → tera code → COMMIT/ROLLBACK
    public SurveyResponse createSurvey(SurveyRequest req) {
        User creator = userRepo.findById(req.getUserId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + req.getUserId()));
        // orElseThrow = nahi mila? RuntimeException → rollback → 404

        Survey survey = new Survey(); // TRANSIENT state
        survey.setTitle(req.getTitle()); // DTO se data → Entity mein
        survey.setStatus(SurveyStatus.DRAFT);
        survey.setCreatedBy(creator);

        Survey saved = surveyRepo.save(survey); // INSERT → PERSISTENT state
        return mapToResponse(saved); // Entity → DTO convert
    }

    @Cacheable(value = "surveys", key = "#id")
    public SurveyResponse getSurvey(Long id) {
        // Pehli baar: DB fetch + Redis store
        // Doosri baar: Redis se return, DB SKIP
        Survey survey = surveyRepo.findByIdWithResponses(id)
            .orElseThrow(() -> new ResourceNotFoundException("Survey not found"));
        return mapToResponse(survey);
    }

    @CacheEvict(value = "surveys", key = "#id")
    @Transactional
    public SurveyResponse updateSurvey(Long id, SurveyRequest req) {
        Survey s = surveyRepo.findById(id).orElseThrow(...);
        s.setTitle(req.getTitle());
        // save() NAHI chahiye! Dirty checking – @Transactional commit pe auto UPDATE
        return mapToResponse(s);
    }
    // @CacheEvict = cache DELETE → next get pe fresh DB fetch
}

```

WHY: Constructor injection KYUN? (1) final = immutable. (2) Test = new Service(mockRepo). (3) Circular dependency = startup FAIL = pata chale.

WHY: Dirty Checking: @Transactional mein entity modify kiya = commit pe Hibernate COMPARE = different? AUTO UPDATE. save() call ZARURI NAHI.

[TRAP] @Transactional same class mein doosra method call = PROXY BYPASS = txn/cache/async DEAD. ALAG @Service mein rakho.

[TRAP] Catch block mein exception swallow kiya? Proxy ko pata nahi = COMMIT ho jayega = galat state save!

6. Controller — HTTP Handle, ZERO Logic

```

@RestController // @Controller + @ResponseBody = auto JSON
@RequestMapping("/api/v1/surveys") // Base URL + API versioning (/v1)
public class SurveyController {

    private final SurveyService surveyService;

    public SurveyController(SurveyService surveyService) {
        this.surveyService = surveyService;
    }

    @PostMapping // POST /api/v1/surveys = CREATE
    public ResponseEntity<SurveyResponse> create(
        @Valid @RequestBody SurveyRequest request) {
        // @RequestBody = JSON → Java (Jackson)
        // @Valid = validation TRIGGER (@NotBlank, @Size check)
        // Bina @Valid = validation DEAD!
        SurveyResponse response = surveyService.createSurvey(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response); // 201
    }

    @GetMapping("/{id}") // GET /api/v1/surveys/123
    public SurveyResponse getById(@PathVariable Long id) {
        return surveyService.getSurvey(id);
    }

    @GetMapping // GET /api/v1/surveys?status=ACTIVE&page=0&size=20
    public Page<SurveyResponse> getAll(
        @RequestParam(required = false) SurveyStatus status,
        // required=false = optional parameter. Default=true = nahi bheja toh 400
        @PageableDefault(size = 20) Pageable pageable) {
        return surveyService.getByStatus(status, pageable);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        surveyService.deleteSurvey(id);
        return ResponseEntity.noContent().build(); // 204 No Content
    }
}

```

WHY: Controller = THIN. Sirf HTTP handle. Validation? @Valid karo. Logic? Service mein. Exception? @ControllerAdvice mein. Controller mein if-else = code smell.

WHY: ResponseEntity KYUN? HTTP status + headers + body CONTROL karo. POST = 201, DELETE = 204, GET = 200. Plain return = hamesha 200.

[TRAP] @Controller use kiya @RestController ki jagah? JSON nahi jayega — View name dhundhega. @RestController = @Controller + @ResponseBody.

7. @Transactional — Proxy Internal Working

→ Tu likhe:

```
@Transactional
public void placeOrder(OrderRequest req) {
    orderRepo.save(order);
    paymentService.charge(req);
    inventoryService.reduce(req);
}
```

→ Spring ACTUALLY ye chala raha:

```
// PROXY generated (tu nahi likhta):
public void placeOrder(OrderRequest req) {
    tx.begin(); // Step 1: START
    try {
        orderRepo.save(order); // TERA CODE
        paymentService.charge(req);
        inventoryService.reduce(req);
        tx.commit(); // Step 2: SUCCESS = COMMIT
    } catch (RuntimeException e) {
        tx.rollback(); // Step 3: FAIL = UNDO SAB
        throw e;
    }
}
```

WHY: Bina @Transactional: save() COMMIT, charge() FAIL = order DB mein hai, payment nahi. DATA CORRUPT. @Transactional = sab ya kuch nahi = ATOMIC.

→ REQUIRED vs REQUIRES_NEW:

```
// OrderService
@Transactional // REQUIRED (default)
public void placeOrder(OrderRequest req) {
    orderRepo.save(order);
    auditService.log("Order placed"); // ALAG @Service mein!
}

// AuditService (ALAG class!)
@Transactional(propagation = REQUIRES_NEW) // NAYA transaction
public void log(String message) {
    auditRepo.save(new AuditLog(message));
    // Order fail bhi ho = audit SAVE rahega (alag txn)
}
```

WHY: REQUIRED = existing txn JOIN karo. Inner fail = dono rollback. REQUIRES_NEW = HAMESHA naya txn. Inner fail = sirf inner rollback. Audit log = HAMESHA save.

[TRAP] readOnly=true → dirty checking OFF → setTitle() kiya bhi toh DB mein NAHI jayega. GET endpoints pe lagao.

8. @Cacheable — READ Cache, WRITE Evict

```

// READ – cache se do
@Cacheable(value = "surveys", key = "#id")
public SurveyResponse getSurvey(Long id) {
    // Pehli baar: DB fetch → Redis store
    // Doosri baar: Redis se return → method EXECUTE hi nahi hua!
    return mapToResponse(repo.findById(id).orElseThrow(...));
}

// UPDATE – cache HATAO
@CacheEvict(value = "surveys", key = "#id")
@Transactional
public SurveyResponse updateSurvey(Long id, SurveyRequest req) {
    Survey s = repo.findById(id).orElseThrow(...);
    s.setTitle(req.getTitle()); // Dirty checking = auto UPDATE
    return mapToResponse(s);
}
// Update ke baad cache DELETE → next get = fresh DB fetch

// DELETE – cache HATAO
@CacheEvict(value = "surveys", key = "#id")
@Transactional
public void deleteSurvey(Long id) {
    repo.deleteById(id);
}

```

WHY: Pattern: READ = @Cacheable. WRITE/DELETE = @CacheEvict. Simple.

[TRAP] null return kiya @Cacheable se = null CACHED = har future call null = DB hit ZERO. FIX: orElseThrow() use karo, null mat return.

[TRAP] @EnableCaching MISSING = @Cacheable DEAD. Same: @EnableAsync zaruri @Async ke liye.

9. @Async — User Ko Wait Mat Karwao

→ Bina @Async: saveOrder(50ms) + email(3s) + notification(2s) = 5 sec user wait.

→ @Async se: saveOrder(50ms) → RESPONSE. Background: email | notification | analytics PARALLEL.

```

// Step 1: Enable + configure thread pool
@Configuration
@EnableAsync
public class AsyncConfig {
    @Bean
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
        e.setCorePoolSize(5); // 5 threads always ready
        e.setMaxPoolSize(10); // Max 10 under load
        e.setQueueCapacity(25); // 25 tasks wait in queue
        e.initialize();
        return e;
    }
}

// Step 2: Async methods – ALAG @Service mein!
@Service
public class NotificationService {
    @Async
    public void sendEmail(Order order) {
        emailClient.send(order.getEmail(), ...); // 3 sec – ALAG thread
    }
    @Async
    public CompletableFuture<Boolean> sendSMS(Order order) {
        boolean sent = smsClient.send(...); // Result chahiye? CompletableFuture
        return CompletableFuture.completedFuture(sent);
    }
}

// Step 3: Service mein use
@Transactional
public OrderResponse placeOrder(OrderRequest req) {
    Order order = orderRepo.save(new Order(req));
    notifService.sendEmail(order); // fire & forget – ALAG thread
    return mapToResponse(order); // 50ms mein response!
}

```

[TRAP] @Async void mein exception = SILENTLY LOST. CompletableFuture return karo + .exceptionally() handle karo.

[TRAP] Same class call = proxy BYPASS = async NAHI. @Transactional, @Cacheable, @Async — TEENO ka trap SAME. Alag @Service.
[TRAP] Default SimpleAsyncTaskExecutor = har call NAYA thread = 10000 calls = OOM. Custom ThreadPoolTaskExecutor HAMESHA configure.

10. N+1 Problem — SABSE Common Performance Bug

→ 10 doctors fetch, har ek ke patients chahiye:

→ GALAT: 1 query (doctors) + 10 queries (har doctor ke patients) = 11 queries!

→ SAHI: JOIN FETCH = 1 query mein sab.

```
// ❌ N+1 — GALAT
@Entity
public class Doctor {
    @OneToMany(mappedBy = "doctor", fetch = FetchType.LAZY)
    private List<Patient> patients;
}
// findAll() = 1 query
// for loop mein getPatients() = 10 queries = N+1!

// ✅ FIX 1: JOIN FETCH
@Query("SELECT d FROM Doctor d JOIN FETCH d.patients")
List<Doctor> findAllWithPatients();
// 1 query mein doctors + patients DONO!

// ✅ FIX 2: @EntityGraph
@EntityGraph(attributePaths = {"patients"})
List<Doctor> findAll();
// Spring internally JOIN FETCH add kare

// ✅ FIX 3: @BatchSize
@BatchSize(size = 20)
private List<Patient> patients;
// 10 alag queries ke jagah 1 batch:
// WHERE doctor_id IN (1,2,3,...,10)
```

WHY: N+1 = loop mein lazy field access = extra queries. Fix = JOIN FETCH = 1 query mein sab. LAZY default rakho + jab chahiye tab JOIN FETCH.

[TRAP] EAGER mein bhi N+1! EAGER ne sirf timing badla (TURANT fetch), query count NAHI. Fix same: JOIN FETCH.

[TRAP] JOIN FETCH se DUPLICATE entities aa sakte. FIX: SELECT DISTINCT s FROM Survey s JOIN FETCH s.responses

WHY: Detect kaise? spring.jpa.show-sql=true → log mein dekho kitne queries. 1 expected, 11 dikhe = N+1!

11. Request Flow — Client Se DB Tak Aur Wapas

```
CLIENT POST /api/v1/surveys {"title":"Health Survey"}
↓
JwtAuthFilter (OncePerRequestFilter)
  | Token valid? → SecurityContext set
  | Token invalid? → 401 Unauthorized RETURN
  ↓
SurveyController.create(@Valid @RequestBody req)
  | @Valid trigger → Bean Validation check
  | title blank? → 400 Bad Request RETURN
  ↓
SurveyService.createSurvey(req) ← @Transactional START
  | PROXY: Transaction BEGIN
  | userRepo.findById() → SELECT from users
  | Not found? → ResourceNotFoundException → 404
  | survey.save() → INSERT into surveys
  | PROXY: Transaction COMMIT
  | mapToResponse(entity → DTO)
  ↓
SurveyController
  | ResponseEntity.status(201).body(response)
  ↓
CLIENT ← 201 Created {"id":1, "title":"Health Survey", "status":"DRAFT"}
```

INTERVIEW: "Request flows through JWT filter for auth, controller validates input, service manages transactions and business logic, repository handles data access, DTO prevents entity exposure. Exceptions caught centrally by @RestControllerAdvice."

12. Custom Exception — Apni Exceptions Banao

→ Generic RuntimeException throw karo toh catch mein pata nahi KYA fail hua. Custom exception = meaningful naam + specific HTTP status.

```
// Custom exception — 404 Not Found
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String resource, Long id) {
        super(resource + " not found with id: " + id);
    }
}

// Custom exception — 409 Conflict
@ResponseStatus(HttpStatus.CONFLICT)
public class DuplicateEmailException extends RuntimeException {
    public DuplicateEmailException(String email) {
        super("Email already exists: " + email);
    }
}
```

→ Service mein use — CLEAN pattern:

```
@Service
public class UserService {

    public User getUser(Long id) {
        return repo.findById(id)
            .orElseThrow(() ->
                new ResourceNotFoundException("User", id));
        // → 404: "User not found with id: 5"
    }

    public User createUser(UserDto dto) {
        if (repo.existsByEmail(dto.getEmail())) {
            throw new DuplicateEmailException(dto.getEmail());
        }
        // → 409: "Email already exists: arpan@mail.com"
        return repo.save(mapper.toEntity(dto));
    }
}
```

WHY custom? ResourceNotFoundException = meaningful naam. Catch mein pata chale KYA fail hua. Plus @ResponseStatus se HTTP code automatic map. RuntimeException extend = @Transactional auto rollback.

WHY orElseThrow? Optional.get() = NoSuchElementException (generic). orElseThrow = TERA exception, TERA message, TERA HTTP status.

[TRAP] Checked exception (extends Exception) throw kiya @Transactional mein? DEFAULT mein rollback NAHI hoga! Sirf RuntimeException pe rollback. FIX: @Transactional(rollbackFor = Exception.class)

INTERVIEW: "I create custom exceptions per business rule, throw from service layer which triggers @Transactional rollback, and catch centrally in @RestControllerAdvice. Client gets structured JSON errors."

13. GlobalExceptionHandler — @RestControllerAdvice

→ Sab controllers ke liye EK GLOBAL error handler. Koi bhi controller se exception aaye → pehle yahan check → match mila → handle.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // 404 – Resource not found
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handle404(
        ResourceNotFoundException ex) {
        return ResponseEntity.status(404).body(
            new ErrorResponse("Not Found", ex.getMessage()));
    }

    // 409 – Duplicate/Conflict
    @ExceptionHandler(DuplicateEmailException.class)
    public ResponseEntity<ErrorResponse> handle409(
        DuplicateEmailException ex) {
        return ResponseEntity.status(409).body(
            new ErrorResponse("Conflict", ex.getMessage()));
    }

    // 400 – Validation errors (field-level)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handle400(
        MethodArgumentNotValidException ex) {
        Map<String, String> errors = ex.getBindingResult()
            .getFieldErrors().stream()
            .collect(Collectors.toMap(
                FieldError::getField,
                FieldError::getDefaultMessage));
        return ResponseEntity.status(400).body(
            new ErrorResponse("Validation Failed", errors));
        // → {"error":"Validation Failed",
        //     "details":{"title":"Title is required","maxPanelists":"must be >= 1"}}
    }

    // 500 – Generic fallback (LAST mein rakho)
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(
        Exception ex) {
        log.error("Internal error: ", ex); // LOG karo server mein
        return ResponseEntity.status(500).body(
            new ErrorResponse("Internal Server Error",
                "Something went wrong"));
        // Actual error client ko mat bhejo – security risk!
    }
}
```

→ ErrorResponse DTO — clean JSON response:

```
public record ErrorResponse(
    String error,
    Object details,
    String timestamp
) {
    public ErrorResponse(String error, Object details) {
        this(error, details, Instant.now().toString());
    }
}
```

WHY @RestControllerAdvice? Bina ye = har controller mein try-catch = code DUPLICATE. Ek jagah handle = DRY + consistent error format.
Flow: Service throw → @Transactional rollback → Exception propagate → @RestControllerAdvice catch → proper HTTP status + JSON.

[TRAP] @ExceptionHandler(Exception.class) = catch-all. Ye LAST mein rakho. Pehle specific exceptions handle karo (ResourceNotFound, Validation). Specific pehle, generic baad mein.

[TRAP] 500 error mein actual exception message client ko mat bhejo — "NullPointerException at line 42" = security risk. Generic message do, actual error LOG karo server mein.

INTERVIEW: "I use @RestControllerAdvice for centralized exception handling. Custom exceptions map to specific HTTP status codes. Validation errors return field-level messages. Internal errors are logged but not exposed to clients."

14. SecurityConfig — JWT + BCrypt + CORS + Endpoint Rules

→ POORA security setup ek file mein. Ye file Spring ko batati: kaun access kar sakta, kaise authenticate karo, kaunsa filter lagao.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(
        HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            // JWT use karte = CSRF off
            // (token header mein, browser auto nahi bhejta)

            .cors(cors -> cors.configurationSource(corsConfig()))

            .sessionManagement(s -> s
                .sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))
            // REST = stateless. Har request mein token aana chahiye.
            // Server pe koi session NAHI stored.

            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                // Login/Register = PUBLIC – token nahi chahiye
                .requestMatchers("/actuator/health").permitAll()
                // Health check = PUBLIC
                .requestMatchers("/api/admin/**")
                .hasRole("ADMIN")
                // Admin endpoints = sirf ADMIN role
                .anyRequest().authenticated()
                // Baaki SAB = token ZARURI
            )

            .addFilterBefore(jwtFilter,
                UsernamePasswordAuthenticationFilter.class)
            // JWT filter PEHLE chale – token check kare
            // phir Spring Security authorize kare

            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
        // BCrypt = one-way hash. Plain text KABHI store mat karo.
        // Same password = DIFFERENT hash har baar (salt).
    }

    @Bean
    public CorsConfigurationSource corsConfig() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of(
            "http://localhost:3000")); // Frontend URL
        config.setAllowedMethods(List.of(
            "GET", "POST", "PUT", "DELETE"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source =
            new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }
}
```

WHY CSRF disable? CSRF attack = browser automatically cookies bhejta. JWT header mein jaata, browser auto nahi bhejta. Toh CSRF protection ZARURI nahi JWT mein.

WHY STATELESS? REST API = har request independent. Server pe session store karna = memory waste + scaling problem. JWT mein SAB info hai — session ki zarurat nahi.

WHY addFilterBefore? JWT filter PEHLE chale = token validate kare + SecurityContext set kare. PHIR Spring Security authorize kare (permitAll/authenticated check). Order ULTA kiya = authentication SKIP.

[TRAP] permitAll BAAD mein likha anyRequest().authenticated() ke? Spring PEHLA match use karta — sab block! Specific rules PEHLE, anyRequest LAST.

[TRAP] BCrypt plain text compare se nahi hota — passwordEncoder.matches(raw, hashed) use karo. equals() = HAMESHA false.

INTERVIEW: "SecurityConfig disables CSRF for JWT, sets stateless session policy, defines endpoint authorization rules with permitAll for public endpoints, requires authentication for everything else, and adds JWT filter before Spring's default authentication filter."

15. JwtAuthFilter — OncePerRequestFilter, Token Validate

→ HAR request pe ek baar chale. Token check kare → valid hai toh SecurityContext set kare → Spring Security ko bole "ye user authenticated hai."

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    // OncePerRequestFilter = HAR request pe EXACTLY ek baar chale

    private final JwtUtil jwtUtil;
    private final UserDetailsService userService;

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain chain)
        throws ServletException, IOException {

        // Step 1: Header se token nikalo
        String header = request.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(request, response);
            return; // No token = let Spring Security handle
            // Public endpoints = OK. Protected = 401.
        }

        String token = header.substring(7);
        // "Bearer eyJhbGc..." → "eyJhbGc..." (7 chars hatao)

        // Step 2: Token se email nikalo
        String email = jwtUtil.extractEmail(token);

        // Step 3: Validate + SecurityContext set karo
        if (email != null && SecurityContextHolder
            .getContext().getAuthentication() == null) {
            // Authentication null = abhi tak koi set nahi kiya

            UserDetails user = userService
                .loadUserByUsername(email);
            // DB se user load karo

            if (jwtUtil.isTokenValid(token, user)) {
                // Token valid = SecurityContext mein set karo
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        user, null, user.getAuthorities());
                SecurityContextHolder.getContext()
                    .setAuthentication(auth);
                // Ab Spring Security jaanta: ye user AUTHENTICATED hai
            }
        }

        // Step 4: Next filter ko bhejo
        chain.doFilter(request, response);
        // chain.doFilter HAMESHA call karo — warna request STUCK
    }
}
```

WHY OncePerRequestFilter? Normal Filter forward/redirect pe DOBARA chale. OncePerRequestFilter = GUARANTEE ek baar. JWT validate ek baar karo — dobara kyun?

WHY SecurityContext? Spring Security SecurityContext check karta — set hai toh authenticated, nahi toh 401. JWT filter ka kaam = token valid hai toh SecurityContext FILL karo. Invalid/missing = EMPTY chhodo → Spring 401 return kare.

[TRAP] chain.doFilter() call BHOOL gaye? Request STUCK — koi response nahi jayega. HAMESHA call karo — valid ho ya invalid.

[TRAP] Token expired ya tampered? isValid() false return kare → SecurityContext EMPTY rahe → Spring Security 401 return kare. Manually 401 mat bhejo — Spring handle kare.

INTERVIEW: "JWT filter runs once per request via OncePerRequestFilter. It extracts token from Authorization header, validates it, loads user from DB, and sets SecurityContext. If token is invalid or missing, request proceeds without authentication and Spring Security returns 401 for protected endpoints."

16. application.yml — Poora Config Ek Jagah

→ Application ka POORA configuration ek file mein. Interview mein poochhte: "application.yml mein kya kya configure karte ho?" — har line ka matlab pata hona chahiye.

```
server:
  port: 8081                                # Default 8080, change kiya

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/iris_db
    username: root
    password: root                          # Production mein ENV variable!
    hikari:
      maximum-pool-size: 20                 # Max 20 connections
      minimum-idle: 5                       # Min 5 idle connections ready
      connection-timeout: 30000             # 30sec wait, phir exception

  jpa:
    hibernate:
      ddl-auto: validate                    # PRODUCTION = validate (sirf check)
                                           # create = DROP + CREATE (data gone!)
                                           # update = ALTER (column add, delete NAHI)
                                           # none = kuch mat karo
    show-sql: false                        # true = debugging, false = prod
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQL8Dialect

  data:
    redis:
      host: localhost
      port: 6379
      timeout: 2000                        # 2sec timeout

  cache:
    type: redis
    redis:
      time-to-live: 3600000                # 1 hour TTL (milliseconds)

  jwt:
    secret: mySecretKey123456789           # Production mein ENV variable!
    expiration: 900000                    # Access token = 15 min
    refresh-expiration: 604800000          # Refresh token = 7 days

  management:                             # Spring Boot Actuator
    endpoints:
      web:
        exposure:
          include: health,metrics,info # Sirf ye 3 expose — sab mat karo
```

WHY application.yml? Poora app ka config ek jagah. Environment specific = application-dev.yml, application-prod.yml. Spring profile se switch: spring.profiles.active=prod. Hierarchical format = readable.

WHY HikariCP config? Default pool size = 10. Production mein 100 requests ek saath aaye = connection wait = timeout. Pool size badhao, lekin DB handle kar sake utna hi. Tune HikariCP exhaustion production mein debug kiya — ye story interview mein ZAROOR bol.

[TRAP] jwt.secret hardcoded = SECURITY RISK. Production: environment variable ya Spring Cloud Config se inject karo. application.yml Git mein jaati = secret EXPOSED.

[TRAP] ddl-auto: update production mein = DANGEROUS. Column add OK lekin delete/rename NAHI karega = schema out of sync. Production = validate use karo + Flyway/Liquibase for migrations.

[TRAP] show-sql: true production mein = PERFORMANCE hit + log FLOOD. Debugging ke liye ON karo, production mein OFF.

INTERVIEW: "I configure datasource with HikariCP pool, set ddl-auto to validate in production with Flyway for migrations, use Redis for caching with TTL, externalize secrets via environment variables, and expose only health/metrics/info Actuator endpoints."

17. pom.xml — Dependencies, Kaunsi Kya Kaam Karti

→ Project mein kaunsi libraries chahiye = pom.xml. Interview mein poochhte: "aapke project mein kaunsi dependencies hain?" — har starter ka matlab pata hona chahiye.

```
<!-- PARENT – versions manage karta, tu version mat likh -->
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
</parent>

<dependencies>
  <!-- WEB = Spring MVC + Tomcat + Jackson (JSON) -->
  <dependency>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- JPA = Hibernate + Spring Data JPA -->
  <dependency>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- SECURITY = Spring Security + filter chain -->
  <dependency>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- REDIS = Cache + Session store -->
  <dependency>
    <artifactId>spring-boot-starter-data-redis</artifactId>
  </dependency>

  <!-- VALIDATION = @NotNull, @NotBlank, @Size, @Email -->
  <dependency>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- ACTUATOR = /health, /metrics, /info endpoints -->
  <dependency>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>

  <!-- MySQL Driver – runtime scope -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
    <!-- runtime = compile time nahi chahiye, sirf run time -->
  </dependency>

  <!-- Lombok – boilerplate reduce -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
    <!-- optional = dependent projects ko inherit NAHI hoga -->
  </dependency>

  <!-- TEST = JUnit 5 + Mockito + MockMvc -->
  <dependency>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
    <!-- test = sirf test mein available, production JAR mein NAHI -->
  </dependency>
</dependencies>
```

WHY starter-parent? Version management. Tu individual library versions mat likh — parent compatible versions KHUD decide kare. Version conflict = ZERO.

WHY starter concept? starter-web = Spring MVC + Tomcat + Jackson = TEEN libraries ek dependency mein. Bina starter = 10 alag dependencies manually add + version manage. Starter = "ek button mein sab ready."

WHY scope runtime? MySQL driver compile time pe NAHI chahiye — code mein directly use nahi karta. Hibernate internally use karta runtime pe. Compile time pe laga = unnecessary dependency leak.

[TRAP] starter-validation ALAG add karna padta. starter-web mein NAHI aata (Spring Boot 3+). Bina ye = @NotBlank, @Valid DEAD = validation CHALTA HI NAHI.

[TRAP] Lombok optional=true KYUN? Agar koi tera project dependency mein use kare, unko Lombok FORCE nahi hoga. Internal use ke liye = optional.

INTERVIEW: "My project uses starter-web for REST APIs, starter-data-jpa for Hibernate, starter-security for JWT auth, starter-data-redis for caching, starter-validation for request validation, and starter-actuator for health monitoring. MySQL connector at runtime scope, Lombok for boilerplate reduction."

18. @Configuration — Bean Factory, Singleton Guarantee

→ Tune Day 2 mein doubt poocha tha: "@Configuration kya hai aur KYUN?" — Simple answer:

→ Spring ko bolo: "is class mein BEANS banane ki RECIPES hain." Jaise kitchen mein recipe book — @Configuration = recipe book, @Bean methods = individual recipes.

```
@Configuration // "Ye class config hai — andar @Bean methods hain"
public class AsyncConfig {

    @Bean // "Ye method jo return kare = Spring bean ban jaaye"
    public Executor taskExecutor() {
        return new ThreadPoolTaskExecutor();
    }
}
```

WHY @Configuration specifically? @Configuration mein @Bean methods ko Spring ek baar call karta = SINGLETON guarantee. @Component mein @Bean likha toh har call pe NAYA object — singleton NAHI. Ye IMPORTANT fark hai.

```
// @Configuration mein:
@Bean
public DataSource dataSource() { return new HikariDataSource(); }

@Bean
public JdbcTemplate jdbcTemplate() {
    return new JdbcTemplate(dataSource());
    // dataSource() call kiya — lekin SAME object milega (singleton)
    // Spring proxy intercept kare — cached bean return kare
}

// @Component mein SAME code likhte:
// dataSource() call = NAYA HikariDataSource har baar!
// 2 alag pools = CONNECTION LEAK = production crash
```

[TRAP] @Bean method @Component class mein likha? Singleton guarantee NAHI. Doosra @Bean call kare toh NAYA object. @Configuration mein rakho — proxy singleton ensure kare.

WHY bina @Configuration @Bean NAHI kaam karta? Spring ko pata nahi ki yahan beans dhundhni hain. @Configuration = marker ki "yahan dekho, beans hain." Bina ye = @Bean methods IGNORED.

19. Entity Direct Return — 3 REAL Problems (Code Examples)

→ Section 3 mein WHY bataya. Ab ACTUAL code dikhata hoon — kya TOOTTA hai jab Entity directly return karo:

→ Problem 1: PASSWORD EXPOSE

```
// Controller Entity DIRECTLY return kar raha
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return userRepo.findById(id).orElseThrow();
}

// Client ko ye JSON milta:
// GET /api/v1/users/123
// Response:
{
    "id": 123,
    "name": "Arpan",
    "email": "arpan@iris.com",
    "password": "$2a$10$xyz...", // ← PASSWORD EXPOSE!
    "role": "ADMIN",             // ← INTERNAL role EXPOSE!
    "createdAt": "2024-01-01"
}
// Attacker ke liye goldmine — BCrypt hash bhi sensitive hai
```

→ Problem 2: LazyInitializationException

```
// Entity mein:
@OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
private List<Survey> surveys;

// Controller Entity return kare:
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return userRepo.findById(id).orElseThrow();
}

// Jackson serialize kare → surveys field access kare
// Hibernate session BAND ho chuki → BOOM!
// LazyInitializationException:
// could not initialize proxy - no Session
// App CRASH – 500 Internal Server Error
```

→ Problem 3: Tight Coupling — DB change = API TOOTA

```
// DB mein column rename kiya: title → survey_title
// Entity update kiya:
@Column(name = "survey_title")
private String surveyTitle; // field naam bhi badla

// Bina DTO: API response BHI badla!
// Pehle: {"title": "Health Survey"}
// Ab:    {"surveyTitle": "Health Survey"}
// Client ka code TOOTA – "title" field nahi mila!

// DTO ke saath: Entity badle, DTO SAME rahe
public class SurveyResponse {
    private String title; // Client ko YE dikhta – DB se decouple
}
// Client ko PATA bhi nahi DB mein kya hua
```

WHY DTO = FILTER (sirf needed fields) + PROTECTION (password hide) + DECOUPLING (DB change, API same) + SAFETY (LazyInit avoid). Chaar problems ek solution.

INTERVIEW: "I never expose entities directly. DTOs decouple the API contract from the database schema, prevent accidental data exposure like passwords, and avoid LazyInitializationException when serializing lazy associations."

20. Entity Mapping — Relationships Kaise Define Karte

→ 3 relationships: @ManyToOne (bahut se ek), @OneToMany (ek se bahut), @ManyToMany (bahut se bahut).

→ @ManyToOne — SABSE common (Survey → User):

```
@Entity
public class Survey {

    @ManyToOne(fetch = FetchType.LAZY)    // LAZY karo HAMESHA!
    @JoinColumn(name = "created_by")      // FK column = created_by
    private User createdBy;
    // Survey = OWNING SIDE (jisme FK hai = owner)
    // DEFAULT = EAGER (One side) – explicitly LAZY set karo
}

// DB: surveys table: id | title | created_by (FK → users.id)
```

→ @OneToMany — Reverse side (User → Surveys):

```
@Entity
public class User {

    @OneToMany(
        mappedBy = "createdBy",          // Survey mein FIELD naam
        // mappedBy = "main OWNER pe dekho, mere paas FK nahi"
        cascade = CascadeType.ALL,        // User save → surveys bhi save
        orphanRemoval = true              // List se hataya → DB se DELETE
    )
    private Set<Survey> surveys = new HashSet<>();
    // Set KYUN List nahi? Set = duplicate nahi + efficient
    // List + orphanRemoval = Hibernate DELETE+INSERT bug!
}
```

→ @ManyToMany — Join Table (User ↔ Role):

```
@Entity
public class User {

    @ManyToMany
    @JoinTable(
        name = "user_roles", // THIRD table
        joinColumns = @JoinColumn(name = "user_id"), // IS entity FK
        inverseJoinColumns = @JoinColumn(name = "role_id") // DOOSRI entity FK
    )
    private Set<Role> roles = new HashSet<>();
}

// DB: 3 tables – users, roles, user_roles (join table)
```

WHY Owning Side? Jis entity mein @JoinColumn ya @JoinTable = OWNING SIDE. DB changes SIRF owning side se hote. Non-owning side (mappedBy) mein add kiya = DB mein KUCH NAHI hoga! Dono side sync karo — FK set karo + collection mein add karo.

[TRAP] List + @OneToMany + orphanRemoval = Hibernate pehle SARI rows DELETE kare, phir bachi hui WAPAS INSERT. Set use karo — sirf removed wali DELETE = efficient.

[TRAP] mappedBy mein FIELD naam likho (createdBy), COLUMN naam NAHI (created_by). Galat = MappingException on startup.

[TRAP] Non-owning side se add kiya (user.getSurveys().add(survey)) lekin owning side mein FK set nahi kiya (survey.setCreatedBy(user)) = FK NULL rahega DB mein!

INTERVIEW: "@ManyToOne is the owning side with @JoinColumn. @OneToMany uses mappedBy pointing to the owning field. I use Set over List for collections to avoid Hibernate's batch delete-reinsert with orphanRemoval. @ManyToMany uses @JoinTable with two foreign keys."

21. Hibernate States — Object Ka Lifecycle (4 States)

→ TRANSIENT → PERSISTENT → DETACHED → REMOVED. Dirty Checking sirf PERSISTENT mein kaam karta.

```
// == STATE 1: TRANSIENT – Hibernate ko PATA nahi ==
Survey s = new Survey(); // TRANSIENT
s.setTitle("Health Survey"); // Java object, DB mein NAHI
// id = null. Session se koi connection NAHI.
// Dirty checking? NAHI. DB mein jayega? NAHI.

// == STATE 2: PERSISTENT – Hibernate WATCH kar raha ==
Survey saved = repo.save(s); // PERSISTENT!
// INSERT query fire. id = generated. Session mein tracked.

saved.setTitle("Updated Title"); // KUCH mat karo!
// Dirty Checking ACTIVE = commit pe Hibernate COMPARE kare
// Different? AUTO UPDATE. save() call ZARURI NAHI.

// == STATE 3: DETACHED – Hibernate BHool gaya ==
// @Transactional method end = session close = DETACHED
// Ab setTitle() karo = DB mein NAHI jayega
// Wapas PERSISTENT? → repo.save(detached) = merge internally

// == STATE 4: REMOVED – DELETE marked ==
repo.delete(saved); // REMOVED
// DELETE query commit pe fire hoga
// Object memory mein hai lekin DB se HATEGA
```

WHY Dirty Checking? PERSISTENT state mein Hibernate SNAPSHOT rakhta load time ka. Commit pe COMPARE karta — different? AUTO UPDATE SQL fire. save() ZARURI NAHI @Transactional mein. Ye Hibernate ka SABSE important concept hai.

WHY States matter? Spring Data JPA mein: repo.save() = TRANSIENT hai toh INSERT (persist). ID hai toh MERGE (update). repo.delete() = REMOVED. @Transactional end = session close = sab DETACHED.

[TRAP] @Transactional(readOnly=true) → dirty checking OFF. setTitle() kiya bhi toh UPDATE NAHI hoga. GET endpoints pe lagao — performance BETTER (no snapshot compare).

[TRAP] DETACHED entity pe LAZY field access (getResponses()) = LazyInitializationException! Session band = proxy dead. FIX: @Transactional mein access karo ya JOIN FETCH.

[TRAP] evict(obj) = PERSISTENT → DETACHED (L1 cache se hatao). DB se DELETE NAHI karta! clear() = SARI entities DETACHED. Dono tracking band karte, delete NAHI.

[TRAP] Ek session mein 2 objects same id = NonUniqueObjectException. update() risky. merge() ALWAYS safe — state COPY kare managed object mein. Spring save() = internally merge if id exists.

INTERVIEW: "Hibernate tracks entities through 4 states. Dirty checking in PERSISTENT state auto-detects changes and generates UPDATE on commit — no explicit save needed. @Transactional boundary controls session lifecycle. readOnly=true skips dirty checking for better performance on read endpoints."

22. Exception Flow — Service Se Client Tak, POORA Journey

→ Request → Service mein galat hua → Exception throw → @Transactional ROLLBACK → @RestControllerAdvice catch → Client ko clean JSON error.

→ Step 1: Custom Exceptions banao (RuntimeException extend = auto rollback):

```
// 404 – Resource not found
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String resource, Long id) {
        super(resource + " not found with id: " + id);
    }
}

// 409 – Duplicate data
public class DuplicateEmailException extends RuntimeException {
    public DuplicateEmailException(String email) {
        super("Email already exists: " + email);
    }
}

// RuntimeException extend = @Transactional AUTO ROLLBACK
// Checked (extends Exception) = rollback NAHI (TRAP!)
```

→ Step 2: Service mein throw karo:

```
@Service
public class UserService {

    @Transactional
    public UserResponse createUser(UserRequest req) {
        if (repo.existsByEmail(req.getEmail())) {
            throw new DuplicateEmailException(req.getEmail());
            // → @Transactional ROLLBACK (RuntimeException)
            // → Propagate → @RestControllerAdvice catch
        }
        User user = new User();
        user.setName(req.getName());
        user.setEmail(req.getEmail());
        user.setPassword(passwordEncoder.encode(req.getPassword()));
        // BCrypt hash – plain text KABHI save mat karo
        return mapToResponse(repo.save(user));
    }

    public UserResponse getUser(Long id) {
        User user = repo.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("User", id));
        // → 404: "User not found with id: 5"
        // .get() KABHI use mat karo – generic NoSuchElementException
        return mapToResponse(user);
    }
}
```

→ Step 3: @RestControllerAdvice GLOBALLY catch kare:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // 404 – Not Found
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handle404(
        ResourceNotFoundException ex) {
        return ResponseEntity.status(404).body(
            new ErrorResponse("Not Found", ex.getMessage()));
    }

    // 409 – Conflict/Duplicate
    @ExceptionHandler(DuplicateEmailException.class)
    public ResponseEntity<ErrorResponse> handle409(
        DuplicateEmailException ex) {
        return ResponseEntity.status(409).body(
            new ErrorResponse("Conflict", ex.getMessage()));
    }

    // 400 – @Valid validation errors (field-level)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handle400(
        MethodArgumentNotValidException ex) {
        Map<String, String> errors = ex.getBindingResult()
            .getFieldErrors().stream()
            .collect(Collectors.toMap(
                FieldError::getField,
                FieldError::getDefaultMessage));
        return ResponseEntity.status(400).body(
            new ErrorResponse("Validation Failed", errors));
        // → {"error":"Validation Failed",
        //     "details":{"title":"required","email":"invalid"}}
    }

    // 500 – Generic fallback (LAST mein rakho!)
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        log.error("Internal error: ", ex); // LOG server mein
        return ResponseEntity.status(500).body(
            new ErrorResponse("Internal Server Error",
                "Something went wrong"));
        // Client ko actual error MAT bhejo = security risk!
    }
}
```

→ Step 4: ErrorResponse DTO (Java 17 record):

```
public record ErrorResponse(
    String error,
    Object details,
    String timestamp
) {
    public ErrorResponse(String error, Object details) {
        this(error, details, Instant.now().toString());
    }
}
// 404: {"error":"Not Found","details":"User not found with id: 5","timestamp":"..."}
// 400: {"error":"Validation Failed","details":{"title":"required","email":"invalid"}}
// 500: {"error":"Internal Server Error","details":"Something went wrong"}
```

WHY @RestControllerAdvice? Bina ye = har controller mein try-catch = code DUPLICATE. Ek jagah handle = DRY + consistent error format.
Flow: Service throw → @Transactional rollback → Propagate → @RestControllerAdvice catch → proper HTTP status + JSON.

WHY orElseThrow()? .get() = NoSuchElementException (generic, useless). orElseThrow() = TERI exception, TERA message, TERA HTTP status. .get() KABHI use mat karo.

[TRAP] Checked exception (extends Exception) @Transactional mein throw kiya? DEFAULT rollback NAHI hoga! Sirf RuntimeException pe rollback. FIX: @Transactional(rollbackFor = Exception.class) ya RuntimeException use karo.

[TRAP] @ExceptionHandler(Exception.class) = catch-all. LAST mein rakho. Pehle specific (ResourceNotFound, Validation). Spring SPECIFIC handler prefer karta, lekin readability ke liye order maintain karo.

[TRAP] 500 mein actual exception client ko mat bhejo — 'NullPointerException at UserService.java:42' = security risk. Attacker ko internal structure pata chale. Generic message do, actual error LOG karo server mein.

[TRAP] @Valid BHOOL gaye @RequestBody ke saath? Validation DEAD. @NotBlank, @Email, @Size — sab BEKAAR. Empty/null sab ACCEPT ho jayega. @Valid @RequestBody HAMESHA saath likho.

INTERVIEW: "I create custom RuntimeExceptions per business rule. Service throws them, @Transactional auto-rolls back, and @RestControllerAdvice catches centrally — mapping each exception to a specific HTTP status. Validation errors return field-level messages. Internal errors are logged but never exposed to clients."

23. Annotations Reference — Sab Annotations Ek Jagah, Quick Revision

→ Interview se pehle ek baar padh lo — har annotation ka naam, kya karti, kyun karti. Quick reference.

Entity / JPA Annotations

@Entity — Ye class = database TABLE. Hibernate isko manage karega.

WHY: Bina @Entity = Hibernate ignore. Table nahi banega.

WHERE: Entity classes pe (Survey.java, User.java)

@Table(name = "surveys") — Table ka naam specify. Nahi likha = class naam = table naam.

WHY: Explicit BETTER — confusion nahi.

WHERE: Entity class pe

@Id — Ye field = PRIMARY KEY. Har entity mein ZARURI.

WHY: Bina @Id = startup error: 'No identifier specified'.

WHERE: Entity mein ek field pe

@GeneratedValue(strategy = IDENTITY) — DB auto-increment kare. Tu id SET mat kar.

WHY: IDENTITY = MySQL auto_increment. SEQUENCE = PostgreSQL/Oracle.

WHERE: Id field pe

@Column(nullable=false, length=200) — DB column constraints. NOT NULL + VARCHAR(200).

WHY: Java level validation ALAG, DB level constraint ALAG. Dono chahiye.

WHERE: Entity fields pe

@Enumerated(EnumType.STRING) — Enum ko STRING store karo DB mein ('DRAFT', 'ACTIVE').

WHY: ORDINAL = position store (0,1,2). Enum order badle = DATA CORRUPT silently!

WHERE: Enum fields pe

@CreationTimestamp — INSERT pe current time AUTO set.

WHY: Manually set karne ki zarurat nahi. Hibernate handle kare.

WHERE: LocalDateTime field pe

@ManyToOne(fetch = LAZY) — Bahut se ek. FK IS entity mein. OWNING SIDE.

WHY: Default EAGER — explicitly LAZY karo HAMESHA. EAGER = N+1 trap.

WHERE: Relationship field pe

@OneToMany(mappedBy, cascade, orphanRemoval) — Ek se bahut. NON-OWNING side. mappedBy = owner field naam.

WHY: Set use karo List nahi — orphanRemoval + List = DELETE+INSERT bug.

WHERE: Collection field pe

@ManyToMany + @JoinTable — Bahut se bahut. THIRD join table banti. 2 FKs.

WHY: joinColumns = IS entity FK. inverseJoinColumns = DOOSRI entity FK.

WHERE: Collection field pe

@JoinColumn(name = "created_by") — FK column naam specify. OWNING SIDE mein lagao.

WHY: Bina ye = Hibernate default naam dega (ugly + confusing).

WHERE: Relationship field pe

@Transient — Ye field DB mein save NAHI hoga. Hibernate SKIP kare.

WHY: Calculated/temporary fields ke liye. transient keyword = serialization skip (ALAG!).

WHERE: Entity field pe

@Version — Optimistic Locking. Update pe version check — mismatch = OptimisticLockException.

WHY: Concurrent updates prevent. Pehle padha wala version match nahi = REJECT.

WHERE: Integer/Long field pe

Spring Core Annotations

@Component — Generic Spring bean. Spring scan kare, bean banaye.

WHY: Base annotation. @Service, @Repository, @Controller = specialized @Component.

WHERE: Any class pe

@Service — Business logic class. @Component ka specialized version.

WHY: Naming convention — Spring ko bhi pata ye service layer hai.

WHERE: Service classes pe

@Repository — Data access class. @Component + exception translation.

WHY: SQLException → DataAccessException auto translate. Consistent errors.

WHERE: Repository classes pe

@Controller — Web controller. VIEW return karta (JSP/HTML).

WHY: @RestController chahiye REST API ke liye, @Controller nahi.

WHERE: MVC controller pe

@RestController — @Controller + @ResponseBody. Auto JSON return.

WHY: REST API = HAMESHA @RestController. @Controller = view resolver dhundhega.

WHERE: REST controller pe

@Configuration — Bean factory class. @Bean methods andar.

WHY: Singleton GUARANTEE — @Component mein @Bean = har call NAYA object!

WHERE: @Bean methods wali class pe

@Bean — Method ka return value = Spring bean.

WHY: @Configuration mein = singleton. @Component mein = NAHI singleton.

WHERE: Config class ke methods pe

@Autowired — Dependency inject karo. FIELD injection (BAD practice).

WHY: Constructor injection BEST — final, testable, circular detect.

WHERE: Field/Constructor/Setter pe

@Qualifier("name") — Multiple beans same type — kaunsa inject karo specify.

WHY: @Primary = default pick. @Qualifier = SPECIFIC naam se pick.

WHERE: Inject point pe

@Primary — Multiple beans mein DEFAULT ye use ho.

WHY: @Qualifier se override ho sakta. @Primary = fallback default.

WHERE: Bean definition pe

@Value("\${property}") — Single property inject karo application.yml se.

WHY: 5+ properties = @ConfigurationProperties use karo (type-safe).

WHERE: Field pe

@Scope("prototype") — Har injection pe NAYA object. Default = singleton.

WHY: Singleton = 1 instance. Prototype = har baar naya. Request/Session bhi.

WHERE: Bean class/method pe

@Profile("dev") — Sirf specific profile mein bean active.

WHY: Dev/Prod alag config. spring.profiles.active=dev se switch.

WHERE: Bean class pe

@Conditional... — @ConditionalOnClass, @ConditionalOnProperty, @ConditionalOnMissingBean.

WHY: Auto-configuration ka backbone. Jar hai? Configure. Nahi? Skip.

WHERE: Config class/method pe

@SpringBootApplication — @Configuration + @EnableAutoConfiguration + @ComponentScan. TEEN-IN-ONE.

WHY: Entry point. Ek annotation = poora Spring Boot setup.

WHERE: Main class pe

Web / REST Annotations

@RequestMapping("/api/v1/surveys") — Base URL path. Class level pe lagao.

WHY: API versioning (/v1) yahan set karo.

WHERE: Controller class pe

@GetMapping / @PostMapping / @PutMapping / @DeleteMapping — HTTP method specific mapping.

WHY: GET=read, POST=create, PUT=full update, DELETE=remove. PATCH=partial.

WHERE: Controller methods pe

@PathVariable — URL se value nikalo: /users/{id} mein id.

WHY: Path mein required hota. Optional nahi.

WHERE: Method parameter pe

@RequestParam — Query string se value: ?status=ACTIVE.

WHY: required=true default. required=false = optional (nahi bheja toh null).

WHERE: Method parameter pe

@RequestBody — JSON body → Java object convert (Jackson).

WHY: @Valid saath lagao HAMESHA — bina @Valid validation DEAD.

WHERE: Method parameter pe

@Valid — Bean Validation trigger karo (@NotBlank, @Size, @Email check).

WHY: BHOOL gaye = validation CHALTA HI NAHI. Sab accept ho jayega.

WHERE: @RequestBody ke saath

@ResponseStatus(HttpStatus.NOT_FOUND) — Exception pe HTTP status auto set.

WHY: Custom exception pe lagao — @RestControllerAdvice mein bhi use.

WHERE: Exception class pe

@RestControllerAdvice — GLOBAL exception handler. Sab controllers ke liye.

WHY: DRY — har controller mein try-catch NAHI. Ek jagah handle.

WHERE: Exception handler class pe

@ExceptionHandler(Exception.class) — Specific exception catch karo.

WHY: Specific pehle, Exception.class (catch-all) LAST mein.

WHERE: @RestControllerAdvice methods pe

@CrossOrigin(origins = "...") — CORS allow karo specific origin se.

WHY: Per controller. Global = WebMvcConfigurer.addCorsMappings().

WHERE: Controller class/method pe

Validation Annotations (@Valid trigger kare)

@NotNull — null REJECT. Empty string ("") ACCEPT.

WHY: Sirf null check. Blank allow.

WHERE: DTO fields pe

@NotEmpty — null + empty REJECT. Whitespace ACCEPT.

WHY: null + "" reject. " " accept.

WHERE: DTO fields pe

@NotBlank — null + empty + whitespace REJECT. SABSE strict.

WHY: HAMESHA ye use karo String ke liye. Teeno cover.

WHERE: String fields pe

@Size(min=3, max=200) — Length check. Min/max characters.

WHY: String + Collection dono pe kaam.

WHERE: String/Collection fields pe

@Min(1) / @Max(100000) — Numeric range check.

WHY: Integer/Long fields pe. int default=0 trap — Integer use karo.

WHERE: Numeric fields pe

@Email — Email format check.

WHY: Basic format check. Production = regex pattern bhi.

WHERE: Email string field pe

Transaction / Cache / Async Annotations

@Transactional — Method ko transaction mein wrap karo. Proxy: BEGIN → code → COMMIT/ROLLBACK.

WHY: RuntimeException = auto rollback. Checked = NAHI. Same class call = proxy BYPASS.

WHERE: Service methods pe

@Transactional(readOnly = true) — Dirty checking OFF. Performance BETTER.

WHY: GET endpoints pe. setTitle() kiya bhi toh UPDATE NAHI hoga.

WHERE: Read-only methods pe

@Transactional(propagation = REQUIRES_NEW) — HAMESHA naya transaction. Purana SUSPEND.

WHY: Audit log, notification — main fail bhi ho toh ye SAVE rahe.

WHERE: Independent operations pe

@Transactional(rollbackFor = Exception.class) — Checked exceptions pe bhi rollback karo.

WHY: Default sirf RuntimeException. Ye FORCE kare sab pe rollback.

WHERE: Methods with checked exceptions

@Cacheable(value, key) — Pehli baar DB + cache store. Doosri baar cache se.

WHY: Proxy based — same class call = DEAD. null cache TRAP!

WHERE: Read methods pe

@CacheEvict(value, key) — Cache DELETE karo. Update/delete pe use.

WHY: Stale data prevent. Write ke baad cache hatao.

WHERE: Write/delete methods pe

@EnableCaching — Caching activate karo. Bina ye = @Cacheable DEAD.

WHY: Config class pe lagao. Same: @EnableAsync, @EnableScheduling.

WHERE: Config class pe

@Async — Method ALAG thread mein run karo. User wait nahi.

WHY: Proxy based — same class = DEAD. Default executor = OOM trap. Custom ThreadPool configure.

WHERE: Background task methods pe

@EnableAsync — Async activate karo. Bina ye = @Async DEAD.

WHY: Config class pe. Custom ThreadPoolTaskExecutor bhi yahan.

WHERE: Config class pe

@Scheduled(fixedRate/cron) — Periodic task run karo. Cron expression ya fixed rate.

WHY: @EnableScheduling ZARURI. fixedRate = ms mein.

WHERE: Scheduled task methods pe

Security Annotations

@EnableWebSecurity — Spring Security activate karo.

WHY: SecurityConfig class pe. FilterChain define karo.

WHERE: Security config class pe

@PreAuthorize("hasRole('ADMIN')") — Method level authorization. Role check PEHLE.

WHY: AOP proxy based — same class bypass YAHAN bhi.

WHERE: Controller/Service methods pe

SecurityFilterChain @Bean — HTTP security rules define — CSRF, session, endpoints, filters.

WHY: permitAll PEHLE, anyRequest LAST. Order MATTERS!

WHERE: SecurityConfig mein

Hibernate Specific Annotations

@BatchSize(size = 20) — N+1 fix — IN clause batch mein. 10 queries → 1.

WHY: size = ek batch mein kitne IDs. Zyada = kam queries.

WHERE: Collection field pe

@EntityGraph(attributePaths = {"patients"}) — N+1 fix — Spring internally JOIN FETCH add kare.

WHY: Repository method pe lagao. @Query likhne ki zarurat nahi.

WHERE: Repository methods pe

@NamedQuery / @Query — Custom HQL/JPQL query define karo.

WHY: @Query repository mein. HQL = ENTITY naam (Survey), SQL nahi (surveys).

WHERE: Repository methods pe

Spring Boot Config Annotations

@ConfigurationProperties(prefix = "app") — Group of related properties bind karo. Type-safe.

WHY: 5+ properties = ye use karo. @Value = single property.

WHERE: Config POJO class pe

@EnableAutoConfiguration — Classpath se auto configure. @SpringBootApplication mein included.

WHY: @ConditionalOnClass internally. Jar hai = configure.

WHERE: Main class pe (auto)

@ComponentScan — Package scan karo beans dhundhne ke liye. @SpringBootApplication mein included.

WHY: Default = same package + sub-packages. Alag package = miss!

WHERE: Main class pe (auto)

Lombok Annotations (Boilerplate Reduce)

@Getter / @Setter — Auto generate getter/setter methods.

WHY: Entity/DTO pe. Less code.

WHERE: Class pe

@NoArgsConstructor — No-argument constructor generate.

WHY: @Entity mein ZARURI — Hibernate reflection se create karta.

WHERE: Entity class pe

@AllArgsConstructor — All fields constructor generate.

WHY: Test mein useful. @Builder ke saath.

WHERE: Class pe

@Builder — Builder pattern auto generate. User.builder().name("X").build().

WHY: DTO creation clean. new User(a,null,b,null) se BETTER.

WHERE: Class pe

@RequiredArgsConstructor — final fields ka constructor auto generate.

WHY: Constructor injection ke liye PERFECT. @Autowired bhi zarurat nahi.

WHERE: Service/Controller pe

@Data — @Getter + @Setter + @ToString + @EqualsAndHashCode + @RequiredArgsConstructor.

WHY: DTO pe OK. Entity pe AVOID — equals/hashCode issues with Hibernate.

WHERE: DTO class pe

@Slf4j — Logger auto generate: log.info(), log.error().

WHY: Har class mein manually Logger declare nahi karna padta.

WHERE: Any class pe

[TRAP] PROXY BYPASS — @Transactional, @Cacheable, @Async, @PreAuthorize — CHARO ka trap SAME. Same class mein call = proxy ko pata nahi = annotation DEAD. ALAG @Service mein rakho.

[TRAP] @Enable... BHOOL gaye? @EnableCaching bina = @Cacheable DEAD. @EnableAsync bina = @Async DEAD. @EnableScheduling bina = @Scheduled DEAD. @EnableWebSecurity bina = Security DEAD.

INTERVIEW: "I use annotations for declarative programming — @Transactional for transactions, @Cacheable for caching, @Async for background tasks, @Valid for validation, @RestControllerAdvice for global error handling. All proxy-based via Spring AOP. Key trap: same-class calls bypass the proxy."

24. Eureka Server — Service Discovery REAL Code

→ Eureka = phone directory. Services register, Gateway poochhe 'kahan hai?', Eureka bata de.

→ Step 1: Eureka Server (ALAG microservice):

```
@SpringBootApplication
@EnableEurekaServer // Eureka SERVER ban gaya
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
// pom.xml: spring-cloud-starter-netflix-eureka-server
```

→ Eureka Server application.yml:

```
server:
  port: 8761 # Default port

eureka:
  client:
    register-with-eureka: false # Khud ko register NAHI
    fetch-registry: false # Doosron ki list NAHI chahiye
    # Ye SIRF server hai — client nahi
```

→ Step 2: Har microservice = Eureka CLIENT:

```
# Survey Service application.yml:
spring:
  application:
    name: survey-service          # Eureka mein YE naam dikhega
                                   # FeignClient ISI naam se call karega
  server:
    port: 8081

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
  instance:
    prefer-ip-address: true      # Docker mein IP use karo

# pom.xml: spring-cloud-starter-netflix-eureka-client
```

WHY Eureka? Bina = hardcoded URLs (http://192.168.1.5:8081). IP badle = code change. Eureka = naam se dhundho, IP badle fark nahi. Load balance free — multiple instances, round-robin auto.

[TRAP] spring.application.name BHOOL gaye? Eureka mein UNKNOWN. FeignClient naam se dhundhta — naam nahi = 503 Service Unavailable.

[TRAP] Eureka server pe register-with-eureka: true rakha? Khud ko khud mein register = error logs. Server pe false, clients pe true (default).

[TRAP] Heartbeat 30s. 90s miss = DEAD mark. Self-preservation: 85%+ miss = NETWORK issue → registry FREEZE — stale > empty.

INTERVIEW: "Eureka handles service discovery. Services register with name+port. Gateway queries Eureka to route. Heartbeat 30s, 90s miss = dead. Self-preservation at 85% miss = network issue, freeze registry."

25. FeignClient — Service-to-Service Call (Declarative HTTP)

→ RestTemplate = 10 lines boilerplate. FeignClient = interface likho, Spring PROXY handle. Same pattern as @Transactional!

→ Step 1: @EnableFeignClients:

```
@SpringBootApplication
@EnableFeignClients          // FeignClient scanning ON
public class SurveyServiceApplication { ... }

// pom.xml: spring-cloud-starter-openfeign
```

→ Step 2: FeignClient interface:

```
@FeignClient(
  name = "user-service",      // Eureka registered naam
  // Eureka se IP+port dhundh ke call karega
  fallback = UserClientFallback.class // Circuit Breaker fallback
)
public interface UserClient {

  @GetMapping("/api/v1/users/{id}")
  UserResponse getUserById(@PathVariable Long id);
  // Interface likho — implementation Spring PROXY banaye
  // HTTP GET to user-service/api/v1/users/5
  // JSON → UserResponse (Jackson auto convert)

  @PostMapping("/api/v1/users")
  UserResponse createUser(@RequestBody UserRequest req);
}
```

→ Step 3: Service mein inject + use:

```
@Service
public class SurveyService {
    private final UserClient userClient; // Inject like any bean

    public SurveyResponse getSurvey(Long id) {
        Survey survey = repo.findById(id).orElseThrow();

        // User Service se user data lao:
        UserResponse user = userClient.getUserById(
            survey.getCreatedBy());
        // Lagta local method – internally HTTP via Eureka!

        return SurveyResponse.builder()
            .title(survey.getTitle())
            .createdByName(user.getName()).build();
    }
}
```

WHY FeignClient? Declarative = interface likho, Spring proxy HTTP kare. RestTemplate = 10 lines URL+headers+exchange. FeignClient = interface + annotations = DONE. Same PROXY pattern — @Transactional, @Cacheable, @FeignClient — sab proxy.

[TRAP] @EnableFeignClients BHOOL gaye? Bean nahi banega = injection FAIL = startup error.

[TRAP] FeignClient name = Eureka naam EXACT match. 'user-service' vs 'userService' = NAHI milega = 503.

[TRAP] Default timeout 60s. Production: feign.client.config.default.connectTimeout=5000, readTimeout=10000. Slow service = POORI chain block.

INTERVIEW: "FeignClient = declarative HTTP client. Interface likho, Spring proxy generate kare. Eureka integration for discovery, Resilience4j for circuit breaking. Same proxy pattern as @Transactional."

26. Circuit Breaker — Cascade Failure Roko (Resilience4j)

→ Service DOWN → 60s timeout → thread pool exhaust → CASCADE. Circuit Breaker = fuse box — ek fuse udhe, baaki safe.

→ 3 States: CLOSED (normal) → 50%+ fail → OPEN (sab block, fallback) → 30s → HALF-OPEN (test calls) → success? CLOSED. fail? OPEN.

→ Config:

```
# pom.xml: spring-cloud-starter-circuitbreaker-resilience4j

# application.yml:
resilience4j:
  circuitbreaker:
    instances:
      userService:
        # Circuit Breaker naam
        sliding-window-size: 10      # Last 10 calls track
        failure-rate-threshold: 50   # 50% fail = OPEN
        wait-duration-in-open-state: 30s # 30s baad HALF-OPEN
        permitted-number-of-calls-in-half-open-state: 3
        # HALF-OPEN mein 3 test calls
        # Success? → CLOSED. Fail? → OPEN again.
```

→ FeignClient fallback class:

```
@FeignClient(name = "user-service",
    fallback = UserClientFallback.class)
public interface UserClient {
    @GetMapping("/api/v1/users/{id}")
    UserResponse getUserById(@PathVariable Long id);
}

// Fallback – Circuit OPEN pe ye chale:
@Component
public class UserClientFallback implements UserClient {

    @Override
    public UserResponse getUserById(Long id) {
        // User Service DOWN? Default response:
        return UserResponse.builder()
            .name("Unknown User") // Graceful degradation
            .email("N/A").build();
        // App CRASH nahi – degraded lekin ALIVE
    }
}
```

→ @CircuitBreaker annotation (alternative):

```
@Service
public class UserServiceClient {

    @CircuitBreaker(name = "userService",
        fallbackMethod = "getUserFallback")
    public UserResponse getUser(Long id) {
        return restTemplate.getForObject(
            "http://user-service/api/v1/users/" + id,
            UserResponse.class);
    }

    // Fallback – SAME return type + Exception param ZARURI
    public UserResponse getUserFallback(Long id, Exception ex) {
        log.warn("User service down, fallback: {}", id);
        return UserResponse.builder()
            .name("Unknown User").build();
    }
    // AOP proxy – same class call = BYPASS!
}
```

WHY Circuit Breaker? Bina = ek service down → 60s wait → thread exhaust → CASCADE → POORA system DOWN. IRIS mein 500+ incidents. Circuit Breaker = fail fast, fallback, cascade prevent.

[TRAP] Fallback method return type + params EXACT match + Exception param ZARURI. Mismatch = fallback CALL nahi = exception propagate.

[TRAP] @CircuitBreaker = AOP proxy. Same class call = BYPASS — same trap as @Transactional, @Cacheable, @Async, @PreAuthorize!

[TRAP] HALF-OPEN = test calls. Success → CLOSED (recovered). Fail → wapas OPEN. Fuse box — check bijli aayi? Haan = ON. Nahi = OFF wapas.

INTERVIEW: "I use Resilience4j Circuit Breaker with FeignClient fallbacks. 3 states — CLOSED, OPEN, HALF-OPEN. Prevents cascade failures. Graceful degradation — partial data instead of crash. IRIS = 15+ microservices, circuit breaker prevented cascades."

27. API Gateway — Spring Cloud Gateway REAL Code

→ Client ko sirf EK URL pata: http://localhost:8080. Gateway decide kare kaunsi request kahan jaaye. 15 services ke 15 URLs yaad rakhne ki zarurat NAHI.

→ 5 kaam EK jagah (ORDER matters): Auth check → Rate Limit → Route to correct service → Load Balance across instances → Log requests.

WHY ye order? Auth PEHLE — invalid token toh route hi mat karo (resource waste). Rate limit DOOSRA — authenticated user bhi spam kar sakta. Route TEESRA — ab pata kahan jaana. Load balance CHAUTHA — kaunsa instance free. Log LAST — sab info available.

WHY Gateway? Cross-cutting concerns (auth, logging, rate limit) har service mein chahiye. 15 jagah copy-paste = DRY violation MASSIVE. Gateway pe ek baar likho = clean. Spring Security filter chain yaad hai? Gateway = POORE system ka filter chain.

→ Step 1: Dependencies (pom.xml):

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

→ Gateway = WebFlux based (reactive, non-blocking). Mono/Flux return karta. Spring MVC (servlet) NAHI. WebFlux KYUN? Non-blocking = 1000s concurrent connections bina thread block kiye. Gateway pe SABSE zyada traffic — blocking = bottleneck.

→ Step 2: application.yml — Route config:

```
server:
  port: 8080

spring:
  application:
    name: API-GATEWAY
  cloud:
    gateway:
      routes:
        - id: survey-route
          uri: lb://SURVEY-SERVICE    # lb = load balanced via Eureka
          predicates:
            - Path=/api/surveys/**    # URL pattern match

        - id: user-route
          uri: lb://USER-SERVICE
          predicates:
            - Path=/api/users/**

        - id: auth-route
          uri: lb://AUTH-SERVICE
          predicates:
            - Path=/api/auth/**

  eureka:
    client:
      service-url:
        defaultZone: http://eureka:8761/eureka
```

→ lb://SURVEY-SERVICE = Load Balanced. Gateway Eureka se poochhe "SURVEY-SERVICE kahan?" → Eureka 3 instances bataye → Gateway Round Robin se ek choose kare. Hardcode URL NAHI — dynamic discovery + load balancing.

→ predicates = condition. Path=/api/surveys/** match kare toh SURVEY-SERVICE pe route. /** = wildcard (kuch bhi match).

→ SURVEY-SERVICE = exact spring.application.name jo Eureka mein registered. Eureka case INSENSITIVE default mein, lekin EXACT naam likho consistency ke liye.

→ Step 3: Custom JWT Filter on Gateway:

```
@Component
public class JwtAuthFilter implements GatewayFilterFactory<Object> {

    @Override
    public GatewayFilter apply(Object config) {
        return (exchange, chain) -> {
            // 1. Header se token nikalo
            String token = exchange.getRequest()
                .getHeaders().getFirst("Authorization");

            // 2. No token? 401 Unauthorized
            if (token == null || !token.startsWith("Bearer ")) {
                exchange.getResponse()
                    .setStatusCode(HttpStatus.UNAUTHORIZED);
                return exchange.getResponse().setComplete();
            }

            // 3. Validate + extract userId
            String jwt = token.substring(7);
            String userId = jwtUtil.extractUserId(jwt);

            // 4. X-User-Id header downstream ko bhejo
            ServerHttpRequest modified = exchange.getRequest()
                .mutate()
                .header("X-User-Id", userId)
                .build();

            // 5. ZARURI: chain.filter() — aage bhejo!
            return chain.filter(
                exchange.mutate().request(modified).build());
        };
    }
}
```

→ Har request PEHLE is filter se guzre. Token nahi? 401. Valid? userId extract → X-User-Id header → downstream. Tera JWT Filter Chain WAHI concept — bas ab GATEWAY level pe.

→ Downstream services ko JWT validate NAHI karna — Gateway already kar chuka. Sirf X-User-Id header padho. DRY principle.

→ Step 4: Route config — Public vs Protected (filter lagao):

```
spring:
  cloud:
    gateway:
      routes:
        # PUBLIC — NO auth filter
        - id: auth-route
          uri: lb://AUTH-SERVICE
          predicates:
            - Path=/api/auth/**
        # No filters — login/register public

        # PROTECTED — JWT required
        - id: survey-route
          uri: lb://SURVEY-SERVICE
          predicates:
            - Path=/api/surveys/**
          filters:
            - JwtAuth # custom filter lagao
```

→ /api/auth/** = public (login, register). /api/surveys/** = protected (JWT required). Route-level control — kahan auth chahiye kahan nahi.

→ Gateway STATELESS — JWT validate karo, kuch store nahi. 3 instances chalao = ek down toh baaki handle. Scaling EASY kyunki no state.

WHY Gateway pe JWT? 15 services mein JWT validation code copy = DRY violation MASSIVE. Gateway pe EK baar validate, downstream ko sirf X-User-Id bhejo. Service ko JWT library import bhi nahi karni.

[TRAP] Gateway mein spring-boot-starter-web ADD mat karna! Gateway = WebFlux (reactive, Mono/Flux). MVC + WebFlux = CONFLICT = BeanCreationException = app START nahi hoga.

[TRAP] chain.filter() BHOOL gaye = request SILENTLY DROP. Client wait karta... TIMEOUT. Koi error nahi, koi response nahi — sirf silence. Debug MUSHKIL. HAMESHA chain.filter() call karo! Same trap as JwtAuthFilter mein chain.doFilter().

[TRAP] Gateway = SINGLE POINT OF FAILURE. Ek gateway down = POORA system down. FIX: 2+ gateway instances + Nginx/AWS ALB upar. Gateway STATELESS — JWT validate, kuch store nahi. 3 instances = ek down toh baaki handle.

[TRAP] Gateway pe BUSINESS LOGIC mat likho! Sirf cross-cutting: auth, rate limit, CORS, logging. Business logic (validation, DB calls) = SERVICE mein. Gateway THIN, service THICK.

[QUIZ GALT 1] Service Name case: Maine socha 503 aayega agar lowercase likhe. GALAT — Eureka case INSENSITIVE hai default mein. lb://survey-service aur lb://SURVEY-SERVICE dono kaam karte. Lekin BEST PRACTICE = EXACT naam likho jaise spring.application.name mein hai. Consistency ke liye, bug se bachne ke liye.

[QUIZ GALT 2] chain.filter() BHOOL: Maine socha 500 error aayega. GALAT — chain.filter() nahi call kiya toh request SILENTLY DROP hota. Koi error nahi, koi log nahi, client bas wait karta aur TIMEOUT. Debug MUSHKIL kyunki silence. Same trap as chain.doFilter() in JwtAuthFilter. HAMESHA chain call karo!

INTERVIEW: “API Gateway = single entry point. 5 kaam ORDER: Auth → Rate Limit → Route → Load Balance → Log. Spring Cloud Gateway with WebFlux for non-blocking high throughput. lb:// URI = Eureka-backed load balancing. JWT centralized on gateway, X-User-Id header forwarded downstream. At IRIS, API Gateway routes across 15+ microservices.”

INTERVIEW: “Gateway pe auth kyun, service pe kyun nahi? DRY principle. 15 services mein JWT code copy = maintenance nightmare. Gateway pe EK baar validate, downstream ko X-User-Id header bhejo. Service ko JWT library bhi import nahi karni.”

28. docker-compose — Multi-Service Orchestration (Concepts Only)

WHY docker-compose? 15 services hain — har ek ko manually start kare? MySQL start, Redis start, Eureka start... 15 baar terminal khole? docker-compose = EK command se SAB start. docker-compose up = sab start. docker-compose down = sab band. POORA infrastructure EK file mein define.

→ services: har service = ek container. MySQL, Redis, Eureka, Survey, User, Gateway = 6 containers ek file mein.

→ image: mysql:8 = Docker Hub se pre-built image. build: ./survey-service = tera code + Dockerfile se image BUILD karo.

→ environment: container ke andar env variables set. application.yml mein \${DB_HOST} se read karo.

→ ports: “3307:3306” = LEFT host machine port, RIGHT container port. Host pe 3307 se access, container mein 3306 pe chalta.

→ volumes: mysql_data:/var/lib/mysql = NAMED VOLUME. Container delete karo = data SAFE. Bina volume = container restart = DATA GONE.

→ depends_on: START order. MySQL pehle, phir survey-service. Lekin READY ka guarantee NAHI — sirf start order.

→ networks: sab containers ek network pe. Ek doosre ko SERVICE NAAM se call kar sakte — Docker internal DNS resolve karta.

[TRAP] DB_HOST: localhost NAHI! Docker mein localhost = APNA container. MySQL ALAG container mein hai. DB_HOST: mysql likhna = Docker DNS resolve karega. LOCALHOST = CONNECTION REFUSED!

[TRAP] Container-to-container port = INTERNAL port (3306), mapped port NAHI (3307). 3307 sirf HOST se access ke liye hai. DB_PORT: 3306 likhna, 3307 NAHI.

[TRAP] depends_on = start ORDER, READY nahi. MySQL start hota 10 sec lagta. App 2 sec mein start = MySQL ready NAHI = connection fail. FIX: app mein retry logic ya healthcheck + condition: service_healthy.

[TRAP] volumes BHOOL gaye = container restart = MySQL ka data GONE. Production mein HAMESHA named volume ya bind mount.

INTERVIEW: “docker-compose defines multi-container apps in one YAML file. Services communicate via service names as DNS hostnames — not localhost. Named volumes persist data across restarts. depends_on controls start order but not readiness. At IRIS, 15+ services orchestrated with containers.”

INTERVIEW: “Docker mein services kaise communicate karti? Service naam = DNS hostname. survey-service se mysql ko call = DB_HOST: mysql. Docker internal DNS resolve karta. localhost = apna container, doosre container ka nahi.”